

грокаем

конкурентность

Кирилл Бобров



Дорогие русскоязычные читатели!

Я рад приветствовать вас на страницах перевода моей книги «Grokking Concurrency». Для меня большая честь представить вам эту работу на моем родном языке.

Конкурентное программирование — штука не из легких, но чертовски важная. В начале карьеры инженеры часто стараются обходить эту тему стороной, как кот старается избежать воды. Но рано или поздно столкнуться с ней все равно придется. Возможно, вам понадобится масштабировать вашу систему, написать потокобезопасный код или исправить баг в уже существующем. Многоядерные процессоры и распределенные системы уже давно стали нормой, и умение эффективно использовать и правильно работать с конкурентностью необходимо для создания высокопроизводительных приложений.

На русском языке термин «concurrency» может означать много разных вещей, включая «параллелизм», «многозадачность», «асинхронность» и «согласованность». Все эти вещи важны для понимания конкурентности, и я постарался объяснить их как можно проще и понятнее в этой книге. А команда издательства «Питер» постаралась перевести их максимально точно для вас, читателей.

Вместо сухого перечня API и списка классов я знакомлю вас с концепциями, сущностями, паттернами и моделями, которые не зависят от языка программирования и всегда будут актуальны. А чтобы чтение было веселее, я добавил пару историй и примеров из жизни. Надеюсь, они помогут вам лучше понять материал и сделают процесс обучения интереснее.

Эта книга написана с целью сделать сложные концепции доступными и понятными для каждого в стиле серии Grokking. Надеюсь, что она поможет вам разобраться в конкурентном программировании и вдохновит на создание новых крутых проектов. Ваш интерес и стремление к знаниям — это то, что движет мир технологий вперед.

С нетерпением жду ваших отзывов и впечатлений. Желаю вам успешного изучения и применения полученных знаний!

Кирилл Бобров

grokking concurrency

Kirill Bobrov


MANNING
SHELTER ISLAND

грокаем

конкурентность

Кирилл Бобров



Санкт-Петербург • Москва • Минск

2025

Кирилл Бобров
Грокаем конкурентность

Перевел с английского Е. Матвеев

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Е. Строганова</i>
Литературный редактор	<i>Р. Чебыкин</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>С. Беляева, М. Котова</i>

ББК 32.973.2-018

УДК 004.424

Бобров Кирилл

Б72 Грокаем конкурентность. — СПб.: Питер, 2025. — 272 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-4102-9

Конкурентность позволяет эффективно выполнять компьютерные программы, разделяя их на задачи, которые можно запускать независимо. Такой подход помогает ускорить игровую графику, обучать большие модели искусственного интеллекта, быстро масштабировать веб-приложения, оптимизировать обработку больших данных и решать многие другие задачи. Работать с конкурентностью непросто, так что эта книга постепенно введет вас в курс дела, а помогут в этом интересные примеры, забавные иллюстрации и понятный код на Python.

Вы изучите приемы, с помощью которых сможете программировать многоядерные и графические процессоры, а также другие высокопроизводительные системы. Кирилл Бобров обходится без сложной математики, технического жаргона и тяжеловесных научных рассуждений, предпочитая простые и доступные объяснения.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ISBN 978-1633439771 англ.

Authorized translation of the English edition © 2024

This translation is published and sold by permission of Manning Publications, the owner of all rights to publish and sell the same.

ISBN 978-5-4461-4102-9

© Перевод на русский язык ООО «Прогресс книга», 2024

© Издание на русском языке, оформление ООО «Прогресс книга», 2025

© Серия «Библиотека программиста», 2025

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес:

190444, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 09.2024. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 —

Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 19.07.24. Формат 70×100/16. Бумага офсетная. Усл. п. л. 21,930. Тираж 1500. Заказ 0000.

Оглавление



Предисловие	10
Благодарности	12
Об этой книге	14
Для кого эта книга	14
Как устроена эта книга	15
О коде в книге	15
Форум liveBook	16
Об авторе	17
От издательства	18
ЧАСТЬ I. ОРКЕСТР ОСЬМИНОВ. СИМФОНΙΑ КОНКУРЕНТНОСТИ	19
Глава 1. Знакомство с конкурентностью	20
Почему конкурентность важна	21
Уровни конкурентности	27
Что вы узнаете из этой книги	29
Итоги	31
Глава 2. Последовательное и параллельное выполнение	32
Обзор основных понятий: что такое программа	33
Последовательное выполнение	35
Требования к параллельным вычислениям	41

Параллельные вычисления	43
Закон Амдала	49
Закон Густафсона	54
Конкурентность и параллелизм	55
Итоги	57
Глава 3. Как работают компьютеры	59
Процессор	60
Система выполнения	64
Архитектура компьютерных систем	66
Уровни конкурентного оборудования	67
Итоги	72
Глава 4. Структурные элементы конкурентности	74
Этапы конкурентного программирования	75
Потоки	80
Итоги	86
Глава 5. Межпроцессная коммуникация	87
Типы коммуникации	88
Пул потоков	98
Снова о подборе паролей	101
Итоги	103
ЧАСТЬ II. ЩУПАЛЬЦА КОНКУРЕНТНОСТИ: МНОГОЗАДАЧНОСТЬ, ДЕКОМПОЗИЦИЯ И СИНХРОНИЗАЦИЯ	105
Глава 6. Многозадачность	106
Приложения с интенсивными вычислениями (CPU-bound) и интенсивным вводом/выводом (I/O-bound)	107
Почему необходима многозадачность	109
Общие принципы многозадачности	112
Многозадачные среды	119
Итоги	121
Глава 7. Декомпозиция	123
Анализ зависимостей	124
Декомпозиция задач	125
Декомпозиция задач: паттерн «Конвейер»	128
Декомпозиция данных	133

Уровень детализации	143
Итоги	145
Глава 8. Как решать проблемы конкурентности: ситуации гонки и синхронизация	146
Общие ресурсы	147
Ситуация гонки	148
Синхронизация	153
Итоги	161
Глава 9. Как решать проблемы конкурентности: взаимные блокировки и нехватка ресурсов	162
Обедающие философы	163
Взаимные блокировки	165
Нехватка ресурсов	173
Как проектировать синхронизацию	175
Несколько слов напоследок	181
Итоги	181
ЧАСТЬ III. АСИНХРОННЫЕ ОСЬМИНОГИ. ИСТОРИЯ О КОНКУРЕНТНОМ ВЫПОЛНЕНИИ СО ВКУСОМ ПИЦЦЫ	183
Глава 10. Неблокирующий ввод/вывод	184
Мир распределенных вычислений	185
Модель «клиент — сервер»	185
Служба заказа пиццы	188
Блокирующий ввод/вывод	195
Неблокирующий ввод/вывод	198
Итоги	201
Глава 11. Событийная конкурентность	203
События	204
Обратные вызовы	205
Цикл событий	206
Мультиплексирование ввода/вывода	208
Событийный сервер для заказа пиццы	209
Паттерн «Реактор»	212
Синхронизация при передаче сообщений	214
Модели ввода/вывода	217
Итоги	219

Глава 12. Асинхронная коммуникация	220
Зачем нужна асинхронность	221
Асинхронные вызовы процедур	222
Кооперативная многозадачность	223
Предназначенные объекты	229
Кооперативный сервер для заказа пиццы	232
Асинхронная пиццерия	237
Асинхронная модель: выводы	243
Итоги	244
Глава 13. Как писать конкурентные приложения	245
Так что же такое конкурентность?	246
Методология Фостера	247
Умножение матриц	249
Распределенный подсчет слов	258
Итоги	271
Эпилог	272

*Посвящаю родителям — Елене и Андрею,
замечательной паре, которая создала этот шедевр (меня);
а также моей жене Кате,
которая помогла мне не сойти с ума в этом мире,
полном багов и накладок.*

Предисловие



Представьте себе мир, в котором за развитием технологии угнаться сложнее, чем за гепардом, накачанным стимуляторами, а спрос на эффективное конкурентное программирование неудержимо растет. В этом мире разработчики сталкиваются с серьезной задачей: как создавать системы, которые способны справиться с колоссальными объемами данных и обрабатывать их достаточно проворно, чтобы удовлетворить неутолимые потребности пользователей? В этом мире конкурентность не только пленила умы, но и стала головоломкой, которую приходится разгадывать. И в этом мире мы живем сегодня.

Когда-то и я попался на эту приманку. Затем мне встретились понятия «конкурентность» и «асинхронность». И то, что я обнаружил, оказалось настоящей золотой жилой: местом неосвоенной силы, которая может превратить обычную программу во впечатляющую манифестацию вычислительной мощи. Однако сокровища залежали под массивными пластами сложности, а разные части головоломки: конкурентность, параллелизм, потоки, процессы, многозадачность, корутины — были разбросаны по всему технологическому ландшафту. Мне отчаянно не хватало наставника, который собрал бы все составляющие воедино и представил полную картину. Я так и не смог найти ресурс, который бы заполнял пробелы между теорией и практикой в разных языках программирования, поэтому решил создать такой ресурс своими силами. Так родилась книга «Грокаем конкурентность», которая станет вашим верным помощником, будет сопровождать вас, нашептывать секреты и освещать дорогу сквозь лабиринты на пути к мастерству.

Эта книга — не типичное техническое пособие; она призвана увлечь вас, вплетая в материал занимательные истории и пасхалки. Книга выросла из теоретического руководства, насыщенного побасенками, культурными отсылками (сумеете обнаружить их все?) и забавными иллюстрациями. Эта книга не стесняется ни юмора, ни глубокой привязанности к пельменям и пицце — кто сказал, что изучать конкурентность нужно уныло и формально?

Совместными усилиями мы справимся со сложностями конкурентности и раскроем тайны асинхронности. На всем пути, от азов конкурентности до `asunc` и `await`, нас будет сопровождать верный спутник Python — язык программирования, который я выбрал для этой книги. Не беспокойтесь, если это не ваш основной язык: понятия и приемы, о которых пойдет речь, не привязаны ни к какому конкретному языку.

«Но почему Python?» — спросите вы. Дело в том, что этот язык выдерживает идеальный баланс между простотой и выразительностью, благодаря чему мы сможем сосредоточиться на сути конкурентности и не отвлекаться на второстепенные мелочи. Кроме того, не буду скрывать, что Python мне просто нравится.

В этой книге найдется что-то полезное для самых разных читателей — как для опытных разработчиков, которые стремятся глубже разобраться в конкурентных системах, так и для любознательных новичков, которые только осваивают основы конкурентности. Вместе мы раскроем секреты конкурентного выполнения, и вы научитесь строить масштабируемые, эффективные и гибкие программные системы, которые способны справиться с любыми проблемами.

Приготовьтесь отправиться в совершенно уникальное путешествие, где размываются границы времени и пространства, а программы танцуют под ритмы осьминогов. Да, вы правильно прочитали — осьминогов. Эти очаровательные обитатели морских глубин, чьи восемь щупалец извиваются в идеальной гармонии, символизируют нетривиальную, завораживающую природу конкурентных систем, которые нам с вами предстоит исследовать. Итак, в путь!

Благодарности



Прежде чем погружаться в тонкости конкурентности, я хочу выразить благодарность замечательным людям, без которых эта книга не увидела бы свет. Говорят, что написать книгу — все равно что пробежать марафонскую дистанцию, но я бы сказал, что это больше похоже на головокружительную поездку на американских горках. И как же я рад, что все эти люди были рядом!

Прежде всего я безгранично благодарен своей жене Екатерине Кривец, которая считает, будто я — творческая личность, хотя именно ей принадлежат все потрясающие иллюстрации в этой книге.

Хочу также поблагодарить всех родных и близких. Их любовь и поддержка неустанно вдохновляли меня в этом проекте, и мне повезло, что такие люди есть в моей жизни.

Я глубоко признателен группе поддержки, которая содействовала мне на всех этапах работы. Отдельные благодарности — Кристине Ялышевой, Михаилу Полторацкому, Татьяне Бородиной, Андрею Гаврилову и Александру Бельницкому, которые всегда верили в меня и были готовы прийти на помощь. Хочу особо поблагодарить Веру Кривец за то, что помогала мне совершенствоваться в английском.

Отдельно упомяну Берта Бейтса (Bert Bates) и Брайана Ханафи (Brian Hanafee); их наставничество и конструктивные соображения научили меня по-новому преподавать и излагать сложные концепции. Спасибо за ваши бесценные советы и вклад в мое становление как автора.

Я глубоко благодарен потрясающей команде издательства Manning Publications. Майк Стивенс (Mike Stephens), мое приключение началось благодаря тебе, и я неимоверно рад, что ты поверил в меня. Иэн Хаф (Ian Hough) был моим наставником, терпеливо прорабатывал со мной главу за главой и приводил в порядок мой английский. Он заслуживает самой высокой награды за то, что пережил процесс редактирования! Спасибо Артуру Зубареву, который одолел мои неуклюжие черновики и поделился ценной обратной связью. Лу Кови (Lou Covey), прости за периодические размолвки, но твои истории всегда были интересными и поддерживали меня в ра-

боте. Марк Томас (Mark Thomas), твоя научная редактура и помощь с программным кодом сильно изменили книгу к лучшему. Тиффани Тейлор (Tiffany Taylor), без твоей внимательности и опыта текст не получился бы таким ясным и связным. Кэти Теннант (Katie Tennant), благодаря твоей скрупулезности и ценным редакторским замечаниям материал удалось отшлифовать и подготовить к публикации.

Мои рецензенты, спасибо вам всем: Абхиджит Наяк (Abhijith Nayak), Амрах Умудлу (Amrah Umudlu), Андрес Сакко (Andres Sacco), Арно Байи (Arnaud Bailly), Балбир Сингх (Balbir Singh), Биджит Комалан (Bijith Komalan), Клиффорд Тёрбер (Clifford Thurber), Дэвид Якобович (David Yakobovitch), Дмитрий Воробьев, Эдду Мелендес (Eddu Melendez), Эрнесто Арройо (Ernesto Arroyo), Эрнесто Босси (Ernesto Bossi), Эшан Тендон (Eshan Tandon), Эзра Шрёдер (Ezra Schroeder), Франс Ойлинки (Frans Oilinki), Ганеш Свамнатан (Ganesh Swaminathan), Гленн Госсенс (Glenn Goossens), Грегори Варгис (Gregory Varghese), Имакулет Ресто Моша (Imaculate Resto Mosha), Джеймс Чжицзюнь Лю (James Zhijun Liu), Иржи Чинчура (Jiří Činčura), Джонатан Ривз (Jonathan Reeves), Лаванья М. К. (Lavanya M K), Люк Рогге (Luc Rogge), Манодж Редди (Manoj Reddy), Мэтт Гуковски (Matt Gukowsky), Мэтт Вельке (Matt Welke), Микаэль Дотри (Mikael Dautrey), Нолан То (Nolan To), Оливер Кортен (Oliver Korten), Патрик Гётц (Patrick Goetz), Патрик Реган (Patrick Regan), Рагунат Джавахар (Ragunath Jawahar), Саи Хегди (Sai Hegde), Серхио Арбео Родригес (Sergio Arbeo Rodríguez), Широшика Кулатилаке (Shiroshica Kulatilake), Венката Нагиндра Бабу Янамадала (Venkata Nagendra Babu Yanamadala), Виталий Ларченков и Уильям Джамир (William Jamir). Ваши замечания помогли улучшить книгу.

Кроме того, хочу обратиться к безымянным героям, которые неустанно трудились за кулисами, чтобы эта книга вышла в свет. Вы знаете, о ком я говорю, и хотя у вас нет эффектного супергеройского плаща, вы — тот «секретный ингредиент», без которого ничего бы не получилось. Вы — настоящие суперзвезды!

Наконец, как сказал бы Снуп Догг, я хочу поблагодарить себя, без которого эта книга никогда бы не написалась!

Об этой книге



Эта книга была написана для того, чтобы развеять мистический ореол, который окружает конкурентность, асинхронность и параллельное программирование, познакомить вас с фундаментальными основами этих понятий и продемонстрировать, как они применяются на практике. В отличие от научных исследований или работ, посвященных конкретным языкам, эта книга стремится объяснять базовые идеи и принципы, а не отдельные реализации. Здесь рассматриваются высокоуровневые понятия, материал изложен доступным языком, а вместо сложных математических выкладок используются наглядные диаграммы; все это помогает лучше разобраться в предмете. Знания, которые вы получите, помогут ориентироваться в конкурентных фреймворках и разрабатывать масштабируемые решения в интересующей вас области. Книга в некоторой степени восполняет нехватку доступных ресурсов по этой теме; это подробное и доходчивое руководство для читателей, которые хотят разобраться в понятиях конкурентности и асинхронности. Она экономит время программистам, которым понадобились бы годы практической работы, чтобы самостоятельно получить те же знания.

Для кого эта книга

Книга идеально подойдет для всех, кто хочет изучить основы конкурентности. Чтобы извлечь из материала максимальную пользу, желательно в общих чертах разбираться в том, как работают компьютерные системы, понимать основные концепции и структуры данных языков программирования, а также иметь опыт написания последовательных программ. Предварительные знания об операционных системах не нужны: вся необходимая информация есть в книге. Хотя здесь фигурируют понятия из области компьютерных сетей, они не объясняются во всех подробностях: книга предполагает, что вы немного представляете себе, как

устроены сети. Вам не обязательно глубоко разбираться во всех этих областях, а если понадобится подтянуть свои знания по той или иной теме, этим можно будет заниматься по ходу дела.

Как устроена эта книга

Книга делится на три части. В части I «Оркестр осьминогов. Симфония конкурентности» рассматриваются фундаментальные понятия и примитивы написания конкурентных программ. Главы 1–5 посвящены основным принципам конкурентности на разных уровнях — от аппаратного до прикладного.

В части II «Щупальца конкурентности. Многозадачность, декомпозиция и синхронизация» идет речь о том, как абстракции и популярные паттерны помогают сделать код более производительным, масштабируемым и гибким. В главах 6–9 вы научитесь преодолевать распространенные проблемы, которые возникают при разработке конкурентных систем.

Часть III «Асинхронные осьминоги. История о конкурентном выполнении со вкусом пиццы» расширяет ваши знания в области конкурентности: вместо одной машины вы будете иметь дело с несколькими машинами, которые связаны по сети. В этом контексте события могут происходить асинхронно, то есть разные события не обязательно привязаны друг к другу по времени. Понятие асинхронности занимает центральное место в главах 10–12, где эта тема освещается в разных измерениях. С помощью асинхронности создается иллюзия конкурентного или параллельного выполнения задач, однако в современных реализациях можно совмещать асинхронные операции с действительно конкурентными, что позволяет значительно повысить производительность системы. Книгу завершает глава 13, где собраны задачи из области конкурентности. Мы решим их шаг за шагом, чтобы убедиться в том, что вы действительно освоили тему конкурентности.

О коде в книге

Исполняемые фрагменты кода можно загрузить из онлайн-версии этой книги по адресу <https://livebook.manning.com/book/grokking-concurrency>. Полный код примеров из книги доступен на сайте издательства Manning (<https://www.manning.com>) и на GitHub (https://github.com/luminousmen/grokking_concurrency). Исходный код задуман как справочник о том, как можно реализовывать программы. Эти примеры оптимизированы для *обучения*, а не для *реальной эксплуатации*. Код специально составлен так, чтобы служить учебным пособием. В реальных проектах стоит использовать проверенные библиотеки и фреймворки, потому что они обычно оптимизированы для производительности, хорошо протестированы и качественно поддерживаются.

Форум liveBook

Приобретая книгу «Грокаем конкурентность», вы также получаете бесплатный доступ к платформе для онлайн-чтения liveBook издательства Manning (на английском языке). Эксклюзивные возможности liveBook позволяют оставлять комментарии как к книге в целом, так и к отдельным ее разделам или абзацам. Можно легко делать заметки для себя, задавать технические вопросы и отвечать на них, а также получать помощь от авторов и других пользователей. Чтобы получить доступ к форуму, посетите страницу <https://livebook.manning.com/book/grokking-concurrency/discussion>. Информацию о форумах Manning и правилах поведения на них см. на <https://livebook.manning.com/discussion>.

В рамках своих обязательств перед читателями издательство Manning предоставляет ресурс для содержательного общения читателей и авторов. Эти обязательства не подразумевают конкретную степень участия автора, которое остается добровольным (и неоплачиваемым). Задавайте авторам хорошие вопросы, чтобы им было интересно участвовать в диалоге! Форум и архивы обсуждений доступны на сайте Manning, пока книга продолжает издаваться.

Об авторе



Кирилл Бобров — опытный разработчик с непростым характером, который отлично разбирается в разработке и проектировании высоконагруженных приложений. Питая страсть к инженерии данных, он направляет усилия на то, чтобы реализовывать передовые практики инженерии данных для компаний по всему миру. А еще этот угрюмый тип выступает в образе симпатичного кота, который ведет популярный иллюстрированный техноблог <https://luminousmen.com>.

От издательства



Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Оркестр осьминогов. Симфония конкурентности



Вот вы сидите в кофейне за чашечкой капучино, и вдруг ваше внимание привлекает группа программистов за соседним столиком, которые с энтузиазмом беседуют о конкурентности. Они легко жонглируют разными терминами — «параллельные вычисления», «потoki», «межпроцессная коммуникация», — от которых у вас голова идет кругом. Не огорчайтесь, вы в этом не одиноки.

Все, кто бывал на концертах симфонической музыки, слышали характерный эффект, когда во время настройки перед концертом музыканты одновременно играют на разных инструментах разные мелодии. Это прекрасный хаос, который сам по себе подобен своеобразному музыкальному произведению. У него много общего с явлением конкурентности: существуют несколько процессов или потоков, которые выполняются одновременно, чтобы достичь общей цели.

В главах 1–5 представлены основы конкурентности, некоторые принципы работы компьютеров и разные виды примитивов конкурентности. Мы рассмотрим последовательные и параллельные вычисления, узнаем, какие аппаратные и программные компоненты нужны для конкурентного выполнения, а также изучим разные типы межпроцессной коммуникации, которые позволяют нескольким процессам органично работать совместно.

Итак, возьмите чашечку капучино и присоединяйтесь к беседе. Обещаю, что вы не пожалеете о потраченном времени.

1 | Знакомство с конкурентностью

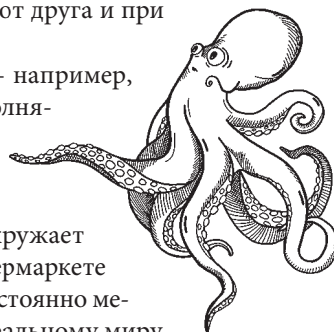


В этой главе

- ✓ Почему конкурентность — важная тема, которую стоит изучить
- ✓ Как измерять производительность систем
- ✓ Какие бывают уровни конкурентности

Выгляните в окно и повнимательнее присмотритесь к окружающему миру. Разве все вокруг происходит по линейной, последовательной схеме? Нет, скорее, вы убедитесь, что мир больше похож на сложное переплетение компонентов, которые все одновременно функционируют независимо друг от друга и при этом взаимодействуют друг с другом.

Хотя люди склонны мыслить последовательно — например, когда они просматривают список текущих дел и выполняют задачи одну за другой, — реальный мир устроен намного сложнее. В нем гораздо больше параллельного, чем последовательного. Взаимосвязанные события случаются одновременно. Конкурентность окружает нас повсюду — от хаотичной толкотни в людном супермаркете до слаженных перемещений футболистов на поле и постоянно меняющегося потока дорожного движения. Подобно реальному миру, компьютеры должны уметь выполнять операции в конкурентном



режиме, чтобы с их помощью можно было моделировать, имитировать и анализировать сложные явления реального мира.

Принцип конкурентности в компьютерных технологиях заключается в том, что система работает сразу с несколькими задачами. Такой системой может быть программа, компьютер или сеть компьютеров. Без конкурентных вычислений наши приложения неизбежно отстали бы от сложности окружающего мира.

Но когда мы начинаем глубже погружаться в тему конкурентности, могут возникнуть определенные вопросы. Первый вопрос (на случай, если я вас еще не убедил) — почему вообще стоит уделять внимание конкурентности?

Почему конкурентность важна

Конкурентность играет чрезвычайно важную роль в разработке программных продуктов. Спрос на высокопроизводительные приложения и конкурентные системы таков, что конкурентное программирование становится критически важным навыком для разработчиков.

Конкурентное программирование — не новое понятие, но за последние годы внимание к нему существенно усилилось. По мере того как в современных компьютерных системах растет количество ядер и процессоров, конкурентное программирование превращается в необходимое требование для разработки ПО. Компании ищут разработчиков, которые уверенно владеют конкурентным программированием, потому что оно часто оказывается единственным выходом в ситуациях, когда нужна высокая производительность, но вычислительные ресурсы ограничены.

Самое значительное преимущество конкурентного программирования (и исторически первая причина для исследований в этой области) состоит в том, что оно помогает повысить производительность систем. Разберемся, почему это важно.

Конкурентность помогает повысить производительность

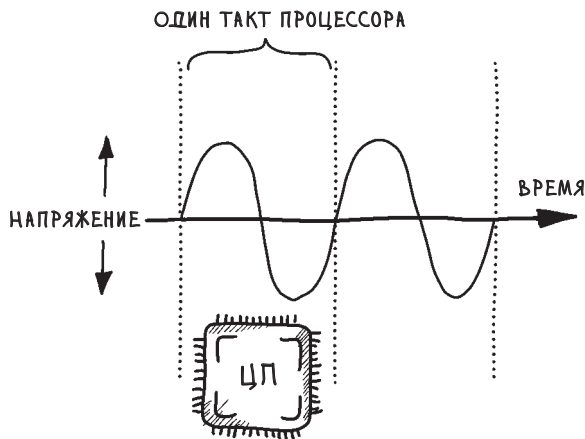
Если нужно повысить производительность, почему бы просто не купить более быстрые компьютеры? Десятки лет назад так и поступали, но в какой-то момент выяснилось, что быстрые компьютеры уже не решают проблему.

Закон Мура

В 1965 году Гордон Мур, один из основателей Intel, обнаружил любопытную закономерность. Новые модели процессоров появлялись примерно через два года после своих предшественников, и каждый раз количество транзисторов на них приблизительно удваивалось. Мур заключил, что количество транзисторов — и, как следствие, тактовая частота процессоров — удваивается через каждые 24 месяца. Это наблюдение стало известно как *закон Мура*. Для разработчиков это означало, что нужно было подождать всего два года, чтобы приложение стало работать вдвое быстрее.



Проблема в том, что около 2002 года правила изменились. Как сказал знаменитый эксперт по C++ Херб Саттер (Herb Sutter), «бесплатные обеды закончились»¹. Обнаружилась фундаментальная связь между физическим размером процессора и скоростью обработки (частотой процессора). Время, за которое выполняется операция, зависит от длины цепи и скорости света. Проще говоря, транзисторы (главные структурные блоки компьютерных микросхем) нельзя добавлять в процессор бесконечно. Немаловажную роль играет и повышение температуры. Стало невозможно улучшать производительность только за счет того, чтобы повышать частоту процессора. С этого момента начался так называемый *кризис многоядерности*.



Развитие отдельных процессоров (в смысле повышения тактовой частоты) остановилось из-за физических ограничений, а необходимость повышать производительность систем осталась. Производители переключились на горизонтальное расширение в форме многопроцессорных систем, что заставило разработчиков, архитекторов и создателей языков адаптироваться к архитектурам со множественными вычислительными ресурсами.

Главный вывод из этого исторического обзора таков: важнейшим преимуществом конкурентности (и исторически первой причиной для исследований в этой

¹ Herb Sutter. «The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software», <http://www.gotw.ca/publications/concurrency-ddj.htm>.

области) стало то, что она позволяет повысить производительность системы так, чтобы при этом эффективно использовать дополнительные вычислительные ресурсы. И здесь возникают два важных вопроса: как измерить эту производительность и как ее улучшить?

Задержка и пропускная способность

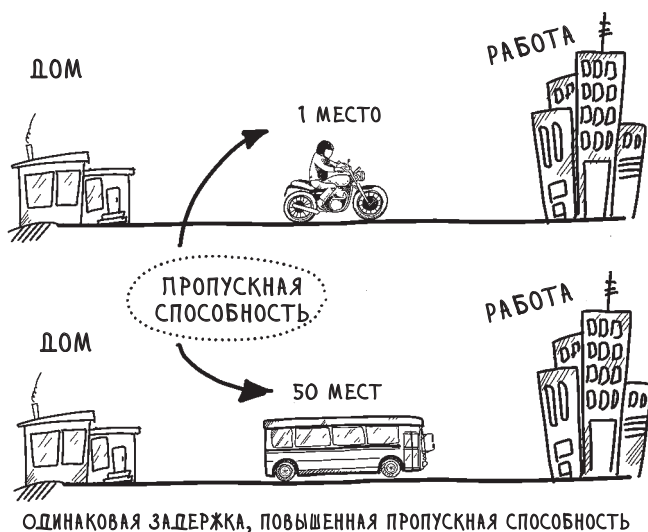
В компьютерных технологиях производительность можно измерять по-разному в зависимости от того, как мы рассматриваем вычислительную систему. Один из способов повысить объем работы — сократить время, за которое выполняются отдельные операции.

Допустим, вы ездите из дома на работу на мотоцикле, и дорога в одну сторону занимает один час. Для вас важно, насколько быстро вы добываетесь до работы, поэтому вы измеряете производительность системы по этой метрике. Если вы будете ехать быстрее, то быстрее окажетесь на работе. С точки зрения компьютерной системы такой эффект называется *задержкой* (latency). Задержка — это метрика, которая измеряет, сколько времени занимает выполнение отдельной задачи от начала до конца.



Теперь представьте, что вы работаете в управлении общественного транспорта и вам поручено повысить производительность автобусной сети. На этот раз ваша задача не в том, чтобы один человек доехал до работы быстрее, а в том, чтобы увеличить количество людей, которые добираются из дома на работу в единицу времени. Такой эффект называется *пропускной способностью* (throughput): эта метрика измеряет количество задач, которые система может выполнить за тот или иной период времени.

Очень важно понимать, чем задержка отличается от пропускной способности. Даже если мотоцикл едет вдвое быстрее автобуса, пропускная способность автобуса в 25 раз больше. (Мотоцикл перевозит одного человека на заданное расстояние за 1 час, тогда как автобус перевозит 50 человек на то же расстояние за 2 часа: если усреднить по времени, получается 25 человек в час.) Иначе говоря, более высокая пропускная способность системы не обязательно означает более низкую задержку. Если оптимизировать производительность, то улучшение по одному показателю (например, по пропускной способности) может привести к ухудшению по другому (например, по задержке).



Конкурентность помогает снизить задержку. Например, если задача занимает много времени, ее можно разбить на меньшие задачи, которые выполняются параллельно, что сократит общее время выполнения. Конкурентность также помогает повысить пропускную способность, если несколько задач обрабатываются одновременно.

Кроме того, конкурентность может скрывать задержку. Когда вы ждете важного звонка, автобуса до работы и т. д., можно просто ждать, а можно направить вычислительные ресурсы на другую деятельность. Например, в ожидании автобуса можно проверить электронную почту. Таким образом вы фактически выполняете несколько задач одновременно и скрываете задержку благодаря тому, что эффективно используете время ожидания. Скрытие задержки играет ключевую роль при разработке систем с малым временем отклика и применяется в задачах, где фигурирует ожидание.

Итак, конкурентность может улучшить производительность системы по трем основным направлениям:

- сократить задержку (чтобы единица работы выполнялась быстрее);
- скрыть задержку (чтобы система делала что-то еще во время операции с высокой задержкой);
- повысить пропускную способность (чтобы система выполняла больше работы в единицу времени).

Мы увидели, как конкурентность помогает повысить производительность, а теперь рассмотрим, для чего еще она применяется на практике. Ранее в этой главе упоминалось, что конкурентность необходима, чтобы моделировать сложный мир, который нас окружает. Теперь можно конкретнее поговорить о том, как с ее помощью решать большие или сложные задачи вычислительными средствами.

Конкурентность помогает решать сложные и большие задачи

Многие задачи, которые стоят перед разработчиками систем, имеющих отношение к реальному миру, настолько сложны, что решать их в последовательной системе практически нереально. Сложность может быть обусловлена размером задачи или тем, насколько трудно разобраться в отдельной части разрабатываемой системы.

Масштабируемость

Размер задачи связан с *масштабируемостью* (scalability); это характеристика системы, производительность которой можно повысить, если добавить дополнительные ресурсы. Масштабировать систему можно в двух направлениях: по вертикали или по горизонтали.

Вертикальное масштабирование повышает производительность, увеличивая объем памяти (чтобы нарастить существующие вычислительные ресурсы) или заменяя процессор на более мощный. В этом случае масштабирование ограничено: очень сложно повысить скорость отдельного процессора, зато очень легко упереться в потолок производительности. Кроме того, переход на более мощные вычислительные ресурсы (например, покупка суперкомпьютера) обходится недешево, потому что вам приходится все больше платить за мощные облачные экземпляры или оборудование, получая при этом все меньше выигрыша.

Можно еще немного выгадать, если удастся сократить процессорное время, которое необходимо для той или иной единицы работы, но в конечном счете придется масштабировать систему по горизонтали. *Горизонтальное масштабирование* увеличивает производительность программ или систем за счет того, что нагрузка распределяется между существующими и новыми вычислительными ресурсами. Пока есть возможность добавлять новые ресурсы, можно наращивать производительность системы. В этом случае проблемы с масштабированием возникнут не так скоро, как при вертикальном масштабировании.



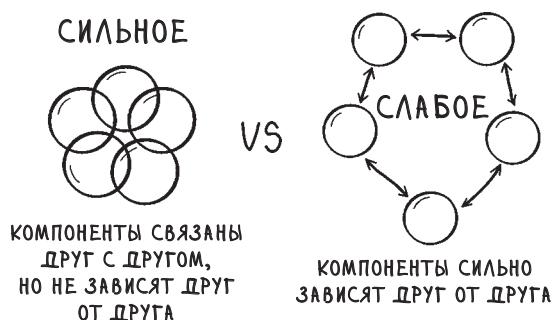
Компьютерная отрасль в основном тяготеет к горизонтальному масштабированию. Основные причины этого — потребность в системах реального времени, большие объемы данных, популярный подход «надежность через избыточность», а также оптимизированное потребление ресурсов за счет их совместного использования при переходе на облачные и SaaS-среды.

Горизонтальное масштабирование требует конкурентности на уровне системы, и одного компьютера может оказаться недостаточно. Комплексы соединенных машин — так называемые *вычислительные кластеры* — решают задачи обработки данных за разумное время.

Ослабление сцепления

У больших задач есть еще один важный аспект — сложность. К сожалению, со временем системы не становятся менее сложными, если только разработчики не прилагают специальных усилий. Компании хотят, чтобы их продукты были более мощными и функциональными, а от этого неизбежно усложняется и кодовая база, и инфраструктура, и работа по сопровождению. Специалистам приходится искать и реализовывать разные архитектурные приемы, чтобы упрощать системы и делить их на более простые независимые единицы, которые взаимодействуют друг с другом.

В разработке ПО практически всегда приветствуется разделение обязанностей. Создавать слабо сцепленные системы позволяет основной инженерный принцип — «разделяй и властвуй». Если группировать логически и функционально связанные фрагменты кода (компоненты с *сильным сцеплением*, *tightly coupled*) и разделять не связанные фрагменты (компоненты со *слабым сцеплением*, *loosely coupled*), то приложения становится проще понимать и тестировать, а количество ошибок сокращается — по крайней мере теоретически.



Так что конкурентность можно рассматривать и как стратегию ослабления сцепления. Если разделять функциональность между модулями или единицами конкурентности, то отдельные части смогут лучше сосредоточиться на конкретной функциональности, их будет проще сопровождать, а общая сложность системы уменьшится.

Разработчики отделяют то, *что* нужно сделать, от того, *когда* это будет сделано. Это кардинально улучшает производительность, масштабируемость, надежность и внутреннюю структуру приложения.

Конкурентность играет важную роль и широко используется в современных вычислительных системах, операционных системах и больших распределенных кластерах. Она помогает моделировать реальный мир, максимизирует эффективность систем с точки зрения пользователей и разработчиков, а также позволяет разработчикам решать большие и сложные задачи.

Путешествие по миру конкурентности изменит ваши представления о компьютерных системах и их возможностях. Посмотрим на то, как устроен этот мир с точки зрения разных уровней конкурентности.

Уровни конкурентности

Как и многие сложные концепции проектирования, конкурентность строится на нескольких уровнях. В многоуровневой архитектуре важно понимать, что структуры, которые на первый взгляд противоречат друг другу или взаимно исключают друг друга, могут *конкурентно* сосуществовать на разных уровнях. Например, ничто не мешает организовать конкурентное выполнение на последовательной машине.

Мне нравится представлять многоуровневую архитектуру конкурентности как симфонический оркестр, который играет, допустим, Чайковского:

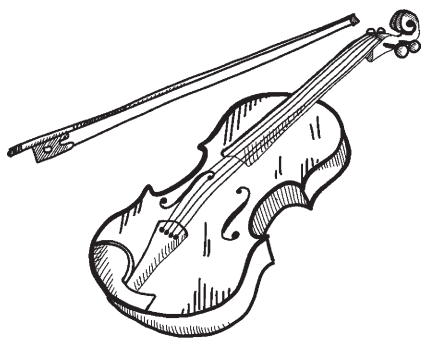
- На самом верху располагается концептуальный уровень, или уровень проектирования (*прикладной уровень*). Его можно сравнить с нотами, которые композитор написал для оркестра. В компьютерной системе роль такой партитуры играют алгоритмы, которые указывают компонентам системы, что им делать.



- Далее идет механизм многозадачности в среде выполнения (*уровень среды выполнения*). Он подобен музыкантам, которые совместно играют разные партии одного и того же произведения на разных инструментах. Музыкальное исполнение переходит от одной группы инструментов к другой в соответствии с жестами дирижера. В компьютерной системе этот уровень представлен разными процессами, каждый из которых выполняет свою часть работы в рамках общей цели.



- Наконец, мы приходим к низкоуровневому исполнению кода (*аппаратный уровень*). Здесь мы детализируем исполнение до уровня конкретных инструментов, например скрипки. Каждая нота, которую играет скрипач, возникает в результате того, что от одной до четырех струн колеблются с определенной частотой, которая зависит от длины, диаметра, натяжения и плотности струны. В компьютерной системе каждый процесс выполняет задачи в соответствии с инструкциями, которые относятся к этому конкретному процессу.

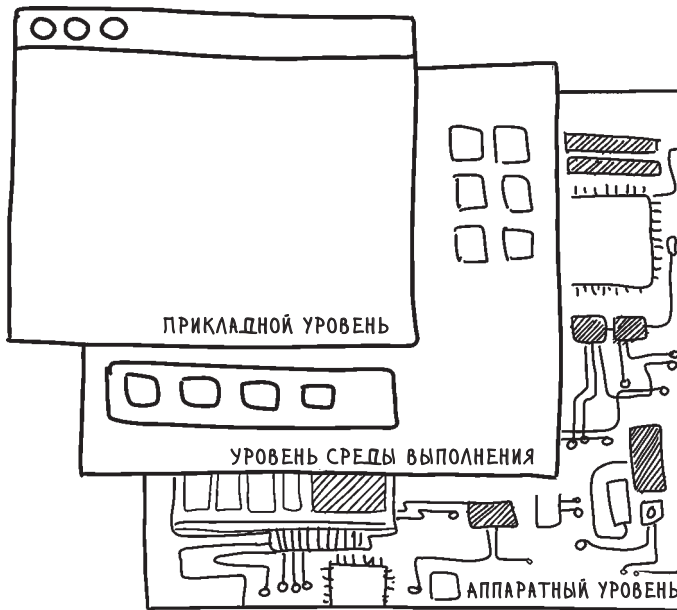


Каждый уровень описывает один и тот же процесс с разной степенью детализации, но подробности описания различаются, а иногда и противоречат друг другу.

То же самое происходит с конкурентностью:

- На *аппаратном уровне* мы имеем дело непосредственно с машинными командами: их выполняют вычислительные ресурсы, которые с помощью сигналов обращаются к периферийным устройствам. Современные архитектуры становятся все сложнее, и поэтому, чтобы оптимизировать производительность приложений в таких архитектурах, нужно глубоко разбираться в том, как приложения взаимодействуют с аппаратными компонентами.

- При переходе на *уровень среды выполнения* многие высокоуровневые сущности, связанные с программными абстракциями, скрываются за таинственными системными вызовами, драйверами устройств и алгоритмами планирования, которые существенно влияют на конкурентные системы, — а значит, в них тоже нужно глубоко разбираться. Этот уровень часто представлен операционной системой, и мы поговорим об этом подробнее в главе 3.
- Наконец, на прикладном уровне становятся доступными абстракции, которые по духу ближе к тому, как устроен физический мир. Программисты пишут исходный код, который реализует сложные алгоритмы и отражает бизнес-логику. Кроме того, этот код может влиять на порядок выполнения при помощи специальных языковых средств; в общем случае он представляет очень абстрактные сущности, которые понятны только разработчикам.



Далее мы будем широко использовать эти уровни как ориентиры в нашем восхождении к вершинам мастерства в области конкурентности.

Что вы узнаете из этой книги

Конкурентность заслужила репутацию непростой дисциплины. Ее сложность отчасти связана с тем, что по этой теме не хватает литературы, которую написали бы опытные практикующие специалисты. Сфера конкурентности больше опирается на устные традиции, чем на знания, закрепленные в письменном виде, и поэтому она

до сих пор окутана тайнами. Я написал эту книгу, чтобы хоть немного приподнять завесу над этими тайнами.



Не стоит ожидать, что эта книга научит всему, что вам когда-либо понадобится знать о конкурентности. Она всего лишь поможет сделать первые шаги и понять, какие темы придется изучать подробнее. Мы рассмотрим некоторые задачи из области конкурентного программирования и опишем передовые методы, которые пригодятся, чтобы создавать конкурентные и масштабируемые приложения.

Программисты начального и среднего уровня получают базовые представления о том, как разрабатывать конкурентные системы. Чтобы извлечь максимум пользы из материала, желательно иметь некоторый опыт программирования, но вовсе не обязательно быть экспертом. Мы будем сперва объяснять ключевые понятия в общих чертах на конкретных примерах, а затем демонстрировать их в действии на языке программирования Python.

Книга делится на три части, в которых рассматриваются разные уровни конкурентности. Первая часть посвящена фундаментальным понятиям и примитивам написания конкурентных программ; она охватывает все уровни — от аппаратного до прикладного.

Тема второй части — проектирование конкурентных приложений и популярные паттерны конкурентности. В этой части также рассказывается о том, как преодолевать распространенные проблемы, которые возникают при разработке конкурентных систем.

Третья часть книги расширит ваши знания в области конкурентности: мы не ограничимся одной машиной, а будем масштабировать приложения на несколько машин, которые связаны по сети. Мы рассмотрим асинхронную коммуникацию между задачами — тему, которая чрезвычайно важна в этом контексте. Кроме того,

здесь вы найдете пошаговое руководство о том, как писать конкурентные приложения.

К концу книги вы возьмете уверенный курс к вершинам мастерства в области конкурентности и будете ориентироваться в современных методах асинхронного и конкурентного программирования. Мы пройдем путь от низкоуровневых аппаратных операций до высокоуровневого проектирования приложений и переведем теорию на язык практической реализации.

Весь код в книге написан на языке Python 3.9 и протестирован в операционных системах macOS и Linux. Материал не привязан ни к какому конкретному языку программирования, хотя опирается на подсистему ядра Linux. Весь исходный код примеров доступен в репозитории GitHub (https://github.com/luminousmen/grokking_concurrency) и на сайте книги (www.manning.com/books/grokking-concurrency).



Итоги

- Конкурентная система — это система, которая способна работать сразу с несколькими задачами.
- В реальном мире многие события происходят одновременно. Если вы хотите моделировать реальный мир, вам понадобится конкурентное программирование.
- Конкурентность радикально улучшает производительность и пропускную способность системы, потому что при этом задержка сокращается или скрывается, а существующие ресурсы эксплуатируются эффективнее.
- В книге используются понятия *масштабируемости* и *ослабления сцепления*:
 - Масштабируемость бывает вертикальной или горизонтальной. Вертикальное масштабирование повышает производительность за счет обновления существующих вычислительных мощностей. Горизонтальное масштабирование повышает производительность за счет того, что нагрузка распределяется между существующими и новыми вычислительными ресурсами. Архитектуры компьютерных систем в основном тяготеют к горизонтальному масштабированию, при котором обязательно задействуется конкурентность.
 - Сложные задачи можно разделить на простые компоненты, связанные друг с другом. В определенном смысле конкурентность — это стратегия ослабления сцепления, которая помогает решать сложные и большие задачи.
- Отправляясь в незнакомое место, обычно стоит захватить с собой карту, которая поможет вам не заблудиться. В этой книге нашим ориентиром станут уровни конкурентности: *прикладной уровень*, *уровень системы выполнения* и *аппаратный уровень*.

2 | Последовательное и параллельное выполнение



В этой главе

- ✓ Терминология, относящаяся к выполнению программ
- ✓ Низший уровень конкурентности: различные подходы к физическому выполнению задач
- ✓ Наброски нашей первой параллельной программы
- ✓ Ограничения технологии параллельных вычислений

.....

Тысячелетиями (ладно, это небольшое преувеличение, — но уже очень давно) работчики писали программы, используя простейшую модель вычислений — последовательную. Последовательное выполнение лежит в основе последовательного программирования, которое станет отправной точкой нашего знакомства с конкурентностью. В этой главе я представлю разные схемы выполнения, которые относятся к самому низкому уровню выполнения.

Обзор основных понятий: что такое программа

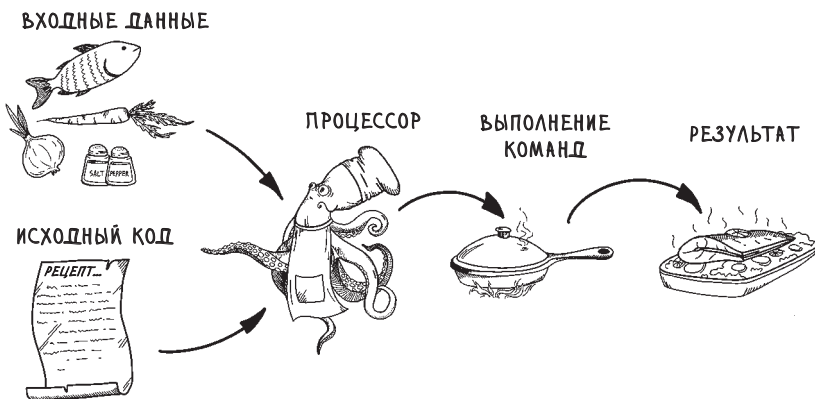
Первая проблема с конкурентностью (и вообще с computer science) заключается в том, что нам очень плохо удается присваивать имена. Иногда одно слово обозначает несколько разных понятий, иногда разные слова обозначают одно и то же, а иногда разные слова соответствуют разным понятиям, но их смысл зависит от контекста. А бывает, что мы просто выдумываем новые слова.

ПРИМЕЧАНИЕ А вы знали, что CAPTCHA — заумный акроним для «Completely Automated Public Turing test to tell Computers and Humans Apart», то есть «Полностью автоматизированный общедоступный тест Тьюринга для различения компьютеров и людей»?

Таким образом, прежде чем разговаривать о процессе выполнения, имеет смысл разобраться, что же конкретно выполняется, и задать общую терминологию, которая будет использоваться в книге. В общем случае *программа* — это последовательность команд, которую *выполняет* компьютерная система.

Чтобы программу можно было выполнять, ее сначала нужно написать. Для этого пишут *исходный код* на одном из многих существующих языков программирования. Исходный код можно рассматривать как рецепт в кулинарной книге — последовательность действий, которые сообщают повару, как приготовить блюдо из исходных ингредиентов. В процессе приготовления задействовано несколько компонентов: сам рецепт, повар и ингредиенты.

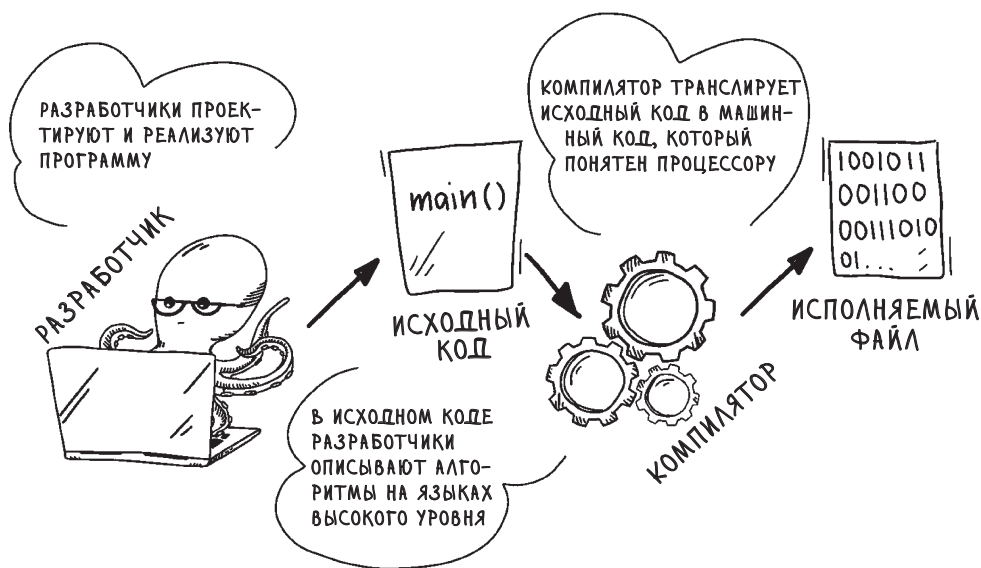
Выполнение программы похоже на приготовление блюда по рецепту: у нас есть исходный код программы (рецепт), повар (процессор) и ингредиенты (входные данные).



Процессор сам по себе не может выполнить ни одной содержательной задачи. Он не умеет сортировать список или искать объекты с заданными характеристиками, а способен только выполнять ограниченный набор простых операций. Вся «интеллектуальная» мощь процессора определяется программами, которые он выполняет. Сколько бы вычислительных мощностей ни было в вашем распоряжении, вы не

получите полезных результатов, если не направите эти мощности в нужное русло. Преобразовать задачу в последовательность операций, которую может выполнить процессор,— это функция *разработчика*, который делает примерно то же самое, что автор кулинарной книги.

Задачу, которую нужно выполнить, разработчики обычно описывают на языке программирования. Однако процессор не поймет исходный код, который написан на обычном языке программирования. Сначала его надо преобразовать в машинный код — язык, которым владеет процессор. Этим занимаются специальные программы, которые называются *компиляторами*. Компилятор создает файл, который часто называется *исполняемым файлом* и содержит низкоуровневые команды, которые процессор поймет и сможет выполнить.



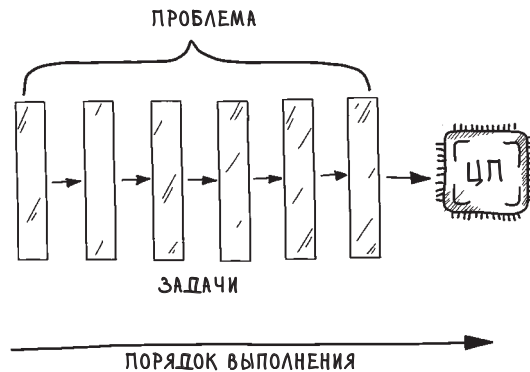
Процессор может выполнять машинный код по-разному. Самый простой способ выполнить набор команд — *последовательное выполнение*, на котором основан принцип последовательных вычислений.

Последовательное выполнение

Как упоминалось ранее, программа представляет собой перечень команд, и в общем случае их порядок имеет значение. Вернемся к примеру с кулинарией: допустим, вы начали готовить блюдо по своему любимому рецепту и выполнили все предписанные действия, но в неправильном порядке: например, сначала поджарили яйцо, а потом попытались смешать его с мукой. Скорее всего, результат вас не обрадует. Для многих задач порядок действий чрезвычайно важен.

То же самое относится к программированию. Когда мы решаем задачу программирования, мы разбиваем ее на ряд меньших задач, которые затем выполняются одна за другой, то есть *последовательно*. Программирование, ориентированное на задачи, позволяет рассматривать вычисления независимо от машинного уровня и помогает разрабатывать программы в модульном стиле.

Задачу можно рассматривать как единицу работы. Если мы говорим о выполнении кода процессором, эту задачу можно назвать *командой*. Задача также может быть последовательностью операций, которые формируют *абстракцию* модели из реального мира; примеры таких моделей — запись данных в файл, поворот графического изображения или вывод сообщения на экран. Задача может состоять из одной или нескольких операций (об этом мы подробнее поговорим в дальнейших главах), но это логически независимый блок работы. Мы будем использовать термин «задача» как обобщенную абстракцию для единицы выполнения.



Последовательное выполнение задач похоже на цепочку: за первой задачей следует вторая, за второй — третья и т. д., и задачи не перекрываются по времени. Представьте, что на сегодня у вас запланирована стирка, и накопилась большая груда грязных вещей. К сожалению, как и у большинства людей, у вас дома только одна

стиральная машина, и вы с содроганием вспоминаете, как однажды постирали свою любимую белую футболку вместе с цветной рубашкой. Какая досада!

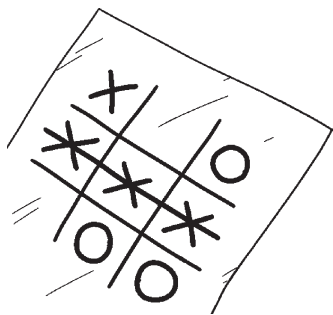
ПОСЛЕДОВАТЕЛЬНОЕ ВЫПОЛНЕНИЕ



Наученные горьким опытом, вы сначала стираете в машине белые вещи, потом цветные, потом постельное белье и наконец полотенца. Минимальное время стирки определяется производительностью стиральной машины и количеством вещей. Даже если у вас целая гора грязного белья, ее придется обрабатывать последовательно, одну кучу за другой. Каждое выполнение блокирует весь вычислительный ресурс; стиральная машина не может постирать половину белой одежды, а затем перейти к стирке цветной. Никто не ожидает от нее такого поведения.

Последовательные вычисления

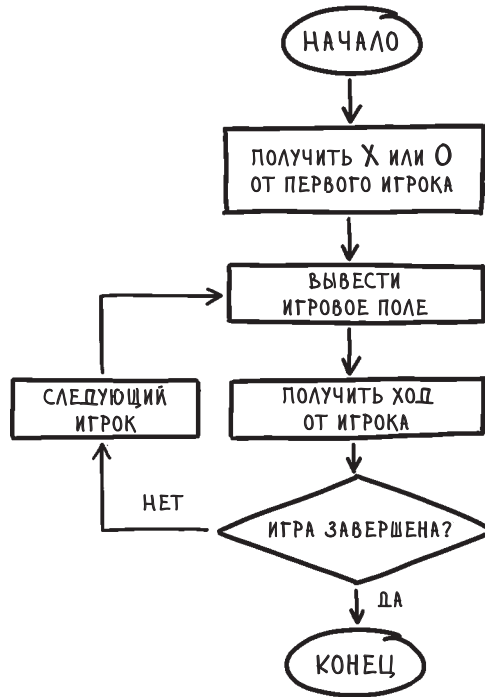
Слово «последовательный» характеризует динамические явления, которые развиваются во времени. Программы и системы по своей природе являются последовательными — правда, этот термин скорее относится к тому, как программа или система спроектирована и описана в исходном коде, а не к тому, как она фактически выполняется.



Представьте, что вам нужно реализовать игру в крестики-нолики. Правила игры достаточно просты: есть два игрока, один игрок ставит в клетки крестики, а другой — нолики. Игроки поочередно заполняют клетки игрового поля, по одной клетке за ход. Если игрок выстроит три своих знака по вертикали, по горизонтали или по одной из двух диагоналей, он выиграл. Если игровое поле заполнено, но никто не выиграл, засчитывается ничья. Сможете ли вы написать такую игру?

Начнем с логики игры. Игроки ходят по очереди; для этого каждый игрок вводит номер строки и номер столбца ячейки, в которую он хочет поставить знак. После того как игрок сделал ход, программа проверяет, не выиграл ли он или не наступила ли ничья, а затем передает ход другому игроку. Игра продолжается до тех пор, пока либо один игрок не выиграет, либо не наступит ничья. Если игрок выиграл, то программа выдает сообщение о результате игры, после чего ее можно завершить, нажав любую клавишу.

На следующей иллюстрации показано, как может выглядеть блок-схема игры в крестики-нолики.



Программа последовательно выполняет шаги, чтобы решить проблему. Каждый шаг зависит от результата предыдущего шага, а следовательно, каждый шаг *блокирует* выполнение последующих шагов. Такую программу можно реализовать только в парадигме последовательного программирования.

Как видите, вычислительная модель этой программы определяется правилами игры — алгоритмом. Между задачами существует очевидная зависимость, которую невозможно нарушить никаким образом. Нельзя проверить ход, который игрок еще не сделал, и нельзя позволить первому игроку сделать два хода подряд, потому что это будет нечестно.

ПРИМЕЧАНИЕ На самом деле существует не так много задач, в которых каждый следующий шаг зависит от завершения всех предыдущих. А значит, конкурентность можно относительно легко применять во многих задачах, с которыми программисты сталкиваются в повседневной работе. Мы еще поговорим об этом в следующих главах.

Попробуйте сами привести примеры задач, в которых требуется последовательное выполнение. Подсказка: это значит, что ни один шаг задачи нельзя выполнять, пока не будет завершен предыдущий шаг.

Противоположность последовательного программирования — *конкурентное программирование*. Конкурентность основана на идее о том, что существуют *независимые* вычисления, которые можно выполнять в произвольном порядке с тем же результатом.

Плюсы и минусы последовательных вычислений

У последовательных вычислений есть ряд важных преимуществ, но есть и свои недостатки.

Простота (плюс)

Любую программу можно написать последовательно. Последовательное выполнение очевидно и предсказуемо, поэтому эта концепция наиболее популярна. Когда мы думаем о задачах, то вполне естественно представляем их себе в виде последовательности. Сначала приготовить еду, потом поесть, а после этого вымыть посуду — разумная последовательность задач. Сначала поесть, потом вымыть посуду, а потом приготовить еду — это выглядит уже не так разумно.

Последовательные вычисления — прямолинейный подход с понятным набором пошаговых инструкций о том, что и когда делать. При такой схеме заведомо не требуется проверять, завершился ли шаг, от которого зависит текущий шаг: следующая операция не начнет выполняться, пока предыдущая не завершится.

Масштабируемость (минус)

Масштабируемость характеризует, насколько система способна справляться с возрастающим объемом работы или насколько она может повысить эту способность, чтобы адаптироваться к росту. Система считается масштабируемой, если быстродействие улучшается, после того как добавляются новые вычислительные ресурсы. В случае последовательных вычислений повышать производительность можно, только если наращивать мощность используемых системных ресурсов — процессора, памяти и т. д. Это вертикальное масштабирование, которое ограничено производительностью процессоров, доступных на рынке.

Нерациональное использование ресурсов (минус)

Последовательные вычисления не требуют коммуникации или синхронизации между разными шагами выполнения программы. Тем не менее существуют невидимые потери из-за того, что доступные вычислительные ресурсы используются неэффективно: даже если вас полностью устраивает последовательный подход внутри программы, она может не задействовать все доступные ресурсы системы, отчего снижается эффективность и возникают ненужные затраты. Даже если система оснащена одним одноядерным процессором, она все равно может быть недозагружена. В главе 6 мы подробнее разберемся, почему это происходит.

Параллельное выполнение

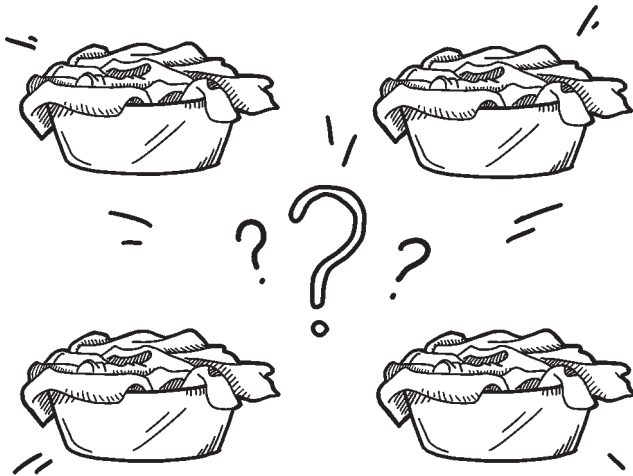
Если вы знакомы с огородничеством, то, возможно, знаете: чтобы вырастить помидор, обычно требуется около четырех месяцев. Но значит ли это, что можно вырастить всего три помидора в год? Разумеется, нет, потому что можно выращивать несколько помидоров одновременно.

Как вы уже видели, при последовательном выполнении в любой момент времени выполняется только одна команда. Большинство разработчиков сначала овладевают последовательным программированием, и многие программы пишутся по однотипной схеме: выполнение начинается с начала главной функции (например, `main`) и продолжается последовательно: по одной задаче, функции, операции, по одному вызову за раз.

Если отказаться от допущения, что все операции должны выполняться по одной, открывается возможность *параллельной* работы — как в примере с выращиванием сразу нескольких помидоров. Тем не менее написать параллельную программу нередко бывает сложнее. Рассмотрим простую аналогию.

Как ускорить стирку

Поздравляем! Вы выиграли в лотерею бесплатную поездку на Гавайи. Прекрасно! Но есть одна загвоздка: до вылета пара часов, а вам нужно выстирать четыре загрузки белья. В вашу стиральную машину, какой бы эффективной она ни была, не помещается больше одной загрузки за раз, и вы не хотите смешивать вещи разных типов.



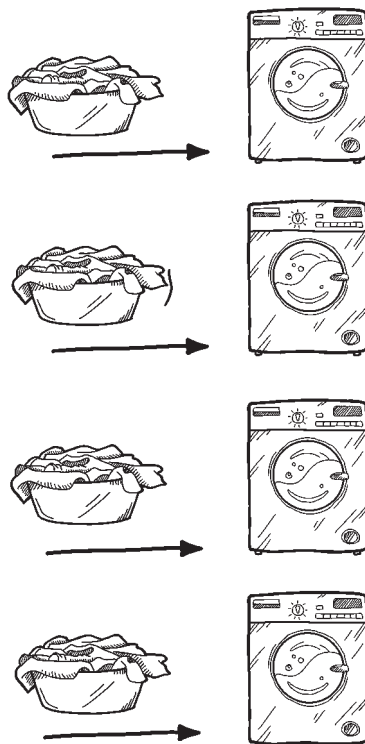
В программировании, как и при стирке, время выполнения последовательной программы ограничивается скоростью процессора и тем, насколько быстро он

может выполнить соответствующую серию команд. Но что если использовать несколько стиральных машин? Поскольку каждую загрузку можно обработать независимо от любой другой, с несколькими машинами вы явно справитесь быстрее, верно?

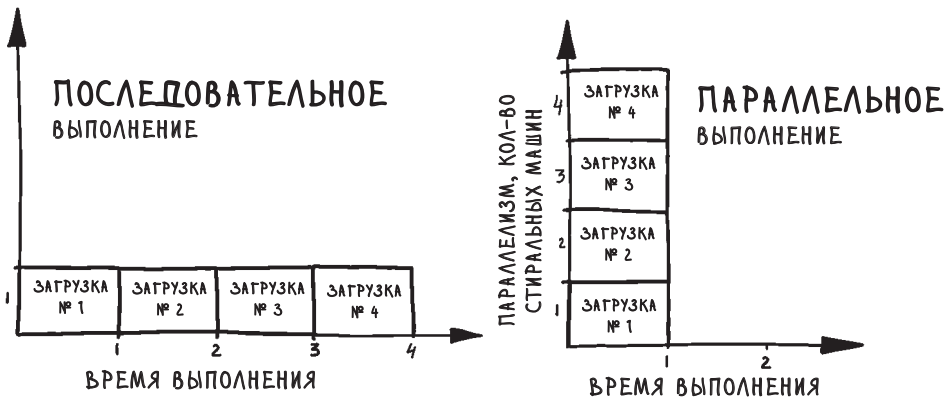
И вы решаете посетить ближайшую прачечную самообслуживания. В ней установлено несколько стиральных машин, и вы легко постираете все четыре загрузки белья в четырех разных машинах одновременно. В таком случае можно сказать, что стиральные машины работают *параллельно*: в одно и то же время они обрабатывают несколько загрузок. Таким образом, пропускная способность повышается в 4 раза.

Помните горизонтальное масштабирование, о котором мы рассказывали в главе 1? Здесь мы применили этот подход на практике.

ПАРАЛЛЕЛЬНОЕ ВЫПОЛНЕНИЕ



Параллельное выполнение означает, что задачи выполняются физически одновременно. Это противоположность последовательного выполнения. *Параллелизм* можно измерить количеством задач, которые могут выполняться параллельно. В нашем примере используются четыре стиральные машины, так что степень параллелизма равна 4.



Итак, вы узнали, что такое параллельное выполнение. Теперь давайте разберемся, при каких условиях оно возможно.

Требования к параллельным вычислениям

Прежде чем углубляться в параллельное выполнение, рассмотрим его необходимые требования: независимость задач и аппаратную поддержку.

Независимость задач

При последовательных вычислениях все операции ускорятся, если повысить скорость процессора. Так можно проще всего снизить задержку, причем это решение не требует никакой специальной структуры программы. Все, что нужно, — более мощный процессор. А параллельные вычисления в основном используются, чтобы снизить задержку, разделив проблему на задачи, которые можно выполнять одновременно и независимо друг от друга.

ПРИМЕЧАНИЕ Большие программы часто состоят из множества меньших. Например, веб-сервер обрабатывает запросы от браузеров и отвечает веб-страницами в формате HTML. Каждый запрос обрабатывается как небольшая программа, и было бы идеально, если бы такие программы выполнялись одновременно.

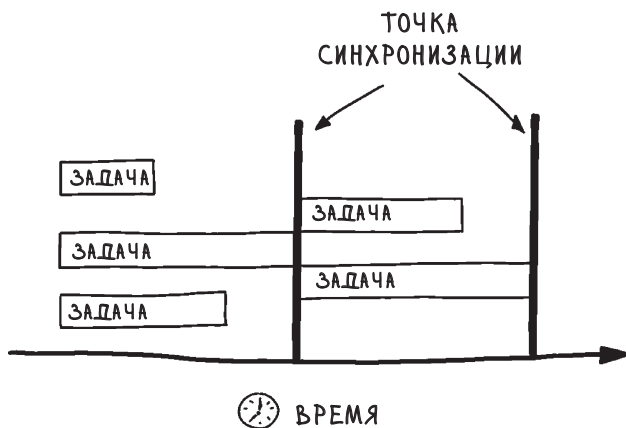
Применение параллельных вычислений определяется спецификой проблемы. Чтобы их применить, проблема должна раскладываться на множество независимых задач так, чтобы каждый вычислительный ресурс мог выполнять часть алгоритма одновременно с другими. Независимость здесь означает то, что ресурсы могут обрабатывать задачи в любом порядке и в любое время, — при условии, что результат получится одним и тем же. Если это требование не соблюдается, проблему нельзя параллелизовать.

Для того чтобы понять, можно ли выполнить программу параллельно, главное — проанализировать, какие задачи можно разбить на части и какие задачи могут выполняться независимо. В главе 7 мы подробнее расскажем, как декомпозировать задачи.

ПРИМЕЧАНИЕ В данном случае логика работает только в одну сторону: программу, которая выполняется параллельно, всегда можно сделать последовательной, но последовательную программу далеко не всегда можно сделать параллельной.

Независимость задач не всегда возможна, потому что не каждую программу или алгоритм можно разделить на независимые задачи от начала до конца. Одни задачи могут быть независимыми, а другие — нет, потому что они опираются на ранее выполненные задачи. Поэтому, чтобы получить правильные результаты, разработчикам приходится *синхронизировать* разные зависимые части программы.

Синхронизация означает, что выполнение задачи *блокируется* в ожидании зависимостей. В примере с «крестиками-ноликами» выполнение программы блокировалось ходами игроков. Если координировать взаимозависимые параллельные вычисления посредством синхронизации, это может серьезно ограничить параллелизм программы — поэтому писать параллельные программы гораздо сложнее, чем простые последовательные (мы подробнее рассмотрим эту тему в главе 8).



Эта дополнительная работа оправдывает усилия. Если параллельное выполнение правильно реализовано, оно повышает общую пропускную способность программы, позволяя разбивать большие задачи на части, чтобы решать их быстрее или чтобы программа успела выполнить больше задач за определенный промежуток времени.

Задачи, которые обходятся минимальной синхронизацией (или вообще без нее), иногда называются *тривиально параллелизуемыми* (embarrassingly parallel)¹. Их легко разбить на независимые задачи, которые выполняются параллельно. Такие задачи часто встречаются в научных вычислениях: например, чтобы декомпозировать работу по поиску простых чисел, числовые подмножества можно легко распределить по разным вычислительным ресурсам.

ПРИМЕЧАНИЕ В тривиально параллельных задачах нет ничего плохого! Наоборот, приложения с такими задачами хороши тем, что их легко программировать. В последние годы значение термина «тривиально параллелизуемый» несколько изменилось. В тривиально параллелизуемых алгоритмах коммуникация между процессами обычно минимальна, что способствует хорошей производительности, так что *тривиально параллелизуемыми* обычно называются алгоритмы с низкими потребностями в коммуникации. Эта тема подробнее затрагивается в главе 5.

Таким образом, степень параллелизма больше зависит от проблемы, чем от специалистов, которые ее решают.

Аппаратная поддержка

Параллельные вычисления требуют аппаратной поддержки: для параллельных программ нужно оборудование с несколькими вычислительными ресурсами. Если нет хотя бы двух вычислительных ресурсов, добиться истинного параллелизма не удастся. В следующей главе мы поговорим про оборудование и о том, как оно позволяет одновременно выполнять несколько операций. Узнав, какие требования предъявляются к параллельным вычислениям, можно теперь разобраться, что же они собой представляют.

Параллельные вычисления

Параллельные вычисления используют декомпозицию, чтобы разбивать большие или сложные задачи на меньшие, которые затем можно будет эффективно решить, параллельно запустив в системе выполнения. В следующем примере мы продемонстрируем, как параллелизм может спасти мир.

Представьте, что вы работаете в IT-отделе ФБР, и ваша миссия — написать программу, чтобы взломать пароль (комбинацию цифр заданной длины) и получить доступ к системе, которая грозит уничтожить весь мир.

Чтобы угадать пароль, обычно перебирают все возможные варианты (так называемым *методом грубой силы*, или *брутфорсом*), вычисляют свернутую форму каждого варианта (его *криптографический хеш*) и сравнивают ее со значением,

¹ Иногда можно найти определение «очевидно параллельные», «чрезвычайно параллельные» или «смущающе параллельные». — *Примеч. пер.*

которое хранится в системе. (Допустим, у вас уже есть криптографический хеш пароля.)



Как реализовать такую программу?

Метод грубой силы обычно заключается в том, что мы составляем список всех возможных комбинаций, перебираем его и для каждой комбинации проверяем, является ли она решением проблемы. В нашем случае нужно перебрать все возможные комбинации цифр и для каждой из них проверить, совпадает ли ее криптографический хеш с тем, который хранится в системе.



Через пару бессонных ночей вы разобрались в том, как проверить криптографический хеш, перебрали все возможные комбинации цифр и реализовали программу на своем любимом языке. Алгоритм генерирует комбинацию цифр и проверяет хеш. Если значения совпадают, то найденный пароль выводится, а программа завершает работу; если нет, то она переходит к следующей комбинации, и цикл повторяется заново.

Фактически вы перебираете все возможные варианты пароля с помощью последовательных вычислений. Вы поручаете процессору обработать сначала одну задачу, потом другую и делать это последовательно, пока все задачи не будут завершены. Предыдущая картинка и следующий код демонстрируют, как решить задачу путем последовательного выполнения:

```
# Chapter 2/password_cracking_sequential.py
import time
import math
import hashlib
import typing as T
```

```
def get_combinations(*, length: int, min_number: int = 0) -> T.List[str]:
    combinations = []
    max_number = int(math.pow(10, length) - 1)
    for i in range(min_number, max_number + 1):
        str_num = str(i)
        zeros = "0" * (length - len(str_num))
        combinations.append("".join((zeros, str_num)))
    return combinations
```

В заданном диапазоне генерирует список всех возможных паролей с заданным количеством цифр

```
def get_crypto_hash(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()
```

```
def check_password(expected_crypto_hash: str,
                    possible_password: str) -> bool:
    actual_crypto_hash = get_crypto_hash(possible_password)
    return expected_crypto_hash == actual_crypto_hash
```

Сравнивает криптографический хеш потенциального пароля с хешем, который хранится в системе

```
def crack_password(crypto_hash: str, length: int) -> None:
    print("Комбинации цифр обрабатываются в последовательном режиме")
    start_time = time.perf_counter()
    combinations = get_combinations(length=length)
    for combination in combinations:
        if check_password(crypto_hash, combination):
            print(f"ПАРОЛЬ НАЙДЕН: {combination}")
            break

    process_time = time.perf_counter() - start_time
    print(f"ЗАТРАЧЕННОЕ ВРЕМЯ: {process_time}")

if __name__ == "__main__":
    crypto_hash = \
        "e24df920078c3dd4e7e8d2442f00e5c9ab2a231bb3918d65cc50906e49ecaef4"
    length = 8
    crack_password(crypto_hash, length)
```

Последовательно генерирует и проверяет все возможные пароли заданной длины. Останавливается, если обнаружен пароль, которому соответствует ожидаемый хеш

Результат выглядит примерно так:

```
Комбинации цифр обрабатываются в последовательном режиме
ПАРОЛЬ НАЙДЕН: 87654321
ЗАТРАЧЕННОЕ ВРЕМЯ: 64.60886170799999
```

С полной уверенностью в том, что задача решена, вы отдаете программу следующему герою, который отправится на миссию. Агент 008 кивает и отставляет бокал с коктейлем.

Но мы ведь знаем, что никому нельзя доверять, верно? Не проходит и часа, как агент 008 врывается в ваш кабинет и говорит, что программа работает слишком долго: ей понадобится больше часа, чтобы перебрать все возможные комбинации на его шпионском компьютере. «У меня будет всего пара минут, прежде чем здание будет охвачено огнем», — мрачно говорит агент 008, отхлебнув еще глоток коктейля. Он уходит, на прощание скомандовав: «Ускорь программу!» Ох...

И как ее ускорить?

Очевидным решением было бы повысить производительность процессора: если увеличить тактовую частоту шпионского компьютера, можно будет обрабатывать больше паролей за то же время. К сожалению, мы уже выяснили, что этот подход связан с ограничениями: у скорости процессоров есть физические пределы. Кроме того, не забывайте, что у нас ФБР, а значит, вам уже выдали самый быстрый процессор, и его производительность больше никак не увеличить. Это самый значительный недостаток последовательных вычислений: они плохо масштабируются, даже если компьютерная система оснащена несколькими вычислительными ресурсами.



Другой способ ускорить программу — разбить ее на независимые задачи и распределить их между вычислительными ресурсами, которые будут обрабатывать их одновременно. Чем больше ресурсов и чем меньше задачи, тем быстрее идет обработка. Эта идея лежит в основе параллельных вычислений и подробнее рассматривается в главах 8 и 12.

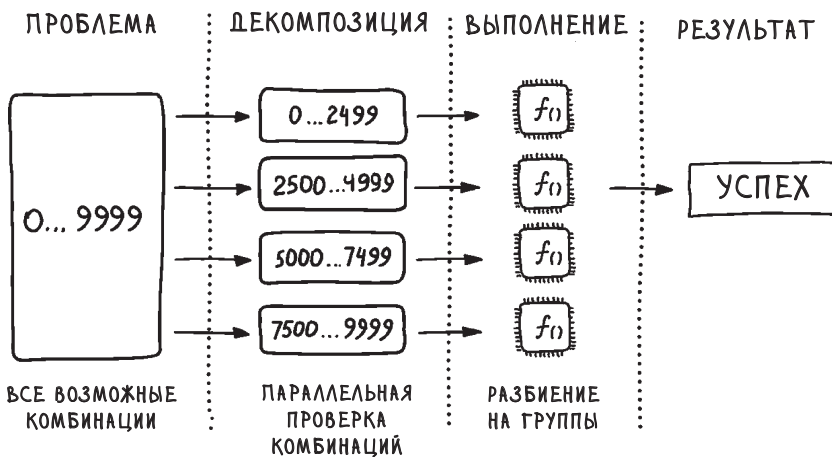
Как вы думаете, получится ли применить параллельное выполнение в нашем случае? Шпионский компьютер оснащен сверхсовременным процессором со множеством ядер. Таким образом, первое условие выполняется: у вас есть подходящее оборудование, которое способно выполнять задачи параллельно.

Можно ли разложить всю проблему на задачи, которые не зависят друг от друга? Проверку отдельных комбинаций можно рассматривать именно как такие задачи: не обязательно проверять все предшествующие пароли перед текущим. Неважно, какой пароль обрабатывать первым, потому что проверки можно выполнять полностью независимо друг от друга. Отлично!

Значит, все требования для параллельных вычислений выполнены. У нас есть аппаратная поддержка и независимость задач. Давайте спроектируем итоговое решение!

Первый шаг для каждой проблемы такого рода — декомпозировать ее на отдельные задачи. Как вы уже знаете, отдельные проверки паролей можно считать независимыми задачами, которые можно выполнять параллельно. Между ними нет зависимостей, а значит, в программе не будет точек синхронизации; следовательно, мы имеем дело с тривиально параллелизуемой проблемой.

На следующей диаграмме изображено, как решение разбивается на несколько шагов. На первом шаге создаются диапазоны паролей (блоки), которые затем распределяются между доступными вычислительными ресурсами. В результате с каждым вычислительным ресурсом связывается свой диапазон паролей. На следующем шаге начинается фактическое выполнение.



Код выглядит так:

```
ChunkRange = T.Tuple[int, int]
```

Большой диапазон целых чисел делится на меньшие блоки, содержащие приблизительно одинаковое количество паролей, которые можно обрабатывать параллельно на разных ядрах или процессорах

```
def get_chunks(num_ranges: int,
               length: int) -> T.Iterator[ChunkRange]:
    max_number = int(math.pow(10, length) - 1)
    chunk_starts = [int(max_number / num_ranges * i)
                    for i in range(num_ranges)]
    chunk_ends = [start_point - 1
                  for start_point in
                    chunk_starts[1:]] + [max_number]
    return zip(chunk_starts, chunk_ends)
```

```
def crack_password_parallel(crypto_hash: str, length: int) -> None:
    num_cores = cpu_count()
    chunks = get_chunks(num_cores, length)
    # ВЫПОЛНЯЕТСЯ ПАРАЛЛЕЛЬНО
    # for chunk_start, chunk_end in chunks:
    #     crack_chunk(crypto_hash, length, chunk_start, chunk_end)}
```

Получает количество доступных процессоров

Псевдокод для одновременной обработки каждого блока в отдельном процессе

Мы добавили новую функцию `crack_password_parallel`, которая должна выполнять функцию `crack_password` параллельно на нескольких ядрах. На других языках программирования реализация может выглядеть иначе, но идея остается той же самой: программа должна создавать набор параллельных единиц работы и распределять между ними диапазоны паролей для параллельного выполнения. Для этого нам понадобился *псевдокод* (понятное для человека представление программной логики на стилизованном языке, имитирующем реальный код), который мы еще рассмотрим в главах 4 и 5.

ПРИМЕЧАНИЕ Хотя мы и задействуем псевдокод, наш пример чрезвычайно реалистичен в отношении использования. Например, в языке MATLAB есть конструкция `parfor`, которая позволяет тривиально организовать параллельные циклы `for`. В языке Python есть пакет `joblib` с классом `Parallel`, который значительно упрощает параллелизм. Язык R содержит библиотеку `Parallel` с той же функциональностью. Стандартная библиотека Scala предлагает параллельные коллекции, которые облегчают параллельное программирование, избавляя пользователей от низкоуровневых подробностей параллелизации.

Благодаря параллелизации вычислений агент 008 в очередной раз спасает мир за секунды до катастрофы. К сожалению, большинство из нас не располагает ресурсами ФБР; параллельное выполнение связано с определенными ограничениями и затратами, которые нужно учитывать, прежде чем применять его в практических задачах. Мы рассмотрим их в следующем разделе.



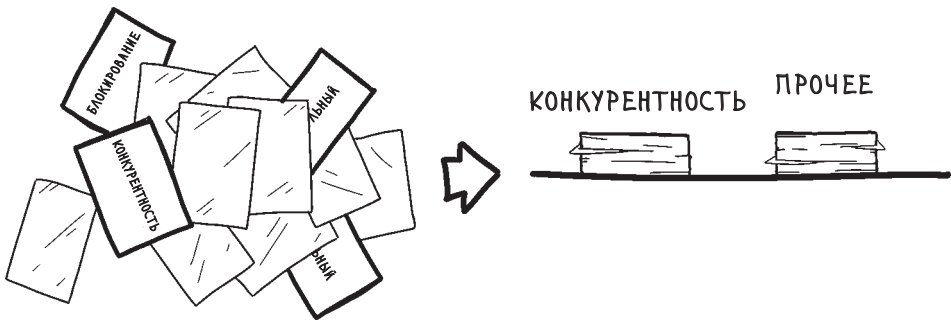
Закон Амдала

Мать может выносить ребенка за девять месяцев. Значит ли это, что девять матерей совместными усилиями родят ребенка через месяц?

Может показаться, что количество процессоров можно увеличивать до бесконечности, чтобы система работала как угодно быстро. К сожалению, это не так. Эту проблему наглядно демонстрирует знаменитое наблюдение Джина Амдала, известное как *закон Амдала*.

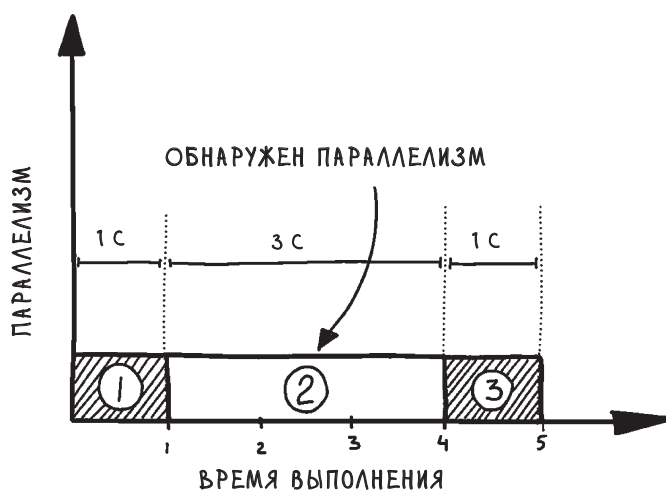
До настоящего момента мы рассматривали, как выполняется параллельный алгоритм. Хотя в нем могут быть некоторые последовательные части, выполнение принято представлять как набор частей, одни из которых полностью параллельны, а другие полностью последовательны. Последовательные части могут быть фазами, которые еще не параллелизованы, а могут быть последовательными по своей природе, как было показано ранее.

Представьте, что перед вами огромная куча карточек, на которых написаны термины. Вы хотите найти карточки, которые относятся к конкурентности, и отложить их в отдельную стопку, но карточки перемешаны. К счастью, у вас есть два друга, так что можно разделить карточки на три стопки, выдать каждому другу по стопке и сказать, что нужно искать. Тогда каждый из вас сможет провести поиск в собственной части. Когда кто-то находит карточку на тему конкурентности, он объявляет об этом и откладывает ее в отдельную стопку.

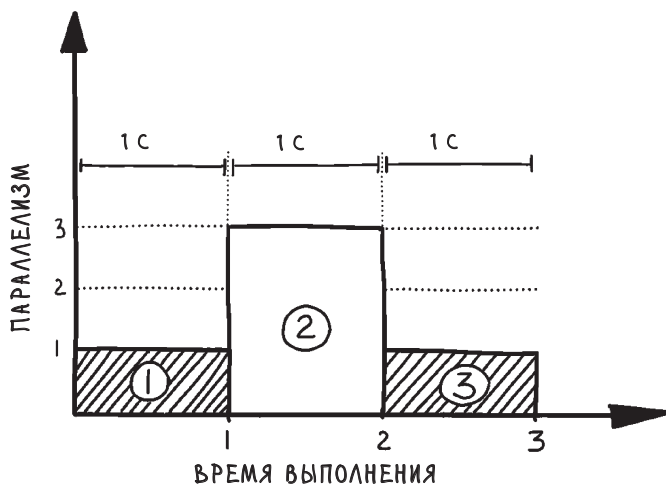


Алгоритм может выглядеть так:

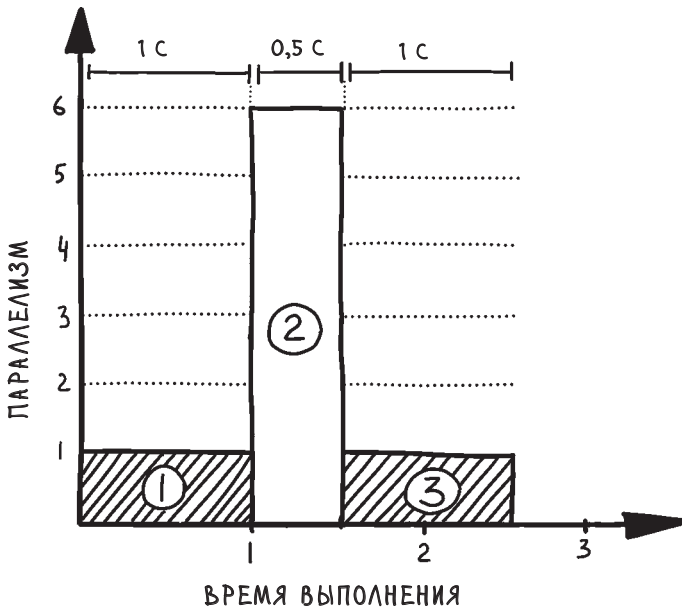
1. Разделить карточки на стопки и передать по одной стопке каждому участнику (последовательная часть).
2. Каждый участник ищет у себя карточки на тему конкурентности (параллельная часть).
3. Сложить найденные карточки в отдельную стопку (последовательная часть).



Шаги 1 и 3 этого алгоритма занимают по 1 секунде, а шаг 2 — 3 секунды. Таким образом, весь алгоритм от начала до конца выполняется за 5 секунд. Шаги 1 и 3 — алгоритмически последовательные: их невозможно разделить на независимые задачи и выполнять параллельно. С другой стороны, на шаге 2 легко организовать параллельное выполнение, если разделить карточки на любое количество стопок, при условии что у вас найдется достаточно помощников, чтобы независимо выполнять этот шаг. С помощью двух друзей время его выполнения уменьшается до 1 секунды. Вся программа теперь занимает всего 3 секунды, что на 40 % быстрее последовательного варианта. Ускорение рассчитывается как отношение времени выполнения в параллельном режиме с определенным количеством вычислительных ресурсов ко времени выполнения в оптимальном последовательном режиме с одним вычислительным ресурсом.

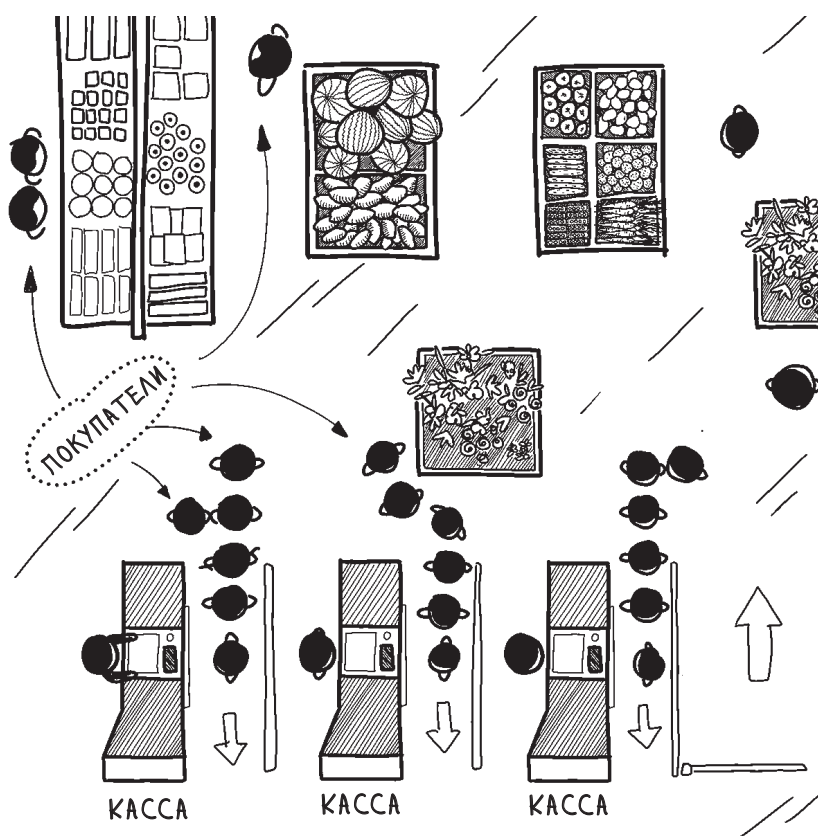


Что произойдет, если привлечь еще больше друзей? Допустим, вы пригласили еще трех друзей, так что всего их стало шестеро. Теперь шаг 2 программы занимает всего 0,5 с, а весь алгоритм выполняется всего за 2,5 с, что соответствует ускорению на 50 %.



По той же логике можно пригласить все население вашего города, в результате чего параллельная часть алгоритма будет выполняться практически мгновенно (хотя теоретически возникнут коммуникационные затраты, о которых мы поговорим в главе 6). При этом остается задержка продолжительностью не менее 2 с — последовательная часть алгоритма.

Параллельная программа выполняется не быстрее самой медленной последовательной части. Вы много раз наблюдали это явление на примере магазина. Сотни людей могут выбирать покупки одновременно, практически не мешая друг другу. Но когда приходит пора платить за покупки, образуется очередь, потому что касс меньше, чем покупателей.



Все это актуально и для программирования. Сколько бы мы ни увеличивали количество ресурсов, последовательные части программы не ускорятся. Это главное, что нужно знать, чтобы понять закон Амдала. Потенциальная скорость программы с параллельными вычислениями ограничена скоростью ее последовательных частей. Закон описывает максимальное ускорение, которого можно ожидать, если добавлять ресурсы в систему с расчетом на параллельные вычисления. В нашем примере закон Амдала предсказывает, что если программа последовательна на $\frac{2}{3}$, то сколько бы процессоров мы ни добавили, выполнение никогда не ускорится более чем в 1,5 раза.

В более формальном виде закон выглядит так:

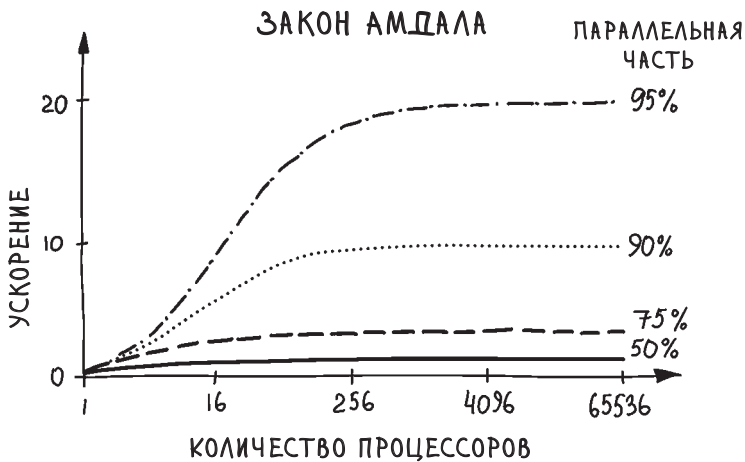
ЗАКОН АМДАЛА УТВЕРЖДАЕТ, ЧТО ТЕОРЕТИЧЕСКОЕ
УСКОРЕНИЕ ОПРЕДЕЛЯЕТСЯ ЧАСТЬЮ КОДА P , КОТОРУЮ
МОЖНО ПАРАЛЛЕЛИЗИРОВАТЬ

$$\text{УСКОРЕНИЕ} = \frac{1}{(1-P) + P/N}$$

ПОСЛЕДОВАТЕЛЬНАЯ ЧАСТЬ РАБОТЫ = 1 (100 %) - ПАРАЛЛЕЛЬНАЯ ЧАСТЬ

ПАРАЛЛЕЛЬНАЯ ЧАСТЬ ДЕЛИТСЯ МЕЖДУ N ИСПОЛНИТЕЛЯМИ

Формула кажется безобидной, пока мы не начинаем подставлять в нее конкретные значения. Например, если $\frac{1}{3}$ программы составляют последовательные вычисления, то добавление 1 миллиона процессоров не сделает ее быстрее более чем в 3 раза. Даже если остальной код выполняется мгновенно, ускорить эту $\frac{1}{3}$ невозможно, так что прирост производительности ограничивается 300 %. Если добавить несколько процессоров, часто удается достичь значительного ускорения, но с дальнейшим ростом количества процессоров выигрыш становится все меньше. На следующем графике показано, как ускорение зависит от количества процессоров для разных частей кода, которые можно параллелизовать (без учета дополнительных затрат на алгоритмы или координацию).



Расчеты также можно выполнять в другом направлении: например, если у вас есть 2500 процессоров, то какой процент программы должен идеально параллелизоваться, чтобы достичь 100-кратного ускорения? Если подставить значения в закон

Амдала, мы получим $100 \leq 1/(S + (1 - S)/2500)$. Вычисляя S , мы видим, что последовательный код должен составлять менее 1 % программы.

Подведем итог: закон Амдала демонстрирует, почему использовать множество процессоров для параллелизации вычислений целесообразно только в программах с высокой степенью параллелизации. Даже если вы можете написать программу в параллельном стиле, это не значит, что так всегда следует делать, потому что затраты, связанные с параллелизмом, иногда перевешивают выигрыш. Закон Амдала — удобный инструмент, который позволяет оценить преимущества параллелизации программы, чтобы определить, стоит ли ее параллелизовать.

Закон Густафсона

После таких разочаровывающих результатов появляется искушение отказаться от параллелизма как от способа повысить производительность. Тем не менее не стоит отчаиваться. Параллелизм позволяет реально ускорить те части программы, которые критичны по быстродействию, но ускорить *все* части программы не удастся — если только проблема не тривиально параллелизуема. Для других задач возможный выигрыш жестко ограничен.

Однако закон Амдала можно рассматривать и с другой точки зрения. В нашем примере программа выполнялась за 5 с; а что если удвоить объем работы, которая выполняется в параллелизуемой части — не 3, а 6 задач? Если 6 задач выполняются одновременно, а программа занимает те же 5 с, то в результате будут выполнены 8 задач: с двумя процессорами это означает ускорение в 1,6 раза. А если добавить еще пару процессоров, каждый из которых будет выполнять тот же объем работы, то за те же 5 с будут выполняться 11 задач, то есть получится ускорение в 2,6 раза.

Согласно закону Амдала, ускорение показывает, насколько быстрее будет выполняться параллельная программа при условии, что объем работы остается постоянным. Однако ускорение также можно рассматривать как больший объем выполняемой работы за одно и то же время (пропускная способность). Из этой интерпретации появился закон Густафсона.

Закон Густафсона позволяет оптимистичнее смотреть на ограничения параллелизма. Если повышать объем работы, то последовательные части будут все меньше влиять на выполнение, и можно будет считать, что ускорение пропорционально количеству доступных процессоров.

Так что если вам приходится слышать, как законом Амдала объясняют, почему в вашем случае параллелизм не будет работать, то закон Густафсона подскажет, что нужно делать. Успех суперкомпьютеров и распределенных систем в области параллелизма в первую очередь объясняется тем, что они позволяют наращивать объем данных.

Познакомившись с параллельными вычислениями, можно переходить к разговору о том, какое отношение они имеют к конкурентности.

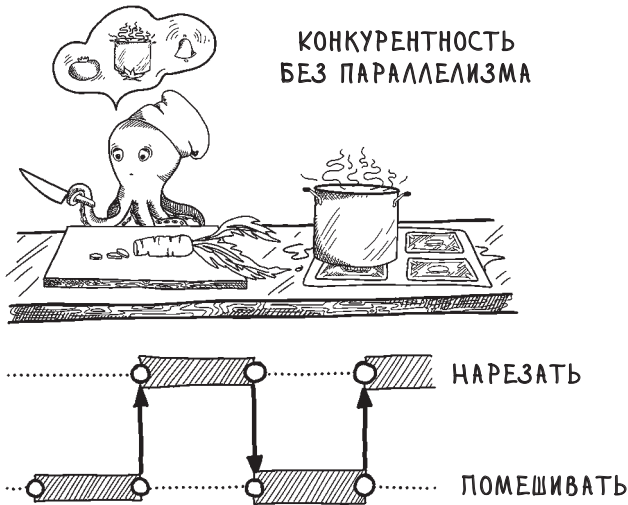
Конкурентность и параллелизм

В бытовой речи термины «параллельный» (parallel) и «одновременный» (concurrent)¹ часто употребляются как синонимы, что создает серьезную путаницу, которая проникла даже в компьютерную литературу. Однако важно отличать параллельное программирование от конкурентного, потому что они преследуют разные цели на разных концептуальных уровнях.

Конкурентность означает, что множество задач запускаются, выполняются и завершаются в перекрывающиеся периоды времени, не выстраиваясь в определенный порядок. А параллелизм означает, что множество задач выполняются в одно и то же время на оборудовании с несколькими вычислительными ресурсами (например, на многоядерном процессоре). Конкурентность и параллелизм — не одно и то же.

Представьте, что повар нарезает салат, время от времени помешивая суп на плите. Повару приходится откладывать нож, проверять состояние супа, снова возвращаться к нарезке и повторять процесс, пока все не будет готово.

Как видите, здесь только один «вычислительный» ресурс (повар), а конкурентность в основном связана с логистикой. Без конкурентности повару пришлось бы сначала дождаться, пока суп сварится, и только после этого начинать нарезать салат.



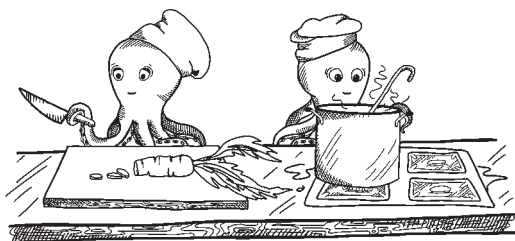
Параллелизм — это *свойство реализации*. Он заключается в том, что несколько задач физически выполняются одновременно, а для этого требуется оборудование с несколькими вычислительными ресурсами. Параллелизм работает на аппаратном уровне.

Предположим, что на кухне появляются два повара: один помешивает суп, а другой нарезает салат. Мы разделили работу, введя дополнительный «вычислительный»

¹ Concurrent означает и «одновременный», и «конкурентный». — *Примеч. пер.*

ресурс — второго повара. Параллелизм — это подкласс конкурентности: прежде чем выполнять несколько задач одновременно, нужно сначала уметь управлять несколькими задачами.

КОНКУРЕНТНОСТЬ И ПАРАЛЛЕЛИЗМ



НАРЕЗАТЬ



ПОМЕШИВАТЬ

Как связаны между собой конкурентность и параллелизм? Конкурентные вычисления можно параллелизовать так, чтобы получить тот же самый правильный результат, но конкурентность сама по себе не подразумевает обязательного параллелизма. Более того, параллелизм тоже не подразумевает обязательной конкурентности; оптимизирующий компилятор часто берет программу, в которой нет семантической конкурентности, и разбивает ее на параллельные компоненты с помощью таких методов, как конвейерная обработка, операции с широкими векторами, операции SIMD (Single Instruction, Multiple Data) и стратегия «разделяй и властвуй» (некоторые из этих методов мы рассмотрим позднее в книге).

Как заметил легенда Unix и Go Роб Пайк: «Суть конкурентности — в том, как держать в поле зрения много операций одновременно. Суть параллелизма — в том, как выполнять много операций одновременно»¹. Конкурентность программы зависит от языка программирования и от того, как написана программа, тогда как параллелизм зависит от фактической среды выполнения. На одноядерном процессоре можно реализовать конкурентность, но не параллелизм. Однако оба этих понятия выходят за пределы традиционной последовательной модели, в которой все операции выполняются по одной за раз.

Чтобы лучше понять, чем конкурентность отличается от параллелизма, обратите внимание на такие факторы:

¹ Роб Пайк в 2012 году выступил на конференции Waza (под эгидой Heroku) с докладом «Concurrency is not parallelism» («Конкурентность — это не параллелизм») (<https://go.dev/blog/waza-talk>).

- Приложение может быть конкурентным и при этом не параллельным: оно обрабатывает более одной задачи за заданный период (то есть манипулирует сразу несколькими задачами, даже если никакие две задачи не выполняются в один момент времени — этот подход подробнее описан в главе 6).
- Приложение может быть параллельным и при этом не конкурентным. Это означает, что оно одновременно обрабатывает множество подзадач одной задачи.
- Приложение может быть ни параллельным, ни конкурентным: оно последовательно обрабатывает одну задачу за другой, и задача никогда не делится на подзадачи.
- Приложение может быть и параллельным, и конкурентным: оно одновременно обрабатывает несколько задач или подзадач одной задачи, и при этом они выполняются параллельно.

Представьте, что ваша программа вставляет значения в хеш-таблицу. Если рассматривать распределение операции вставки между несколькими ядрами, это параллелизм. Однако если рассматривать координацию доступа к хеш-таблице, это конкурентность. Но если вы все еще не понимаете последний пример, не огорчайтесь: в следующих главах мы объясним все в подробностях.

Конкурентность охватывает целый ряд тем, включая взаимодействие между процессами, совместный доступ и состязания за ресурсы (память, файлы, устройства ввода/вывода и т. д.), синхронизацию между процессами и распределение процессорного времени между ними. Эти проблемы возникают не только в многопроцессорных средах и средах распределенной обработки, но и в однопроцессорных системах. В следующей главе мы начнем с того, что рассмотрим среду, в которой выполняется программа, — аппаратное обеспечение и систему выполнения.

Итоги

- Каждая проблема, которая сформулирована в виде приложения, делится на набор задач, которые в простейшем случае выполняются последовательно.
- *Задачу* можно рассматривать как логически независимую единицу работы.
- *Последовательное вычисление* (sequential computing) означает, что каждая задача в программе зависит от выполнения всех предыдущих задач в том порядке, в котором они перечисляются в коде.
- *Последовательное выполнение* (serial execution) заключается в том, что команды из упорядоченного набора выполняются одна за другой на одном вычислительном устройстве. Последовательное выполнение неизбежно, если на вход каждой задачи должны поступать результаты выполнения предыдущей задачи.
- *Параллельное выполнение* заключается в том, что несколько вычислений выполняются в одно и то же время. Этот режим доступен в том случае, когда задачи можно выполнять независимо.
- При *параллельных вычислениях* для решения задач одновременно используются несколько вычислительных устройств. Часто для этого приходится значительно

изменять структуру программы: декомпозировать проблему, создавать или адаптировать алгоритм, добавлять в программу точки синхронизации и т. д.

- *Конкурентность* означает, что мы оперируем несколькими задачами в одно и то же время. *Параллелизм* зависит от фактической среды выполнения и требует нескольких вычислительных ресурсов, а также того, чтобы задачи в декомпозированном алгоритме были независимы. Конкурентность программы зависит от языка программирования и от того, как написана программа, тогда как параллелизм зависит от фактической среды выполнения.
- Закон Амдала — удобный инструмент, который позволяет оценить преимущества параллелизации программы, чтобы определить, стоит ли ее параллелизировать.
- Закон Густафсона описывает, как заставить систему выполнять больше работы, несмотря на ограничения закона Амдала.



В этой главе

- ✓ Как процессор выполняет код
- ✓ Функции и цели системы выполнения
- ✓ Как выбрать оборудование для вашей задачи

Еще двадцать лет назад программист мог за долгие годы работы ни разу не столкнуться с системой, в которой было бы более двух вычислительных ресурсов. Сегодня даже мобильные телефоны оснащены многоядерными процессорами. Мышление современного программиста должно учитывать ситуации, когда несколько процессов выполняются на разных вычислительных ресурсах в одно и то же время.

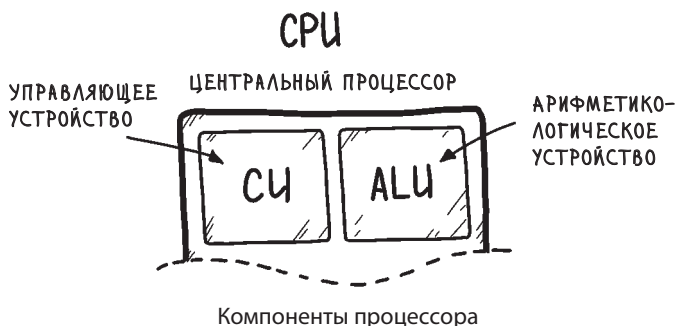
Описывая конкурентные алгоритмы, не обязательно знать конкретный язык программирования, на котором реализуется программа. Однако при этом важно понимать возможности компьютерной системы, где будет выполняться алгоритм. Чтобы сконструировать самый эффективный конкурентный алгоритм, следует выбирать операции, которые наиболее полно используют оборудование компьютерной системы. А значит, нужно ориентироваться в потенциальных возможностях разных аппаратных архитектур.

Поскольку параллельное оборудование используется для того, чтобы повысить производительность, на первый план выходит эффективность кода. В свою очередь это означает, что нужно хорошо разбираться в оборудовании, для которого мы программируем. В этой главе приведен обзор параллельного оборудования, который поможет вам принимать осознанные решения при разработке ПО.

Процессор

Термин «*центральный процессор*» (ЦП, или CPU), или просто «процессор», появился в далекую эпоху первых компьютеров, когда одна большая стойка содержала всю электронику, необходимую для того, чтобы интерпретировать и выполнять машинные команды. Процессор также выполнял все операции с подключенными периферийными устройствами, например принтерами, устройствами чтения перфокарт и ранними накопителями вроде дисковых и ленточных приводов.

Современный процессор несколько изменился. Теперь он в основном занимается своим основным делом — выполняет машинные команды с помощью *управляющего устройства* (CU), которое интерпретирует эти команды, и *арифметико-логического устройства* (ALU), которое выполняет арифметические и поразрядные операции. Благодаря совместной работе CU и ALU процессор способен выполнять более сложные программы, чем простой калькулятор.

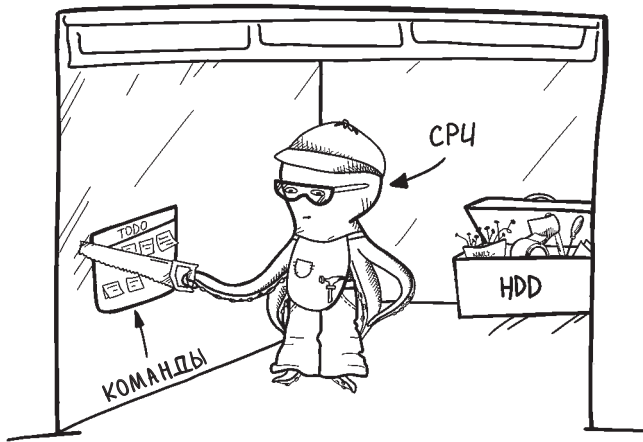


Однако у процессора есть еще один компонент, который играет важную роль в ускорении выполнения, — кэш.

Кэш

Кэш — это временная память процессора. Эта функциональность реализуется на уровне микросхем и позволяет обращаться к информации быстрее, чем к основной памяти компьютера.

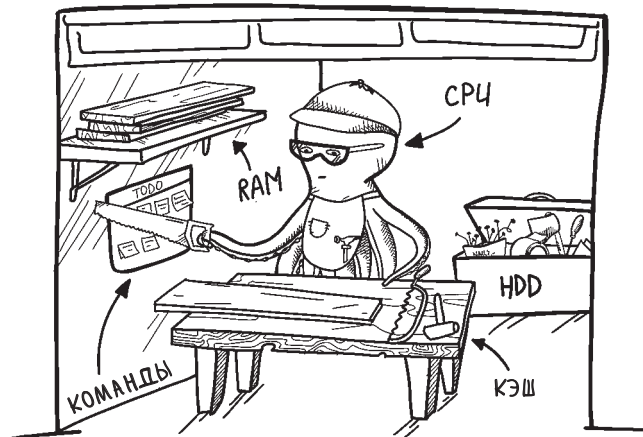
Представьте столярную мастерскую с одним работником. Этот работник (процессор) обрабатывает заказы (команды), которые поступают от клиентов. Чтобы быстрее изготавливать нужные изделия, столяр устроил поблизости временное хранилище для материалов и других ресурсов, чтобы ему не приходилось ходить на склад, где хранятся все запасы (жесткий диск, HDD).



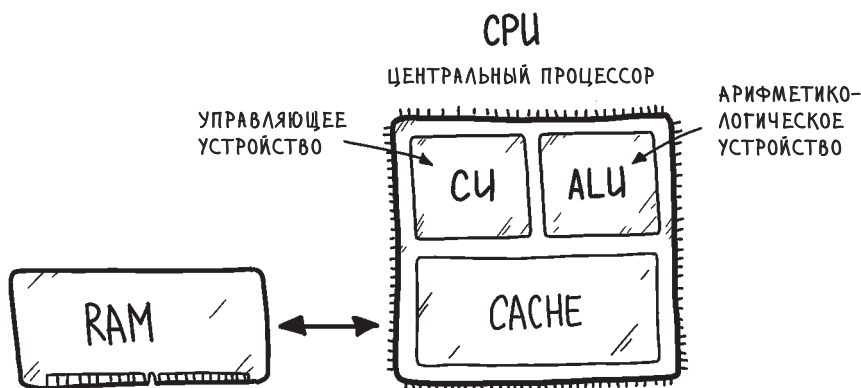
Временное хранилище — это аналог памяти, подключенной к процессору, которая называется *оперативной памятью*, или ОЗУ (RAM), и хранит данные и команды. Когда программа начинает выполняться, исполняемые файлы и данные копируются в оперативную память и хранятся в ней, пока программа не завершится.

Однако процессор никогда не обращается к оперативной памяти напрямую. Он способен выполнять вычисления гораздо быстрее, чем память может передавать ему данные. Чтобы ускорить доступ, современные процессоры оснащаются одним или несколькими уровнями *кэш-памяти*.

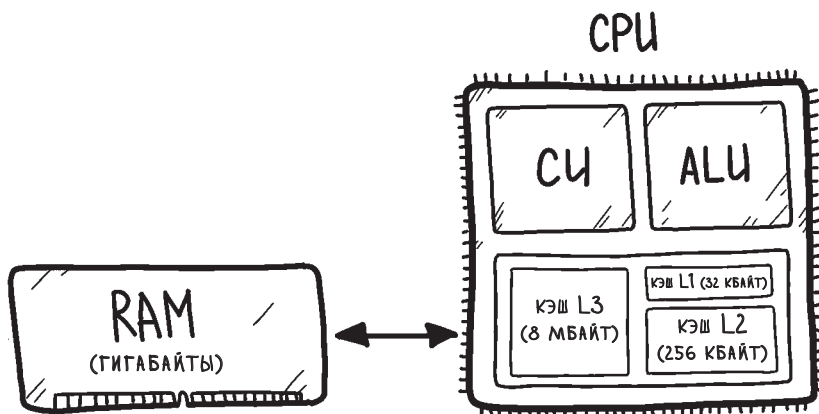
Возвращаясь к примеру со столярной мастерской, наряду с материалами во временном хранилище столяру нужно, чтобы необходимые инструменты постоянно были под рукой. Столяр хранит их на верстаке, где к ним можно быстро получить доступ. Кэш-память можно рассматривать как своего рода верстак для процессора.



Кэш-память быстрее оперативной памяти, и она находится ближе к процессору, потому что размещена непосредственно на его микросхеме. Кэш-память предоставляет место для хранения данных и команд, чтобы процессор не ждал, пока данные загрузятся из оперативной памяти. Когда процессору понадобятся данные — а программные команды считаются данными, — контроллер кэша определяет, присутствуют ли данные в кэше, и предоставляет их процессору. Если запрошенных данных нет в кэше, они загружаются из оперативной памяти и помещаются в кэш. Контроллер кэша анализирует запрашиваемые данные, прогнозирует, какие еще данные понадобятся запросить из оперативной памяти, и загружает их в кэш.



Процессор содержит кэши трех уровней: L1, L2 и L3. Уровни L2 и L3 предназначены для того, чтобы прогнозировать, какие данные и команды могут понадобиться на следующем шаге; эти данные и команды загружаются из оперативной памяти в кэш L1, чтобы они были ближе к процессору и готовы, когда потребуются. Чем выше уровень, тем больше памяти в нем доступно и тем медленнее канал связи. Кэш L1 находится ближе всего к процессору. Благодаря дополнительным уровням кэша процессор остается занятым и не теряет время на то, чтобы ждать необходимых данных.



Почти все обращения к данным и коммуникация добавляются к *задержке выполнения*; эти коммуникационные затраты — одна из самых больших угроз для производительности системы, и кэш помогает их устранить или по крайней мере ослабить. Посмотрим, как они влияют на задержку, если масштабировать все в повседневные единицы, которые интуитивно понятны человеку (это называется *масштабированной задержкой*).

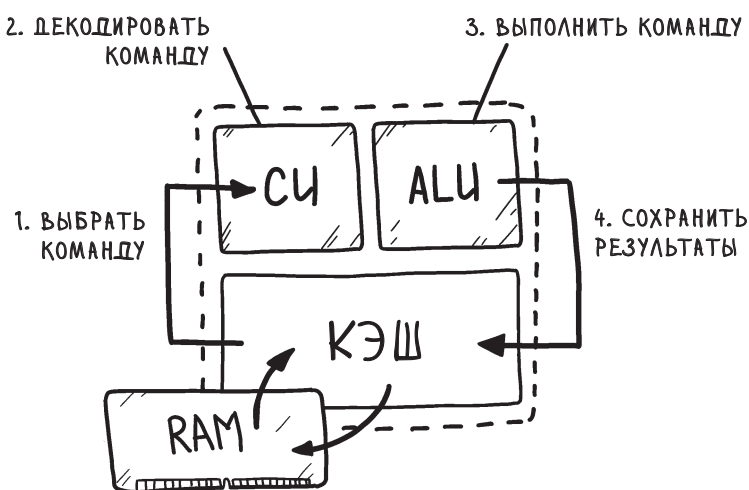
Системное событие	Фактическая задержка	Масштабированная задержка
Один такт процессора	0,4 нс	1 с
Обращение к кэшу L1	0,9 нс	2 с
Обращение к кэшу L2	2,8 нс	7 с
Обращение к кэшу L3	28 нс	1 мин
Обращение к оперативной памяти	~100 нс	4 мин
Скоростной ввод/вывод SSD	< 10 мкс	7 ч
Ввод/вывод SSD	50–150 мкс	1,5–4 дня
Ввод/вывод HDD	1–10 мс	1–9 месяцев
Сетевой запрос из Сан-Франциско в Нью-Йорк	65 мс	5 лет

Разобравшись с кэшами, обратимся к фактическому циклу выполнения.

Цикл выполнения (цикл процессора)

Вернемся к примеру с мастерской. Один столяр выполняет всю работу — от общения с клиентами до обработки дерева. Он получает запросы от клиентов, преобразовывает эти запросы в рабочие задачи, выполняет задачи и передает результаты клиентам. Столяр проводит все свое время в этом цикле, и именно благодаря этой деятельности мастерская приносит доход.

Аналогичным образом процессор занимается непрерывной деятельностью по выполнению команд, которая состоит из нескольких стадий. Эта последовательность называется *циклом процессора*. В простейшей форме процессор повторяет четыре стадии:



1. *Выбрать команду.* Управляющее устройство загружает команду из памяти или кэша и копирует ее в процессор. При этом устройство использует различные счетчики, чтобы понять, какую команду нужно загрузить и где ее найти.
2. *Декодировать команду.* Ранее выбранная команда декодируется и отправляется для обработки. Команды разных типов выполняют разные операции, так что в зависимости от типа команды и кода операции необходимо знать, каким обрабатывающим блокам будет отправлена команда.
3. *Выполнить команду.* Команда перемещается в арифметико-логическое устройство, и начинается ее выполнение.
4. *Сохранить результат.* Когда команда выполнена, результат записывается в оперативную память и начинается обработка следующей команды. Процесс возвращается к шагу 1, и это повторяется, пока не останется ни одной команды для выборки.

Процессор проводит в этом цикле все свое время: он неустанно загружает следующую команду, декодирует ее, выполняет и сохраняет результат.

Система выполнения

Работать с процессором — дело далеко не простое. Разработчикам приходится заниматься всеми вопросами самостоятельно, включая различные текущие операции: управлять аппаратными ресурсами, координировать доступ к этим ресурсам, управлять конкретной функциональностью, которую нужно выполнить, обеспечивать изоляцию между программами на случай сбоев, обращаться к общим ресурсам и т. д.

Современные системы обычно бывают многоцелевыми и, как следствие, сложными. Со временем они обрastaют множеством программных подсистем, которые относятся к конкретным операциям управления: управление файлами, графикой,

задачами и т. д. Все это примеры микропрограммных управляющих систем, которые со временем эволюционировали в дополнительный уровень абстракции между приложением и системой: *систему выполнения*, самым распространенным примером которой является *операционная система*.

Вернемся к примеру со столярной мастерской. Столяр начал получать от клиентов странные заказы: доставить древесину, построить корабль или мост или изготовить продукты, которые выходят за пределы возможностей его инструментов. Столяр догадался, что эти заказы поступили по ошибке, потому что ими должны заниматься другие специалисты. Он решил нанять кого-нибудь, кто будет проверять заказы и передавать столяру только ту работу, которую тот может выполнить. Он договорился с другими владельцами мастерских на той же улице, чтобы привлечь менеджера для обработки входящих заказов. Менеджер предлагает клиенту заполнить стандартную анкету, после чего определяет, кому передавать заказ, и передает его столяру или другому мастеру.

Роль менеджера выполняет *операционная система* (ОС) — низкоуровневый системный интерфейс между аппаратными компонентами компьютерной системы и разработчиком. Эти интерфейсы называются *системными функциями* (или *системными вызовами*). Они взаимодействуют с оборудованием и предоставляют различные службы и возможности, которые могут использоваться пользовательскими приложениями.



Например, когда программа хочет записать данные на диск, она делегирует эту задачу ОС. ОС отдает указания диску через контроллер диска, который может отправить диску правильные сигналы. В таком режиме программе, которая хочет использовать диск, не нужно беспокоиться о том, какой именно диск установлен в системе, или понимать, как он работает. ОС берет на себя все технические подробности и по возможности старается защитить оборудование и другие ресурсы от некорректного использования. Это создает дополнительные накладные расходы, потому что программа использует функциональность ОС, не взаимодействуя напрямую с оборудованием. Иногда это может быть критично, и какие-то операции

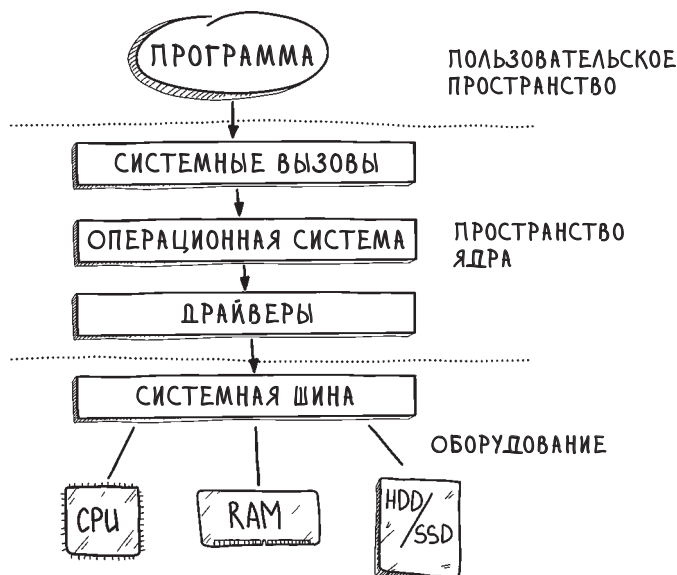
выгоднее выполнять на уровне приложения вместо того, чтобы вызывать системную функцию. Примеры будут приведены в следующих главах.

Чтобы выполнить программу с использованием ОС, прежде всего надо загрузить исполняемый файл и все статические данные (например, инициализированные переменные) в память. Затем программа запускается через точку входа `main()`. Когда ОС передает управление `main()`, контроль над процессором переходит к программе; так она начинает выполняться под управлением и защитой ОС.

По этой схеме работают все современные компьютерные системы. В конкретных случаях процесс может быть сложнее, но в целом его составляющие остаются теми же.

Архитектура компьютерных систем

Взглянув на схему компьютерной системы, вы увидите один или несколько процессоров; оперативную память, доступную для процессоров; различные периферийные устройства (принтеры, устройства для чтения карт, жесткие диски, мониторы и т. д.), а также контроллеры устройств, или *драйверы*, которые позволяют всем этим устройствам взаимодействовать с процессором или оперативной памятью. Также есть *системная шина* — канал, который обеспечивает передачу данных между процессором, оперативной памятью и периферийными устройствами.



Обратите внимание на *пользовательское пространство* и *пространство ядра* — две разные области компьютерной системы. В пользовательском пространстве выполняются приложения пользовательского уровня, а в пространстве ядра — базовые функции ОС и системные вызовы. Это различие важно, потому что приложения

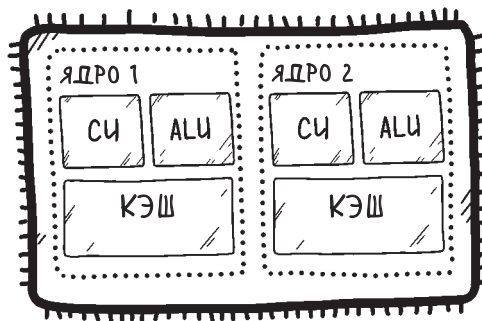
в пользовательском пространстве не могут обратиться к нижележащей системе или изменить ее, тогда как ядро полностью контролирует систему и ее ресурсы. Внутренняя структура компьютерной системы остается в целом неизменной независимо от специфики аппаратной платформы: форм-фактора, структуры ОС, предполагаемого использования и т. д.

Разобравшись в общих компонентах архитектуры, можно рассмотреть разные уровни конкурентного оборудования, которые может представлять эта архитектура.

Уровни конкурентного оборудования

Процессоры состоят из множества логических блоков (арифметико-логических устройств), которые способны выполнять базовые арифметические операции (например, сложение или умножение). Благодаря этому процессор может разбивать сложные математические операции на составные части, которые одновременно выполняются на разных арифметических устройствах. Это называется *параллелизмом на уровне команд*. Иногда этот тип параллелизма переходит на еще более глубокий уровень — *параллелизм на уровне битов*. (Многие разработчики редко задумываются об этом уровне. Упорядочиванием команд в наиболее удобной для процессора последовательности занимается компилятор. Этот уровень представляет интерес лишь для немногих инженеров, которые пытаются выжать всю возможную производительность из процессора или компилятора.)

Создать параллельное оборудование можно и по-другому: установить в компьютерной системе несколько процессоров-дубликатов — по аналогии с наймом менеджера, — чтобы все мастера могли вместе обрабатывать заявки клиентов. Такая конфигурация называется *многопроцессорной системой* — собственно, так можно назвать любую компьютерную систему, в которой больше одного процессора.



Многоядерный процессор

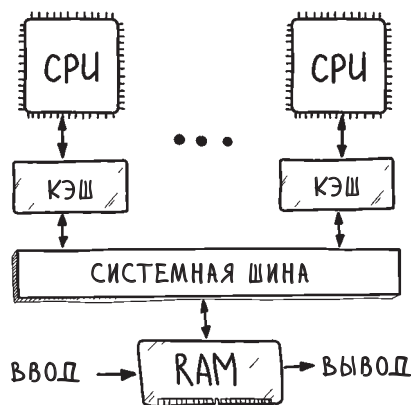
Многоядерный процессор — особая разновидность многопроцессорной конфигурации, в которой все процессоры установлены на одной микросхеме. Каждое ядро работает независимо, а ОС воспринимает каждое ядро как отдельный процессор. Между этими двумя подходами существуют некоторые различия в том, насколько

быстро процессоры могут работать вместе и как они обращаются к памяти, но в этой книге они будут рассматриваться как эквивалентные.

Симметричная multiprocessing (SMP)

Компьютерная память обычно работает гораздо медленнее, чем процессор, из-за чего возникают коммуникационные затраты, которые упоминались в главе 2. Вот почему многие современные multiprocessing системы используют архитектуру *симметричной multiprocessing*, или *SMP* (symmetric multiprocessing). SMP — это набор идентичных процессоров, которые подключены к общей памяти с одним адресным пространством и работают под управлением одной ОС.

Процессоры в архитектуре SMP связываются соединительной сетью через системную шину. Эти сети работают быстро, но если процессорам потребуется обменяться данными, этот обмен не происходит мгновенно, потому что ему приходится проходить через одно или несколько соединительных звеньев. Затратами на коммуникацию нельзя пренебречь, и эта проблема ухудшает задержку по мере того, как растет число взаимодействующих ресурсов и расстояния между ними. В результате в архитектуре SMP у каждого процессора есть собственный частный кэш, который позволяет сократить трафик по системной шине и тем самым снизить задержку.

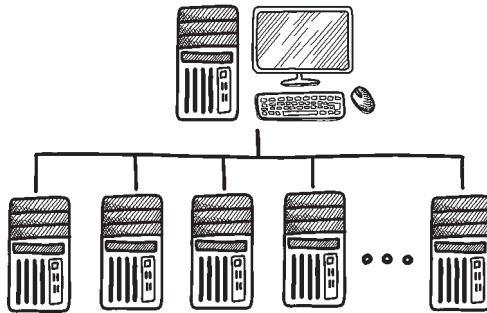


Архитектура симметричной multiprocessing (SMP) состоит из нескольких связанных процессоров с общей памятью

Самая замечательная особенность SMP заключается в том, что наличие нескольких процессоров прозрачно для конечного пользователя. ОС обеспечивает планирование процессов на отдельных процессорах и синхронизацию между ними. Однако если в таких системах повышается количество процессоров, которые подключены к общей системной шине, она становится узким местом. Проблема усугубляется *согласованием кэшей*, когда несколько процессорных ядер совместно используют одну иерархию памяти, но у каждого ядра собственные данные L1 и кэши команд.

ПРИМЕЧАНИЕ Разработка протокола MESI в 1980-е годы решила проблему согласования кэшей в многопроцессорных системах. Отслеживая состояние каждой записи кэша, MESI гарантирует, что у всех процессоров будет целостное представление данных, что обеспечит эффективное взаимодействие без конфликтов. В наши дни протокол MESI стал важнейшей частью компьютерных технологий.

Перейти от SMP к большим компьютерам с массовым параллелизмом можно только одним способом: отказаться от архитектуры общей памяти и ввести системы с распределенной памятью, которые называются *вычислительными кластерами*. Кластеры — это распределенные группы машин с собственными процессорами, которые объединены в сеть и образуют очень мощные параллельные системы. Ни одна машина не может использовать память совместно с другой, потому что каждая работает независимо. Если у одной машины изменяется локальная память, это изменение не отражается автоматически в памяти процессоров на других машинах. Поэтому в кластерах обычно используется распределенная память, что повышает коммуникационные затраты, так как данные необходимо передавать по сети, а это намного медленнее, чем обмен данными между процессами на локальной машине.



Кластеры подходят для задач со *слабым сцеплением* (которые не требуют частых взаимодействий между процессорами, но требуют большей вычислительной мощности), тогда как для задач с *сильным сцеплением* больше подходят одномашинные системы. Преимущество кластеров — хорошая масштабируемость, а их недостаток — высокие коммуникационные затраты. Мы подробнее рассмотрим распределенные системы в следующих главах, а пока сосредоточимся на разновидностях многопроцессорных архитектур.

Таксономия параллельных компьютеров

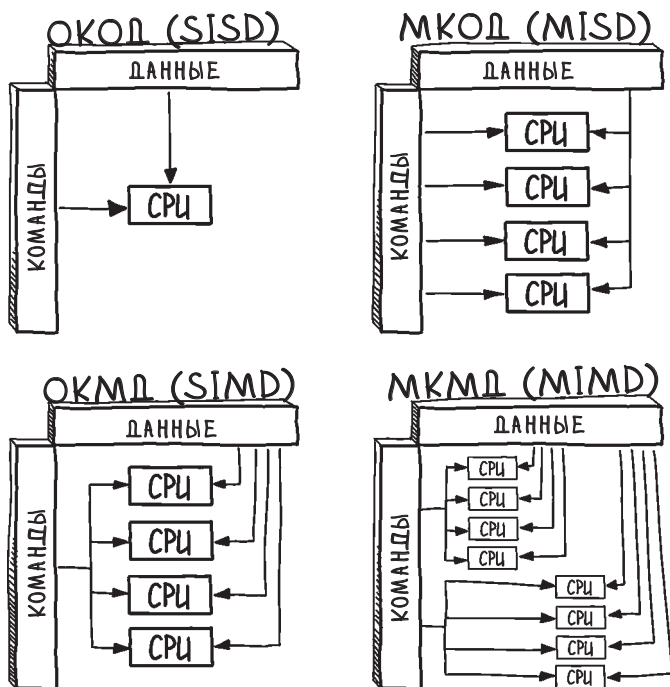
Одна из самых распространенных систем классификации многопроцессорных архитектур — *таксономия Флинна*. Она выделяет четыре класса компьютерных архитектур по двум независимым признакам: *потокам команд* и *потокам данных*.

Первая и вторая категории архитектур — с одним *потокком команд* и одним *потокком данных* (ОКОД, SISD) и со *множественными потоками команд* и одним *потокком данных* (МКОД, MISD) — подразумевают, что один блок данных обрабатывается

одной или несколькими командами соответственно. Поскольку эти категории не поддерживают параллелизацию, они неактуальны для конкурентных систем и упоминаются здесь только для полноты изложения.

Третью категорию составляют архитектуры с *одним потоком команд и множественными потоками данных* (ОКМД, SIMD), где общие управляющие устройства работают с несколькими ядрами. С такой архитектурой на всех доступных вычислительных ресурсах можно в каждый момент времени выполнять только одну команду, что позволяет одновременно производить одну и ту же операцию над большим количеством элементов данных. С другой стороны, набор команд на машинах ОКМД ограничен, в результате чего они подходят для специфических задач, которые требуют высокой вычислительной мощности, но не подразумевают особой гибкости. Типичный пример ОКМД в наши дни — графические процессоры (GPU).

К четвертой категории относятся архитектуры со *множественными потоками команд и множественными потоками данных* (МКМД, MIMD). В этом случае у каждого вычислительного ресурса есть независимое управляющее устройство, благодаря чему ресурс не ограничивается определенными типами команд, а выполняет разные команды независимо на отдельных блоках данных. Таким образом, в эту категорию входят архитектуры с несколькими ядрами, несколькими процессорами и даже несколькими машинами, которые позволяют выполнять разные задачи независимо на разных устройствах.



В архитектуре МКМД шире набор команд, так что отдельные вычислительные ресурсы более универсальны, чем в ОКМД. По этой причине МКМД стала наиболее распространенной архитектурой из таксономии Флинна и встречается где угодно — от персональных компьютеров с многоядерными процессорами до распределенных кластеров.

Центральный процессор и графический процессор

Даже если вы не играете в видеоигры, у вас есть повод сказать спасибо геймерам, потому что благодаря им развился класс очень мощных устройств параллельной обработки — графические процессоры (GPU). У центральных и графических процессоров много общего: и те и другие содержат миллионы транзисторов и могут выполнять огромное количество команд в секунду. Но чем различаются эти два важных компонента и когда следует использовать каждый из них?

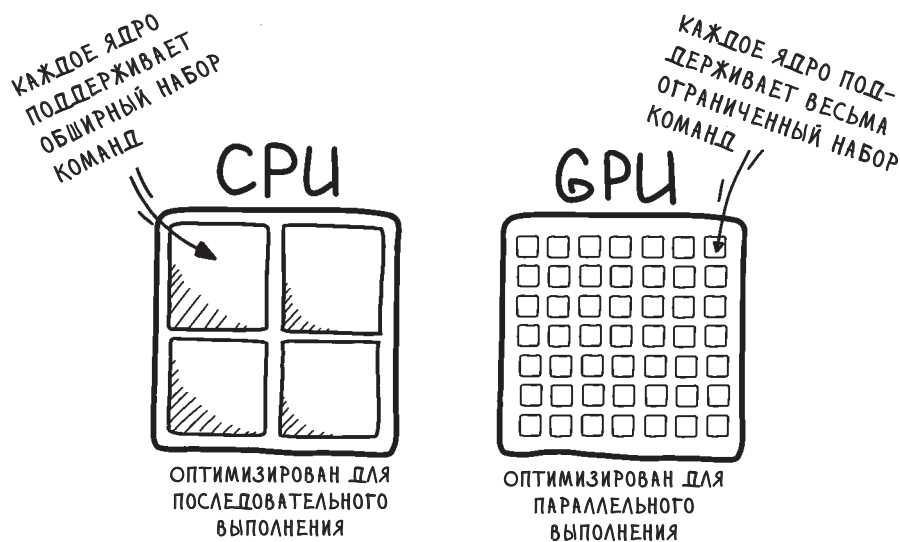
Стандартные центральные процессоры основаны на архитектуре МКМД. Современные процессоры — мощные устройства, потому что инженеры реализовали в них обширный набор команд. Чтобы компьютерная система выполнила ту или иную задачу, нужно, чтобы ее выполнил процессор.

Графический процессор — специализированная разновидность процессора с архитектурой, похожей на ОКМД, которая оптимизирована для очень ограниченного набора команд. Графический процессор работает на более низкой тактовой частоте, чем центральный, но у него больше ядер — сотни и даже тысячи, — которые работают одновременно. Это значит, что он способен выполнять огромное количество простых команд с невероятной скоростью из-за массового параллелизма.

ПРИМЕЧАНИЕ Например, видеокарта NVIDIA GTX 1080 содержит 2560 ядер с тактовой частотой 1607 МГц. Благодаря этим ядрам видеокарта может выполнять 2560 команд за один такт. Если вы хотите сделать изображение на 1 % светлее, графический процессор справится с этим без малейших затруднений. С другой стороны, центральный процессор Intel Core i9-10940X с частотой 3,3 ГГц может выполнять только 14 команд за один такт¹.

Хотя отдельные ядра центральных процессоров работают быстрее в соответствии с тактовой частотой и у них обширный набор команд, общее количество ядер графического процессора и массовый параллелизм, который они обеспечивают, более чем компенсируют разность в тактовой частоте ядер и ограниченный набор команд. Просто центральные процессоры лучше подходят для сложных линейных задач.

¹ Спецификация процессора Intel Core i9-10940X X-series, <http://mng.bz/JgGz>.



Графические процессоры лучше подходят для повторяющихся и сильно параллелизованных вычислительных задач — таких, как обработка видео и изображений, машинное обучение, финансовое моделирование и многие другие типы научных вычислений. Графические процессоры легко выполняют такие операции, как матричное сложение и умножение, потому что большинство подобных операций в ячейках матриц не зависят друг от друга и выполняются сходным образом — а значит, могут параллелизоваться.

Аппаратные архитектуры весьма разнообразны и могут влиять на переносимость программ между разными системами. Кроме того, программы иногда по своей природе ускоряются по-разному в зависимости от того, где они выполняются. Например, многие графические программы выполняются намного быстрее и лучше на ресурсах графического процессора, тогда как обычные программы со смешанной логикой эффективнее работают на центральном процессоре.

В этой книге мы используем термин «процессор» (CPU) в обобщенном смысле, который включает обе разновидности вычислительных ресурсов. Получив представление обо всех компонентах физического выполнения, в следующей главе мы добавим несколько удобных абстракций, которые представляют потоки команд.

Итоги

- Выполнение программ зависит от фактического оборудования. Современное оборудование оснащается несколькими вычислительными ресурсами (*многоядерные системы, многопроцессорные системы или вычислительные кластеры*) и оптимизируется для выполнения программ.

- Таксономия Флинна выделяет четыре разновидности архитектур на основании того, выполняет ли система одну или несколько команд одновременно (ОК или МК) и применяется ли каждая команда к одному или нескольким блокам данных (ОД или МД).
- Графический процессор (GPU) — это пример архитектуры ОКМД (SIMD). Графические процессоры оптимизируются для интенсивного параллельного выполнения задач.
- Современные многопроцессорные системы и многоядерные процессоры — примеры архитектуры МКМД. Они устроены намного сложнее, потому что являются многоцелевыми устройствами.
- Центральный процессор (CPU) — это «мозг» компьютерной системы, но работать с ним напрямую довольно трудно. В программировании между приложением и системой вводится дополнительный уровень абстракции — *система выполнения*.
- Чтобы реализовать параллельное выполнение, разработчику приложения понадобится вычислительное устройство, которое подходит для этой задачи. У центрального процессора выше тактовая частота и шире набор команд, которые можно выполнять параллельно, тогда как графический процессор работает на более низкой тактовой частоте и выполняет только одну команду на всех ядрах, но делает это с невероятной скоростью благодаря массовому параллелизму.

4

Структурные элементы конкурентности



В этой главе

- ✓ Средний уровень конкурентности: система выполнения (распространенным примером такой системы служит ОС)
- ✓ Как устроены две основные абстракции конкурентности: потоки и процессы
- ✓ Как реализовывать конкурентные приложения с помощью потоков и процессов
- ✓ Как выбирать абстракцию конкурентности для конкретной задачи

Конкурентное программирование подразумевает, что приложения разбиваются на независимые блоки конкурентного выполнения. В предыдущих главах мы называли такие блоки *задачами*, чтобы структурировать логику выполнения приложения. Теперь, когда вы знаете, какое оборудование при этом используется, можно связать эти абстракции с физическими устройствами, на которых выполняется код. К счастью, с этой задачей может справиться другая абстрактная прослойка — операционная система. Она помогает использовать доступное оборудование наиболее эффективно, но все-таки не является волшебным решением. В этой главе основное внимание уделено тому, как разработчики могут структурировать свои программы, чтобы помочь ОС оптимизировать нагрузку на оборудование.

Этапы конкурентного программирования

Конкурентное программирование — это набор абстракций, которые позволяют разработчикам структурировать программу так, чтобы формировать небольшие независимые задачи и передавать их системе выполнения, помещая в очередь. Эта система координирует задачи так, чтобы оптимально использовать системные мощности, и передает их для выполнения на соответствующих вычислительных ресурсах. В конкурентном программировании для этого используются две главные абстракции: процессы и программные потоки.

Процессы

Неформальное определение *процесса* относительно прямолинейно: это работающая программа. Программа не работает сама по себе: она находится на диске и состоит из набора команд, которые ожидают выполнения. ОС берет эти команды и выполняет их на оборудовании, превращая программу в нечто полезное.

Представьте себе автомобиль. Это всего лишь набор механических частей. И хотя у него большой потенциал, он не имеет никакой ценности, если стоит на месте. Но когда кто-то поворачивает ключ и двигатель начинает работать, автомобиль приходит в движение. Начинается процесс вождения: это уже не автомобиль, а поездка из точки А в точку В, которая обладает ценностью. Автомобиль позволяет выполнить нужное действие.



Исходный код похож на автомобиль. Это всего лишь пассивная последовательность команд, которые взаимодействуют с абстракциями ресурсов. Когда разработчики пишут исходный код, они не оперируют памятью для хранения фактических временных данных, файлами для чтения или записи либо устройствами, которым они хотят отправлять сигналы. Разработчики пишут код с использованием моделей, построенных на базе абстракций, которые предоставляют языки программирования и среды выполнения. Фактические ресурсы должны обеспечиваться во время выполнения.

Процесс — это тоже абстракция, которую предоставляет ОС для выполняемой программы. На уровне машинных команд не существует понятия процесса.

ОС использует процессы для того, чтобы изолировать задачи и назначать аппаратные ресурсы для их выполнения. Все процессы в ОС совместно используют аппаратные ресурсы и находятся под управлением ОС. Чтобы ОС знала о связях между процессами и ресурсами, у каждого процесса должно быть собственное независимое адресное пространство и файловая таблица. Таким образом, процессы — это единицы распределения ресурсов в ОС.

Операционные системы создают иллюзию того, что каждый процесс полностью распоряжается всей компьютерной системой, хотя обычно несколько процессов работают в конкурентном режиме. Чтобы поддерживать эту иллюзию, ОС прикладывает серьезные усилия, управляя процессами и защищая их, а также изолируя их друг от друга. В частности, ОС распределяет процессорные ядра и память для всех процессов. Главное преимущество процессов — в том, что они полностью независимы и их выполнение изолировано от остальных частей системы, что предотвращает случайные коллизии с глобальными объектами и гарантирует, что сбой одной программы не повлияет на остальные.

Однако из этого преимущества также следует недостаток. Процессы по своей природе не зависят друг от друга, что усложняет взаимодействие между ними. Формально у разных процессов нет почти ничего общего, и для любой нетривиальной коммуникации между процессами требуется использовать другие механизмы, которые обычно на несколько порядков медленнее, чем прямой доступ к данным. Эта тема подробно рассматривается в главе 5, а пока посмотрим, как устроен процесс.

Внутреннее устройство процесса

Как упоминалось ранее, процесс — это просто выполняющаяся программа. В любой момент времени процесс можно охарактеризовать как список разных компонентов компьютерной системы, к которым он обращается или которые он изменяет во время выполнения:

- Данные, которые читает или записывает процесс, хранятся в памяти. Таким образом, память, к которой он может обращаться для чтения или записи (адресное пространство), является частью выполняемого процесса.
- Исполняемый файл со всеми машинными командами тоже является частью процесса.
- Процессу также необходим идентификатор (ID) — уникальное имя,



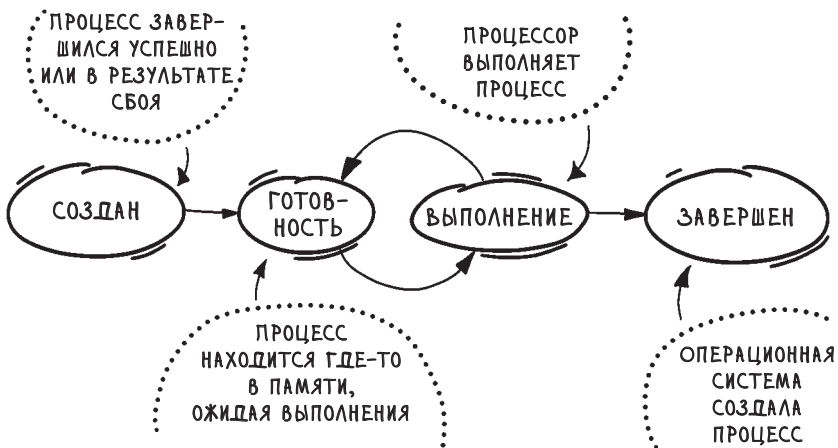
которое однозначно идентифицирует процесс. Идентификатор процесса обычно сокращенно называется *PID* (Process Identifier).

- Наконец, программы часто обращаются к дискам, сетевым ресурсам или другим сторонним устройствам. К соответствующей информации относится список файлов, которые в настоящее время открыты процессом, список открытых сетевых подключений и любая дополнительная информация о ресурсах, которые использует процесс.

Таким образом, процесс инкапсулирует многие компоненты: исполняемый файл, набор используемых ресурсов (файлы, подключения и т. д.) и адресное пространство с внутренними переменными. Все это называется *контекстом выполнения*. Поскольку внутри процессов так много всего, запуск нового процесса — относительно ресурсоемкая задача. Поэтому процессы часто называются *тяжеловесными*.

Состояния процесса

Если рассматривать процесс на высоком уровне, все выглядит элементарно. Сначала процесса не существует. Затем он создается и инициализируется, после чего располагается где-то в памяти компьютера (состояние «Создан»). Когда пользовательский код запускает процесс, он переходит в состояние готовности: он готов выполняться на ядре процессора в любой момент, но пока еще ничего не делает. Чтобы начать выполнение, процессу нужен вычислительный ресурс. Затем из списка процессов, готовых к выполнению, ОС выбирает следующий процесс, который будет выполняться на процессоре. Когда процесс выбран, он переходит в состояние выполнения.



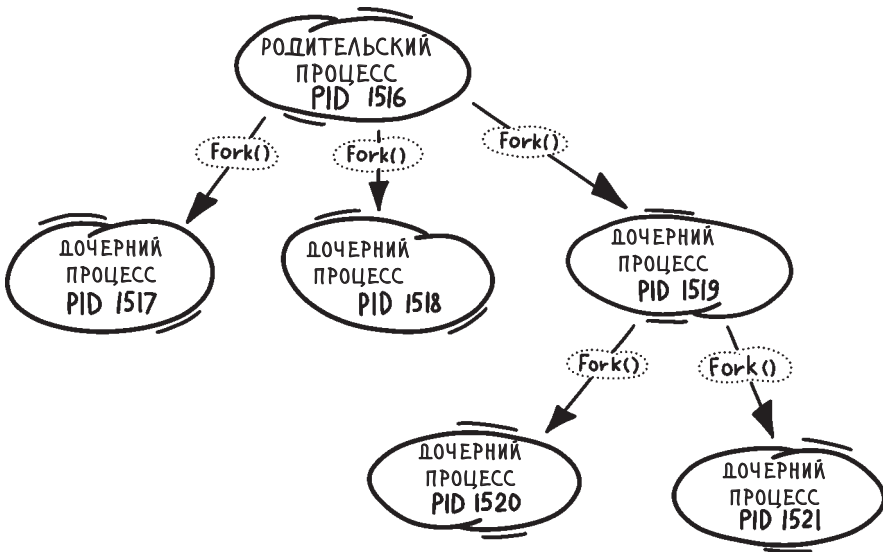
Процессы обычно создает операционная система, и она же отвечает за их завершение. Это нетривиальная задача. ОС должна понять, что процесс завершился — либо задача выполнена, либо он прервался в результате сбоя и нужно освободить соответствующую память и другие ресурсы, либо родительский процесс прекратил существование. Создание или завершение процесса обходится относительно за-

тратно, потому что, как вы убедились, за процессом закрепляется много ресурсов, которые необходимо выделить или освободить. На это уходит время, и возникает дополнительная задержка.

Множественные процессы

Процессы могут создавать другие процессы (так называемые *дочерние процессы*) с помощью соответствующих системных вызовов — таких, как `fork()` или `spawn()`; это называется *порождением новых процессов*. дочерние процессы — это независимые ответвления основного процесса с отдельным адресным пространством памяти. А это снова означает, что процесс работает независимо и изолируется от других процессов под контролем ОС. Он не может напрямую обращаться к данным других процессов, а команды, принадлежащие каждому процессу, исполняются в нем независимо и в идеале параллельно.

Теперь мы перемещаемся на территорию конкурентности. Порождая новые процессы, выполнение можно декомпозировать на несколько процессов, которые можно выполнять одновременно на параллельном оборудовании.



Но, наверное, в коде понять это будет проще, чем в теории. Вот пример программы, которая создает три дочерних процесса с помощью ветвления:

```
# Chapter 4/child_processes.py
import os
from multiprocessing import Process

def run_child() -> None:
    print("Дочерний: Я дочерний процесс")
    print(f"Дочерний: PID дочернего процесса: {os.getpid()}")
    print(f"Дочерний: PID родительского процесса: {os.getppid()}")
```

```
def start_parent(num_children: int) -> None:
    print("Родительский: Я родительский процесс")
    print(f"Родительский: PID родительского процесса: {os.getpid()}")
    for i in range(num_children):
        print(f"Запускается процесс {i}")
        Process(target=run_child).start()

if __name__ == "__main__":
    num_children = 3
    start_parent(num_children)
```

*Порождает новый процесс.
Метод start() запускает
функцию run_child
в отдельном процессе*

Этот код создает родительский процесс с тремя дочерними процессами, которые являются копиями родительского; различаются только идентификаторы процессов. Родительский и дочерние процессы выполняются независимо друг от друга.

ПРИМЕЧАНИЕ Важно заметить, что при ветвлении процесса новый процесс начинает выполняться с точки, в которой произошло ветвление, а его внутреннее состояние копируется. Сценарий не отбрасывается с самого начала.

Программа выводит сообщения от родительского и дочерних процессов с соответствующими PID, как в следующем листинге:

```
Родительский: Я родительский процесс
Родительский: PID родительского процесса: 73553
Родительский: PID дочернего процесса: 73554
Дочерний: Я дочерний процесс
Дочерний: PID дочернего процесса: 73554
Дочерний: PID родительского процесса: 73553
Родительский: Я родительский процесс
Родительский: PID родительского процесса: 73553
Родительский: PID дочернего процесса: 73555
Дочерний: Я дочерний процесс
Дочерний: PID дочернего процесса: 73555
Дочерний: PID родительского процесса: 73553
Родительский: Я родительский процесс
Родительский: PID родительского процесса: 73553
Родительский: PID дочернего процесса: 73556
Дочерний: Я дочерний процесс
Дочерний: PID дочернего процесса: 73556
Дочерний: PID родительского процесса: 73553
```

В языках программирования обычно есть высокоуровневые абстракции или служебные методы для работы с процессами, потому что они проще в сопровождении и более понятны в исходном коде программы.

ПРИМЕЧАНИЕ В нескольких популярных серверных продуктах ветвление и порождение новых процессов реализованы в режиме *предварительного ветвления*. Это значит, что при запуске сервера он создает ответвленные процессы, которые затем обрабатывают входящие запросы. В таком режиме работают nginx, Apache HTTP Server и Gunicorn, что позволяет им обрабатывать сотни запросов одновременно. Но эти решения поддерживают и другие методы.

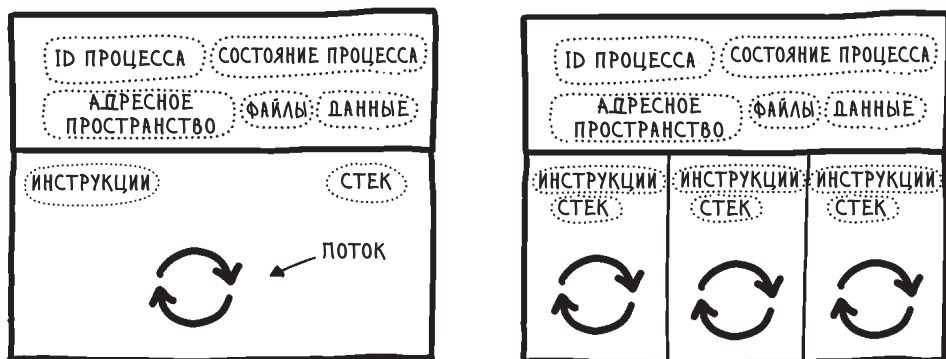
Потоки

В большинстве ОС процессы могут совместно использовать память, но это требует дополнительных усилий (эта тема обсуждается в главе 5). Другая абстракция — *потоки* — несколько упрощает совместное использование ресурсов.

В конечном итоге программа представляет собой набор машинных команд, которые должны последовательно выполняться одна за другой. Чтобы это произошло, ОС использует понятие *программных потоков* (или просто потоков). Формально поток определяется как независимая последовательность команд, исполнение которых может планировать операционная система.

Помните, мы говорили, что процесс — это совокупность выполняющейся программы и ресурсов? Если разбить программу на компоненты, то процесс будет контейнером ресурсов (адресное пространство, файлы, сетевые подключения и т. д.), а поток — динамической частью; это последовательность команд, которая выполняется в этом контейнере. Следовательно, в контексте ОС процесс можно рассматривать как единицу ресурсов, а поток — как единицу выполнения.

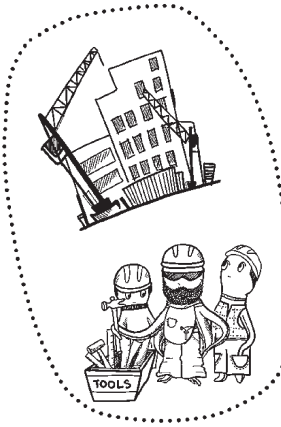
Однако потоки родились из той идеи, что самый эффективный способ обмениваться данными между взаимодействующими процессами — это совместно использовать общее адресное пространство. Таким образом, потоки одного процесса похожи на процессы, которые могут легко использовать ресурсы совместно друг с другом и со своим родительским процессом; к этим ресурсам относятся адресное пространство, файлы, сетевые подключения, общие данные и т. д.



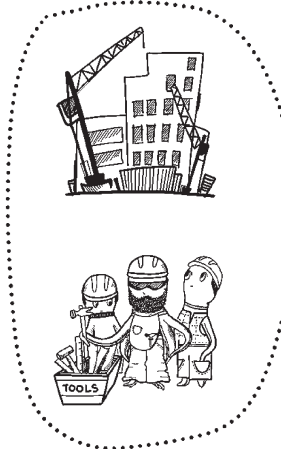
Потоки также поддерживают свое собственное состояние, чтобы иметь возможность выполнять свои команды безопасно, локально и независимо. Каждый поток ничего не знает о других потоках, если только он не будет намеренно вмешиваться в их работу. ОС управляет потоками и может распределять их между доступными ядрами процессора. Таким образом, многопоточная программа может быть удобным способом, чтобы конкурентно выполнять несколько задач.

Чтобы продемонстрировать разницу между процессами и потоками, рассмотрим пример. Представьте, что вы управляете строительной компанией и наняли три бригады строителей для работы над тремя разными проектами.

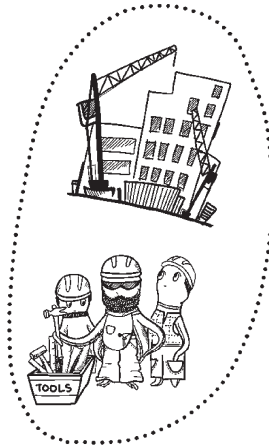
ПРОЕКТ 1



ПРОЕКТ 2

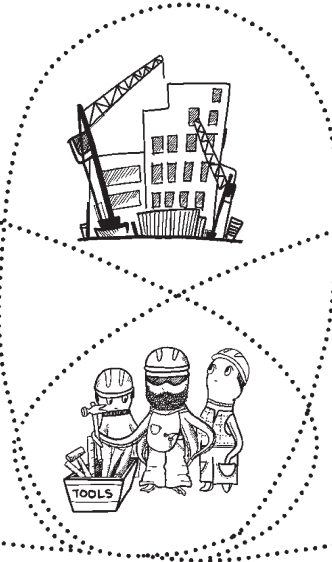


ПРОЕКТ 3

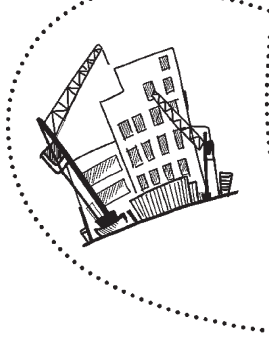


Это напоминает конфигурацию с процессами. Каждая бригада (процесс) специализируется на одном проекте (задаче) с собственными инструментами, планом и ресурсами. С другой стороны, чтобы сэкономить средства, можно нанять одну бригаду для всех трех проектов; она будет использовать общие инструменты и ресурсы, но у каждого проекта будет свой собственный список команд — по аналогии с тем, как работают потоки.

ПРОЕКТ 2



ПРОЕКТ 1



ПРОЕКТ 3



Исторически разные производители оборудования реализовывали собственные версии потоков. Эти реализации значительно различались, что мешало разработчикам писать переносимые многопоточные приложения. Возникла необходимость в стандартизированном интерфейсе программирования.

Для систем UNIX этот интерфейс был определен документом IEEE POSIX¹; для ОС семейства Windows он доступен в виде дополнительной библиотеки. Реализации, которые придерживаются этого стандарта, называются *потоками POSIX* или *Pthreads* (имя реализации в библиотеке C). Многие производители оборудования используют Pthreads, поэтому мы рассмотрим этот стандарт подробнее.

В стандарте каждая программа, которая запускается в системе, заставляет ОС создать процесс, а каждый процесс содержит хотя бы один поток; процесс без потока не может существовать. Каждый поток также поддерживает свой независимый контекст выполнения, который обеспечивает безопасное и независимое выполнение его команд.

Особенности потоков

При правильной реализации потоки обладают как преимуществами, так и недостатками по сравнению с процессами.

Преимущество: потребляется меньше памяти

Процессы полностью независимы. У каждого процесса есть свое адресное пространство, набор потоков и копии переменных, которые никак не зависят от тех же переменных в других процессах. Потоки требуют значительно меньших затрат памяти, чем стандартная функция `fork()`, потому что родительский поток не копируется — потоки используют один и тот же процесс. Из-за этого потоки иногда называются *облегченными* процессами.

В результате в одной и той же системе можно создать больше потоков, чем процессов. Потоки создаются и завершаются быстрее, чем процессы, потому что ОС требуется меньше времени, чтобы выделять ресурсы для потоков и управлять этими ресурсами. Следовательно, потоки можно создавать везде, где это имеет смысл в приложении, не беспокоясь о затратах процессорного времени и памяти.

Преимущество: меньше затрат на коммуникацию

Каждый процесс работает со своей собственной памятью. Процессы могут обмениваться данными только через механизм коммуникации, описанный в главе 5.

Потоки используют общее адресное пространство, а следовательно, могут взаимодействовать друг с другом, читая и записывая данные в общем адресном пространстве своего родительского процесса без каких-либо проблем или лишних затрат: все изменения, которые внес один поток, немедленно становятся доступными для остальных. В результате в популярных системах SMP потоки оказываются намного удобнее, чем процессы.

¹ IEEE POSIX 1003.1c (1995), <https://standards.ieee.org/ieee/1003.1c/1393>.

Недостаток: нужна синхронизация

ОС обеспечивает полную независимость процессов друг от друга, так что если в одном процессе происходит сбой, на других это никак не отразится. С потоками ситуация иная: все потоки в процессе используют одни и те же общие ресурсы, и если в одном из них случится сбой или повреждение, это, скорее всего, отразится на других потоках. Чтобы этого не происходило, разработчику нужно синхронизировать доступ к общим ресурсам и в большей степени контролировать поведение потоков (эта тема рассматривается в главе 8).

Реализация потоков

Конкурентность во многих языках реализуется на основе потоков. Это не значит, что потоки явно используются в языках программирования. Вместо этого среда выполнения отображает другие конкурентные конструкции языка программирования на физические потоки во время выполнения. В языках программирования обычно используются абстракции более высокого уровня для создания процессов, потому что их проще поддерживать и отслеживать в исходном коде программы.

ПРИМЕЧАНИЕ Старайтесь не использовать низкоуровневые потоки, если это возможно. Поищите библиотеки, которые абстрагируют работу с низкоуровневыми потоками. В C/C++ обобщенная реализация POSIX представлена в виде библиотеки функций. Такие современные языки, как Python, Java и C# (.NET), предоставляют на базе платформенных потоков набор абстракций, которые по духу и стилю гармонируют с естественными конструкциями языка. Аналогичным образом свойства множественных потоков могут идиоматически скрываться в таких конструкциях, как горутины Go, параллельные коллекции Scala, GHC в Haskell, процессы Erlang, OpenMP и т. д. Эти реализации переносятся в любую ОС, которая представляет реализацию исполнительской среды, необходимую для этих языков.

В следующем примере на Python создаются пять дочерних потоков:

```
# Chapter 4/multithreading.py
import os
import time
import threading
from threading import Thread

def cpu_waster(i: int) -> None:
    name = threading.current_thread().getName()
    print(f"{name} выполняет работу {i}")
    time.sleep(3)

def display_threads() -> None:
    print("-" * 10)
    print(f"PID текущего процесса: {os.getpid()}")
    print(f"Количество потоков: {threading.active_count()}")
    print("Активные потоки:")
    for thread in threading.enumerate():
        print(thread)
```

```
def main(num_threads: int) -> None:
    display_threads()

    print(f"Запускаем {num_threads} нагрузок на CPU...")
    for i in range(num_threads):
        thread = Thread(target=cpu_waster, args=(i,))
        thread.start()

        display_threads()

if __name__ == "__main__":
    num_threads = 5
    main(num_threads)
```

Выводит сведения о текущем процессе: PID, количество потоков и список активных потоков

Создает и запускает новый поток

Результат выглядит так:

```
-----
PID текущего процесса: 35930
Количество потоков: 1
Активные потоки:
<_MainThread(MainThread, started 8607733248)>
Запускаем 5 нагрузок на CPU...
Thread-1 выполняет работу 0
Thread-2 выполняет работу 1
Thread-3 выполняет работу 2
Thread-4 выполняет работу 3
Thread-5 выполняет работу 4
-----
PID текущего процесса : 35930
Количество потоков: 6
Активные потоки:
<_MainThread(MainThread, started 8607733248)>
<Thread(Thread-1, started 12940410880)>
<Thread(Thread-2, started 12945666048)>
<Thread(Thread-3, started 12950921216)>
<Thread(Thread-4, started 12956176384)>
<Thread(Thread-5, started 12961431552)>
```

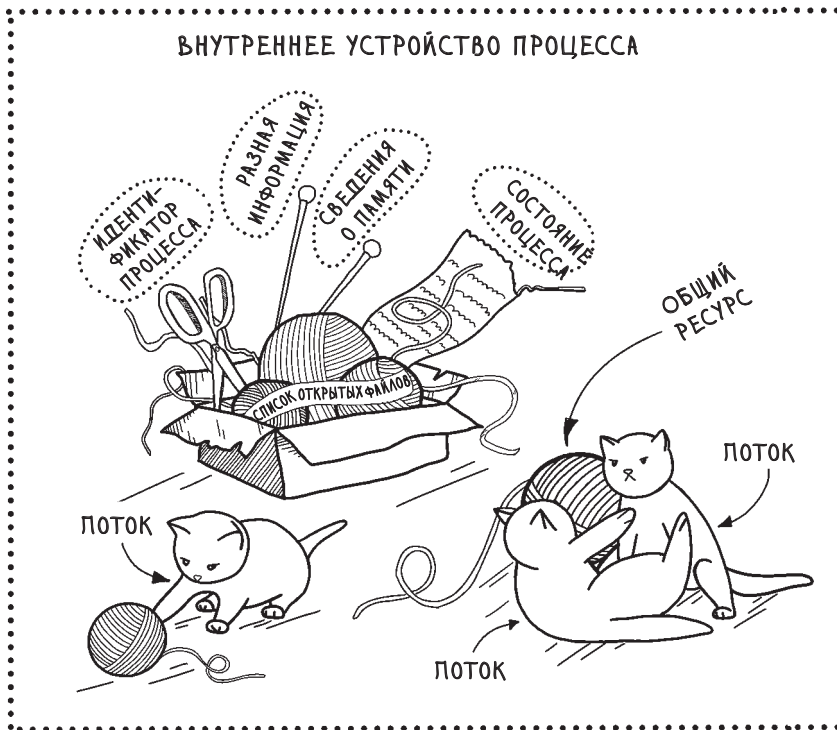
Когда программа запускается, она создает процесс, в котором формируется главный поток выполнения. Обратите внимание, что каждый поток, включая главный, может в любой момент создавать дочерние потоки (отсюда счетчик Количество потоков: 6 в выводе). В этом примере мы создаем пять новых потоков и запускаем их в конкурентном режиме.

Процессы и потоки — это основные структурные элементы конкурентности, и в книге мы уделяем им много внимания. Однако независимо от того, работаете ли вы с потоками или с процессами, ко всем ним можно относиться как к потокам, потому что в каждом процессе есть хотя бы один поток. Далее в книге мы будем называть процессы и потоки обобщенным термином «задача» (task), если конкретная реализация не важна.

Дочитав до этого места, вы, вероятно, уже поняли, что реализовывать конкурентность далеко не просто. У вас могут возникнуть сомнения, подходит ли она вам. Но мне хотелось бы немного вас подбодрить.

Представьте себе котят, а рядом корзину с клубками пряжи. Котят любознательны, склонны к экспериментам и любят развлекаться. Они не смотрят на корзину в сомнении и тревоге; они видят перед собой игровую площадку, которую можно исследовать, осваивать и использовать в свое удовольствие.

ПРОЦЕСС



Хороший программист во многом похож на котенка. В его распоряжении находятся процессы, общие ресурсы, потоки, открытые файлы и данные, и с помощью всего этого он создает программы, которые решают реальные проблемы, автоматизируют задачи или развлекают миллионы пользователей.

Так что не бойтесь и двигайтесь дальше. Хватайте клубки и смело их разматывайте¹. Новые возможности, которые перед вами открываются, могут изменить мир.

¹ Игра слов: thread переводится как «поток» и как «нить» (для шитья или вязания), отсюда метафора с клубками пряжи. — Примеч. пер.

Итоги

- Главная задача операционной системы — отобразить выполнение программы на реальное оборудование.
- *Процесс* — это экземпляр программы, который выполняется в компьютерной системе. Каждый процесс содержит один или несколько потоков, и никакой поток не может существовать вне процесса.
- *Поток* — это единица вычислений, то есть независимый набор программных команд, которые предназначены для достижения определенного результата; ОС управляет каждым потоком независимо от остальных.
- В одном процессе может быть сразу несколько потоков, которые совместно используют ресурсы, тогда как процессы практически не зависят друг от друга.
- Конкурентные приложения легче создавать с помощью потоков, потому что переключаться между ними проще, чем между процессами. Кроме того, потоки используют общее адресное пространство, что ускоряет доступ к общим данным. Однако при этом также возникает риск повредить данные, так что при управлении доступом и синхронизации общих объектов нужна определенная осторожность.



В этой главе

- ✓ Как организовывать эффективную коммуникацию между задачами
- ✓ Как выбрать тип коммуникации для ваших приложений
- ✓ Популярный паттерн программирования для конкурентных приложений: пул потоков

Не всегда можно гарантировать, что задачи, которые одновременно выполняются на компьютере, полностью независимы. Часто для эффективного выполнения нужно, чтобы задачи взаимодействовали между собой. Например, если одна задача зависит от результата другой, приложение должно знать, в какой момент приостановить выполнение, чтобы дождаться завершения другой задачи.

Таким образом, коммуникация лежит в основе любой конкурентной системы. Если не обеспечить правильного взаимодействия между задачами, выигрыш производительности от конкурентности теряет смысл. В этой главе рассказывается о том, какие средства предоставляет ОС для взаимодействия и координации выполнения процессов или потоков. Для начала рассмотрим разные типы коммуникации, с которыми вы, скорее всего, столкнетесь в конкурентных системах.

Типы коммуникации

ОС предоставляет механизмы, которые позволяют процессам и потокам взаимодействовать друг с другом. Такие механизмы называются *межпроцессной коммуникацией*, или IPC (interprocess communication). Если вы пришли к выводу, что она будет полезна в вашем приложении, необходимо решить, какой из доступных методов IPC использовать в вашей системе.

ПРИМЕЧАНИЕ Коммуникация называется межпроцессной, но это не значит, что потребность в коммуникации возникает только у процессов. Независимо от того, работаете ли вы с потоками или с процессами, их можно рассматривать как потоки, потому что в каждом процессе есть хотя бы один поток, так что фактически коммуникация происходит только между потоками. Не обращайтесь внимания на терминологическую путаницу; мы используем термин «задача» как обобщенную абстракцию для единицы выполнения.

Самые популярные типы IPC — *общая память* и *передача сообщений*.

IPC с общей памятью

Простейший способ обмениваться данными между задачами — использовать общую память. Одна или несколько задач взаимодействуют через общую память, которая присутствует во всех их адресных пространствах так, будто операции чтения и записи выполняются с локальными переменными в пространстве каждой отдельной задачи. При этом изменения, которые вносит один процесс или поток, моментально отражаются во всех остальных без взаимодействия с ОС.

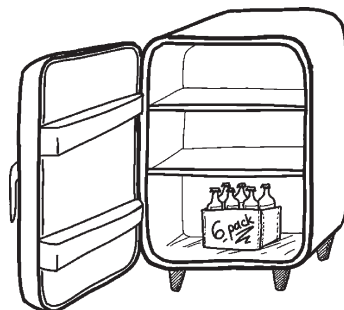
Представьте, что вы живете в одном доме с несколькими друзьями. На общей кухне стоит холодильник, которым пользуются все жильцы. Вы можете купить ящик пива и сообщить друзьям, что он стоит на нижней полке. Холодильник выполняет функции общей памяти, которую используют все друзья (задачи), чтобы хранить пиво (общие данные).

Межпроцессная коммуникация с общей памятью применяется тогда, когда два процессора (или процессорных ядра) на одном компьютере обращаются к одной физической области памяти или когда потоки в одной программе совместно используют одни и те же объекты. В коде это выглядит так:

```
# Chapter 5/shared_ipc.py
import time
from threading import Thread, current_thread

SIZE = 5
shared_memory = [-1] * SIZE
```

```
class Producer(Thread):
    def run(self) -> None:
```



Создает общую память
размером SIZE

```

self.name = "Производитель"
global shared_memory
for i in range(SIZE):
    print(f"{current_thread().name}: Записывает {int(i)}")
    shared_memory[i - 1] = i
class Consumer(Thread):
    def run(self) -> None:
        self.name = "Потребитель"
        global shared_memory
        for i in range(SIZE):
            while True:
                line = shared_memory[i]
                if line == -1:
                    print(f"{current_thread().name}: Данные недоступны\n"
                        f"Ожидание 1 с перед повторной попыткой")
                    time.sleep(1)
                    continue
                print(f"{current_thread().name}: Читает: {int(line)}")
                break
def main() -> None:
    threads = [
        Consumer(),
        Producer(),
    ]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
if __name__ == "__main__":
    main()

```

Запускает все дочерние потоки

Поток-потребитель непрерывно читает данные из общей памяти и ожидает, если данных еще нет

Поток-производитель записывает данные в общую память

Ожидает, пока все дочерние потоки завершатся

Здесь создаются два потока: производитель (Producer) и потребитель (Consumer). Производитель вырабатывает данные и сохраняет их в общей памяти, а потребитель, соответственно, потребляет данные из этой памяти. Таким образом, потоки взаимодействуют через общий массив. Результат выполнения программы выглядит так:

```

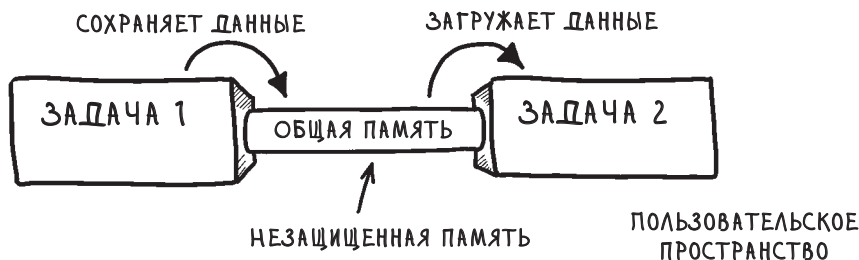
Потребитель: Данные недоступны
Ожидание 1 с перед повторной попыткой
Производитель: Записывает 0
Производитель: Записывает 1
Производитель: Записывает 2
Производитель: Записывает 3
Производитель: Записывает 4
Потребитель: Читает: 1
Потребитель: Читает: 2
Потребитель: Читает: 3
Потребитель: Читает: 4
Потребитель: Читает: 0

```

У совместного использования памяти есть как преимущества, так и недостатки.

Преимущества

Важное достоинство совместного использования памяти состоит в том, что это самый быстрый и наименее ресурсоемкий механизм коммуникации. ОС помогает выделять общую память, но не участвует в обмене данными между задачами. Таким образом, в этом случае ОС полностью исключена из процесса коммуникации и не тратит на него ресурсы, отчего работа ускоряется, а данные меньше копируются.



Недостатки

К сожалению, такая коммуникация не всегда оказывается самой безопасной, потому что ОС не предоставляет интерфейсы и не обеспечивает защиту общей памяти. Представьте, что каждый из двух друзей решил выпить последнюю бутылку пива. Это приводит к конфликту (а иногда даже к войне). Аналогичным образом задачам, которые выполняют одну и ту же программу, может понадобиться прочитать или обновить одни и те же структуры данных. Из-за этого общая память иногда считается более рискованным решением, и разработчикам приходится переделывать код, добавляя защиту объектов в общей памяти (эта тема подробнее рассматривается в главе 8).

У общей памяти есть еще один недостаток: она не масштабируется за пределы одной машины. Общую память можно использовать только для локальных задач. Это создает проблемы в больших распределенных системах, где данные для обработки не помещаются на одном компьютере. С другой стороны, общая память — идеальный вариант для систем с симметричной многопроцессностью (SMP).

В системах SMP все процессы и потоки на разных процессорах совместно используют уникальное логическое адресное пространство, которое отображается на физическую память. И именно поэтому метод общей памяти популярен в системах SMP, особенно при использовании потоков: они с самого начала внедрились с учетом идеи общей памяти. Однако если в таких системах повышается количество процессоров, которые подключены к общей системной шине, она становится узким местом (см. главу 3).

Межпроцессная коммуникация с передачей сообщений

Вероятно, в наши дни самый распространенный механизм межпроцессной коммуникации — это *передача сообщений*; она часто поддерживается в ОС. В IPC с передачей сообщений каждая задача идентифицируется уникальным именем, и именованные задачи отправляют и принимают сообщения друг от друга. ОС создает канал связи и предоставляет необходимые системные вызовы, с помощью которых задачи могут обмениваться сообщениями по каналу.

Этот механизм хорош тем, что ОС управляет каналом связи, обеспечивая удобные интерфейсы, с помощью которых можно отправлять и принимать данные без конфликтов. С другой стороны, передача сообщений требует значительных коммуникационных затрат. Чтобы передать любую информацию между задачами, ее нужно с помощью системных вызовов (как упоминалось в главе 3) скопировать из пользовательского пространства задачи в канал ОС, а затем скопировать обратно в адресное пространство задачи-получателя.

У передачи сообщений есть еще одно преимущество: она легко масштабируется за пределы одной машины в распределенные системы. Впрочем, это еще не все.

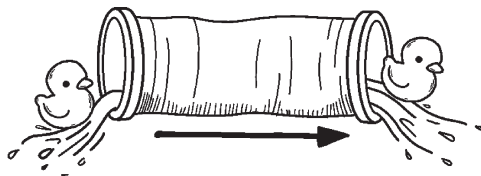
ПРИМЕЧАНИЕ Во многих языках программирования межпроцессная коммуникация ограничена только передачей сообщений. Совместное использование памяти посредством коммуникации — один из принципов философии языка Go. Эта идея сформулирована в лозунге из документации Go: «Не коммуницируйте через общую память, а используйте общую память посредством коммуникации». Другой пример — язык Erlang, в котором процессы не используют данные совместно, а взаимодействуют друг с другом исключительно посредством передачи сообщений.

Существует много разных технологий коммуникации через передачу сообщений. Некоторые технические решения, распространенные в современных ОС, — каналы, сокеты и очереди сообщений — будут описаны в следующих разделах.

Каналы

Вероятно, это простейшая форма межпроцессной коммуникации. *Канал* (pipe) — простой синхронизированный способ передавать информацию из одной задачи в другую. Канал определяет одностороннее движение данных между задачами: данные записываются на одном конце и читаются на другом. Канал обеспечивает передачу данных в одном направлении; если нужна двусторонняя передача, придется создать два канала.

Канал в IPC можно сравнить с водопроводной трубой. Если запустить в трубу резиновую уточку, она проплывет по течению до конца. Сторона записи — там, где вы запускаете уточку в канал, а сторона чтения — там, откуда она выплывает.



В коде одна секция на стороне записи вызывает методы, которые отправляют данные, тогда как другая секция читает входящие данные. Канал — это временный объект, который может использоваться только двумя задачами и будет закрыт, когда отключится отправитель или получатель.

ПРИМЕЧАНИЕ Каналы — популярный тип данных в языке Go; они обеспечивают синхронизацию и коммуникацию между примитивами конкурентности Go (горутинами).

Каналы делятся на две разновидности: *безымянные* и *именованные*. Безымянные каналы бывают только между связанными задачами (то есть между родительским и дочерним процессом, между дочерними процессами одного и того же родителя или между потоками в одном процессе), потому что связанные задачи совместно используют файловые дескрипторы. Безымянные каналы исчезают после того, как задачи перестают их использовать.

Поскольку канал фактически представляет собой файловый дескриптор (в системах UNIX), операции с каналами похожи на операции с файлами, но не связаны с файловой системой. Когда сторона записи хочет записать данные в канал, она использует системный вызов `write()` для канала, а чтобы прочитать данные из канала, используется системный вызов `read()`: он работает с каналами так же, как с файлами, но блокируется до тех пор, пока не будут прочитаны все данные. Каналы могут по-разному реализовываться в разных системах.

Если создать канал в главном потоке, а затем передавать файловые дескрипторы дочерним потокам, можно передавать данные от одного потока другому через канал. Именно так работают стандартные каналы. Взгляните на пример кода:

```
# Chapter 5/pipe.py
from threading import Thread, current_thread
from multiprocessing import Pipe
from multiprocessing.connection import Connection

class Writer(Thread):
    def __init__(self, conn: Connection):
        super().__init__()
        self.conn = conn
        self.name = "Сторона записи"

    def run(self) -> None:
        print(f"{current_thread().name}: Передаю резиновую утку...")
        self.conn.send("Резиновая утка")

class Reader(Thread):
    def __init__(self, conn: Connection):
        super().__init__()
        self.conn = conn
        self.name = "Сторона чтения"

    def run(self) -> None:
        print(f"{current_thread().name}: Читаю...")
        msg = self.conn.recv()
        print(f"{current_thread().name}: Получил: {msg}")
```

Записывает сообщение в канал

Читает сообщение из канала

```
def main() -> None:
    reader_conn, writer_conn = Pipe()
    reader = Reader(reader_conn)
    writer = Writer(writer_conn)

    threads = [
        writer,
        reader
    ]

    for thread in threads:
        thread.start()

    for thread in threads:
        thread.join()

if __name__ == "__main__":
    main()
```

Создает безымянный канал для коммуникации между двумя потоками с двумя канальными соединениями для чтения и записи

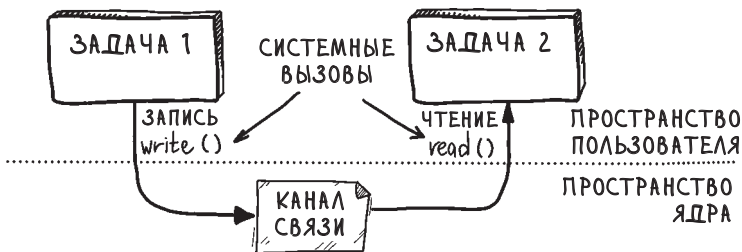
Здесь создается безымянный канал с двумя потоками. Записывающий поток записывает в канал сообщение для читающего потока. Вот что выводит программа:

Сторона записи: Передаю резиновую уточку...
 Сторона чтения: Читаю...
 Сторона чтения: Получил: Резиновая уточка

ПРИМЕЧАНИЕ `pipe()` и `fork()` обеспечивают широко известную функциональность оператора `|`, который применяется в популярной оболочке UNIX и командном языке `bash` (например, в конструкции `ls | more`).

Именованные каналы позволяют передавать данные между задачами по принципу *FIFO* (First In, First Out, то есть «первым вошел — первым вышел»); это значит, что запросы обрабатываются в порядке поступления. Поэтому именованные каналы часто называются каналами *FIFO*.

В отличие от безымянных каналов, каналы *FIFO* — это не временные объекты; они входят в файловую систему, и их могут свободно использовать посторонние задачи, если у них есть необходимые права доступа. Именованные каналы позволяют задачам взаимодействовать даже тогда, когда они не знают, какие задачи находятся на другом конце канала, и даже по сети. В остальном *FIFO* работают точно так же, как безымянные каналы, и используют те же системные вызовы.



Из-за односторонней природы каналов, вероятно, с их помощью лучше всего передавать данные от программ-производителей программам-потребителям. Для других применений возможности каналов чересчур ограничены, и обычно лучше подходят альтернативные методы межпроцессной коммуникации.

Очереди сообщений

Другая популярная реализация IPC с передачей сообщений — *очередь сообщений*. Как и именованные каналы, очереди упорядочивают данные по принципу FIFO (поэтому они и называются очередями), однако при этом они также позволяют записывать или читать сообщения многим задачам.

Очереди сообщений предоставляют мощные средства для того, чтобы ослаблять сцепление между задачами в системе, позволяя производителям и потребителям взаимодействовать друг с другом через очередь, а не напрямую. Это дает разработчикам значительную свободу в управлении выполнением. Например, исполнители могут возвращать сообщения обратно в очередь, если они почему-либо их не обработали. Вот как очереди сообщений выглядят в коде:

```
# Chapter 5/message_queue.py
import time
from queue import Queue
from threading import Thread, current_thread

class Worker(Thread):
    def __init__(self, queue: Queue, id: int):
        super().__init__(name=str(id))
        self.queue = queue

    def run(self) -> None:
        while not self.queue.empty():
            item = self.queue.get()
            print(f"Поток {current_thread().name} "
                  f"обрабатывает элемент {item} из очереди")
            time.sleep(2)

def main(thread_num: int) -> None:
    q = Queue()
    for i in range(10):
        q.put(i)

    threads = []
    for i in range(thread_num):
        thread = Worker(q, i + 1)
        thread.start()
        threads.append(thread)

    for thread in threads:
        thread.join()

if __name__ == "__main__":
    thread_num = 4
    main(thread_num)
```

Получает следующий элемент из очереди для обработки. Метод блокируется до тех пор, пока в очереди не будет доступен элемент

Создает очередь и помещает в нее значения, которые будут обрабатываться потоками

В этом примере мы создаем очередь и помещаем в нее 10 сообщений, которые должны обрабатываться четырьмя дочерними потоками. Потоки обрабатывают все сообщения в очереди, пока та не опустеет. Обратите внимание, что очередь — это не только единая точка взаимодействия потоков; она также хранит сообщения, пока они не обрабатываются, в результате чего получается система со слабым сцеплением. Результат выполнения программы выглядит так:

```
Поток 1 обрабатывает элемент 0 из очереди
Поток 2 обрабатывает элемент 1 из очереди
Поток 3 обрабатывает элемент 2 из очереди
Поток 4 обрабатывает элемент 3 из очереди
Поток 1 обрабатывает элемент 4 из очереди
Поток 2 обрабатывает элемент 5 из очереди
Поток 3 обрабатывает элемент 6 из очереди
Поток 4 обрабатывает элемент 7 из очереди
Поток 1 обрабатывает элемент 8 из очереди
Поток 3 обрабатывает элемент 9 из очереди
```

Как вы убедились, очереди сообщений позволяют реализовывать системы со слабым сцеплением. Они используются повсюду: например, в ОС, чтобы планировать процессы, или в маршрутизаторах, чтобы хранить пакеты перед обработкой. Даже в облачных приложениях, которые состоят из микросервисов, обмен данными происходит с помощью очереди сообщений. Кроме того, очереди широко применяются при асинхронной обработке. Мы рассмотрим их практическое применение в конце главы, а пока перейдем к UDS.

Сокеты UDS

Сокеты позволяют обмениваться данными в широком спектре областей, и в этой главе мы поговорим о *доменных сокетах UNIX*, или *UDS* (Unix Domain Sockets), с помощью которых могут взаимодействовать потоки в одной системе. Сети и сетевые сокеты (другая популярная разновидность доменных сокетов) рассматриваются в главе 10.

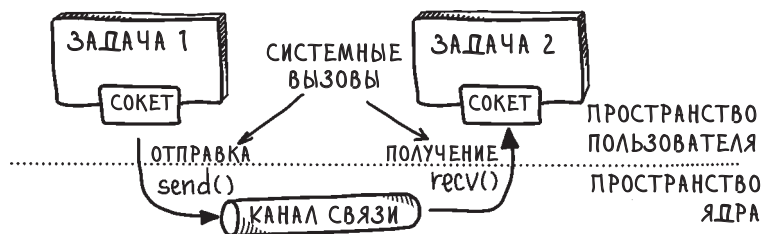
С помощью сокетов, которые реализуют IPC с передачей сообщений, возможен двусторонний обмен данными в режиме FIFO. В такой межпроцессной коммуникации один поток может записывать информацию в сокет, а второй — читать ее из сокета. *Сокет* (socket) — это объект, который представляет конечную точку такого соединения. У каждого из потоков на обоих концах соединения есть свой сокет, который соединен с другим сокетом. Таким образом, чтобы отправить информацию от одного потока к другому, мы записываем ее в выходной поток данных одного сокета и читаем из входного потока данных другого сокета.

Как работает передача сообщений между двумя объектами? Представьте, что вам нужно отправить маме новогоднюю открытку. Вы пишете на ней милые праздничные пожелания, а также имя и адрес получателя, после чего опускаете открытку в почтовый ящик. На этом ваша часть работы завершена, а почта сделает все остальное: она переправит открытку в почтовое отделение, которое обслуживает мамин адрес, а почтальон доставит ее на дом и увидит, как мама обрадуется.



В случае новогодней открытки вы пишете на ней адреса отправителя и получателя. В случае сокетов нужно сначала установить соединение, после чего начинается обмен сообщениями.

Поток Sender (Отправитель) помещает в сообщение информацию, которую хочет отправить, и явно отправляет это сообщение потоку Receiver (Получатель) по выделенному каналу связи, после чего Receiver его читает. Для этого понадобятся как минимум два примитива: `send(сообщение, получатель)` и `receive()`. Потоки, которые участвуют в этой коммуникации, могут выполняться как на одной машине, так и на разных машинах, соединенных сетью.



Программный код выглядит так:

```
# Chapter 5/sockets.py
import socket
import os.path
import time
from threading import Thread, current_thread
```

```
SOCK_FILE = "./mailbox"
BUFFER_SIZE = 1024
```

```
class Sender(Thread):
    def run(self) -> None:
        self.name = "Отправитель"
        client = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
        client.connect(SOCK_FILE)
```

В UNIX все является файлом. Этот файл сокета будет обеспечивать коммуникацию между потоками

Создает новый сокет для потока-отправителя. Константы `AF_UNIX` (доменный сокет UNIX) и `SOCK_STREAM` представляют семейство и тип сокета соответственно

Размер буфера для получения данных из соединения через сокет

Соединяет сокет потока-отправителя с «каналом связи» — файлом сокета UNIX

```

messages = ["Hello", " ", "world!"]
for msg in messages:
    print(f"{current_thread().name}: Отправил: '{msg}'")
    client.sendall(str.encode(msg))
client.close()

```

*Отправляет последовательность сообщений
через сокет потока-отправителя*

```

class Receiver(Thread):
    def run(self) -> None:
        self.name = "Получатель"
        server = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
        server.bind(SOCK_FILE)
        server.listen()

        print(f"{current_thread().name}: Прослушиваю входящие сообщения...")
        conn, addr = server.accept()

        while True:
            data = conn.recv(BUFFER_SIZE)
            if not data:
                break
            message = data.decode()
            print(f"{current_thread().name}: Получил: '{message}'")

        server.close()

def main() -> None:
    if os.path.exists(SOCK_FILE):
        os.remove(SOCK_FILE)

    receiver = Receiver()
    receiver.start()
    time.sleep(1)
    sender = Sender()
    sender.start()

    for thread in [receiver, sender]:
        thread.join()

    os.remove(SOCK_FILE)

if __name__ == "__main__":
    main()

```

*Создает новый сокет для потока-получателя с та-
кой же конфигурацией, как у сокета отправителя*

*Связывает и запускает сокет потока-получателя,
который будет прослушивать входящие соединения*

*Принимает соединение
через сокет
потока-получателя
и возвращает новое
соединение и адрес
отправителя*

*Принимает данные из
присоединенного сокета
отправителя до тех
пор, пока соединение не
будет закрыто*

Здесь создаются два потока — `Sender` и `Receiver`, и у каждого есть собственный сокет. Единственное различие между ними заключается в том, что `Receiver` находится в режиме прослушивания и ожидает, пока отправители отправят свои сообщения. Вывод выглядит так:

```

Получатель: Прослушиваю входящие сообщения...
Отправитель: Отправил: "Hello"
Отправитель: Отправил: " "
Получатель: Получил: "Hello"
Получатель: Получил: " "
Отправитель: Отправил: "world!"
Получатель: Получил: "world!"

```

Вероятно, это самый простой и известный способ реализовать межпроцессную коммуникацию, однако он обходится дорого, потому что требует сериализации, что в свою очередь заставляет разработчиков думать о том, какие данные нужно передавать. Зато сокеты обычно отличаются большей гибкостью, и при необходимости их можно расширять до сетевых сокетов почти без изменений, что позволит легко масштабировать вашу программу на несколько машин. Мы подробнее поговорим об этом в третьей части книги.

ПРИМЕЧАНИЕ Это далеко не полный список разновидностей IPC: здесь перечислены лишь самые популярные из них и те, которые понадобятся нам позднее в книге. Например, среди старейших методов IPC можно отметить *сигналы*; также есть уникальные методы вроде *почтовых слотов*¹, которые доступны только в Windows.

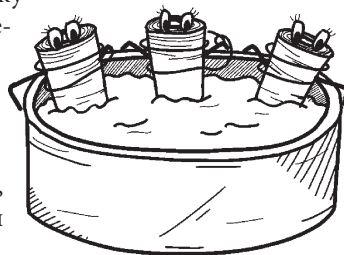
Познакомившись с методами межпроцессной коммуникации, мы закончили осваивать фундаментальные принципы конкурентности. Теперь мы готовы изучать свой первый паттерн конкурентности — *пул потоков*.

Пул потоков

Разрабатывать программное обеспечение с использованием потоков может быть весьма непростым делом. Мало того что потоки — это низкоуровневые конструкции конкурентности, которые требуют ручного управления; кроме этого, механизмы синхронизации, которые обычно применяются с потоками, иногда усложняют структуру кода, не обязательно улучшая быстродействие. Более того, поскольку оптимальное количество потоков для приложения может динамически изменяться в зависимости от текущей нагрузки и конфигурации оборудования, создать надежное решение по управлению потоками в высшей степени сложно.

Несмотря на все сложности, в большинстве конкурентных приложений активно используются множественные потоки. Однако это не значит, что потоки явно выражаются в конструкциях языков программирования. Вместо этого среда выполнения может отображать другие языковые конструкции на реальные потоки во время выполнения. В частности, во многих фреймворках и языках программирования широко применяется паттерн *пула потоков*.

Паттерн заключается в том, что при запуске программы создается небольшой набор долгоживущих рабочих потоков, которые затем помещаются в пул (контейнер). Когда нужно выполнить задачу, пул берет один из заранее созданных потоков и выполняет его; разработчикам не приходится создавать потоки самостоятельно. Задачи отправляются в пул потоков так же, как они добавляются в список задач для потоков-исполнителей.



¹ Документация Microsoft, «Mailslots», <https://learn.microsoft.com/en-us/windows/win32/ipc/mailslots>.

Повторное использование потоков из пула избавляет от затрат на создание новых потоков и защищает от непредвиденных сбоев — например, возбуждение исключения не повлияет на исполнитель. Повторное использование потоков становится ощутимым преимуществом, когда для того, чтобы выполнить задачу, требуется меньше времени, чем для того, чтобы создать новый поток.

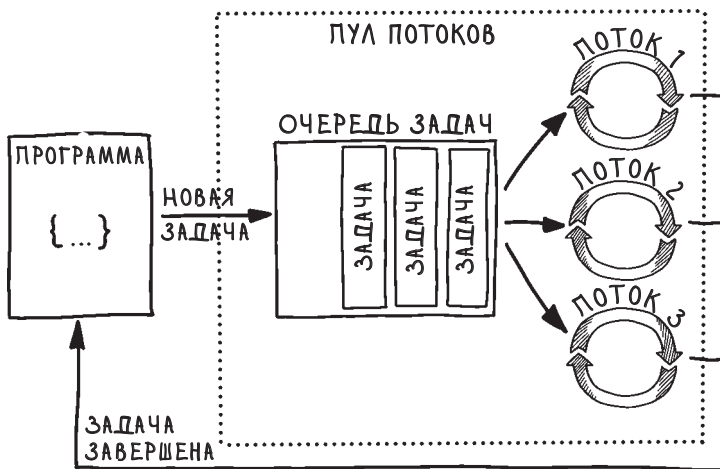
ПРИМЕЧАНИЕ Пул потоков создает потоки-исполнители, управляет ими и планирует их, а все эти задачи могут стать сложными и затратными, если не обращаться с ними должным образом. Пулы потоков бывают разных типов, с разными механизмами планирования и выполнения. В одних пулах фиксированное количество потоков, а в других оно может динамически изменяться в зависимости от рабочей нагрузки.

Допустим, у нас есть большая группа задач, которые нужно обработать с помощью множественных потоков (например, чтобы подобрать пароль, как описывалось в главе 2). Если разделить все возможные пароли на блоки и распределить их между разными потоками, можно добиться конкурентности в обработке. В этом сценарии понадобится главный поток, который будет генерировать задачи для потоков-исполнителей, выполняющихся в фоновом режиме.

Чтобы упростить взаимодействие между главным потоком и исполнителями, понадобится механизм хранения данных, через который они смогут связаться друг с другом. Этот механизм должен назначать задачам приоритеты в порядке их поступления. Более того, у любого свободного исполнителя должна быть возможность взять и обработать следующую доступную задачу из этого хранилища.

Как организовать такую коммуникацию между потоками?

Потоки в пуле взаимодействуют через очередь сообщений. На логическом уровне очередь представляет собой список задач. Потоки в пуле извлекают задачи из очереди и обрабатывают их в конкурентном режиме.



Реализации на разных языках программирования могут выглядеть по-разному. Вот как можно реализовать пул потоков на Python.

```
# Chapter 5/thread_pool.py
import time
import queue
import typing as T
from threading import Thread, current_thread
```

```
Callback = T.Callable[... , None]
Task = T.Tuple[Callback, T.Any, T.Any]
TaskQueue = queue.Queue
```

```
class Worker(Thread):
    def __init__(self, tasks: queue.Queue[Task]):
        super().__init__()
        self.tasks = tasks

    def run(self) -> None:
        while True:
            func, args, kargs = self.tasks.get()
            try:
                func(*args, **kargs)
            except Exception as e:
                print(e)
            self.tasks.task_done()
```

```
class ThreadPool:
    def __init__(self, num_threads: int):
        self.tasks: TaskQueue = queue.Queue(num_threads)
        self.num_threads = num_threads
```

```
    for _ in range(self.num_threads):
        worker = Worker(self.tasks)
        worker.setDaemon(True)
        worker.start()
```

```
    def submit(self, func: Callback, *args, **kargs) -> None:
        self.tasks.put((func, args, kargs))
```

```
    def wait_completion(self) -> None:
        self.tasks.join()
```

```
def cpu_waster(i: int) -> None:
    name = current_thread().getName()
    print(f"{name} выполняет работу {i}")
    time.sleep(3)
```

```
def main() -> None:
    pool = ThreadPool(num_threads=5)
    for i in range(20):
        pool.submit(cpu_waster, i)
```

```
    print("Все запросы на обработку отправлены")
    pool.wait_completion()
    print("Вся обработка завершена ")
```

```
if __name__ == "__main__":
    main()
```

Поток-исполнитель получает задачу из очереди, запускает функцию, которая связана с задачей, и помечает задачу как выполненную, когда она завершается. Это повторяется в бесконечном цикле

Сохраняет в очереди задачу, которая поступила в пул потоков

Создает несколько исполнителей и переводит их в режим демона, чтобы они автоматически завершались, когда завершается главный поток. После этого запускает потоки, чтобы они могли начать выполнять задачи из очереди

Блокирует вызывающий поток, пока не завершатся все задачи в очереди

Создает пул с 5 потоками-исполнителями

Добавляет 20 задач в пул потоков

Когда мы создаем этот пул потоков, он автоматически создает несколько потоков и очередь сообщений, в которой сохраняются входящие задачи. Затем в главном потоке мы добавляем несколько задач в пул и ждем, пока исполнители их обработают и завершат.

Когда поступает новая задача, свободный поток активизируется, выполняет задачу и возвращается в состояние готовности. Это позволяет предотвратить относительно затратные операции создания и завершения потока для каждой незавершенной задачи, а управление потоками перестает быть заботой разработчика и передается библиотеке или ОС, которые способны лучше оптимизировать выполнение программы.

ПРИМЕЧАНИЕ За реализацией паттерна «Пул потоков» из библиотеки Python обращайтесь к файлу `Chapter 5/library_thread_pool.py` из сопроводительных файлов книги.

Пулы потоков — хорошее стандартное решение для большинства конкурентных приложений, но бывают сценарии, в которых эффективнее самостоятельно создавать потоки и управлять ими, чем использовать пул. Это имеет смысл, если:

- Вам нужно управлять приоритетами потоков.
- Ваши задачи приводят к тому, что потоки надолго блокируются. Во многих реализациях пулов максимальное количество потоков ограничено, и если заблокированных потоков будет слишком много, это может помешать запускать задачи в этом пуле.
- Вам нужно, чтобы с потоками связывались статические идентификаторы.
- Вы хотите выделить целый поток под одну конкретную задачу.

Как я уже обещал, мы рассмотрим реализацию различных видов коммуникации и обобщим все, что вы узнали о выполнении конкурентных приложений.

Снова о подборе паролей

Вы узнали много нового, так что вернемся к незавершенной реализации программы для подбора паролей из главы 2 и воспользуемся пулами и процессами (в Python существует ограничение на использование потоков¹, но во многих других языках такой проблемы нет):

```
# Chapter 5/password_cracking_parallel.py
def crack_chunk(crypto_hash: str, length: int, chunk_start: int,
               chunk_end: int) -> T.Union[str, None]:
    print(f"Обрабатывается блок от {chunk_start} до {chunk_end}")
    (reformat)
    combinations = get_combinations(
        length=length,
        min_number=chunk_start,
        max_number=chunk_end)
    for combination in combinations:
```

¹ Документация Python, «Thread State and the Global Interpreter Lock», <http://mng.bz/wvDB>.

```

    if check_password(crypto_hash, combination):
        return combination
    return

```

Пароль найден
Пароль не найден в этом блоке

```

def crack_password_parallel(crypto_hash: str, length: int) -> None:
    num_cores = os.cpu_count()
    print("Комбинации цифр обрабатываются в конкурентном режиме")
    start_time = time.perf_counter()

    with Pool() as pool:
        arguments = ((crypto_hash, length, chunk_start, chunk_end) for
                     chunk_start, chunk_end in
                     get_chunks(num_cores, length))
        results = pool.starmap(crack_chunk, arguments)
        print("Ожидаем завершения обработки блоков")
        pool.close()
        pool.join()

    result = [res for res in results if res]
    print(f"ПАРОЛЬ НАЙДЕН: {result[0]}")
    process_time = time.perf_counter() - start_time
    print(f"ЗАТРАЧЕННОЕ ВРЕМЯ: {process_time}")

if __name__ == "__main__":
    crypto_hash = \
        "e24df920078c3dd4e7e8d2442f00e5c9ab2a231bb3918d65cc50906e49ecaef4"
    length = 8
    crack_password_parallel(crypto_hash, length)

```

Получает количество доступных процессорных ядер в системе

Обрабатывает каждый блок в отдельном процессе в конкурентном режиме

Закрывает пул, чтобы обозначить, что ему больше не будут передаваться задачи

Ожидает, пока все переданные задачи завершатся, прежде чем выполнять программу дальше

Здесь главный поток создает столько же потоков-исполнителей, сколько процессорных ядер доступно в системе. Каждый исполнитель делает то же, что в исходной версии из главы 2, и все блоки паролей обрабатываются в конкурентном режиме. Результат выполнения выглядит примерно так:

```

Комбинации цифр обрабатываются в конкурентном режиме
Обрабатывается блок от 0 до 12499998
Обрабатывается блок от 12499999 до 24999998
Обрабатывается блок от 24999999 до 37499998
Обрабатывается блок от 37499999 до 49999998
Обрабатывается блок от 49999999 до 62499998
Обрабатывается блок от 62499999 до 74999998
Обрабатывается блок от 74999999 до 87499998
Обрабатывается блок от 87499999 до 99999999
Ожидаем завершения обработки блоков
ПАРОЛЬ НАЙДЕН: 87654321
ЗАТРАЧЕННОЕ ВРЕМЯ: 17.183910416

```

Нам удалось добиться более чем трехкратного ускорения по сравнению с исходной последовательной реализацией. Отличный результат!



Параллельное оборудование открывает много интересных возможностей, но иногда приходится использовать только одно ядро, и о параллелизме остается только мечтать. Это не повод отказываться от конкурентности, потому что здесь она превосходит параллелизм. Впрочем, эта тема подробнее рассматривается в следующей главе.

Итоги

- Чтобы синхронизироваться друг с другом и обмениваться данными, потоки и процессы используют механизм, который называется *межпроцессной коммуникацией (IPC)*.
- У каждой разновидности IPC есть свои достоинства и недостатки, и каждая служит оптимальным решением для определенной задачи:
 - Механизм общей памяти применяется, когда потокам и процессам нужно эффективно обмениваться большими объемами данных, но им трудно синхронизировать доступ к данным.
 - Каналы — эффективный способ реализовать синхронную коммуникацию между процессами типа «производитель — потребитель». Именованные каналы предоставляют простой интерфейс передачи данных между двумя процессами — на одном компьютере или по сети.
 - Очередь сообщений между процессами или потоками — это механизм асинхронного обмена данными. Очереди позволяют реализовать слабо сцепленные системы.
 - Сокеты образуют двусторонний канал связи, который может использовать сетевые подключения. Данные передаются через интерфейс сокетов вместо файлового интерфейса. В большинстве случаев сокеты оказываются оптимальным сочетанием производительности, масштабируемости и простоты использования.
- *Пул потоков* — это набор потоков-исполнителей, которые эффективно выполняют входящие задачи, назначенные главным потоком программы. Исполнители в пуле потоков рассчитаны на повторное использование после завершения текущих задач и защищены от непредвиденных сбоев (например, возбуждения исключений), которые обрабатываются без каких-либо последствий для самого исполнителя.

Щупальца конкурентности: многозадачность, декомпозиция и синхронизация



Вы видели, как в цирке жонглер удерживает в равновесии несколько тарелок, которые вращаются на тростях? Ему удается без видимых усилий вращать тарелки в идеальной гармонии. Такова сила многозадачности! В конкурентном программировании тоже необходимо жонглировать разными задачами и при этом добиваться, чтобы каждая задача получала достаточно внимания и ресурсов.

В главах 6–9 мы покажем, как на основе этого принципа создать игру в стиле Рас-Ман, а также применить его во многих других реальных сценариях. Мы исследуем тонкости разработки конкурентных программ, включая многозадачность, декомпозицию задач и влияние детализации на производительность.

Однако с большой силой приходит большая ответственность (я это где-то читал): конкурентность может приводить к ситуациям гонки, взаимным блокировкам и нехватке ресурсов. Но не бойтесь: чтобы решить эти проблемы, мы предоставим вам инструменты, включая такие средства синхронизации, как мьютексы, семафоры и атомарные операции. Как и у музыкантов в оркестре, залогом успешной конкурентности становится координация и синхронизация. И мы затронем даже такие классические примеры, как задача об обедающих философах, а также изучим ряд распространенных паттернов. К концу этой части книги вы овладеете необходимыми знаниями, чтобы проектировать и оптимизировать конкурентные программы, которые помогут справиться с любыми проблемами, попадающимися на вашем пути.

Ну что, готовы пожонглировать задачами?

6 | Многозадачность



В этой главе

- ✓ Как выявлять и анализировать потенциальные узкие места в приложениях
- ✓ Как запускать несколько задач в конкурентном режиме, если нет параллельного оборудования
- ✓ Методы вытесняющей многозадачности: их плюсы и минусы, а также как применять их в задачах с интенсивным вводом/выводом

Вас никогда не поражала способность вашего компьютера работать в многозадачном режиме? Просто невероятно, как он умудряется одновременно выполнять множество приложений, пока вы продолжаете набирать текст в текстовом редакторе. Мы воспринимаем это как нечто само собой разумеющееся, но на самом деле это свидетельствует о впечатляющих возможностях современных технологий.

Вас когда-нибудь интересовало, как компьютер управляется со всем этим? Как ему удастся жонглировать столькими задачами сразу? Что еще интереснее — какие типы задач он обрабатывает и как они классифицируются?

В этой главе мы глубже изучим понятие конкурентности и исследуем захватывающий мир многозадачности. Рассматривая многозадачность на уровне среды выполнения, мы лучше поймем, как наши машины выполняют разные задачи одновременно. Но прежде чем погружаться в тонкости многозадачности, сначала нужно подробнее разобраться, с какими типами задач может работать наш компьютер.

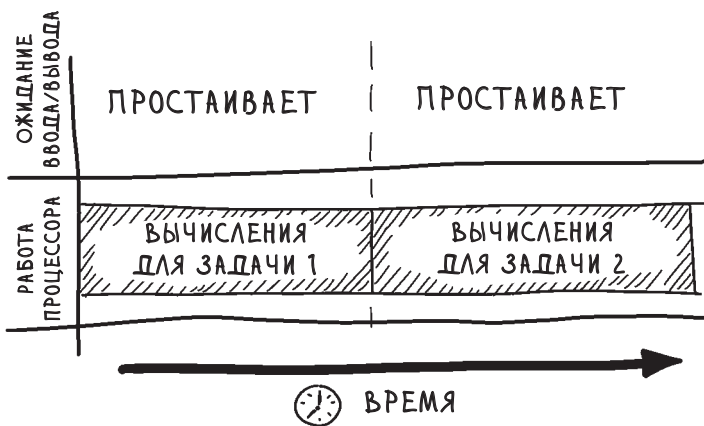
Приложения с интенсивными вычислениями (CPU-bound) и интенсивным вводом/выводом (I/O-bound)

Приложения состоят из числовых, арифметических и логических операций, которые требуют интенсивной работы процессора. Они также могут принимать данные от клавиатуры, жесткого диска или сетевого адаптера и вырабатывают результат в виде файлов, вывода на принтеры или сигналов для видеоадаптера. Эти операции взаимодействуют с устройствами, отправляя и принимая сигналы. В большинстве случаев такие взаимодействия не требуют участия процессора, потому что вычислять ничего не нужно; система просто ожидает ответа от устройства. Такие операции также называются *операциями ввода/вывода (I/O)*. Следовательно, не всегда имеет смысл предоставлять задачам доступ к процессору; сначала необходимо понять тип нагрузки.

Считается, что приложение *интенсивно расходует* тот или иной ресурс, если он становится узким местом, которое ограничивает производительность. Существуют две основные разновидности операций: с *интенсивными вычислениями (CPU-bound)* и с *интенсивным вводом/выводом (I/O-bound)*.

CPU-bound

До сих пор мы в основном рассматривали приложения с интенсивными вычислениями — то есть те, которые интенсивно используют процессор. Если ускорить процессор, такое приложение начнет работать быстрее. Иными словами, оно проводит большую часть времени, используя процессор, чтобы выполнять те или иные вычисления.

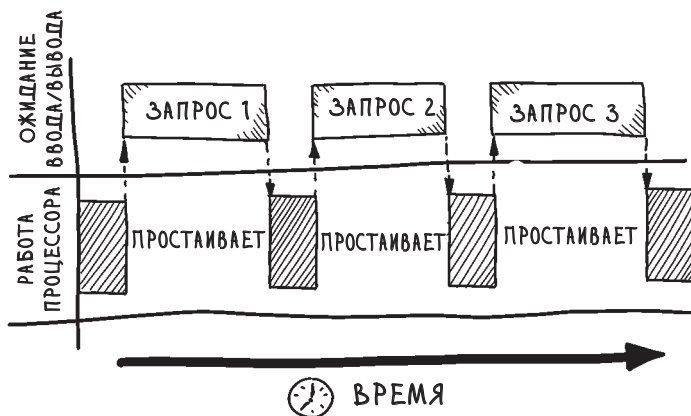


Примеры операций с интенсивными вычислениями:

- Математические операции (например, сложение, вычитание, деление, матричное умножение).
- Алгоритмы шифрования и дешифрования, которые требуют интенсивных вычислительных операций (разложение на простые множители, вычисление криптографических функций).
- Обработка изображений и видео.
- Выполнение таких алгоритмов, как бинарный поиск и сортировка.

I/O-bound

Приложение с интенсивным вводом/выводом ускорится, если подсистема ввода/вывода будет работать быстрее. Эти подсистемы бывают разными — например, они могут читать данные с диска, принимать ввод от пользователя или ожидать ответа по сети. Приложение, которое просматривает огромный файл в поисках определенного текста, может попасть в эту категорию, потому что чтение большого объема данных с диска создает узкое место.



Участки простоя на диаграмме соответствуют периодам времени, когда та или иная задача находится в ожидании и поэтому не может двигаться дальше. Типичная причина простоя — ожидание ввода/вывода. Но когда выполняются различные операции ввода/вывода, процессор часто ничего не делает, а только ждет, пока данные будут переданы внешнему устройству или от него, — а процессорное время стоит дорого. Примеры операций с интенсивным вводом/выводом:

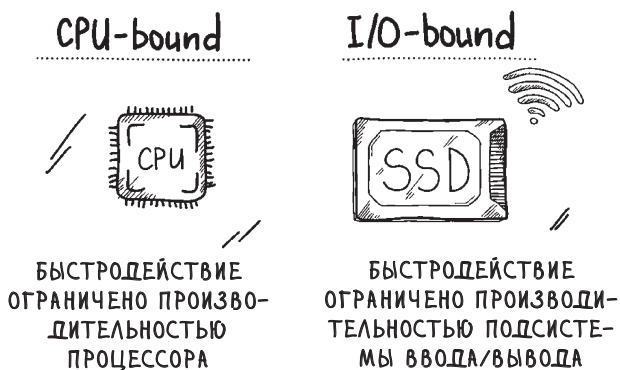
- Большинство приложений с графическим интерфейсом (GUI), даже если они не читают и не записывают никаких данных на диск, потому что они проводят большую часть времени, ожидая взаимодействия с пользователем с помощью клавиатуры или мыши.
- Процессы, которые большую часть времени заняты дисковым или сетевым вводом/выводом, например базы данных или веб-серверы.

Узкие места

Чтобы выявить узкие места приложения, нужно думать о том, какой ресурс следует улучшить, чтобы оно работало эффективнее. Это напрямую относится к тому, как связаны операции и ресурсы, от которых они зависят. Обычно считается, что операции вычисления и ввода/вывода — самые важные, и на них стоит обращать внимание в первую очередь.

ПРИМЕЧАНИЕ Конечно, в выполнении программы участвуют не только процессор и подсистема ввода/вывода; также можно иметь в виду работу памяти и кэша. Однако для большинства разработчиков и целей этой книги достаточно учитывать разницу между нагрузкой на процессор и на ввод/вывод.

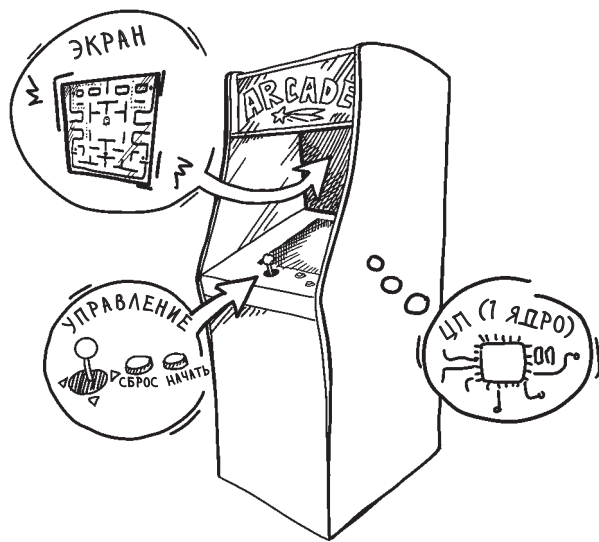
Представьте себе две программы. Первая перемножает две огромные матрицы и возвращает ответ. Вторая программа записывает колоссальный объем информации из сети в файл на диске. Очевидно, что эти программы не ускорятся в равной степени, если повысить тактовую частоту процессора или увеличить количество ядер. Какая разница, сколько будет ядер, если большую часть времени они ожидают, пока следующая партия данных запишется на диск? Неважно, одно ядро или тысяча: при нагрузке с интенсивным вводом/выводом производительность не улучшится. Но если у нас будет нагрузка с интенсивными вычислениями, то есть вероятность, что производительность повысится, если параллелизовать программу так, чтобы она использовала несколько ядер.



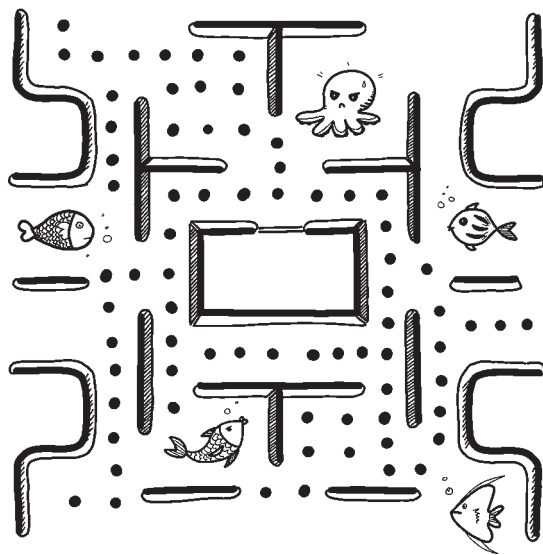
Почему необходима многозадачность

Приложения естественным образом все интенсивнее используют ввод-вывод. Дело в том, что исторически скорость процессоров росла, позволяя выполнять больше команд за заданный период времени, тогда как скорость передачи данных особо не увеличивалась. Таким образом, ограничителем в программах часто становятся операции с интенсивным вводом/выводом, которые блокируют процессор. Но эти операции можно выявить и выполнять в фоновом режиме, и большинство современных систем выполнения поддерживают такую возможность.

Представьте, что ваш друг Алан нашел в родительском чулане древний игровой автомат. У него есть старый одноядерный процессор, экран с низким разрешением и джойстик. Алан обращается к своему единственному другу-программисту — то есть к вам — и просит реализовать для автомата игру в стиле Pac-Man.



Игра интерактивная, и, чтобы перемещать игрового персонажа, требуется ввод от пользователя. В то же время мир внутри игры динамичен: призраки должны двигаться в то же время, когда игрок управляет персонажем. При этом игрок должен видеть, как меняется игровой мир и как перемещается персонаж.



Первым делом нужно создать игровую функциональность, которая разделена между тремя функциями:

- `get_user_input()` — получает ввод от устройств управления и сохраняет его во внутреннем состоянии игры. Это операция с интенсивным вводом/выводом.
- `compute_game_world()` — вычисляет конфигурацию игрового мира в соответствии с правилами игры, вводом пользователя и внутренним состоянием игры. Это операция с интенсивными вычислениями.
- `render_next_screen()` — получает внутреннее состояние игры и отображает игровой мир на экране. Это операция с интенсивным вводом/выводом.

Продумывая эти три функции, вы понимаете, что столкнулись с проблемой: многие события должны происходить одновременно с точки зрения игрока, но в вашем распоряжении только старый одноядерный процессор. Что делать?

Для начала попытаемся создать параллельную программу с помощью одной из абстракций ОС. В приложении будут использоваться потоки, поэтому мы создадим один процесс и три потока. В данном случае потоки полезны, потому что задачи должны работать с общими данными, и они упрощают решение, поскольку могут совместно использовать общее адресное пространство процесса. В итоге наша программа выглядит примерно так:

```
# Chapter 6/arcade_machine.py
import typing as T
from threading import Thread, Event

from pacman import get_user_input, compute_game_world, render_next_screen

processor_free = Event()
processor_free.set()

class Task(Thread):
    def __init__(self, func: T.Callable[..., None]):
        super().__init__()
        self.func = func

    def run(self) -> None:
        while True:
            processor_free.wait()
            processor_free.clear()
            self.func()

def arcade_machine() -> None:
    get_user_input_task = Task(get_user_input)
    compute_game_world_task = Task(compute_game_world)
    render_next_screen_task = Task(render_next_screen)

    get_user_input_task.start()
    compute_game_world_task.start()
    render_next_screen_task.start()

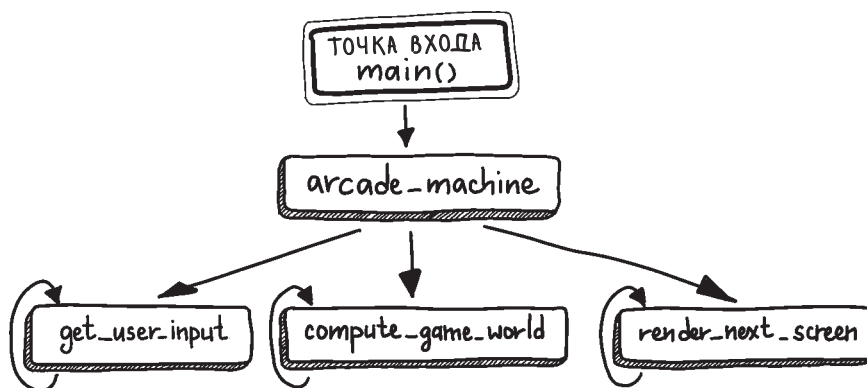
if __name__ == "__main__":
    arcade_machine()
```

Моделирует среду с одним процессором/потоком

Запускает функцию в своем собственном бесконечном цикле. Тело цикла будет повторяться, пока программа не остановится или пока поток не будет завершен

Определяет и запускает задачи в отдельных потоках в конкурентном режиме

Здесь инициализируются три потока, каждый из которых соответствует одной из трех функций. Каждая функция в потоке выполняется в собственном бесконечном цикле (предполагается, что поток не останавливается после одного выполнения), так что потоки работают все время, пока пользователь продолжает играть.



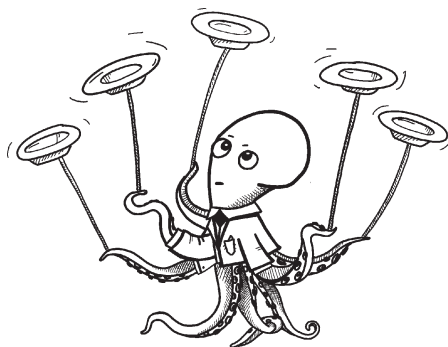
К сожалению, если запустить программу, она застрянет в первом потоке, запрашивая у пользователя ввод в бесконечном цикле. Она не будет делать ничего другого, потому что на процессоре умещается только один поток. Получается, что параллелизма здесь не выйдет, потому что для него необходимо соответствующее оборудование. Но не переживайте: конкурентность все еще можно использовать с многозадачностью!

Однако прежде чем применить многозадачность в нашем примере, необходимо освоить ее основные принципы. Они рассматриваются в следующем разделе.

Общие принципы многозадачности

В современном мире многозадачность встречается на каждом шагу. Мы сами работаем в многозадачном режиме, когда слушаем музыку во время прогулки, отвечаем на телефонные звонки, пока готовим обед, или едим и одновременно читаем книгу.

Многозадачность состоит в том, что несколько задач выполняются в течение одного и того же периода времени в конкурентном режиме. Многозадачную систему можно сравнить с цирковым



жонглером, который удерживает в равновесии несколько тарелок, вращающихся на тростях. Жонглер стремительно перемещается от тарелки к тарелке и старается поддерживать их вращение, чтобы они не упали.

В полноценной многозадачной системе операции выполняются параллельно, но для этого нужна аппаратная поддержка. Тем не менее иллюзию многозадачности можно создать даже на старых процессорах, если использовать соответствующие приемы.

Вытесняющая многозадачность

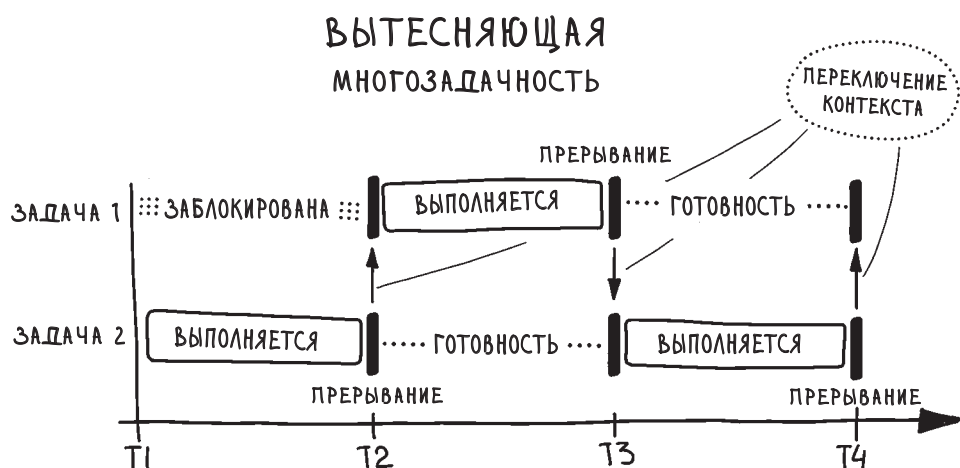
Главная задача ОС — управлять ресурсами, а одним из самых важных ресурсов является процессор. Нужно, чтобы ОС могла обеспечить выполнение каждой программы на процессоре. Это означает, что нужно выполнять задачу в течение некоторого времени, а затем отложить ее и позволить выполняться другой задаче. Проблема в том, что многие приложения написаны без учета того, что в системе выполняются другие приложения. Таким образом, ОС необходим способ принудительно приостанавливать выполнение приложения.

Главное в *вытесняющей многозадачности* — определить, в течение какого периода времени разрешено выполняться каждой задаче. Этот период также называется *квантом времени*, а механизм планирования называется *политикой разделения времени*¹. Процессор выполняет задачу, которая находится в состоянии готовности, в течение этого кванта времени, если в нее не входят никакие блокирующие операции.

Когда квант времени истекает, планировщик *прерывает* задачу (*вытесняет* ее) и позволяет другой задаче выполняться вместо нее, пока первая ожидает своей следующей очереди. Прерывание — это сигнал процессору приостановить задачу и возобновить ее позднее. Прерывания бывают трех видов: аппаратные со специальным контроллером прерываний (например, нажатие клавиши или завершение записи в файл); программные (например, системные вызовы), которые инициирует само приложение; ошибки и прерывания таймера.

Процессор выделяет по небольшому периоду времени для каждой текущей задачи и быстро переключается между задачами, чтобы все они выполнялись в режиме чередования. Быстро чередуя задачи из очереди и передавая им управление, ОС создает иллюзию многозадачности, хотя на самом деле в каждый момент времени выполняется только одна задача. На следующей диаграмме показан ход выполнения двух задач в зависимости от времени. Ось времени идет слева направо, а линии показывают, какая задача выполняется в тот или иной момент. Пользователю кажется, будто обе задачи выполняются одновременно.

¹ Если вам захочется узнать больше, на эту тему есть хороший видеоролик: «1963 Timesharing: A Solution to Computer Bottlenecks» («Разделение времени в 1963 году: борьба с узкими местами на компьютерах»), <https://youtu.be/Q07PhW5sCEk>.



Многие ОС, разработанные за последнее десятилетие, реализуют вытесняющую многозадачность (в главе 12 мы сравним ее с кооперативной многозадачностью). Если вы работаете в Linux, macOS или Windows, вы работаете в ОС с вытесняющей многозадачностью. Чтобы лучше понять, как можно реализовать многозадачность, вернемся к примеру с игровым автоматом.

Игровой автомат с вытесняющей многозадачностью

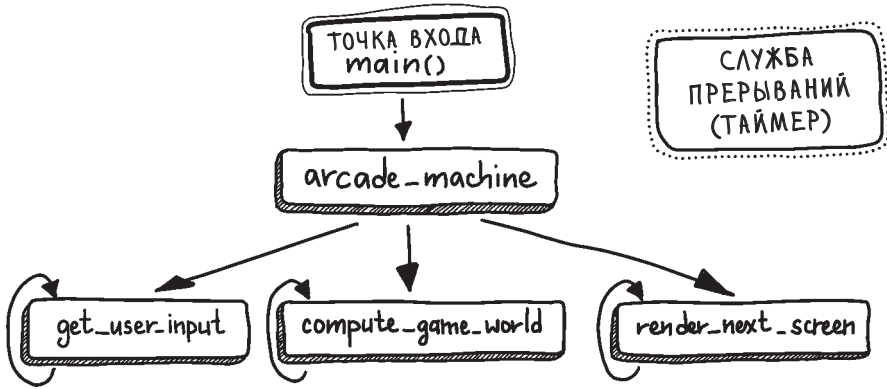
В нашей программе есть две операции с интенсивным вводом/выводом, которые ожидают определенного события и поэтому блокируют процессор. Например, поток `get_user_input_task` ожидает, когда игрок нажмет кнопку на игровом контроллере.

Хотя игровой автомат оснащен старым одноядерным процессором, он работает очень быстро по сравнению с человеческими рефлексам. Пока человек подносит палец к кнопке и нажимает ее, для процессора проходит невообразимо долгое время. Самые быстрые сознательные человеческие реакции занимают около 0,15 с, а процессор с частотой 2 ГГц за это же время обрабатывает 300 миллионов тактов, то есть выполняет примерно столько же команд. Пока мы ждем ввода от игрока, который нажимает на кнопку, мы попусту тратим вычислительные ресурсы процессора, потому что его ядро ничего не делает. Можно использовать период простоя процессора, уступив на это время управление вычислительным задачам.

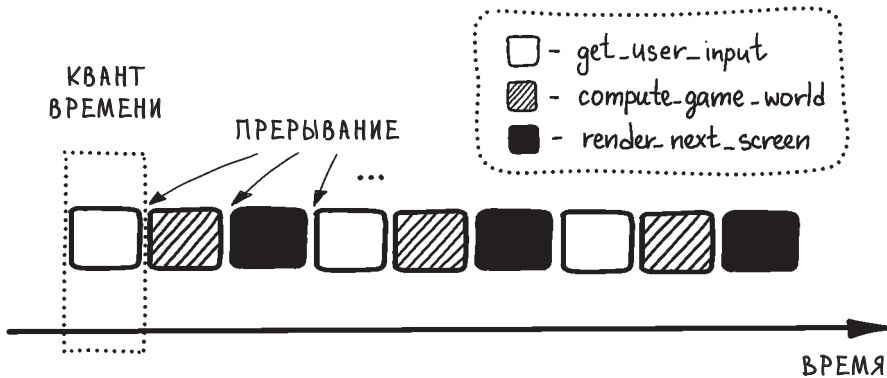
По сути необходимо реализовать часть функциональности ОС. Для этого можно воспользоваться вытесняющей многозадачностью: каждому потоку выделяется процессорное время для выполнения, после чего процессор передается следующему потоку. Можно настроить простую политику разделения времени и разбить все доступное процессорное время на равные кванты.

Здесь на помощь приходит таймер. Он отсчитывает такты с регулярными интервалами, и его можно настроить так, чтобы через каждые N тактов возбуждалось прерывание. Оно приостанавливает текущий поток, позволяя другому потоку

воспользоваться процессором. Таким образом, диаграмма выполнения программы выглядит так:



Реализуя политику разделения времени, система выполнения делит процессорное время между потоками на кванты, чтобы создать иллюзию, будто они выполняются одновременно.



В коде это выглядит так:

```
# Chapter 6/arcade_machine_multitasking.py
import typing as T
from threading import Thread, Timer, Event

from pacman import get_user_input, compute_game_world, render_next_screen

processor_free = Event()
processor_free.set()
TIME_SLICE = 0.5
```

Задаёт квант процессорного времени

```

class Task(Thread):
    def __init__(self, func: T.Callable[..., None]):
        super().__init__()
        self.func = func

    def run(self) -> None:
        while True:
            processor_free.wait()
            processor_free.clear()
            self.func()

class InterruptService(Timer):
    def __init__(self):
        super().__init__(TIME_SLICE, lambda: None)

    def run(self):
        while not self.finished.wait(self.interval):
            print("Тик!")
            processor_free.set()

def arcade_machine() -> None:
    get_user_input_task = Task(get_user_input)
    compute_game_world_task = Task(compute_game_world)
    render_next_screen_task = Task(render_next_screen)

    InterruptService().start()
    get_user_input_task.start()
    compute_game_world_task.start()
    render_next_screen_task.start()

if __name__ == "__main__":
    arcade_machine()

```

Устанавливает таймер, чтобы обозначить, что процессор свободен (таймер прерывает работу процессора)



Мы реализуем многозадачность, поместив потоки в один бесконечный управляющий цикл, где можно поочередно предоставлять каждому потоку квант процессорного времени. Если чередование происходит достаточно быстро (допустим, каждые 10 мс), то у игрока возникает иллюзия одновременного выполнения. Игроку кажется, что все внимание игры посвящено ему, хотя на самом деле процессор и компьютерная система в целом в этот момент могут работать над совершенно другой задачей. Иллюзия параллельного выполнения возникает из-за того, что потоки переключаются чрезвычайно быстро.

Итак, физически задачи все еще выполняются последовательно, потому что ресурсы нашего процессора ограничены. Однако концептуально система работает сразу с тремя потоками, и все они выполняются в конкурентном режиме.

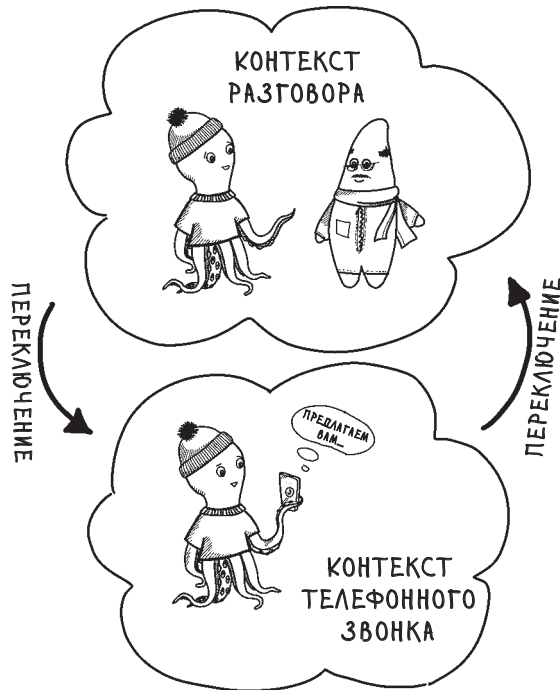
Жизненные циклы конкурентных вычислений перекрываются. Как вы уже видели, с подходящим оборудованием можно добиться настоящего параллелизма, когда задачи физически выполняются одновременно, а многозадачность помогает абстрагировать перекрывающееся выполнение, поручив системе выполнения управлять им. Таким образом, настоящий параллелизм — это фактически особенность реализации выполнения, в то время как многозадачность — часть вычислительной модели.

Однако здесь кроется ловушка, так что давайте сделаем шаг назад.

Переключение контекста

Контекст выполнения задачи содержит код, который сейчас выполняется (в форме указателя команд), и все, что задействовано в его выполнении на ядре процессора (флаги процессора, ключевые регистры, переменные, открытые файлы, сетевые соединения и т. д.). Чтобы возобновить приостановленную задачу, все это нужно загрузить обратно в процессор. Соответственно, *переключение контекста* — это физический акт переключения с контекста одной задачи на контекст другой задачи без потери данных, чтобы их можно было точно восстановить в том же состоянии, в котором они находились на момент переключения. Задача, выбранная из очереди готовности, переходит в состояние выполнения.

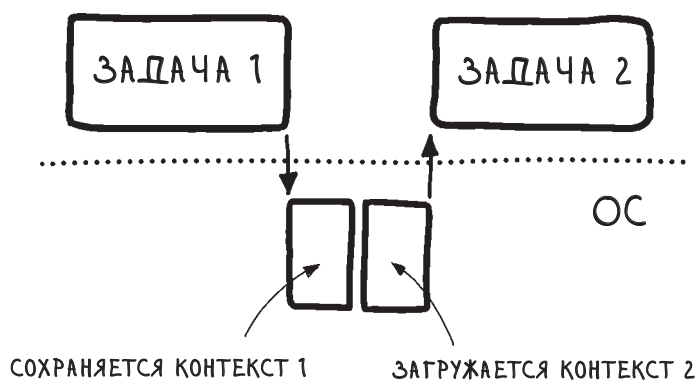
Представьте, что вы разговариваете с другом о чем-то интересном, но тут звонит телефон и вы отвлекаетесь. Вы говорите другу: «Подожди минутку» и берете трубку. Теперь вы вступаете в новую беседу, то есть в новый контекст. Когда вы поняли, кто звонит и что ему нужно, вы можете сосредоточиться на этом вопросе. Завершив звонок, вы возвращаетесь к исходному разговору. Иногда вы забываете, о чем шла речь в момент переключения, но когда ваш друг напомнит тему беседы, вы можете продолжить. Это происходит быстро, но не мгновенно.



Как и вам, процессору нужно найти контекст, на котором остановилась задача, и восстановить его. С точки зрения задачи все вокруг осталось в том же состоянии,

что и прежде, — неважно, была ли задача запущена только что или полчаса назад. Переключением контекста занимается ОС, и это один из ключевых механизмов, благодаря которым она работает в режиме многозадачности.

Переключения контекста считаются затратными операциями, потому что они потребляют системные ресурсы. Чтобы переключиться с одного контекста на другой, требуются определенные действия. Сначала нужно сохранить где-то контекст выполняемой задачи, после чего начинает выполняться новая задача. Если новая задача раньше уже выполнялась, то у нее тоже есть сохраненный контекст; его нужно предварительно загрузить, прежде чем продолжить выполнение. Когда эта задача завершается или вытесняется, планировщик сохраняет ее текущий контекст и восстанавливает контекст ранее вытесненной задачи. Вытесненная задача продолжает выполняться, словно ничего не случилось (кроме сдвига по времени).



Сохранение и восстановление состояния при переключении контекста отрицательно влияют на производительность приложения, потому что во время переключения оно не может выполнять команды. Конкретный объем затрат на переключение зависит от того, какого рода операции выполняет ваша программа.

ПРИМЕЧАНИЕ Продолжительность задержки, которая обусловлена переключением контекста, зависит от разных факторов, но давайте возьмем разумную оценку примерно от 800 до 1300 нс на переключение — я получил эти значения при помощи LMBench (<https://lmbench.sourceforge.net>) на своем ноутбуке. С учетом того, что одно ядро должно быть способно выполнять в среднем 12 команд в наносекунду, переключение контекста обходится приблизительно в 9–15 тысяч выполняемых команд.

Будьте осторожны, если приложение работает с большим количеством задач: когда их слишком много, производительность системы может пострадать, потому что ей придется тратить слишком много полезного времени в цикле переключения контекста.

Итак, теперь вы знаете, что такое многозадачность, так что мы можем интегрировать ее в среду выполнения и объединить все остальные понятия конкурентности.

Многозадачные среды

На заре компьютерной эпохи люди не думали о том, чтобы выполнять сразу несколько задач на одном компьютере, потому что ОС и приложения не были рассчитаны на многозадачность. Пользователям приходилось каждый раз закрывать одно приложение и открывать другое.

В наши дни способность выполнять несколько задач одновременно стала одним из важнейших требований для систем выполнения. Это требование обеспечивается механизмом многозадачности. И хотя при этом нет настоящей параллельной обработки, а переключение задач сопряжено с определенными затратами, выполнение с чередованием дает значительные преимущества в отношении эффективности и структурирования программ.

Для пользователей основное преимущество многозадачной системы состоит в том, что в ней можно запустить сразу несколько приложений. Например, вы можете редактировать документ в одном приложении и одновременно просматривать видео в другом.

Для разработчиков преимущество многозадачности заключается в том, что появляется возможность создавать приложения с несколькими процессами и процессы с несколькими потоками выполнения. Например, в процессе может быть поток пользовательского интерфейса, который обеспечивает взаимодействие с пользователем (ввод с клавиатуры и мыши), и потоки-исполнители, которые выполняют вычислительные задачи, пока поток интерфейса ожидает ввода от пользователя.

Если делегировать планирование и координацию задач системе выполнения, процесс разработки упрощается и приложения становятся гибкими, способными прозрачно адаптироваться к разным аппаратным и программным архитектурам. Использование разных сред выполнения (компьютерных ОС, сред интернета вещей [IoT], производственных ОС и т. д.) позволяет разработчикам оптимизировать приложения для разных целей. Например, для того, чтобы минимизировать энергопотребление, может потребоваться один планировщик, а для того, чтобы максимизировать пропускную способность, — другой.

ПРИМЕЧАНИЕ В 1960-е и 1970-е годы разработка многозадачных ОС (таких, как IBM OS/360 и UNIX) позволила запускать множество программ на одном компьютере, но требовала больше памяти, чем на нем было физически доступно. Чтобы решить эту проблему, была разработана виртуальная память — механизм временной выгрузки данных из оперативной памяти на диск, который позволял компьютерам использовать больше памяти, чем у них было. С помощью этой технологии компьютеры могли запускать больше программ одновременно, и виртуальная память по сей день остается важнейшим компонентом ОС.

Многозадачные ОС

В многопроцессорных средах многозадачность может дополняться тем, что разные задачи распределяются по доступным ядрам процессора. Процессор ничего не знает о процессах или потоках, он умеет только выполнять машинные команды. Таким образом, с точки зрения процессора существует только один поток выполнения —

последовательное выполнение всех машинных команд, полученных от ОС. Чтобы его осуществить, ОС использует абстракции потоков и процессов. А когда на единственное процессорное ядро претендуют несколько потоков, ОС должна каким-то образом жонглировать ими, имитируя параллельное выполнение для пользователя, но фактически выполняя их все в конкурентном режиме.

Многозадачность проявляется на уровне системы выполнения; на аппаратном уровне нет понятия многозадачности. Тем не менее реализация многозадачности — непростое дело, которое часто требует, чтобы задачи строго изолировались, а в системе выполнения был эффективный планировщик.

Изоляция задач

По определению многозадачности в ОС выполняются несколько разных задач. Вероятно, вы подумали, что сейчас речь пойдет об абстракциях процессов и потоков, которые предоставляет ОС, но если вы разрабатываете систему выполнения, то ситуация меняется.

Существует два способа создавать множественные задачи:

- В виде одного процесса с несколькими потоками.
- В виде нескольких процессов, в каждом из которых один или несколько потоков.

Как обсуждалось ранее, у каждого варианта есть свои плюсы и минусы, но оба в большей или меньшей степени обеспечивают изоляцию выполнения задач. В свою очередь, ОС отвечает за то, как эти абстракции отображаются на физические потоки компьютерной системы и как они выполняются на оборудовании.

ОС абстрагирует реальную работу оборудования, и даже если в системе только одно ядро, ОС создает у разработчика иллюзию, будто их несколько. Таким образом, даже если в системе недоступен параллелизм, разработчик все равно может применять методы конкурентного программирования и пользоваться многозадачностью ОС. Программу, разделенную подобным образом, можно писать так, будто процессор находится в ее полном распоряжении.

Обычно многозадачность реализуется эффективнее, если создавать один многопоточный процесс, а не несколько процессов. Причины этого таковы:

- Система может переключать контекст для потоков быстрее, чем для процессов, потому что процесс требует больших затрат (его контекст больше, чем контекст потока).
- Все потоки одного процесса совместно используют адресное пространство и могут обращаться к переменным глобального процесса, что упрощает коммуникации между потоками.

Планирование задач

Планировщик — один из главных компонентов многозадачных ОС. Из списка задач, находящихся в состоянии готовности, планировщик выбирает, какую задачу нужно выполнять следующей.

Планирование выполнения основано на простой идее. Чтобы процессорное время использовалось эффективно, на процессоре всегда должно что-то работать. Если задач в системе больше, чем процессоров (как это обычно бывает), некоторые задачи не выполняются постоянно, а ожидают в состоянии готовности. Чтобы выбрать задачу, которая должна выполняться следующей, планировщик опирается на информацию о задачах, готовых к выполнению.

Поскольку планировщик распределяет ограниченный ресурс (процессорное время), его логика направлена на то, чтобы достичь баланса между конфликтующими целями и приоритетами. Типичные цели — максимизировать пропускную способность (количество задач, которые система может обработать за определенное время), уравновесить распределение (приоритизировать или сбалансировать вычисления), минимизировать задержку (время завершения действия) или время отклика (чтобы система реагировала быстрее).

Планировщик может принудительно отнять управление у задачи (например, по таймеру или когда появится задача с более высоким приоритетом) или дождаться, пока задача передаст ему управление явно (вызвав системную функцию) или неявно (при завершении). Это значит, что работа планировщика не детерминирована в отношении того, какую задачу он выберет для выполнения в тот или иной момент времени. Таким образом, никогда не стоит писать программы, опираясь на ранее наблюдавшееся поведение, потому что нет гарантий, что оно будет повторяться каждый раз. Чтобы приложение вело себя детерминированно, нужно управлять синхронизацией и координацией задач. Эта тема будет рассмотрена в следующих главах.

Что еще важнее, планировщик открывает путь к тому, чтобы реализовывать новые методы улучшения производительности системы, не изменяя программу. Конечно, дополнительная прослойка между приложением и ОС повышает затраты на выполнение. Чтобы этот подход сработал, среда выполнения должна обеспечивать выигрыш по производительности, который превышает затраты на управление этой средой.

ПРИМЕЧАНИЕ В этой главе мы в основном говорили про ОС, но другие среды выполнения реализуют те же концепции многозадачности. Например, в таких языках, как JavaScript и Python, многозадачность задается ключевым словом `await`, а в среде выполнения функционирует однопоточный цикл событий. V8 (один из самых эффективных движков JavaScript на рынке) и язык программирования Go, известный своей масштабируемостью и низким потреблением памяти, используют собственную реализацию многозадачности поверх ОС (на пользовательском уровне). Мы затронем эту тему в главе 12, когда будем рассматривать кооперативную многозадачность и асинхронную коммуникацию.

Итоги

- В программах бывают узкие места двух типов в зависимости от того, какие ресурсы используются наиболее интенсивно — процессор или подсистема ввода/вывода:

- Операциям с интенсивными вычислениями (CPU-bound), чтобы завершить работу, прежде всего необходимы ресурсы процессора. В таком случае выполнение ограничено скоростью, с которой система может что-то вычислить.
- Операции с интенсивным вводом/выводом (I/O-bound), как правило, не зависят от вычислительных ресурсов, а, например, ожидают завершения дисковой операции или ответа от внешней службы. При этом выполнение ограничено скоростью оборудования, например скоростью, с которой данные можно прочитать с диска или передать по сети.
- *Переключение контекста* — физический акт замены контекста одной задачи контекстом другой, чтобы позднее задачу можно было восстановить в том же состоянии, в котором она находилась на момент переключения. Процедуру переключения контекста выполняет ОС, и это один из ключевых механизмов, которые обеспечивают многозадачность в ОС.
- Переключение контекста не обходится бесплатно, так что будьте осторожны, если приложение работает с большим количеством задач: когда их слишком много, производительность системы может пострадать, потому что ей придется тратить значительную часть полезного времени в цикле переключения контекста.
- Для систем выполнения критически важно уметь выполнять сразу несколько задач. Эту возможность обеспечивает *многозадачность* — механизм, который управляет чередованием и попеременным выполнением задач. Постоянно переключая задачи, система поддерживает иллюзию того, что они выполняются одновременно, хотя на самом деле это не так.
- *Многозадачность* состоит в том, что несколько задач выполняются в течение одного и того же периода времени в конкурентном режиме. Многозадачность проявляется на уровне системы выполнения; на аппаратном уровне нет понятия многозадачности.
- При вытесняющей многозадачности планировщик назначает задачам приоритеты и заставляет их уступать управление друг другу.
- В общем случае реализация многозадачности с одним многопоточным процессом (вместо нескольких процессов) работает эффективнее.
- Планировщик системы выполнения должен различать задачи с интенсивными вычислениями и с интенсивным вводом/выводом — тогда он сможет обеспечить, чтобы системные ресурсы использовались оптимально.



В этой главе

- ✓ Приемы декомпозиции, которые позволяют эффективно разбивать проблему программирования на отдельные независимые задачи
- ✓ Популярные паттерны разработки конкурентных приложений: конвейер, отображение, ветвление/соединение и отображение/свертка
- ✓ Как выбрать уровень детализации (гранулярности) для ваших приложений
- ✓ Как использовать агломерацию, чтобы сократить коммуникационные затраты и повысить производительность системы

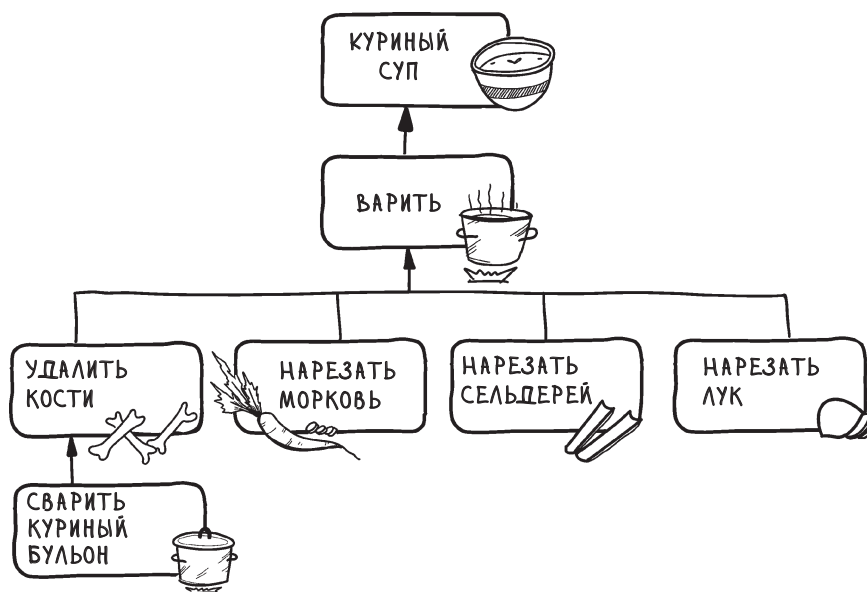
Ранее вы узнали, что при конкурентном программировании проблема разбивается на *задачи* (tasks) — независимые единицы конкурентного выполнения. Как разложить проблему на конкурентные задачи — один из самых трудных, но важных вопросов. Автоматическая декомпозиция программ с использованием средств конкурентного программирования стала темой для сложных исследований. Таким образом, в большинстве случаев декомпозицией приходится заниматься разработчикам.

В этой главе представлены методы и популярные паттерны разработки конкурентных приложений. В ней рассматривается прикладной уровень конкурентности; мы больше сосредоточимся на том, как выявлять независимость задач, а также на том, как структурировать и проектировать программы, чем на том, как они будут выполняться (хотя об этом мы тоже упомянем).

Анализ зависимостей

Декомпозиция проблемы на конкурентные задачи — один из первых необходимых этапов при написании конкурентных приложений, который играет ключевую роль в конкурентном программировании. Занимаясь декомпозицией, следует помнить, что одни задачи могут зависеть от других. Следовательно, декомпозицию нужно начинать с того, чтобы найти все зависимости между задачами и выявить независимые задачи. Чтобы моделировать связи между задачами, можно построить *граф зависимостей задач*.

Рассмотрим этот граф на примере приготовления простого куриного супа. Что нужно сделать, чтобы приготовить суп? Сварить курицу, чтобы получился бульон; удалить кости; нарезать морковь, нарезать сельдерей, нарезать лук, добавить эти ингредиенты в суп и варить, пока курица не будет полностью готова. Нельзя начать варить суп, пока не будут выполнены все предыдущие задачи. Каждый шаг представляет задачу, и, двигаясь от результата назад через все его зависимости, можно построить граф зависимостей, который выглядит примерно так:



Такие вычислительные графы можно строить разными способами, но у них всех одна цель: обеспечить абстрактное представление программы. Они помогают наглядно представить отношения и зависимости между задачами. Каждый узел (вершина) представляет задачу, а ребра представляют зависимости.

Графы зависимостей также помогают получить представление о том, насколько конкурентной может быть программа. Сам факт, что между задачами приготовления бульона и нарезки овощей нет прямых ребер, указывает на то, что в этом месте возможна конкурентность. Таким образом, если эта программа будет реализована с помощью потоков, мы сможем создать четыре отдельных потока: один для приготовления бульона и еще три для нарезки овощей. Все они могут выполняться одновременно. На этом же принципе основано планирование отдельных задач в системах выполнения.

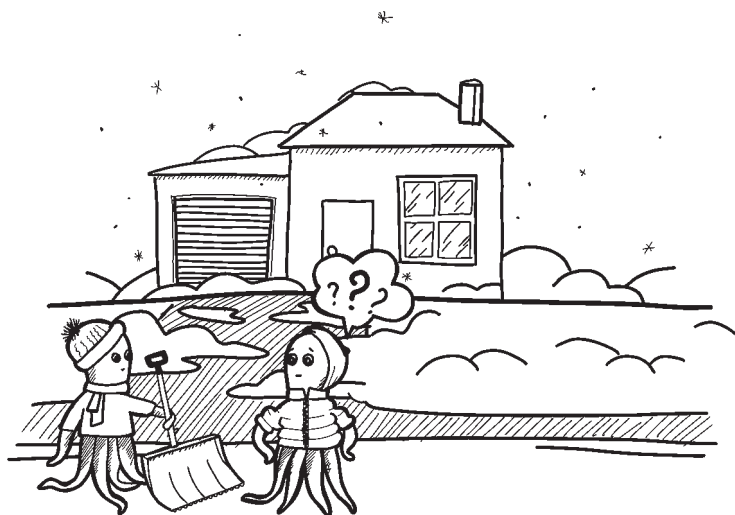
Построение графа зависимостей — первая стадия при проектировании программ или систем. Полезно выявить части работы, которые можно выполнять конкурентно. На этом этапе мы игнорируем детали практической реализации — например, сколько процессоров или ядер нам доступно; все внимание сосредоточено на возможной конкурентности исходной проблемы. Отметив эти особенности графа зависимостей, рассмотрим его под другими углами.

В коде бывают зависимости двух типов: зависимости управления (control dependency) и зависимости данных (data dependency). Им соответствуют разные способы деления проблемы на меньшие задачи: *декомпозиция задач* и *декомпозиция данных*.

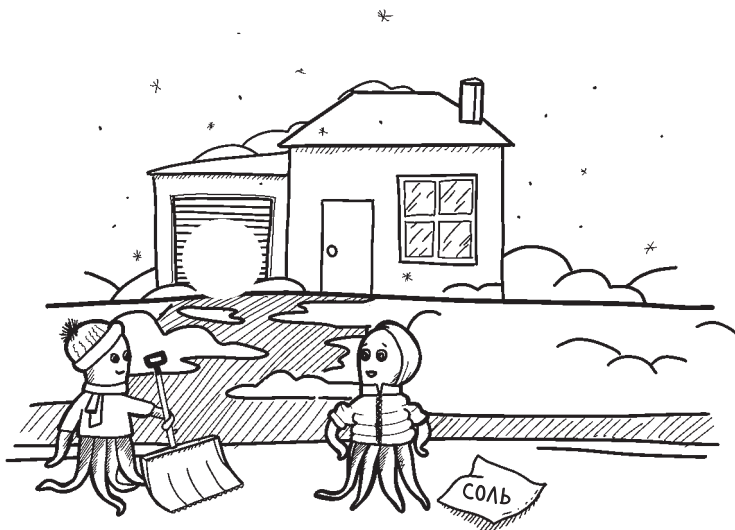
Декомпозиция задач

Декомпозиция задач отвечает на вопрос «Как разложить проблему на независимые блоки функциональности, которые можно выполнять в конкурентном режиме?» или, проще говоря, «Как разбить проблему на задачи, которые можно выполнять все сразу?».

Представьте, что прошел обильный снегопад. Вы хотите расчистить участок вокруг дома, для чего нужно убрать снег лопатой и посыпать дорожки солью. Ваш приятель приходит на помощь, чтобы работа шла быстрее, но у вас только одна лопата. Таким образом, пока один из вас работает лопатой, другой ждет своей очереди. Помните: хотя процесс выглядит логично, с единственным ресурсом (лопатой) работа не ускоряется, а только замедляется. Из-за затрат на переключение контекста процесс становится неэффективным, потому что переключения постоянно прерывают саму работу.



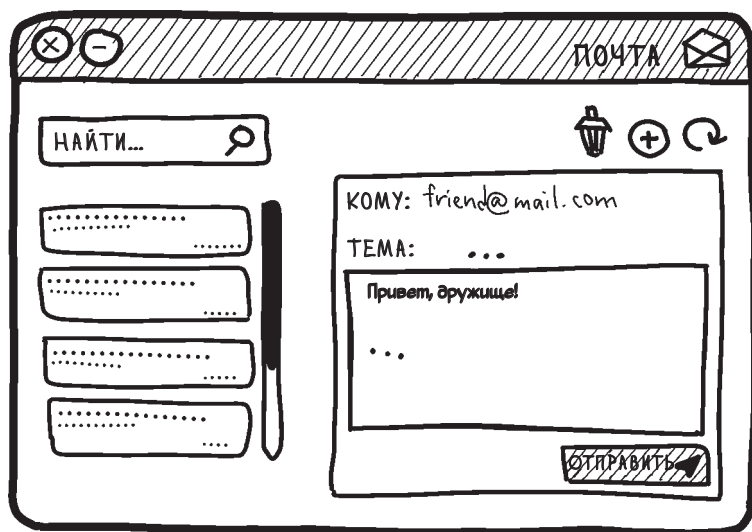
Тогда вы поручаете приятелю другую подзадачу. Пока вы расчищаете снег единственной лопатой, приятель разбрасывает соль. Когда больше не приходится ждать, пока освободится лопата, работа становится эффективнее. Фактически в этом и заключается декомпозиция задач (которая также называется *параллелизмом задач*).



Это пример того, как проблема разбивается на задачи по функциональности. Однако декомпозиция задач далеко не тривиальна: это сложная и крайне субъективная процедура.

Декомпозиция задач подразумевает, что приложение делится на функционально независимые задачи в зависимости от функциональности самого приложения. Такая декомпозиция возможна, когда в проблеме, которую мы решаем, естественным образом выделяются разные типы задач, каждую из которых можно выполнять независимо.

Например, у приложения для работы с электронной почтой может быть множество функциональных требований: оно должно иметь пользовательский интерфейс, механизм надежной доставки новых сообщений и инструменты, с помощью которых пользователь может составлять и отправлять сообщения, а также искать по ним.

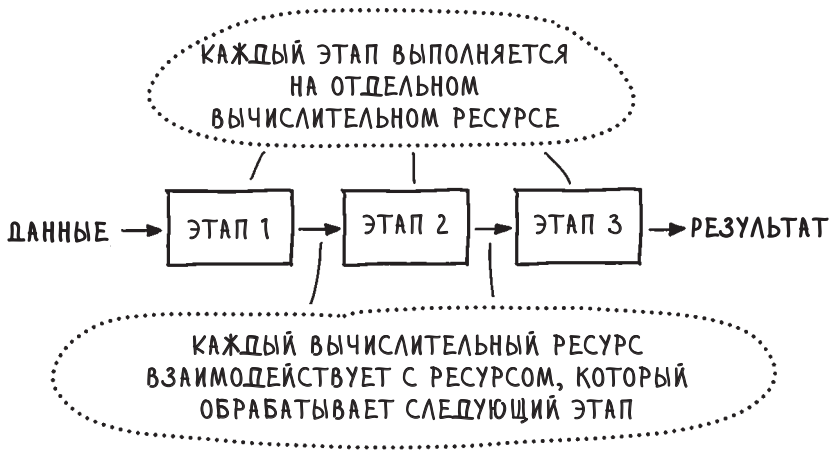


Задача поиска сообщений и пользовательский интерфейс, который выводит список этих сообщений, зависят от одних и тех же данных, но совершенно не зависят друг от друга, поэтому их можно разделить на две задачи и выполнять независимо. То же самое относится к отправке и получению сообщений. Например, можно использовать разные процессоры, которые работают с одними и теми же данными, но выполняют свои задачи в конкурентном режиме.

Как было показано выше, функциональность разных задач при декомпозиции задач весьма разнообразна, и в ней задействован широкий спектр операций. Следовательно, такую декомпозицию можно использовать только в системах с одним потоком команд и одним потоком данных (ОКОД) и со множественными потоками команд и одним потоком данных (МКОД).

Декомпозиция задач: паттерн «Конвейер»

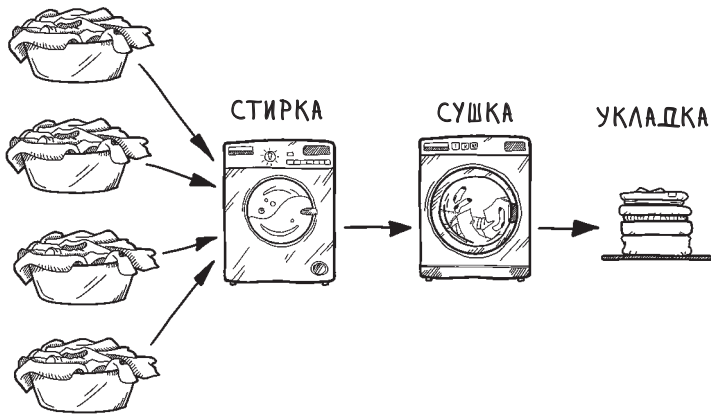
Самый распространенный паттерн в декомпозиции задач — так называемая *конвейерная обработка*. Ее суть заключается в том, чтобы разбить алгоритм на несколько последовательных этапов. Затем отдельные этапы можно распределить между разными ядрами. Каждое ядро можно сравнить с рабочим у конвейерной ленты; завершив свою работу, оно передает результат следующему ядру и получает новую порцию данных. Таким образом, ядра могут обрабатывать несколько блоков данных одновременно, начиная новые вычисления, пока другие все еще выполняются.



ПРИМЕЧАНИЕ Помните, как мы обсуждали бесконечный цикл выполнения на процессоре? Обработка одной команды включает ее выбор, декодирование, выполнение и сохранение результатов. На современных процессорах эти этапы устроены так, чтобы команды могли выполняться с использованием конвейерной обработки даже на таком низком уровне.

В главе 2 мы рассматривали пример со срочной стиркой. Давайте приблизим его к реальности. Кроме стирки, которая занимает некоторое время, вам нужно высушить вещи, а затем аккуратно их уложить — вам ведь не захочется прилететь на Гавайи в мятой одежде, верно?

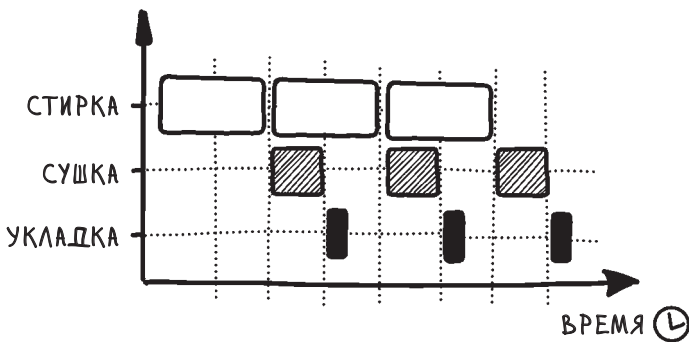
Без конвейерной обработки стирка, сушка и раскладка четырех загрузок с одной стиральной машиной и одной сушилкой выглядят примерно так:



При таком подходе ваши ресурсы (стиральная машина и сушилка) используются не в полной мере: в некоторые моменты они простаивают, пока выполняется какое-то другое действие.

Конвейерная обработка обеспечивает, чтобы стиральная машина и сушилка были загружены постоянно. Три стадии процесса делятся между тремя исполнителями: стиральная машина, сушилка и укладчик (скорее всего, это вы). Каждый исполнитель блокируется по общему ресурсу.

Первая загрузка готова к стирке и отправляется в стиральную машину. Когда вещи будут выстираны, вы достанете их из машины и передадите на следующую фазу конвейера — сушилку. Пока первая загрузка сушится, можно стирать вторую загрузку, потому что стиральная машина простаивает.



Конкурентность возникает тогда, когда вторая загрузка, проходя по конвейеру, начинает обрабатываться одновременно с первой. Если совместить по времени операции, которые раньше были разделены, обработка ощутимо ускорится.

ПРИМЕЧАНИЕ Один из самых популярных паттернов в области больших данных — ETL (Extract, Transform, Load, то есть «извлечение, преобразование, загрузка»). Это распространенный принцип сбора и обработки данных из разных источников, в котором реализуется паттерн конвейерной обработки. Используя средства ETL, мы *извлекаем* данные из источника (или нескольких источников) и *преобразовываем* их в структурированную информацию, которую затем *загружаем* в хранилище данных или другую целевую систему.

Чтобы реализовать подобную функциональность в коде, необходимы две составляющие: способ создавать независимо выполняемые задачи и механизм коммуникации задач друг с другом. В этом нам помогут потоки и очереди. Следующий код показывает, как это может выглядеть:

```
# Chapter 7/pipeline.py
import time
from queue import Queue
from threading import Thread
```

```
Washload = str
```

```
class Washer(Thread):
    def __init__(self, in_queue: Queue[Washload], out_queue:
Queue[Washload]):
        super().__init__()
        self.in_queue = in_queue
        self.out_queue = out_queue
```

```
    def run(self) -> None:
        while True:
```

```
            washload = self.in_queue.get()
```

```
            print(f"Стиральная машина: стирается {washload}...")
```

```
            time.sleep(4)
```

```
            self.out_queue.put(f'{washload}')
```

```
            self.in_queue.task_done()
```

Принимает загрузку
от предыдущей стадии

Моделирует
реальную работу

Передает загрузку
на следующую
стадию

```
class Dryer(Thread):
    def __init__(self, in_queue: Queue[Washload], out_queue:
Queue[Washload]):
        super().__init__()
        self.in_queue = in_queue
        self.out_queue = out_queue
```

```
    def run(self) -> None:
        while True:
```

```
            washload = self.in_queue.get()
```

```
            print(f"Сушилка: сушится {washload}...")
```

```
            time.sleep(2)
```

```
            self.out_queue.put(f'{washload}')
```

```
            self.in_queue.task_done()
```

Принимает загрузку
от предыдущей стадии

Моделирует
реальную работу

Передает загрузку
на следующую
стадию

```
class Folder(Thread):
    def __init__(self, in_queue: Queue[Washload]):
        super().__init__()
        self.in_queue = in_queue
```

```

def run(self) -> None:
    while True:
        washload = self.in_queue.get()
        print(f"Укладчик: укладывается {washload}...")
        time.sleep(1)
        print(f"Укладчик: {washload} готова!")
        self.in_queue.task_done()

class Pipeline:
    def assemble_laundry_for_washing(self) -> Queue[Washload]:
        washload_count = 8
        washloads_in: Queue[Washload] = Queue(washload_count)
        for washload_num in range(washload_count):
            washloads_in.put(f'Загрузка {washload_num}')
        return washloads_in

    def run_concurrently(self) -> None:
        to_be_washed = self.assemble_laundry_for_washing()
        to_be_dried: Queue[Washload] = Queue()
        to_be_folded: Queue[Washload] = Queue()

        Washer(to_be_washed, to_be_dried).start()
        Dryer(to_be_dried, to_be_folded).start()
        Folder(to_be_folded).start()
        to_be_washed.join()
        to_be_dried.join()
        to_be_folded.join()
        print("Все готово!")

if __name__ == "__main__":
    pipeline = Pipeline()
    pipeline.run_concurrently()

```

Принимает загрузку от предыдущей стадии

Моделирует реальную работу

Передает загрузку на следующую стадию

Собирает очередь загрузок для стирки и запускает потоки в правильном порядке, связывая их через очереди

Ожидает, пока все загрузки в очереди будут обработаны

Мы реализуем три основных класса: Washer (стиральная машина), Dryer (сушилка) и Folder (укладчик). В этой программе все наши функции выполняются в конкурентном режиме в разных потоках. Результат выглядит так:

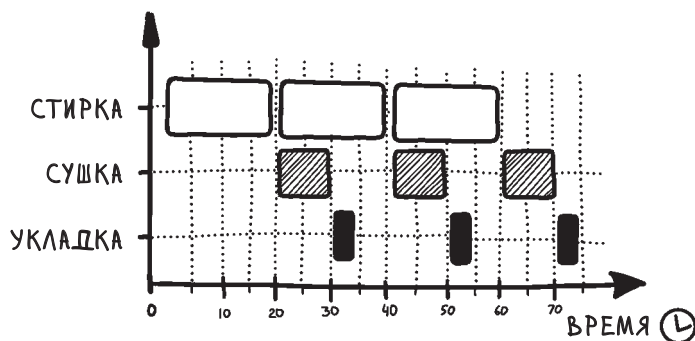
```

Стиральная машина: стирается Загрузка 0...
Стиральная машина: стирается Загрузка 1...
Сушилка: сушится Загрузка 0...
Укладчик: укладывается Загрузка 0...
Укладчик: Загрузка 0 готова!
Стиральная машина: стирается Загрузка 2...
Сушилка: сушится Загрузка 1...
Укладчик: укладывается Загрузка 1...
Укладчик: Загрузка 1 готова!
Стиральная машина: стирается Загрузка 3...
Сушилка: сушится Загрузка 2...
Укладчик: укладывается Загрузка 2...
Укладчик: Загрузка 2 готова!
Сушилка: сушится Загрузка 3...
Укладчик: укладывается Загрузка 3...
Укладчик: Загрузка 3 готова!
Все готово!

```

Поскольку по такой схеме можно одновременно обрабатывать несколько загрузок, с паттерном конвейерной обработки работа становится эффективнее, чем когда мы обрабатываем по одной загрузке за раз. Допустим, что стирка состоит из трех стадий, которые занимают 20, 10 и 5 минут соответственно. Таким образом, если выполнять все три фазы последовательно, то одна загрузка будет готова за 35 минут.

При конвейерной обработке первая загрузка будет готова за 35 минут, а каждая последующая — за 20 минут, потому что сразу же после того, как первая загрузка постирана, вторая переходит на стадию стирки, а первая в то же самое время — на стадию сушки. Таким образом, первая загрузка выйдет из конвейера через 35 минут после начала стирки, вторая — через 55 минут, третья — через 75 минут и т. д.



Может показаться, что конвейерную обработку можно успешно заменить простым параллелизмом. Но даже в нашем примере, чтобы обеспечить параллелизм, понадобятся четыре стиральные машины и четыре сушилки. На мой взгляд, это нереально — хотя бы из-за того, сколько будет стоить все оборудование и сколько места для него понадобится.

Конвейеры наиболее эффективны, когда общие ресурсы ограничены: конвейер позволяет ограничить количество потоков, которые необходимы для конкретной стадии (как в пуле потоков); если этого не сделать, потоки будут напрасно простаивать.

ПРИМЕЧАНИЕ Например, файловые системы обычно могут обслуживать ограниченное количество конкурентных запросов чтения/записи, при превышении которого возникает перегрузка. Отсюда следует верхний предел для количества потоков, что позволяет оптимизировать конкурентность на этом этапе.

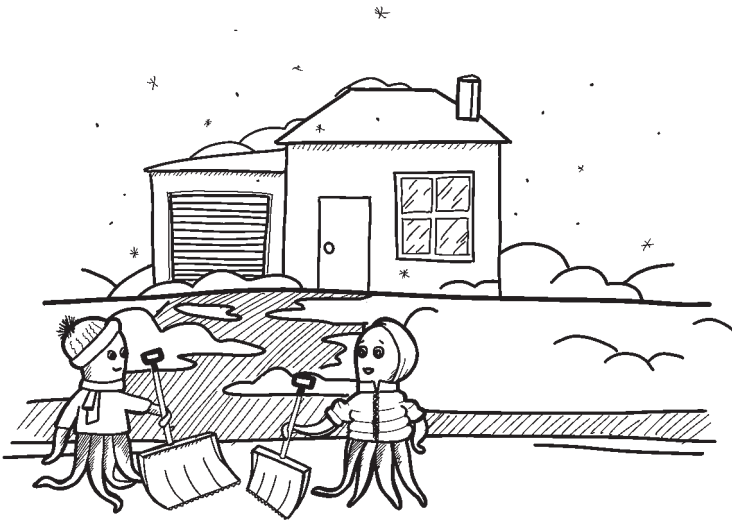
Конвейерную обработку часто совмещают с другими методами декомпозиции, например декомпозицией данных.

Декомпозиция данных

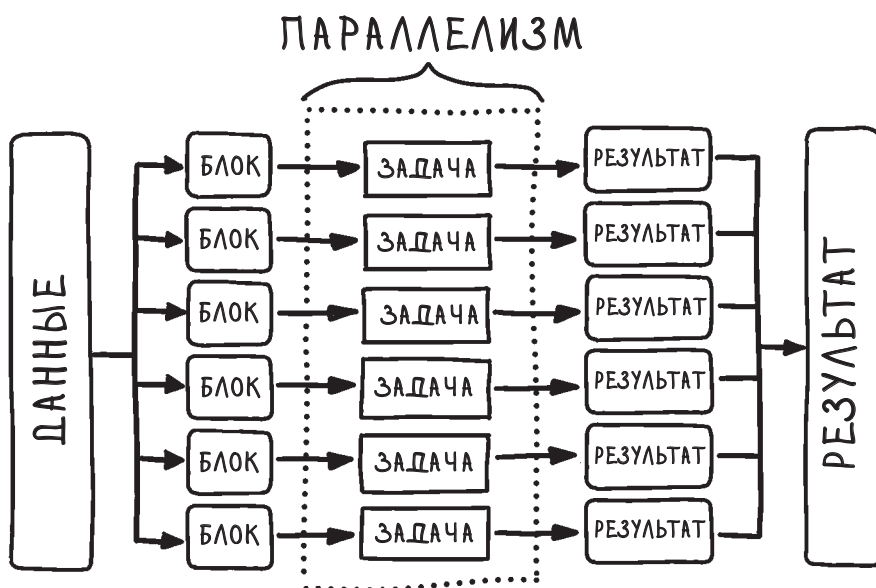
Другая часто встречающаяся модель конкурентного программирования — *декомпозиция данных* — позволяет разработчикам воспользоваться конкурентностью, которая возникает, когда одна операция применяется к разным элементам коллекции, например чтобы умножить все элементы массива на 2 или чтобы повысить налог для всех граждан, чей доход превышает определенный уровень. Каждая задача выполняет один и тот же набор команд, но со своим блоком данных.

Таким образом, декомпозиция данных отвечает на вопрос: «Как разбить данные задачи на блоки, которые можно обрабатывать независимо друг от друга?» То есть декомпозиция данных зависит от данных, а не от типа задачи.

Вернемся к примеру с уборкой снега. У вас была только одна лопата, и цель состояла в том, чтобы расчистить снег вокруг дома. Но если у вас будет две лопаты, то участок (данные) можно будет разделить на две зоны (блоки данных) и расчищать их параллельно, используя независимость операций с разными данными.



При декомпозиции данных они разбиваются на *блоки*. Поскольку каждую операцию с каждым блоком данных можно рассматривать как независимую задачу, полученная конкурентная программа состоит из последовательности таких операций. Мы уже использовали декомпозицию данных в главе 3 в примере с подбором паролей. Возможные пароли (данные) делились на независимые группы (части задачи), которые обрабатывались на разных вычислительных ресурсах.



ПРИМЕЧАНИЕ Хотя в этой главе рассматривается конкурентность на уровне приложения, декомпозиция данных больше зависит от фактического параллелизма на аппаратном уровне, потому что без него нет особого смысла применять этот метод.

В распределенной системе декомпозицию данных можно реализовать, если разделить работу между несколькими компьютерами или несколькими ядрами процессора на одном компьютере. Чтобы повысить производительность системы, независимо от объема входных данных всегда можно горизонтально масштабировать ресурсы, потому что специально настроенная распределенная система выполняет одинаковые действия на всех доступных вычислительных ресурсах одновременно. Это описание кажется вам знакомым? Так и есть: это напоминает архитектуру с одним потоком команд и множественными потоками данных (ОКМД), и она лучше всего подходит для такой категории задач.

Параллелизм на уровне циклов

Главные кандидаты для декомпозиции данных — программы с операцией, которую можно выполнять независимо с каждым блоком данных. В общем случае в эту категорию прекрасно вписываются всевозможные циклы (`for`, `while`, `for-each`), поэтому такой параллелизм часто называется *параллелизмом на уровне циклов*. Он часто позволяет извлекать из циклов конкурентные задачи. Некоторые компиляторы даже умеют применять его автоматически и преобразовывать последовательные части программы в семантически эквивалентный конкурентный код.

Представьте, что вам поручено создать приложение, которое ищет на компьютере файлы, содержащие определенное слово. Пользователь вводит путь к каталогу

и искомое слово, а программа выводит имена файлов, в которых оно есть. Как реализовать такую функциональность?

Если написать программу в простой последовательной форме без конкурентности, это будет обычный цикл `for`:

```
# Chapter 7/find_files/find_files_sequential.py
import os
import time
import typing as T

def search_file(file_location: str, search_string: str) -> bool:
    with open(file_location, "r", encoding="utf8") as file:
        return search_string in file.read()

def search_files_sequentially(file_locations: T.List[str],
                              search_string: str) -> None:
    result = search_file(file_name, search_string)
    if result:
        print(f"Слово найдено в файле: `{file_name}`")

if __name__ == "__main__":
    file_locations = list(
        glob.glob(f"{os.path.abspath(os.getcwd())}/books/*.txt"))
    search_string = input("Какое слово вы хотите найти?: ")

    start_time = time.perf_counter()
    search_files_sequentially(file_locations, search_string)
    process_time = time.perf_counter() - start_time
    print(f"ЗАТРАЧЕННОЕ ВРЕМЯ: {process_time}")
```

Создает список
расположений
файлов для поиска

Получает
от пользователя
слово для поиска

Чтобы использовать этот сценарий, введите по запросу каталог, в котором нужно искать, и искомое слово. Сценарий ищет слово во всех файлах заданного каталога и выводит имена файлов, где оно есть. Пример вывода:

```
Какое слово вы хотите найти?: варкалось
Слово найдено в файле: `Алиса в Зазеркалье.txt`
ЗАТРАЧЕННОЕ ВРЕМЯ: 0.75120013574
```

Присмотревшись к коду, мы видим, что на каждой итерации цикла `for` одни и те же действия выполняются с разными данными (файлами) независимо от других: не обязательно завершать обработку файла `N`, чтобы обрабатывать файл `N + 1`. Почему бы не разделить эти блоки данных и не обрабатывать их в разных потоках? Конечно, это можно сделать:

```
# Chapter 7/find_files/find_files_concurrent.py
import os
import time
import typing as T
from multiprocessing.pool import ThreadPool

def search_file(file_location: str, search_string: str) -> bool:
    with open(file_location, "r", encoding="utf8") as file:
```

```

        return search_string in file.read()

def search_files_concurrently(file_locations: T.List[str],
                             search_string: str) -> None:
    with ThreadPool() as pool:
        results = pool.starmap(search_file,
                               ((file_location, search_string) for
                                file_location in file_locations))
        for result, file_name in zip(results, file_locations):
            if result:
                print(f"Слово найдено в файле: `{file_name}`")

if __name__ == "__main__":
    file_locations = list(
        glob.glob(f"{os.path.abspath(os.getcwd())}/books/*.txt"))
    search_string = input("Какое слово вы хотите найти?: ")

    start_time = time.perf_counter()
    search_files_concurrently(file_locations, search_string)
    process_time = time.perf_counter() - start_time
    print(f"ЗАТРАЧЕННОЕ ВРЕМЯ: {process_time}")

```

Ищет одно
и то же слово
в каждом по-
токе в кон-
курентном
режиме

Этот код ищет заданное слово во всех файлах из заданного каталога и его подкаталогов, используя несколько потоков. Пример вывода:

```

В каком каталоге искать?: /Users/kirill/books/
Какое слово вы хотите найти?: варкалось
Слово найдено в файле: `Алиса в Зазеркалье.txt`
ЗАТРАЧЕННОЕ ВРЕМЯ: 0.04880058398703113

```

ПРИМЕЧАНИЕ В этом примере хотелось бы использовать все доступные ядра процессора, чтобы обрабатывать несколько файлов одновременно. Однако не забывайте, что чтение файлов с жесткого диска — это операция ввода/вывода, так что в начале выполнения данные еще не будут находиться в памяти. А значит, не факт, что мы получим обработанные блоки данных одновременно (даже с параллельным оборудованием). Тем не менее параллелизм на уровне циклов позволяет программе начать полезную работу, как только будет прочитан хотя бы один блок данных. Даже если система выполнения будет однопоточной, она все равно поспособствует многозадачности, при которой работа выполняется без лишних простоев.

В этом примере кода потоки выполняют одну и ту же работу, но на разных итерациях и, как следствие, с разными частями данных. Каждый из N потоков может работать с $1/N$ частей данных в конкурентном режиме.

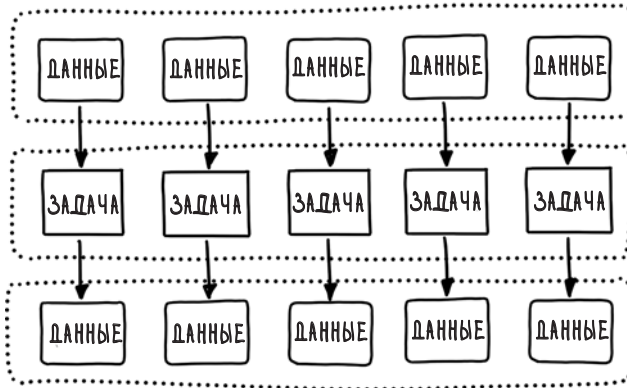
Паттерн «Отображение»

Мы только что реализовали новый паттерн программирования — *отображение* (map), который основан на принципах из функциональных языков программирования. Этот паттерн используется, когда одна операция применяется ко всем элементам коллекции, но каждая отдельная задача обрабатывается автономно и без

побочных эффектов (она не изменяет состояние программы, а только преобразовывает входные данные в выходные).

С помощью паттерна «Отображение» удобно решать тривиально параллелизируемые задачи — то есть такие, которые можно разложить на независимые подзадачи, не требующие коммуникации или синхронизации. Эти подзадачи выполняются в одном или нескольких процессах, потоках, на разных дорожках ОКМД или на нескольких компьютерах.

ПАТТЕРН «ОТОБРАЖЕНИЕ»



Циклы занимают значительную часть времени выполнения во многих программах, особенно в научных и аналитических системах, и могут принимать различные формы. Чтобы понять, соответствует ли ваша проблема этому паттерну, необходимо проанализировать ее на уровне исходного кода или близко к нему. Важно понимать, зависят ли одни итерации циклов от других, то есть используются ли данные предыдущих итераций на последующих.

ПРИМЕЧАНИЕ Параллелизм на уровне циклов используется во многих популярных библиотеках и фреймворках. OpenMP (Open Multi-Processing) применяет его для архитектур многоядерных процессоров. Библиотека CUDA от NVIDIA обеспечивает параллелизм на уровне циклов для архитектур графических процессоров. Паттерны отображения широко реализованы в большинстве современных языков программирования — Scala, Java, Kotlin, Python, Haskell и т. д.

Как видите, декомпозиция данных широко применяется на практике, но есть паттерн, который встречается еще чаще.

Паттерн «Ветвление/соединение»

К сожалению, в большинстве приложений есть как последовательные части (которые зависят друг от друга и должны выполняться в определенном порядке), так и конкурентные части (которые можно выполнять без определенного порядка и даже параллельно). В таких приложениях часто применяется еще один паттерн конкурентности.

Представьте, что вы отвечаете за подсчет голосов на местных выборах (пожалуйста, учтите, что это вымышленный сценарий, и не относитесь к нему слишком серьезно). Работа несложная: просто обработайте бюллетени и подсчитайте количество голосов за каждого кандидата.

Поскольку это ваши первые выборы, вы не слишком глубоко задумываетесь о том, как организовать процесс, и решаете проделать все самостоятельно, после того как закроются участки в день выборов. Вы последовательно перебираете стопки бюллетеней, одну за другой. На это уходит весь день, но вы справляетесь. Последовательное решение выглядит примерно так:

```
# Chapter 7/count_votes/count_votes_sequential.py
import typing as T
import random
```

```
Summary = T.Mapping[int, int]
```

```
def process_votes(pile: T.List[int]) -> Summary:
    summary = {}
    for vote in pile:
        if vote in summary:
            summary[vote] += 1
        else:
            summary[vote] = 1
    return summary
```

```
if __name__ == "__main__":
    num_candidates = 3
    num_voters = 100000
    pile = [random.randint(1, num_candidates) for _ in range(num_voters)]
    counts = process_votes(pile)
    print(f"Общее количество голосов: {counts}")
```

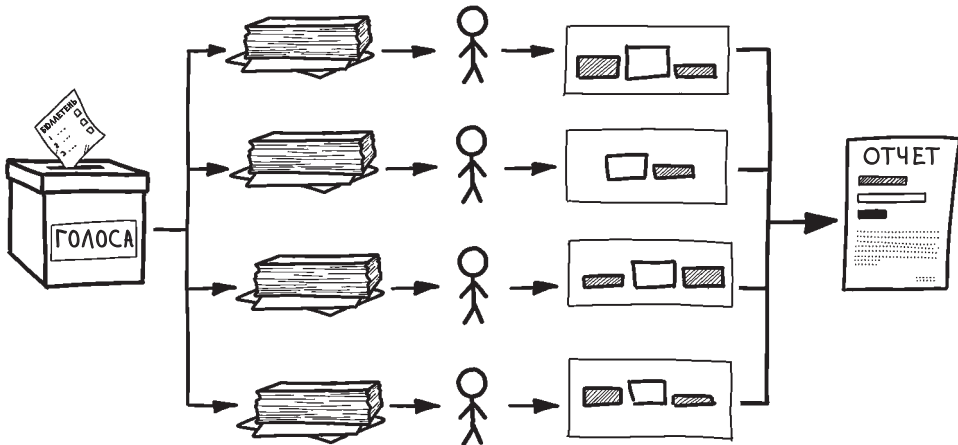
Генерирует огромную кучу голосов, поданных за трех кандидатов, где каждый голос — это целое число, представляющее того или иного кандидата

Функция принимает в качестве аргумента массив голосов, каждый элемент которого представляет голос, поданный за определенного кандидата. Функция возвращает ассоциативный массив с количеством голосов за каждого кандидата.

В результате успешно проведенной кампании вас повышают до организатора подсчета голосов — но уже не на местном уровне, а на выборах президента США! И когда к вам начинает поступать огромное количество голосов из разных штатов, вы понимаете, что тот же последовательный метод уже не сработает.

Как организовать процесс, чтобы гигантскую массу голосов можно было обработать за ограниченное время?

Очевидное решение — разделить голоса на несколько меньших частей и раздать эти части отдельным работникам, чтобы они обрабатывали голоса параллельно. Если распределить работу между многими людьми и даже группами людей, можно легко ускорить процесс. Впрочем, это еще не все. От вас требуется не ворох отчетов об отдельных частях голосов, а единый отчет с общим количеством голосов за каждого кандидата, так что информацию из отдельных отчетов нужно объединить. Поэтому вы решаете сначала разбить голоса и распределить работу между сотрудниками, а когда они ее закончат, самостоятельно объединить отдельные результаты.



Чтобы организовать параллельное выполнение, вы нанимаете дополнительных сотрудников, которые будут считать голоса. Допустим, таких сотрудников четверо. Тогда можно сделать так:

- Поручить первому сотруднику обработать первую четверть бюллетеней.
- Поручить второму сотруднику обработать вторую четверть бюллетеней.
- Поручить третьему сотруднику обработать третью четверть бюллетеней.
- Поручить четвертому сотруднику обработать четвертую четверть бюллетеней.
- Получить результаты от всех четырех сотрудников, объединить их и вернуть сводный отчет.

Первые четыре задачи можно выполнять параллельно, но последняя задача будет последовательной, потому что она зависит от результатов предыдущих шагов.

Прежде чем просматривать следующий код, подумайте, как бы вы сами решили эту задачу:

```
# Chapter 7/count_votes/count_votes_concurrent.py
import typing as T
import random
from multiprocessing.pool import ThreadPool
```

```
Summary = T.Mapping[int, int]
```

```

def process_votes(pile: T.List[int], worker_count: int = 4) -> Summary:
    vote_count = len(pile)
    vpw = vote_count // worker_count

    vote_piles = [
        pile[i * vpw:(i + 1) * vpw]
        for i in range(worker_count)
    ]

    with ThreadPool(worker_count) as pool:
        worker_summaries = pool.map(process_pile, vote_piles)

    total_summary = {}
    for worker_summary in worker_summaries:
        print(f"Голоса от сотрудника: {worker_summary}")
        for candidate, count in worker_summary.items():
            if candidate in total_summary:
                total_summary[candidate] += count
            else:
                total_summary[candidate] = count

    return total_summary

def process_pile(pile: T.List[int]) -> Summary:
    summary = {}
    for vote in pile:
        if vote in summary:
            summary[vote] += 1
        else:
            summary[vote] = 1
    return summary

if __name__ == "__main__":
    num_candidates = 3
    num_voters = 100000
    pile = [random.randint(1, num_candidates) for _ in range(num_voters)]
    counts = process_votes(pile)
    print(f"Общее количество голосов: {counts}")

```

Ветвление:
распределяет
голоса между
исполнителями
и запускает их
в конкурентном
режиме

Соединение:
объединяет сводные
данные исполнителей

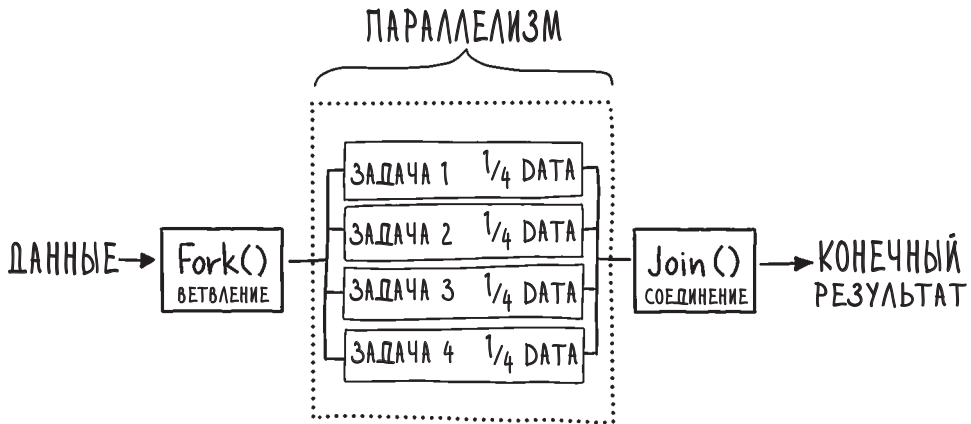
В этом примере используется популярный паттерн разработки конкурентных приложений — «Ветвление/соединение» (fork/join).

Идея заключается в следующем: мы разбиваем данные на меньшие блоки, которые обрабатываются как независимые задачи. В нашем примере меньшие части результатов голосования делятся между сотрудниками. Этот шаг называется *ветвлением* (fork). В параллелизме на уровне циклов его можно масштабировать горизонтально, если добавлять новые вычислительные ресурсы.

Следующий этап — объединить результаты отдельных задач, чтобы получить решение исходной высокоуровневой проблемы. В нашем примере нужно агрегировать голоса за каждого кандидата на основе сводных результатов каждого сотрудника. Считайте это своего рода точкой синхронизации: на этом шаге мы просто ожидаем,

пока завершатся все зависимые задачи, чтобы вычислить итоговый результат. Этот шаг называется *соединением* (join).

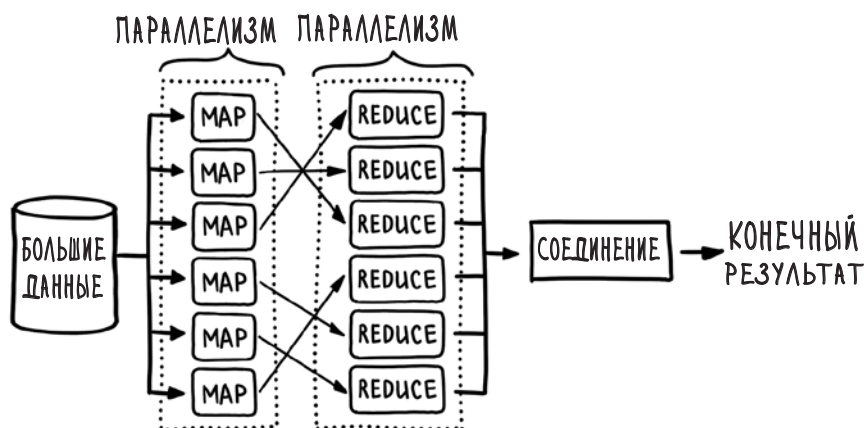
Если объединить два шага, получится паттерн «Ветвление/соединение». Как упоминалось ранее, это один из самых популярных паттернов в современном программировании; в таком стиле написаны многие конкурентные системы и библиотеки.



Паттерн «Отображение/свертка»

«Отображение/свертка» (map/reduce) — еще один паттерн конкурентности, который тесно связан с «Ветвлением/соединением». Идея стадии отображения та же, что у одноименного паттерна: одна и та же функция применяет отображение ко всем входным данным, чтобы получить новые результаты (например, «умножить на 2»). На стадии свертки происходит агрегирование (например, «сложить отдельные результаты» или «найти минимальное значение»). Отображение и свертка обычно выполняются последовательно, при этом отображение производит промежуточные результаты, которые обрабатываются на этапе свертки.

В паттерне «Отображение/свертка», как и в «Ветвлении/соединении», набор входных данных обрабатывается параллельно несколькими вычислительными ресурсами. Затем результаты объединяются, пока не получится единый итог. Хотя структура обоих паттернов одинакова, между ними есть методологические различия, которые проявляются в том, какую работу они выполняют. Фазы отображения (map) и свертки (reduce) более независимы, чем при стандартном ветвлении/соединении, потому что их можно масштабировать за пределы одного компьютера и использовать комплекс машин, чтобы выполнять одну и ту же операцию над большим объемом данных. Другое отличие от «Ветвления/соединения» заключается в том, что отображение иногда может выполняться без свертки и наоборот.



Это один из ключевых принципов, которые лежат в основе фреймворка MapReduce компании Google и версии Apache Hadoop с открытым кодом компании Yahoo. В этих системах разработчик просто задает операции, которые описывают, как отображать и свертывать данные. Система выполняет всю работу, часто используя сотни и тысячи компьютеров, чтобы обработать многие гигабайты и терабайты данных. От разработчика требуется лишь упаковать всю необходимую логику в вычислительные примитивы, которые предоставляет фреймворк, а система выполнения делает все остальное.

ПРИМЕЧАНИЕ Есть еще один популярный фреймворк, вдохновленный моделью MapReduce, — Apache Spark. Чтобы реализовать паттерн «Отображение/свертка», он использует функциональное программирование и конвейерную обработку. Вместо того чтобы записывать данные для каждого задания на диск, как это делает MapReduce, Spark умеет кэшировать результаты между заданиями. Более того, на базе Spark построено много разных систем, включая Spark SQL и DataFrames, GraphX и Streaming Spark, и все эти системы можно легко совмещать в одном приложении. Благодаря этим особенностям Spark становится оптимальным решением для итеративных заданий и интерактивной аналитики, а также обеспечивает более высокую производительность.

Декомпозиция данных и задач не исключают друг друга; их можно реализовать одновременно в одном и том же приложении. При этом приложения добиваются максимального выигрыша от применения конкурентности.

Уровень детализации

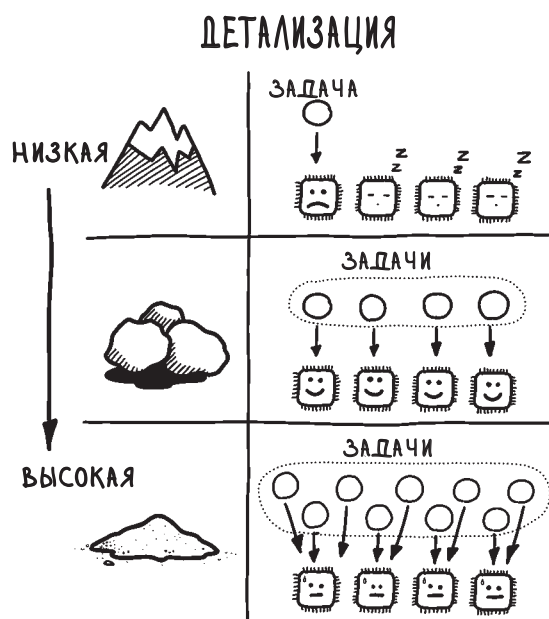
В предыдущем примере с голосованием мы сделали два довольно спорных допущения:

- Мы предположили, что в нашем распоряжении ровно четыре вычислительных ресурса (сотрудника) и все они получают примерно одинаковый объем работы. Однако не имеет смысла ограничивать количество ресурсов, которые занимаются обработкой. Хотелось бы, чтобы конкурентные приложения эффективно эксплуатировали все доступные им вычислительные ресурсы. Постоянно использовать ровно четыре потока — не лучшее решение. В этом случае, если запустить программу в системе с тремя ядрами, выполнение займет больше времени, чем если равномерно распределить задачи между тремя потоками на одном ядре. Наоборот, в системе с восемью ядрами четыре из них будут простаивать.
- Мы предположили, что каждый вычислительный ресурс находится в полном распоряжении нашего приложения во время выполнения. Однако это приложение — не единственное в системе, и некоторые вычислительные ресурсы могут понадобиться другим приложениям или самой системе.

Если отказаться от этих допущений, мы сталкиваемся с проблемой: как использовать все доступные ресурсы в системе, чтобы задачи выполнялись максимально эффективно? В идеале в результате декомпозиции должно получаться хотя бы столько же задач, сколько вычислительных ресурсов нам доступно, — а лучше еще больше, чтобы система выполнения могла работать более гибко.

Количество и размер задач, на которые разбивается проблема, определяет *уровень детализации (гранулярность)* декомпозиции. Этот показатель обычно оценивается по количеству команд, которые выполняются в конкретной задаче. Например, если в нашей предыдущей проблеме разделить работу на восемь потоков вместо четырех, то детализация повысится, и в данном случае программа станет более гибкой. Ее можно будет выполнять на большем количестве вычислительных ресурсов, если они будут доступны. Правда, если в системе доступны только четыре ядра, то не все потоки будут выполняться одновременно, потому что в любой момент времени ядро физически может выполнять только один поток. Но это нормально, потому что система выполнения следит за тем, какие потоки ожидают своей очереди, и обеспечивает, чтобы все ядра были заняты. Например, планировщик может решить, что первые четыре потока начнут выполняться параллельно, а когда они завершатся, то запустятся остальные четыре. Если в системе доступны восемь ядер, то она сможет выполнять все задачи параллельно.

Низкая степень детализации означает, что программа делится на большие задачи, и тогда на долю процессоров приходится большой объем вычислений. Это может разбалансировать нагрузку так, что одни задачи будут обрабатывать большую часть данных, а другие простаивать, что ограничит конкурентность в программе. Однако преимущество такой детализации состоит в том, что снижаются затраты на коммуникации и координацию.



При высокой степени детализации программа разбивается на множество мелких задач. Высокая детализация повышает степень параллелизма, а значит, и производительность системы, потому что задачи равномерно распределяются между несколькими процессорами и с каждой конкурентной задачей связан небольшой объем работы, так что задача выполняется очень быстро.

Впрочем, есть и минус в том, чтобы создавать много мелких задач: чем больше задач, которые должны взаимодействовать друг с другом, тем существеннее затраты на коммуникацию. Взаимодействующим задачам приходится прерывать вычисления, чтобы отправлять и принимать сообщения. Кроме коммуникационных затрат, также приходится учитывать затраты на создание задач. Как упоминалось ранее, создание потоков и процессов тоже расходует ресурсы. Если увеличить количество задач — например, до 1 000 000, — то нагрузка на планировщик ОС значительно повысится и производительность системы пострадает. Таким образом, оптимальная производительность достигается где-то между двумя крайностями — низкой и высокой степенью детализации.

Во многих алгоритмах, разработанных с декомпозицией задач, используется фиксированное количество задач одинакового размера, а также структурированные связи между задачами (как локальные, так и глобальные). В таких случаях эффективное отображение выполняется тривиально. Задачи отображаются на вычислительные ресурсы так, чтобы минимизировать коммуникацию между процессорами. Также можно объединять задачи, которые отображаются на один процессор (если это не было сделано ранее), чтобы получились задачи с низкой степенью детализации, по одной на процессор. Такая группировка задач называется *агломерацией* (подробнее об этом см. в главе 13).

В том, что касается декомпозиции данных, лучше определять как можно больше мелких задач. Это полезно, потому что заставляет разработчиков учитывать широкий спектр возможностей для параллельного выполнения. При необходимости задачи объединяются в большие задачи — такая агломерация помогает повысить производительность или уменьшить коммуникацию.

В более сложных алгоритмах, которые основаны на декомпозиции задач, с переменными рабочими нагрузками на задачу и/или неструктурированными схемами коммуникации, эффективные стратегии агломерации и распределения задач могут быть неочевидны для разработчиков. В этих случаях можно использовать алгоритмы балансирования нагрузки, которые позволяют выявлять эффективные стратегии агломерации и отображения, обычно с помощью эвристических методов.

Итоги

- Не существует волшебной формулы для декомпозиции проблем из области программирования. Один из методов, который может здесь помочь, заключается в том, чтобы наглядно представить зависимости задач в алгоритме, построив *граф зависимостей задач*, и найти по нему независимые задачи.
- Если приложение четко делится на функциональные компоненты, может быть полезно разложить его на функционально независимые задачи с помощью *декомпозиции задач*, а затем выполнять их в системах МКМД/МКОД. Декомпозиция задач отвечает на вопрос: «Как разложить проблему на задачи, которые можно выполнять в конкурентном режиме?»
- *Конвейерная обработка* — популярный паттерн декомпозиции задач, который может повысить пропускную способность системы, когда общие ресурсы ограничены. Его можно использовать совместно с другими методами декомпозиции.
- Если в приложении есть этапы, которые можно выполнять независимо друг от друга с разными блоками данных, можно применить *декомпозицию данных* и выполнять приложение на системах ОКМД. Декомпозиция данных отвечает на вопрос: «Как разбить данные задачи на блоки, которые можно обрабатывать относительно независимо друг от друга?»
- «*Отображение*», «*Ветвление/соединение*» и «*Отображение/свертка*» — популярные паттерны декомпозиции данных, которые широко используются во многих распространенных библиотеках и фреймворках.
- Количество и размер задач определяют *степень детализации* (гранулярность) системы. В идеале в результате декомпозиции должно получаться хотя бы столько же задач, сколько доступно вычислительных ресурсов, — а лучше еще больше, чтобы система выполнения могла работать более гибко.

8

Как решать проблемы конкурентности: ситуации гонки и синхронизация



В этой главе

- ✓ Как выявлять и решать одну из самых распространенных проблем конкурентности — ситуацию гонки
- ✓ Как безопасно и надежно совместно использовать ресурсы между задачами с помощью примитивов синхронизации

В последовательных программах код выполняется по благополучному сценарию предсказуемости и детерминизма; чтобы разобраться в том, что делает код, достаточно понять, как работает каждая функция с учетом текущего состояния программы. Но если программа конкурентная, то ее состояние меняется во время выполнения. Внешние обстоятельства — такие как планировщик ОС, согласование кэша или платформенные компиляторы, — могут влиять на то, в каком порядке выполняется программа и к каким ресурсам она обращается. Кроме того, конкурентные задачи часто состязаются друг с другом за доступ к одним и тем же ресурсам (процессору, общим переменным или файлам), и ОС не всегда удается это контролировать. Все это может влиять на результат программы.

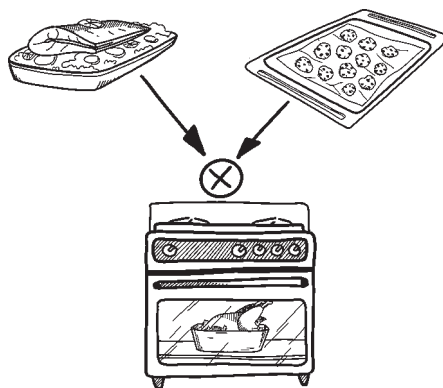
Важность контроля над конкурентным выполнением наглядно проявилась в мае 2012 года во время долгожданного первичного размещения акций (IPO) Facebook. Дефект в системе NASDAQ вызвал 30-минутную задержку. Ситуация гонки привела к хаотичным изменениям и отменам заявок, что принесло трейдерам значительные убытки. Слабые результаты первичного размещения бросили тень на роль NASDAQ в одном из самых крупных IPO в истории США и наглядно продемонстрировали,

что эффективно управлять конкурентностью критически необходимо (<http://mng.bz/Bmr2>).

Таким образом, нельзя просто положиться на то, что система выполнения будет управлять задачами и общими ресурсами вашей программы и координировать их, потому что подробные требования и последовательность выполнения программы могут быть неочевидными. В этой главе вы научитесь писать код, который позволяет синхронизировать доступ к общим ресурсам. Мы рассмотрим типичные проблемы конкурентности, а также обсудим их возможные решения и популярные паттерны конкурентности.

Общие ресурсы

Вернемся к примеру с куриным супом, который приводился ранее. Часто рецепт содержит шаги, которые можно выполнять одновременно, если на кухне несколько поваров. Но если духовка только одна, в ней не получится одновременно запечь индюшку и приготовить другое блюдо при другой температуре. В этом случае духовка является общим ресурсом.



Иными словами, если у нас несколько поваров, это может повысить эффективность, но при этом процесс приготовления тоже затрудняется, потому что между поварами необходима *коммуникация* и *координация*. В программировании все то же самое: ОС выполняет задачи в конкурентном режиме, и они тоже опираются на ограниченные ресурсы. Эти задачи работают независимо, часто не подозревая о существовании и действиях друг друга. Соответственно, если они пытаются использовать общие ресурсы во время выполнения, могут возникать конфликты. Чтобы их предотвратить, важно, чтобы каждая задача оставляла в неизменном виде состояние любых ресурсов, которые она потребляет. Представьте себе, например, что две задачи пытаются в конкурентном режиме обращаться к принтеру. Если не контролировать доступ к нему должным образом, может возникнуть ошибка, и приложение (или даже вся система) рискует оказаться в неизвестном и потенциально некорректном состоянии.

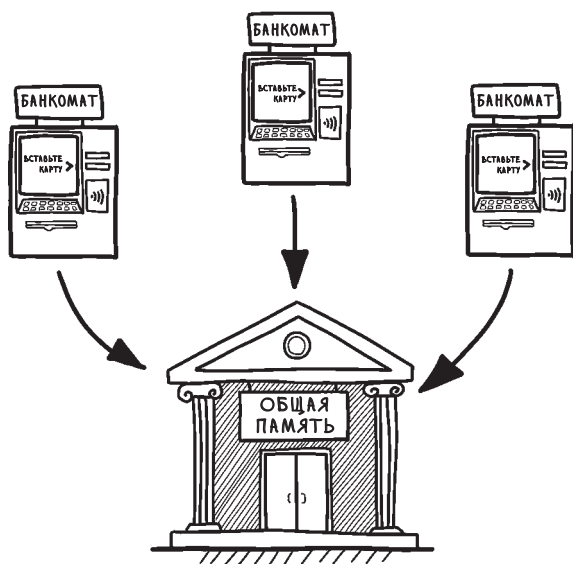
Функция или операция называется *потокowo безопасно*, если она корректно работает, когда к ней обращаются несколько задач, — независимо от того, как среда выполнения их планирует или чередует. Лучшее, что может сделать разработчик, чтобы обеспечить потоковую безопасность, — это качественно спроектировать приложение. Если избегать совместного использования ресурсов и минимизировать коммуникацию между задачами, то у них будет меньше шансов вмешиваться в работу друг друга. Тем не менее не всегда возможно создать приложение, которое не будет обращаться к общим ресурсам.

ПРИМЕЧАНИЕ Обеспечить потоковую безопасность относительно легко позволяют неизменяемые объекты и чистые функции. Поскольку они не могут изменять состояние, их нельзя повредить в результате взаимодействия между потоками или наблюдать в несогласованном состоянии. Язык программирования или приложение могут обеспечивать, чтобы данные не изменялись, если их используют несколько потоков. В этой книге соответствующие методы не рассматриваются.

Чтобы понять, что такое потоковая безопасность, для начала давайте разберемся, что такое небезопасный поток. Как обычно, начать лучше всего с примера.

Ситуация гонки

Представьте себе, что вы пишете банковское ПО, в котором каждый банковский счет представлен объектом. Разные задачи (например, кассиры или банкоматы) могут вносить средства или снимать их с одного и того же счета. Допустим, что все банкоматы банка пользуются общей памятью, благодаря чему могут выполнять операции чтения и записи с одними и теми же объектами счетов.



Допустим, в классе банковского счета есть методы для того, чтобы вносить и снимать средства:

```
# Chapter 8/race_condition/unsynced_bank_account.py
from bank_account import BankAccount

class UnsyncedBankAccount(BankAccount):
    def deposit(self, amount: float) -> None:
        if amount > 0:
            self.balance += amount
        else:
            raise ValueError("Нельзя внести нулевую или отрицательную сумму")

    def withdraw(self, amount: float) -> None:
        if 0 < amount <= self.balance:
            self.balance -= amount
        else:
            raise ValueError("На счете недостаточно средств")
```

Этот класс реализует банковский счет с внутренней переменной `balance`, которая представляет объем средств на счете, и двумя методами, `deposit()` и `withdraw()`, которые увеличивают и уменьшают `balance` соответственно.

Допустим, группа банкоматов выполняет одни и те же транзакции в конкурентном режиме, как обычно происходит в реальном мире. В программном коде это выглядит так:

```
# Chapter 8/race_condition/race_condition.py
import sys
import time
from threading import Thread
import typing as T

from bank_account import BankAccount
from unsynced_bank_account import UnsyncedBankAccount

THREAD_DELAY = 1e-16

class ATM(Thread):
    def __init__(self, bank_account: BankAccount):
        super().__init__()
        self.bank_account = bank_account

    def transaction(self) -> None:
        self.bank_account.deposit(10)
        time.sleep(0.001)
        self.bank_account.withdraw(10)

    def run(self) -> None:
        self.transaction()

def test_atms(account: BankAccount, atm_number: int = 1000) -> None:
    atms: T.List[ATM] = []
    for _ in range(atm_number):
        atm = ATM(account)
        atms.append(atm)
        atm.start()
```

Одна транзакция состоит из последовательного внесения и снятия средств с банковского счета

Создает группу потоков, изображающих банкоматы, которые выполняют транзакции со счетом в конкурентном режиме

```
for atm in atms:
    atm.join()
if __name__ == "__main__":
    atm_number = 1000
    sys.setswitchinterval(THREAD_DELAY)

    account = UnsynchronizedBankAccount()
    test_atms(account, atm_number=atm_number)

    print("Баланс несинхронизированного счета\
        после конкурентных транзакций:")
    print(f"Действительный: {account.balance}\nОжидаемый: 0")
```

Ожидает, пока потоки банкоматов закончат выполняться

Значительно повышает вероятность того, что операция будет прервана из-за переключения контекста, что позволяет эффективно протестировать синхронизацию

Мы реализовали банкомат как поток, который просто вызывает метод `deposit()`, за которым следует `withdraw()` с такой же суммой (допустим, 10 долларов). Мы запускаем 1000 потоков в конкурентном режиме. Таким образом, баланс счета должен остаться неизменным, потому что каждый раз вносится и снимается одинаковая сумма. В конце программы баланс должен быть равен нулю, верно?

Но если выполнить код, часто оказывается, что итоговый баланс отличен от нуля:

Баланс несинхронизированного счета после конкурентных транзакций:
Действительный: 380
Ожидаемый: 0

Как такое возможно?
Проанализируем, как методы разбиваются на низкоуровневые команды:

deposit()	withdraw()		Баланс
Получить баланс		←	0
Прибавить 10			0
Записать результат		→	10
	Получить баланс	←	10
	Вычесть 10		-10
	Записать результат	→	0

Допустим, два банкомата (обозначим их *A* и *B*) вносят средства на один банковский счет в конкурентном режиме. Во многих случаях конкурентное выполнение двух методов обходится без проблем.

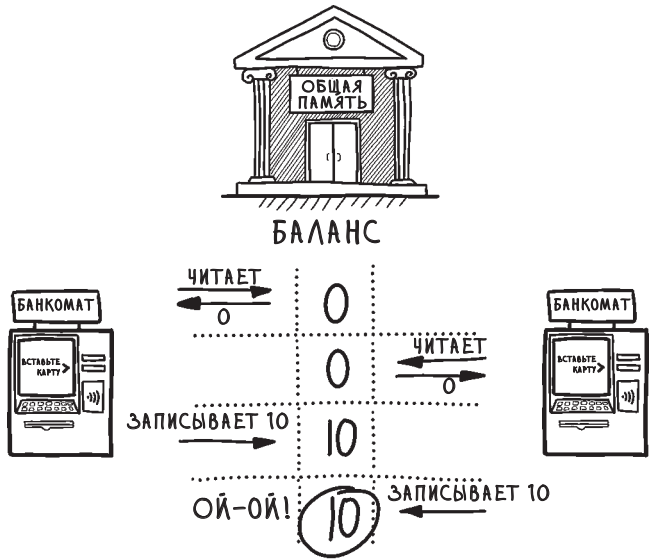
Метод deposit() банкомата A	Метод deposit() банкомата B		Баланс
Получить баланс		←	0
Прибавить 10			0
Записать результат		→	10
	Получить баланс	←	10
	Прибавить 10		10
	Записать результат	→	20

Все отлично: мы получили правильный баланс 20 долларов, а значит, банкоматы А и В выполнили свои транзакции корректно.

Но если А и В работают в конкурентном режиме, то их низкоуровневые команды могут чередоваться, и может произойти нечто такое:

Метод deposit() банкомата А	Метод deposit() банкомата В		Баланс
Получить баланс		←	0
	Получить баланс	←	0
Прибавить 10			
	Прибавить 10		
Записать результат		→	10
	Записать результат	→	10

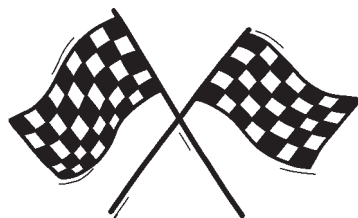
Здесь каждый из банкоматов, А и В, одновременно с другим читает баланс, вычисляет итоговый баланс, а затем сохраняет его новое значение, не учитывая операцию другого банкомата, — так что одно из внесений средств теряется. На балансе всего 10 долларов: одна из операций пропала!



Возможно, в этом случае два потока выполняются одновременно на разных ядрах процессора или планировщик ОС останавливает один поток и запускает другой в произвольные моменты времени, переключаясь между потоками несколько раз. Если несколько вызовов метода deposit() выполняются в конкурентном режиме, то баланс может оказаться в некорректном состоянии. Если один поток вносит

средства, а другой их снимает, то исключение, которое возбуждает снимающий поток, может зависеть от порядка операций.

Перед вами пример *ситуации гонки*. В таких ситуациях разные задачи обращаются к одним и тем же общим ресурсам или общим переменным в конкурентном режиме, и в результате правильность программы зависит от того, в какой последовательности выполняются конкурентные операции. Когда такое происходит, мы говорим, что «одна задача вступает в гонку с другой».



Ситуация гонки может возникать по многим причинам. Компиляторы обычно применяют разного рода оптимизацию, чтобы код выполнялся быстрее, но его семантика не изменялась. Если заставить компиляторы никогда не применять чередование и другие механизмы оптимизации кода, сложно будет добиться, чтобы компиляторы работали эффективно. Аналогичным образом на уровне аппаратного обеспечения не существует единой области общей памяти, которая содержала бы единственную копию всех данных программы. Вместо этого есть различные кэши и буферы, которые позволяют процессору обращаться к одним областям памяти быстрее, чем к другим, как было показано в главе 3. В результате оборудованию приходится отслеживать разные копии данных и перемещать их туда-сюда. При этом потоки могут «видеть» операции с памятью не в том порядке, в котором они происходят в программе. Как и в случае с компиляторами, заставлять оборудование выполнять все операции чтения и записи в том порядке, который предписан в коде, — это слишком обременительно с точки зрения производительности. Вся оптимизация и переупорядочение полностью скрыты от разработчиков, и вам никогда не придется беспокоиться о них, если удастся предотвратить гонку.

Ошибки, которые возникают из-за ситуации гонки, трудно воспроизвести и изолировать. Они относятся к так называемым «гейзенбагам»: ошибкам, которые исчезают или ведут себя по-другому, если пытаться их анализировать. Поскольку гонка — это семантическая ошибка, ее можно обнаружить только во время выполнения, и в ней трудно разобраться, если просто просматривать код, не запуская программу. К сожалению, не существует универсального способа обнаружить ситуацию гонки. Иногда потенциальную гонку удастся выявить, если разместить в разных местах кода операторы `sleep`, благодаря которым изменятся временные характеристики потоков и, как следствие, порядок их выполнения.

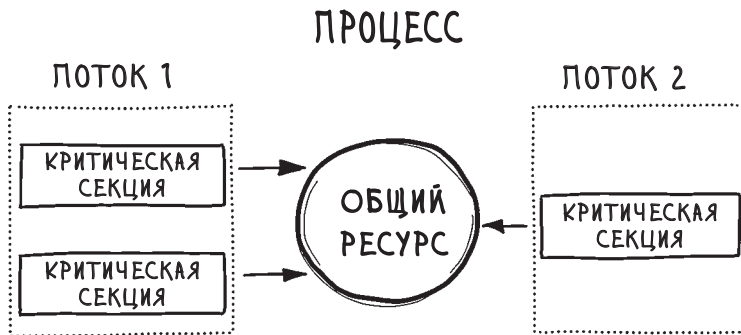
ПРИМЕЧАНИЕ Убедитесь, что библиотеки, которые вы используете, являются потоково безопасными; иначе вам придется синхронизировать библиотечные вызовы. Также учтите, что глобальные переменные в библиотеке могут создать проблемы, если код не рассчитан на то, чтобы поддерживать множественные конкурентные вызовы. В таких случаях, возможно, от библиотеки придется отказаться.

В результате нам понадобятся механизмы, которые обеспечивают потоковую безопасность: синхронизируют доступ и не позволяют разным задачам чередовать свои операции так, чтобы это приводило к некорректным результатам.

Синхронизация

Синхронизация — одно из возможных решений описанных проблем. Это механизм, который управляет доступом разных задач к общим ресурсам. Синхронизация особенно важна, когда нескольким задачам требуется доступ к ресурсам, к которым нельзя обращаться одновременно. Правильно налаженная синхронизация обеспечивает, чтобы задачи обращались к ресурсам монопольно и в нужном порядке. В главах 2 и 6 мы говорили о координации посредством синхронизации точек выполнения и ожидания зависимостей. С помощью синхронизации также можно защищать *критические секции* кода.

Критическая секция — фрагмент кода, который может выполняться одновременно несколькими задачами и обращаться к общим ресурсам. Например, в критической секции разработчик может манипулировать важными структурами данных или использовать ресурс, который поддерживает не более одного клиента одновременно (допустим, принтер).



Нельзя простодушно рассчитывать на то, что ОС поймет это ограничение и обеспечит, чтобы оно соблюдалось, потому что подробные требования могут быть неочевидными для планировщика ОС. Например, в случае принтера любой процесс должен полностью управлять принтером, пока он печатает весь файл, — иначе на бумаге получатся чередующиеся строки из конкурирующих процессов. В пределах критической секции должен быть какой-то механизм взаимного исключения, который позволял бы задачам выполнять операцию печати только в монопольном режиме.



У процессоров есть команды, с помощью которых можно реализовать синхронизацию. Они позволяют временно заблокировать прерывания в отдельных секциях кода, таким образом обеспечив, чтобы их выполнение не прерывалось. Эта возможность помогает защищать критические секции, которые нужно выполнять без прерываний. Хотя эти команды синхронизации особенно востребованы среди разработчиков компиляторов и ОС, они также абстрагируются как библиотечные функции в разных языках программирования. Чтобы обезопасить критические сегменты кода, программисты могут пользоваться этими функциями, не манипулируя напрямую нижележащими командами уровня процессора.

Чтобы управлять доступом к критическим секциям, применяется популярный примитив синхронизации, который называется *блокировкой* (lock). Существуют разные типы блокировок с разным поведением и семантикой.

Мьютекс

Блокировка¹ работает по простому принципу: перед началом операции задача вешает знак «Не беспокоить» на ресурс, с которым она работает, и не снимает (то есть удерживает) этот знак, пока операция не завершится. Все остальные задачи проверяют, висит ли знак, прежде чем пытаться повесить свой такой же и выполнять с ресурсом свои операции. Если знак уже есть, то задача блокируется и ожидает, пока он не будет снят. Тем самым гаранти-



¹ По историческим причинам термины *lock* и *blocking* в русскоязычной литературе переводятся похожими и иногда взаимозаменяемыми словами *блокировка* и *блокирование*, что может вызвать путаницу. В этой книге *lock* — это *блокировка* (программный объект, которым «запирается» другой объект), а *blocking* — это *блокирование* (ситуация, когда разные сеансы или другие объекты конкурируют за установку блокировки на один и тот же объект). Слова *блокировать*, *заблокированный*, *блокирующий* и другие однокоренные (кроме, собственно, слова *блокировка*) относятся к *blocking*, а не к *lock*. — Примеч. ред.

руется, что операцию будет выполнять только одна задача и операции не будут конфликтовать.

Мы только что определили еще одно состояние, в котором может находиться процесс или поток, — «Заблокирован». На следующей диаграмме изображен жизненный цикл потока (или процесса): от создания к готовности, выполнению, возможному блокированию и, наконец, завершению.



Чтобы получить возможность работать с общим ресурсом, задача сначала должна установить для него блокировку. Если другой поток ее уже установил, то первому потоку придется подождать ее снятия, прежде чем он сам сможет ее установить; до этого момента он находится в заблокированном состоянии. Эта схема называется *мьютексом* (mutex) (от «mutual exclusion», то есть «взаимное исключение»), потому что она обеспечивает, что в любой момент времени только у одной задачи есть монопольный доступ к общему ресурсу. Одноименная абстракция конкурентности присутствует во многих языках программирования и ОС.

Мьютекс может находиться только в двух состояниях: заблокированном и разблокированном. Прimitives создается в разблокированном состоянии и содержит два метода: `acquire()` и `release()`. Метод `acquire()` переводит мьютекс в заблокированное состояние и блокирует выполнение до тех пор, пока мьютекс не будет освобожден методом `release()`. Метод `release()` разблокирует мьютекс; его можно вызывать только в заблокированном состоянии. Когда вызывается `release()`, мьютекс переводится в разблокированное состояние, а управление немедленно возвращается вызывающему потоку.

Давайте воспользуемся мьютексом, чтобы решить нашу проблему доступа к банковским счетам. Чтобы мьютекс защищал внутреннюю переменную `balance`, блоки кода, которые с ней работают (критические секции нашей программы), нужно заключить между вызовами методов `acquire()` и `release()`:

```
# Chapter 8/race_condition/synced_bank_account.py
from threading import Lock
from unsynced_bank_account import UnsyncedBankAccount

class SyncedBankAccount(UnsyncedBankAccount):
    def __init__(self, balance: float = 0):
        super().__init__(balance)
```

```

self.mutex = Lock()

def deposit(self, amount: float) -> None:
    self.mutex.acquire()
    super().deposit(amount)
    self.mutex.release()

def withdraw(self, amount: float) -> None:
    self.mutex.acquire()
    super().withdraw(amount)
    self.mutex.release()

```

Освобождает мьютекс

Устанавливает мьютекс на общий ресурс, в результате чего может выполняться только один поток, который удерживает мьютекс

Здесь мьютекс добавляется к двум методам, чтобы в любой момент могла выполняться только одна операция каждого типа. Это избавляет нас от ситуаций гонки: методы `deposit()` и `withdraw()`, которые читают и записывают баланс, делают это под блокировкой. Если поток пытается установить блокировку, которую в это время удерживает другой поток, то первый поток блокируется, пока другой не снимет блокировку. Таким образом, в любой момент мьютекс может принадлежать не более чем одному потоку. А значит, невозможны одновременные операции чтения/записи или записи/записи, которые мы видели в прошлый раз:

Баланс несинхронизированного счета после конкурентных транзакций:
 Действительный: 0
 Ожидаемый: 0

Синхронизация эффективна только тогда, когда ее согласованно используют все потоки приложения. Если вы создаете мьютекс, чтобы ограничить доступ к общему ресурсу, то все потоки должны устанавливать тот же мьютекс, прежде чем пытаться манипулировать ресурсом. Если это не наладить, то мьютекс не будет обеспечивать надежной защиты, что может привести к ошибкам.

Семафоры

Семафор — еще один механизм синхронизации, который позволяет управлять доступом к общим ресурсам подобно мьютексу. Но, в отличие от мьютекса, семафор может разрешить нескольким задачам обращаться к ресурсу одновременно; таким образом, его могут устанавливать и освобождать несколько задач, тогда как мьютекс может блокировать и разблокировать только одна и та же задача.

На внутреннем уровне семафор поддерживает счетчик, который отслеживает, сколько раз он устанавливался и освобождался. Пока значение счетчика остается положительным, любая задача может установить семафор, тем самым уменьшив счетчик. Когда счетчик достигает нуля, задачи, которые пытаются захватить семафор, блокируются и ждут, пока он освободится (то есть когда счетчик станет положительным). Когда задача заканчивает работать с общим ресурсом, она освобождает семафор, увеличивая значение счетчика. Если при этом другие потоки ожидают,

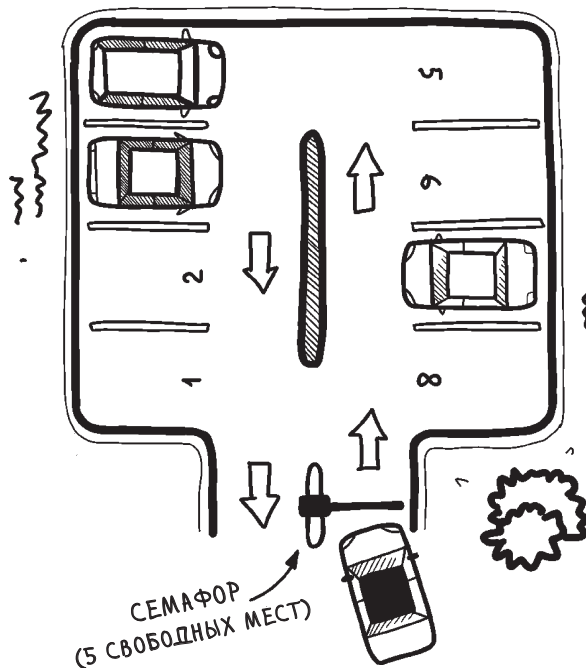
пока семафор освободится, они получают сигнал, который заставляет их выйти из заблокированного состояния и установить свой семафор.

По сути мьютекс можно рассматривать как особую разновидность семафора — так называемый *бинарный семафор*. В случае мьютекса у внутреннего счетчика есть только два допустимых значения: 0 или 1.

ПРИМЕЧАНИЕ Термин «семафор» предложил выдающийся ученый в области computer science Эдсгер Дейкстра в 1960-х годах. Так он называл примитив синхронизации, который позволял передавать сигналы между потоками. Исторически этот термин обозначал систему флагов и сигнальных фонарей, с помощью которых передавались сообщения между кораблями. Позднее Дейкстра признал, что слово «семафор» все-таки не лучшим образом подходило для этого примитива синхронизации, потому что он оказался более общим средством, которое можно было использовать не только для передачи сигналов.



Давайте с помощью семафора смоделируем общественную парковку с определенным количеством мест и двумя въездами, через которые автомобили заезжают на парковку и выезжают с нее. Машина не может заехать, если нет свободных мест, но всегда может выехать.



Чтобы заехать на парковку, автомобилю нужно получить парковочный талон — это аналогично установке семафора. Если есть свободные места, водитель берет талон, и счетчик семафора уменьшается. Но если парковка заполнена, то счетчик уменьшается до нуля, и другие автомобили не могут заехать. Следующая машина сможет установить семафор и заехать на парковку только после того, как семафор освободит одна из машин, которые в настоящее время его удерживают (обычно после того, как она покинет парковку). Код выглядит так:

```
# Chapter 8/semaphore.py
import typing as T
import time
import random
from threading import Thread, Semaphore, Lock

TOTAL_SPOTS = 5

class Garage:

    def __init__(self) -> None:
        self.semaphore = Semaphore(TOTAL_SPOTS)
        self.cars_lock = Lock()
        self.parked_cars: T.List[str] = []

    def count_parked_cars(self) -> int:
        return len(self.parked_cars)

    def enter(self, car_name: str) -> None:
        self.semaphore.acquire()
        self.cars_lock.acquire()
        self.parked_cars.append(car_name)
        print(f"{car_name} припаркован")
        self.cars_lock.release()

    def exit(self, car_name: str) -> None:
        self.cars_lock.acquire()
        self.parked_cars.remove(car_name)
        print(f"{car_name} выезжает")
        self.semaphore.release()
        self.cars_lock.release()
```

Семафор управляет ограниченным количеством свободных парковочных мест

Освобождает семафор, сигнализируя о том, что появилось свободное место

Обеспечивает, чтобы в любой момент времени только один поток мог изменять список припаркованных автомобилей

В этом коде используется и мьютекс, и семафор! Хотя эти конструкции похожи по своим свойствам, здесь они служат для разных целей. Мьютекс нужен, чтобы координировать доступ ко внутренней переменной `parked_cars` — списку припаркованных автомобилей. А с помощью семафора координируются методы парковки `enter()` и `exit()`, чтобы ограничивать количество машин в зависимости от количества свободных мест — в нашем случае их пять.

Если семафор недоступен (потому что его значение равно 0), автомобиль ждет, пока на парковке появится место и семафор освободится. Когда поток устанавливает семафор, он выводит сообщение о том, что автомобиль припаркован, после чего уходит в режим ожидания на случайный период времени. Затем поток выводит сообщение о том, что автомобиль покидает парковку, и освобождает семафор, увеличивая его значение, чтобы его мог установить другой ожидающий поток. Смоделируем напряженный день на парковке:

```
# Chapter 8/semaphore.py
def park_car(garage: Garage, car_name: str) -> None:
    garage.enter(car_name)
    time.sleep(random.uniform(1, 2))
    garage.exit(car_name)
```

Автомобиль встает на парковку, ожидает, а затем выезжает

```
def test_garage(garage: Garage, number_of_cars: int = 10) -> None:
    threads = []
    for car_num in range(number_of_cars):
        t = Thread(target=park_car,
                   args=(garage, f"Автомобиль {car_num}"))
        threads.append(t)
        t.start()

    for thread in threads:
        thread.join()
```

Создает несколько потоков, которые моделируют парковку нескольких автомобилей в конкурентном режиме

```
if __name__ == "__main__":
    number_of_cars = 10
    garage = Garage()
    test_garage(garage, number_of_cars)
```

Моделирует напряженный день на парковке; порождает потоки, которые представляют автомобили, заезжающие и выезжающие с парковки

```
print("Количество припаркованных автомобилей в напряженный день:")
print(f"Действительное: {garage.count_parked_cars()}\nОжидаемое: 0")
```

Как и в случае с мьютексом, мы получаем ожидаемые результаты:

```
Автомобиль 0 припаркован
Автомобиль 1 припаркован
Автомобиль 2 припаркован
Автомобиль 0 выезжает
Автомобиль 3 припаркован
Автомобиль 1 выезжает
Автомобиль 4 припаркован
Автомобиль 2 выезжает
Автомобиль 5 припаркован
Автомобиль 4 выезжает
Автомобиль 6 припаркован
Автомобиль 5 выезжает
Автомобиль 7 припаркован
Автомобиль 3 выезжает
Автомобиль 8 припаркован
Автомобиль 7 выезжает
Автомобиль 9 припаркован
Автомобиль 6 выезжает
Автомобиль 8 выезжает
```

Автомобиль 9 выезжает

Количество припаркованных автомобилей в напряженный день:

Действительное: 0

Ожидаемое: 0

Чтобы решать проблемы синхронизации, также можно реализовывать более мощные операции, которые выполняются за один шаг и таким образом устраняют возможность нежелательных прерываний. Такие операции называются *атомарными*.

Атомарные операции

Атомарные операции — это простейшая форма синхронизации, которая работает с примитивными типами данных. *Атомарность* означает, что никакой другой поток не сможет увидеть операцию в частично завершенном состоянии.

Для некоторых простых действий (таких как увеличение переменной-счетчика на единицу) атомарные операции могут обеспечить значительный выигрыш по производительности по сравнению с традиционными механизмами блокировки. Вместо того чтобы устанавливать блокировку, изменять переменную и снимать блокировку, атомарные операции предоставляют более элегантное решение. Рассмотрим пример на языке ассемблера:

```
add 0x9082a1b, $0x1
```

Здесь ассемблерная команда прибавляет значение 1 к содержимому памяти по адресу 0x9082a1b. Оборудование гарантирует, что эта операция будет выполнена атомарно без каких-либо прерываний. Если происходит прерывание, то операция либо не выполняется вообще, либо выполняется вся целиком; промежуточного состояния не существует.

Преимущество атомарных операций заключается в том, что они не блокируют конкурирующие задачи. Теоретически это может повысить уровень конкурентности и свести к минимуму затраты на синхронизацию. Но эти операции зависят от специальных аппаратных команд, и если между аппаратной и программной частью налажено эффективное взаимодействие, то гарантии атомарности на аппаратном уровне могут расширяться до программного уровня.

ПРИМЕЧАНИЕ Многие языки программирования предоставляют атомарные структуры данных, но будьте осторожны, потому что не все структуры данных атомарны. Например, некоторые коллекции Java являются потоково безопасными; кроме того, в Java существуют такие неблокирующие атомарные структуры данных, как `AtomicBoolean`, `AtomicInteger`, `AtomicLong` и `AtomicReference`. А в стандартную библиотеку C++ входят такие атомарные типы, как `std::atomic_int` и `std::atomic_bool`.

Впрочем, не все операции атомарны, так что на атомарность не стоит чересчур полагаться. Разрабатывая конкурентные приложения, принято делать вид, что мы не знаем ничего, кроме того, что сказано в стандартах языка программирования. Если атомарные операции недоступны, применяйте блокировки.

Вооружившись тем, что вы узнали о синхронизации, в следующей главе рассмотрим некоторые ее распространенные проблемы.

Итоги

- Используя общие ресурсы, как это часто происходит в конкурентных программах, будьте осторожны и избегайте конкурентного доступа к общим ресурсам, потому что любая задача может прерваться на середине выполнения. Такие прерывания могут привести к непредвиденным эффектам и коварным ошибкам, которые порой проявляются лишь много позднее.
- *Критическая секция* кода может выполняться несколькими задачами в конкурентном режиме и обращаться к общим ресурсам. Чтобы гарантировать, что критические секции будут выполняться в монопольном режиме, нужен механизм синхронизации.
- Простейший способ предотвратить непредвиденные эффекты в критической секции — использовать атомарные операции. *Атомарность* означает, что никакой другой поток не увидит операцию в частично завершенном состоянии. Однако такие операции зависят от окружения (то есть поддержки со стороны оборудования и среды выполнения).
- Другой, наиболее распространенный метод синхронизации — использовать *блокировки*. Блокировка — это абстрактное понятие. Предполагается, что она защищает доступ к общему ресурсу. Если вы удерживаете блокировку, то можете обращаться к защищенному общему ресурсу, а если не удерживаете, то не можете.
- Если задачи требуют взаимоисключающих операций, их можно защитить с помощью *мьютексов* (взаимоисключающих блокировок), которые не позволят читать общие данные в одной задаче, а обновлять их в другой.
- *Семафор* — блокировка, которая позволяет управлять доступом к общим ресурсам подобно мьютексу. Но, в отличие от мьютексов, семафор допускает, чтобы несколько задач обращались к ресурсу одновременно; следовательно, его могут устанавливать и освобождать несколько задач, тогда как мьютексом в каждый момент времени манипулирует только одна задача.
- Синхронизация обходится дорого. Поэтому по возможности старайтесь проектировать код так, чтобы ее не требовалось.
- Когда две задачи обращаются к общему ресурсу, манипулируя им в конкурентном режиме, и результат выполнения зависит от порядка обращений, это называется *ситуацией гонки* (один поток вступает в гонку с другим). Чтобы предотвратить такую ситуацию, следует правильно синхронизировать потоки в критических секциях с помощью таких методов, как блокировки или атомарные операции, а также можно прибегнуть к межпроцессной коммуникации с передачей сообщений.

9

Как решать проблемы конкурентности: взаимные блокировки и нехватка ресурсов



В этой главе

- ✓ Как выявлять и решать распространенные проблемы конкурентности: взаимные блокировки, активные блокировки и нехватку ресурсов
- ✓ Популярные паттерны конкурентного проектирования: «Производитель — потребитель» и «Читатели — записыватель».

В предыдущей главе мы говорили о том, какие трудности возникают в конкурентном программировании (например, ситуации гонки) и какие примитивы синхронизации помогают их преодолеть. В этой главе речь пойдет о другой группе распространенных проблем конкурентности: взаимных блокировках, активных блокировках и нехватке ресурсов.

Эти проблемы могут привести к чрезвычайно серьезным последствиям, особенно если учесть, что конкурентность применяется в различных технологиях, которым мы буквально доверяем свои жизни. Два крушения Boeing 737 Max в 2018 и 2019 годах произошли из-за ошибок конкурентного выполнения. Система улучшения характеристик маневренности (MCAS) на этих самолетах предназначалась для того, чтобы предотвратить сваливание, но из-за ситуации гонки она сработала некорректно, что привело к крушениям, в которых погибло 347 человек. Десятилетием ранее, в 2009 и 2010 годах, в автомобилях компании Toyota стали возникать внезапные неконтролируемые ускорения из-за ошибки ПО, которая создавала проблему конкурентности в системе электронного управления дроссельной заслонкой. В результате ошибки заслонка неожиданно открывалась, что привело к нескольким авариям с человеческими жертвами.

В этой главе вы узнаете, как выявлять и решать типичные проблемы конкурентности, а также овладеете инструментами, которые для этого необходимы. К концу главы вы будете глубоко понимать типичные проблемы и популярные паттерны конкурентности, включая паттерны «Производитель — потребитель» (producer-consumer) или «Читатели — записыватель» (readers-writer), которые помогут реализовывать подходящие решения, чтобы предотвращать потенциальные катастрофы.

Обедающие философы

Блокировки (мьютексы и семафоры) весьма сложны в использовании. Если применять их некорректно, работа приложения может нарушиться оттого, что блокировки устанавливаются, но не освобождаются, или блокировки, которые нужно установить, оказываются недоступными. Проблему синхронизации в ситуациях, когда несколько задач конкурируют за блокировки, можно проиллюстрировать с помощью задачи об обедающих философах, которую сформулировал Эдсгер Дейкстра в 1965 году. Этот пример часто служит стандартным тестовым сценарием, с помощью которого оцениваются методы синхронизации.

Пять молчаливых философов сидят за круглым столом перед блюдом с пельменями. Между каждой парой ближайших философов лежит палочка. Философы занимаются тем, что у них обычно получается лучше всего: они размышляют и едят.

Каждую палочку может держать только один философ, так что никто из них не может использовать палочку, которую в данный момент держит кто-то другой. После того как философ закончит есть, он должен положить обе палочки, чтобы их могли взять соседи. Философ может брать только те палочки, которые лежат справа и слева от него, может делать это только тогда, когда палочки доступны, и не может начать есть, пока не возьмет обе палочки.



Требуется разработать ритуал (алгоритм), чтобы каждый философ мог переключаться между двумя состояниями: он либо ест, либо размышляет. Предполагается, что ни один философ не знает, когда другие захотят есть или размышлять, — таким образом, у нас получается конкурентная система.

Взятие пельменей с блюда является критической секцией, и чтобы ее защитить, можно разработать процесс взаимного исключения, используя две палочки как мьютексы. Таким образом, когда философ хочет взять пельмень, он сначала берет палочку слева, если она доступна, и устанавливает для нее блокировку. Затем он берет палочку справа, если она доступна, и тоже устанавливает для нее блокировку. Теперь философ держит две палочки: значит, он находится в критической секции и может съесть пельмень. После этого он кладет на стол правую палочку, чтобы снять блокировку, а затем кладет левую палочку. Наконец, будучи философом, он возвращается к философским размышлениям.

В коде этот процесс выглядит так:

```
# Chapter 9/deadlock/deadlock.py
import time
from threading import Thread

from lock_with_name import LockWithName

dumplings = 20

class Philosopher(Thread):
    def __init__(self, name: str, left_chopstick: LockWithName,
                 right_chopstick: LockWithName):
        super().__init__()
        self.name = name
        self.left_chopstick = left_chopstick
        self.right_chopstick = right_chopstick

    def run(self) -> None:
        global dumplings

        while dumplings > 0:
            self.left_chopstick.acquire()
            print(f"{self.left_chopstick.name} взята: {self.name}, "
                  f"теперь ему нужна {self.right_chopstick.name}")
            self.right_chopstick.acquire()
            print(f"{self.right_chopstick.name} взята: {self.name}")
            dumplings -= 1
            print(f"Философ {self.name} съел пельмень. "
                  f"Осталось пельменей: {dumplings}")
            self.right_chopstick.release()
            print(f"{self.right_chopstick.name} положена: {self.name}")
            self.left_chopstick.release()
            print(f"{self.left_chopstick.name} положена: {self.name}")
            print(f"Философ {self.name} размышляет...")
            time.sleep(0.1)
```

С каждым философом связаны две палочки: одна лежит слева от него, другая — справа

Обед продолжается до тех пор, пока пельмени не кончатся

Берет левую палочку

Берет правую палочку

Пельмень съеден

Освобождает правую палочку

Освобождает левую палочку

В этом коде поток `Philosopher` представляет одного философа. Он содержит его имя и два мьютекса, `left_chopstick` и `right_chopstick`, которые философ захватывает в указанном порядке.

Общая переменная `dumplings` представляет оставшиеся пельмени на общем блюде. В цикле `while` философы берут пельмени, пока они остаются на блюде. В цикле философ устанавливает блокировку сначала для левой, а потом для правой палочки. Затем, если на блюде еще остались пельмени, философ берет один из них, уменьшая переменную `dumplings`, и программа выводит количество оставшихся пельменей.

Философы переключаются между едой и размышлениями. Но поскольку они ведут себя как конкурентные задачи, ни один из них не знает, когда другие захотят есть или размышлять, что может вызвать затруднения. Давайте посмотрим, какие проблемы могут возникнуть при выполнении этого кода и как их можно решить.

Взаимные блокировки

Чтобы упростить объяснения в этом разделе, уменьшим количество философов до двух, сохранив исходный алгоритм:

```
# Chapter 9/deadlock/deadlock.py
if __name__ == "__main__":
    chopstick_a = LockWithName("Палочка А")
    chopstick_b = LockWithName("Палочка В")

    philosopher_1 = Philosopher("Философ 1", chopstick_a,
    chopstick_b)
    philosopher_2 = Philosopher("Философ 2", chopstick_b,
    chopstick_a)

    philosopher_1.start()
    philosopher_2.start()
```

Результат запуска программы выглядит примерно так:

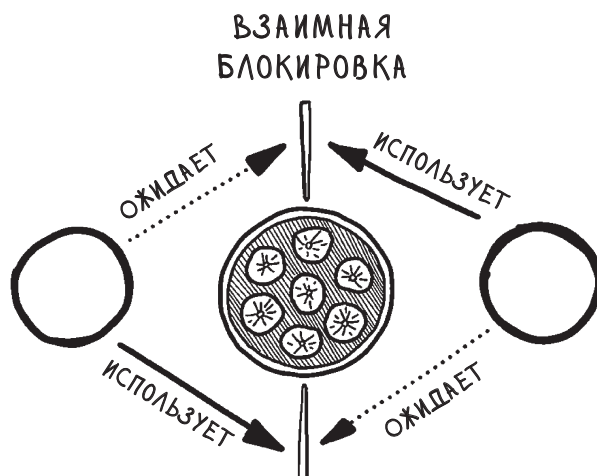
```
Философ 1 ест пельмень. Осталось пельменей: 19
Философ 1 ест пельмень. Осталось пельменей: 18
Философ 2 ест пельмень. Осталось пельменей: 17
...
Философ 2 ест пельмень. Осталось пельменей: 9
```

Программа не завершается: она застревает на одном месте, а на блюде все еще остаются пельмени. Что происходит?

Допустим, первый философ проголодался, и он берет палочку А. В то же время второй философ тоже хочет перекусить и берет палочку В. Каждый поток удерживает одну из двух блокировок, которые ему нужны, но оба бесконечно ожидают, пока другой поток освободит оставшуюся блокировку.

Перед вами пример так называемой *взаимной блокировки* (deadlock). Эта ситуация заключается в том, что несколько задач ожидают ресурсов, которые заняты другими задачами, и ни одна из них не может продолжить выполнение. Программа навечно застревает в этом состоянии, поэтому ее приходится прерывать вручную.

Если запустить программу еще раз, она приведет к взаимной блокировке после другого количества пельменей — точное значение каждый раз зависит от того, как система планирует задачи.

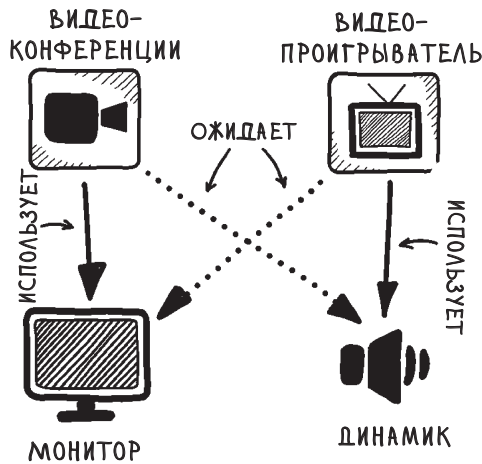


Как и в случае с ситуацией гонки, вам может повезти, и вы никогда не столкнетесь с этой проблемой в своих приложениях. Но даже если взаимная блокировка может возникнуть хотя бы теоретически, ее стоит предотвратить. Опасность взаимной блокировки возникает каждый раз, когда задача пытается получить сразу несколько блокировок. Как избежать взаимных блокировок — типичная проблема в конкурентных программах, где критические секции кода защищаются с помощью мьютексов.

ПРИМЕЧАНИЕ Никогда не рассчитывайте на конкретный порядок выполнения. Если в программе выполняются несколько потоков, как это было в предыдущих примерах, то порядок их выполнения не детерминирован. Если для вас важно, в каком порядке выполняются потоки относительно друг друга, необходимо применить синхронизацию. Но если вам нужна оптимальная производительность, следует избегать синхронизации, насколько это возможно. В частности, стоит создавать задачи с высокой детализацией, которым не нужна синхронизация; это позволит ядрам обрабатывать каждую назначенную им задачу как можно быстрее.

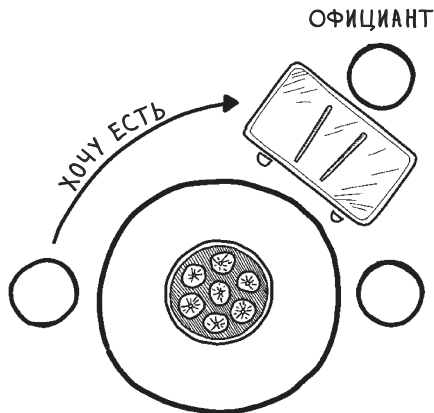
Но давайте рассмотрим более правдоподобный пример — все-таки не каждый день приходится угощать философов пельменями. Представьте себе реальную систему: ваш домашний компьютер, на котором установлены два приложения — одно для видеоконференций (например, Zoom или Skype), а другое — для просмотра фильмов (например, Netflix или YouTube). Приложения предназначены для разных целей — одно позволяет общаться с коллегами или друзьями, а другое — смотреть любимое кино, но оба обращаются к одним и тем же подсистемам компьютера — в частности к экрану и аудиоустройству. Допустим, обеим программам понадобился доступ к экрану и звуку. Они выдают запросы одновременно, и ОС передает экран проигры-

вателю, а звук — видеочату. Обе программы блокируют полученный ресурс, а затем ожидают, пока освободится другой ресурс. И они будут ждать вечно, как и бедные философы! Взаимная блокировка не прекратится, если только ОС не предпримет радикальные меры — например, уничтожит один или несколько процессов или заставит их снять блокировку.



Решение с арбитром

Вернемся к задаче с философами. Чтобы предотвратить взаимную блокировку, можно предусмотреть, чтобы каждый философ мог взять либо две палочки, либо ни одной. Для этого проще всего добавить в систему *арбитра*, который отвечает за палочки (например, официанта). Чтобы взять палочку, философ сначала спрашивает разрешения у официанта. Официант дает разрешение только одному философу за раз, пока тот не возьмет обе палочки. В любой момент философ может вернуть палочку обратно.



Функции официанта можно реализовать с помощью еще одной блокировки:

```
# Chapter 9/deadlock/deadlock_arbitrator.py
import time
from threading import Thread, Lock
```

```
from lock_with_name import LockWithName
```

```
dumplings = 20
```

```
class Waiter:
```

```
    def __init__(self) -> None:
        self.mutex = Lock()
```

```
    def ask_for_chopsticks(self, left_chopstick: LockWithName,
                          right_chopstick: LockWithName) -> None:
```

```
        with self.mutex:
            left_chopstick.acquire()
            print(f"{left_chopstick.name} взята")
            right_chopstick.acquire()
            print(f"{right_chopstick.name} взята")
```

Внутренний мьютекс, который защищает критическую секцию и обеспечивает обращение к ней только одного потока одновременно

```
    def release_chopsticks(self, left_chopstick: LockWithName,
                          right_chopstick: LockWithName) -> None:
```

```
        right_chopstick.release()
        print(f"{right_chopstick.name} освободилась")
        left_chopstick.release()
        print(f"{left_chopstick.name} освободилась\n")
```

Теперь официант отвечает за то, чтобы управлять раздачей и освобождением палочек

И официант может выступать в качестве блокировки:

```
# Chapter 9/deadlock/deadlock_arbitrator.py
```

```
class Philosopher(Thread):
```

```
    def __init__(self, name: str, waiter: Waiter,
                 left_chopstick: LockWithName,
                 right_chopstick: LockWithName):
        super().__init__()
        self.name = name
        self.left_chopstick = left_chopstick
        self.right_chopstick = right_chopstick
        self.waiter = waiter
```

```
    def run(self) -> None:
        global dumplings
```

```
        while dumplings > 0:
            print(f"{self.name} просит палочки у официанта")
            self.waiter.ask_for_chopsticks(
                self.left_chopstick, self.right_chopstick)
```

Философ запрашивает палочки у официанта

```

        dumplings -= 1
        print(f"{self.name} съел пельмень. "
              f"Осталось пельменей: {dumplings}")
        print(f"{self.name} возвращает палочки официанту")
        self.waiter.release_chopsticks(
            self.left_chopstick, self.right_chopstick)
        time.sleep(0.1)
if __name__ == "__main__":
    chopstick_a = LockWithName("Палочка A")
    chopstick_b = LockWithName("Палочка B")

    waiter = Waiter()
    philosopher_1 = Philosopher("Философ 1", waiter, chopstick_a,
                                chopstick_b)
    philosopher_2 = Philosopher("Философ 2", waiter, chopstick_b,
                                chopstick_a)

    philosopher_1.start()
    philosopher_2.start()

```

Философ возвращает палочки официанту после еды

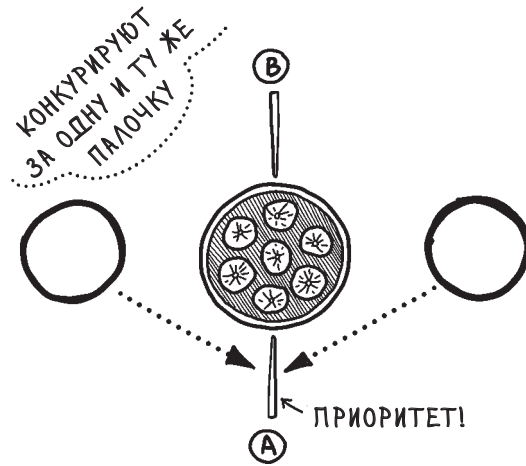
Поскольку это решение вводит новую центральную сущность (официанта), оно может ограничить конкурентность: если философ ест, а один из его соседей запрашивает палочки, то всем остальным философам придется ждать, пока этот запрос будет выполнен, даже если палочки им доступны. В реальной компьютерной системе арбитр делает практически то же самое, упорядочивая доступ со стороны потоков-исполнителей. Такой подход снижает уровень конкурентности, и это не самый эффективный вариант.

Решение с иерархией ресурсов

А что если установить приоритеты для блокировок, чтобы философы сначала пытались брать одну и ту же палочку? Тогда проблема взаимной блокировки не возникнет, потому что философы будут конкурировать за одну первую блокировку.

Оба философа должны договориться о том, что из двух палочек, которые они собираются использовать, палочку с большим приоритетом всегда нужно брать первой. В нашем случае оба философа одновременно конкурируют за высокоприоритетную палочку. Когда один философ побеждает и берет ее, на столе остается только палочка с меньшим приоритетом. Поскольку философы согласились сначала брать высокоприоритетную палочку, второй философ не сможет взять оставшуюся. А у философа, который взял первую палочку, теперь есть доступ к палочке с меньшим приоритетом, так что он может начать есть обеими палочками. Гениально!

Назначим приоритеты для палочек. Допустим, палочка *A* имеет наивысший приоритет, а палочка *B* — следующий. Каждый философ всегда должен сначала брать палочку с большим приоритетом.



В нашем коде философ 2 создавал проблему, когда брал палочку *B* до палочки *A*. Чтобы решить проблему, мы изменим порядок захвата палочек, не изменяя остальной код. Сначала берется палочка *A*, и только после этого — палочка *B*:

```
# Chapter 9/deadlock/deadlock_hierarchy.py
from lock_with_name import LockWithName

from deadlock import Philosopher

if __name__ == "__main__":
    chopstick_a = LockWithName("Палочка A")
    chopstick_b = LockWithName("Палочка B")

    philosopher_1 = Philosopher("Философ 1", chopstick_a,
                                chopstick_b)
    philosopher_2 = Philosopher("Философ 2", chopstick_a,
                                chopstick_b)

    philosopher_1.start()
    philosopher_2.start()
```

Если запустить программу после этих изменений, она отработает до конца без взаимных блокировок.

ПРИМЕЧАНИЕ Упорядочить блокировки не всегда возможно, если заранее неизвестно, какие блокировки могут понадобиться задаче. Чтобы предотвратить взаимные блокировки, можно использовать такие механизмы, как графы распределения ресурсов (RAG) и иерархии блокировок. RAG помогают выявлять и предотвращать циклы в отношениях между процессами и ресурсами. В некоторых языках программирования и фреймворках поддерживаются высокоуровневые примитивы синхронизации, которые упрощают управление блокировками. Тем не менее вам все равно потребуется тщательно проектировать и тестировать свой код, потому что эти методы не гарантируют полного устранения взаимных блокировок.

Еще один способ предотвратить взаимные блокировки — установить тайм-аут для попыток блокирования. Если в течение определенного периода времени задаче не удастся успешно установить все необходимые блокировки, мы заставляем поток освободить все блокировки, которые он удерживает в данный момент. Однако это может создать другую проблему — активную блокировку.

Активные блокировки

Активная блокировка (livelock) отчасти похожа на взаимную блокировку: она возникает, когда две задачи конкурируют за один набор ресурсов. Однако при активной блокировке задача уступает свою первую блокировку в попытке установить вторую. Установив вторую блокировку, она снова пытается установить первую. Задача оказывается в том же заблокированном состоянии, потому что она тратит все время не на реальную работу, а на то, чтобы постоянно освобождать и пытаться установить то одну, то другую блокировку.

Представьте, что вы звоните по телефону, но вызываемый абонент в это же самое время тоже пытается позвонить вам. Вы оба вешаете трубку и одновременно повторяете попытку, что создает ту же ситуацию. В итоге никому из вас не удастся дозвониться до другого.



Активная блокировка возникает тогда, когда задачи активно выполняют конкурентные операции, но выполнение программы от этого не продвигается. Эта ситуация похожа на взаимную блокировку, но отличается тем, что задачи ведут себя «вежливо», позволяя другим выполнить свою работу в первую очередь.

Представьте, что наши философы стали вести себя более учтиво: они кладут палочку на стол, если не могут взять обе:

```
# Chapter 9/livelock.py
import time
from threading import Thread

from deadlock.lock_with_name import LockWithName

dumplings = 20
```

```

class Philosopher(Thread):
    def __init__(self, name: str, left_chopstick: LockWithName,
                  right_chopstick: LockWithName):
        super().__init__()
        self.name = name
        self.left_chopstick = left_chopstick
        self.right_chopstick = right_chopstick

    def run(self) -> None:
        global dumplings

        while dumplings > 0:
            self.left_chopstick.acquire()
            print(f"{self.left_chopstick.name} "
                  f"взята: {self.name}")
            if self.right_chopstick.locked():
                print(f"{self.name} не может взять: "
                      f"{self.right_chopstick.name}, "
                      f"вежливо уступает...")
            else:
                self.right_chopstick.acquire()
                print(f"{self.right_chopstick.name} "
                      f"взята: {self.name}")
                dumplings -= 1
                print(f"{self.name} съел пельмень. Осталось "
                      f"пельменей: {dumplings}")
                time.sleep(1)
                self.right_chopstick.release()
                self.left_chopstick.release()

if __name__ == "__main__":
    chopstick_a = LockWithName("Палочка А")
    chopstick_b = LockWithName("Палочка В")

    philosopher_1 = Philosopher("Философ 1", chopstick_a, chopstick_b)
    philosopher_2 = Philosopher("Философ 2", chopstick_b, chopstick_a)

    philosopher_1.start()
    philosopher_2.start()

```

Философ взял левую палочку. В задаче два философа, поэтому каждый берет со стола одну палочку

Философ пытается взять правую палочку. Если она доступна, то философ берет ее и ест пельмень; иначе он уступает и откладывает левую палочку

К сожалению, этим славным людям так и не суждено полакомиться:

```

Палочка А взята: Философ 1
Философ 1 не может взять: Палочка В, вежливо уступает...
Палочка В взята: Философ 2
Философ 2 не может взять: Палочка А, вежливо уступает...
Палочка В взята: Философ 2
Палочка А взята: Философ 1
Философ 2 не может взять: Палочка А, вежливо уступает...
Философ 1 не может взять: Палочка В, вежливо уступает...
Палочка В взята: Философ 2
Палочка А взята: Философ 1
Философ 2 не может взять: Палочка А, вежливо уступает...
Философ 1 не может взять: Палочка В, вежливо уступает...

```

Кроме того, что при этом не выполняется никакой полезной работы, такая ситуация может привести к тому, что система будет перегружена из-за частых переключений контекста, что снизит ее производительность. К тому же планировщик ОС не сможет справедливо распределять время, потому что не знает, какая задача дольше всего ожидала общего ресурса.

Чтобы избежать блокировок этого типа, можно иерархически упорядочить последовательность блокировок, как мы это делали, чтобы избавиться от взаимных блокировок. В таком случае только один процесс сможет успешно установить обе блокировки.

ПРИМЕЧАНИЕ Выявлять и устранять активные блокировки часто оказывается сложнее, чем взаимные блокировки. Дело в том, что в сценариях активной блокировки участвуют сложные и динамические взаимодействия между несколькими сущностями, отчего бороться с ними становится непросто.

Активные блокировки составляют подмножество более обширной категории проблем *нехватки ресурсов* (starvation).

Нехватка ресурсов

Добавим локальную переменную, чтобы отслеживать, сколько пельменей съел каждый философ:

```
# Chapter 9/starvation.py
from threading import Thread

from deadlock.lock_with_name import LockWithName

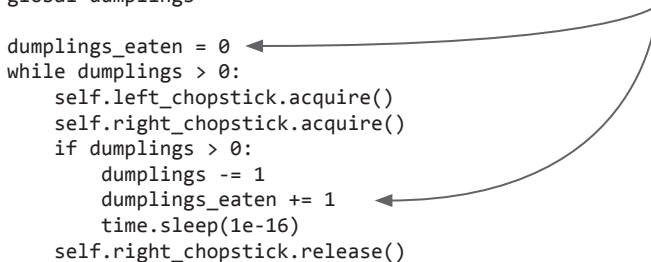
dumplings = 1000

class Philosopher(Thread):
    def __init__(self, name: str, left_chopstick: LockWithName,
                 right_chopstick: LockWithName):
        super().__init__()
        self.name = name
        self.left_chopstick = left_chopstick
        self.right_chopstick = right_chopstick

    def run(self) -> None:
        global dumplings

        dumplings_eaten = 0
        while dumplings > 0:
            self.left_chopstick.acquire()
            self.right_chopstick.acquire()
            if dumplings > 0:
                dumplings -= 1
                dumplings_eaten += 1
                time.sleep(1e-16)
            self.right_chopstick.release()
```

Переменная *dumplings_eaten*
отслеживает, сколько
пельменей съел философ



```

        self.left_chopstick.release()
        print(f"{self.name} съел {dumplings_eaten} пельменей")

if __name__ == "__main__":
    chopstick_a = LockWithName("Палочка A")
    chopstick_b = LockWithName("Палочка B")

    threads = []
    for i in range(10):
        threads.append(
            Philosopher(f"Философ {i}", chopstick_a, chopstick_b))

    for thread in threads:
        thread.start()

    for thread in threads:
        thread.join()

```

Мы присваиваем переменной имя `dumplings_eaten` и инициализируем ее нулем. Также на этот раз мы добавили больше философов. Значение `dumplings_eaten` увеличивается на 1 каждый раз, когда философ съедает пельмень. Когда программа завершается, мы видим, что разные философы съели очень разное количество пельменей, и это несправедливо:

```

Философ 1 съел 417 пельменей
Философ 9 съел 0 пельменей
Философ 6 съел 0 пельменей
Философ 7 съел 0 пельменей
Философ 5 съел 0 пельменей
Философ 0 съел 4 пельменей
Философ 2 съел 3 пельменей
Философ 8 съел 268 пельменей
Философ 3 съел 308 пельменей
Философ 4 съел 0 пельменей

```

Философ 1 съел намного больше пельменей, чем философ 8, — более 400. Похоже, восьмой философ слишком медленно тянется за палочкой, а первый философ действует быстро и снова забирает палочку, пока восьмой в очередной раз застревает в ожидании. Некоторым философам вообще никогда не достаются обе палочки. Если так происходит относительно редко, с этим можно смириться, но когда палочек постоянно не хватает, философы останутся голодными.

Нехватка ресурсов не зря также называется *ресурсным голодом*: поток буквально голодает, не получая доступа к запрашиваемым ресурсам, и никакого прогресса не происходит. Если другая «жадная» задача часто удерживает блокировку по общему ресурсу, то «голодная» задача так и не сможет выполниться.

ПРИМЕЧАНИЕ На принципе нехватки ресурсов основан самый известный тип атак против онлайн-сервисов — атаки отказа в обслуживании (DOS-атаки). В этом случае атакующий пытается исчерпать все ресурсы сервера. Службе начинает не хватать доступных ресурсов (хранилищ данных, памяти, вычислительных мощностей), отчего она отказывает и не может выполнять свои функции.

Нехватка ресурсов обычно происходит из-за чрезмерно упрощенного алгоритма планирования, который, как вы узнали из главы 6, является частью системы выполнения. Он должен равномерно распределять ресурсы между всеми задачами и, в частности, заботиться о том, чтобы ни одна задача не сталкивалась с постоянным блокированием доступа к ресурсам, которые ей необходимы. Обработка задач с разными приоритетами зависит от ОС, но планировщик обычно чаще запускает задачи с более высоким приоритетом, а это может вызвать нехватку ресурсов для низкоприоритетных задач. Другой фактор, который может привести к ресурсному голоду, — слишком много задач в системе, когда задачам приходится чересчур долго ожидать выполнения.

С нехваткой ресурсов можно бороться, если применять алгоритм планирования с очередями приоритетов, где также используется метод *старения* (aging): если поток долго ожидает в системе, его приоритет постепенно повышается. Со временем поток приоритизируется настолько, чтобы система могла запланировать его для обращения к ресурсам/процессорам и полноценно выполнить. Мы не будем обсуждать этот принцип подробно, потому что он весьма специфичен; если вам захочется узнать больше, обращайтесь к книге Эндрю Таненбаума (Andrew Tanenbaum) «Modern Operating Systems»¹. Однако не ограничивайтесь одной-единственной книгой, читайте и изучайте и другие источники информации.

После всего, что вы узнали о синхронизации, рассмотрим некоторые распространенные проблемы проектирования конкурентности.

Как проектировать синхронизацию

Когда вы сталкиваетесь с задачами проектирования систем, полезно сопоставлять эти задачи с уже известными. Некоторые задачи широко описаны в литературе и часто встречаются в реальных ситуациях. Первая из них — задача «Производитель — потребитель».

Задача «Производитель — потребитель»

Допустим, один или несколько производителей генерируют элементы и помещают их в буфер. Некоторые потребители читают элементы из того же буфера, обрабатывая их по одному. Каждый производитель может генерировать и сохранять элементы в буфере в своем собственном темпе. Потребитель действует аналогично, но должен позаботиться о том, чтобы не читать данные из пустого буфера. Систему нужно спроектировать так, чтобы производитель (producer) не пытался добавлять данные в заполненный буфер, а потребитель (consumer) не пытался читать их из пустого буфера. Вы уже знаете о конкурентном программировании достаточно много, поэтому попытайтесь решить проблему самостоятельно, прежде чем читать дальше.

¹ Таненбаум Э., Бос Х. «Современные операционные системы». 4-е изд. СПб., издательство «Питер».

Базовая реализация выглядит так:

```
# Chapter 9/producer_consumer.py
import time
from threading import Thread, Semaphore, Lock
```

```
SIZE = 5
BUFFER = ["" for i in range(SIZE)]
producer_idx: int = 0
mutex = Lock()
empty = Semaphore(SIZE)
full = Semaphore(0)
```

Общий
буфер

```
class Producer(Thread):
    def __init__(self, name: str, maximum_items: int = 5):
        super().__init__()
        self.counter = 0
        self.name = name
        self.maximum_items = maximum_items
```

```
    def next_index(self, index: int) -> int:
        return (index + 1) % SIZE
```

В буфере есть хотя
бы одна пустая
ячейка

```
    def run(self) -> None:
        global producer_idx
        while self.counter < self.maximum_items:
            empty.acquire()
            mutex.acquire()
```

Входит в крити-
ческую секцию,
которая изменя-
ет общий буфер

```
        BUFFER[producer_idx] = f"{self.name}-{self.counter}"
        print(f"{self.name} произвел элемент "
              f"'{BUFFER[producer_idx]}' в ячейку {producer_idx}")
        producer_idx = self.next_index(producer_idx)
        mutex.release()
```

```
        full.release()
        time.sleep(1)
```

В буфер добавился новый
элемент, и теперь в нем на одну
пустую ячейку меньше

```
class Consumer(Thread):
    def __init__(self, name: str, maximum_items: int = 10):
        super().__init__()
        self.name = name
        self.idx = 0
        self.counter = 0
        self.maximum_items = maximum_items
```

```
    def next_index(self) -> int:
        return (self.idx + 1) % SIZE
```

Получает следующий индекс
в буфере для потребления

В буфере есть хотя бы один
элемент, который можно
потреблять

```
    def run(self) -> None:
        while self.counter < self.maximum_items:
            full.acquire()
            mutex.acquire()
            item = BUFFER[self.idx]
            print(f"{self.name} потребил элемент "
                  f"'{item}' из ячейки {self.idx}")
            self.idx = self.next_index()
            self.counter += 1
```

Входит в крити-
ческую секцию, кото-
рая изменяет общий
буфер

```

mutex.release()
empty.release()
time.sleep(2)

if __name__ == "__main__":
    threads = [
        Producer("ГубкаБоб"),
        Producer("Патрик"),
        Consumer("Сквидвард")
    ]

    for thread in threads:
        thread.start()

    for thread in threads:
        thread.join()

```

Входит в критическую секцию, которая изменяет общий буфер

После того как элемент был потреблен, в буфере появляется новая пустая ячейка

Проанализируем этот код. В нем используются три синхронизации:

- Семафор `full` отслеживает пространство, которое заполняет `Producer`. В начале программы он инициализируется как заблокированный (счетчик равен 0), потому что буфер пуст: у производителей еще не было возможности его заполнить.
- Семафор `empty` отслеживает пустые ячейки в буфере. Он инициализируется максимальным значением (`SIZE` в коде), потому что в начале буфер пуст.
- Мьютекс `mutex` используется для взаимного исключения, чтобы с общим ресурсом (буфером) в любой момент времени мог работать только один поток.

Производитель может вставлять данные в буфер в любой момент. В критической секции производитель добавляет элемент в буфер и увеличивает индекс буфера, который используют все производители; доступом к критической секции управляет мьютекс. Но прежде чем помещать данные в буфер, производитель пытается обратиться к семафору `empty` и уменьшить его значение на 1. Если значение семафора равно 0, это означает, что буфер полон, и семафор блокирует всех производителей, пока в буфере не появится свободное место (то есть `empty` станет больше 0). Производитель освобождает семафор `full`, после того как добавил туда один элемент.

С другой стороны, потребитель пытается обратиться к семафору `full`, прежде чем потреблять данные из буфера. Если значение этого семафора равно 0, значит, буфер пуст, и семафор `full` блокирует любого потребителя, пока значение `full` не станет больше 0. Затем потребитель берет элемент из буфера и обрабатывает его в своей критической секции. После того как потребитель обработает все данные из буфера, он освобождает семафор `empty`, увеличивая его значение на 1, чтобы производители знали, что в буфере есть свободное место для нового элемента.

Если производитель опережает потребителя (типичная ситуация), то потребитель редко блокируется по семафору `empty`, потому что буфер обычно не пустует. Соответственно как производитель, так и потребитель работают с общим буфером без каких-либо проблем.

ПРИМЕЧАНИЕ Аналогичная задача возникает в реализации каналов — механизмов межпроцессной коммуникации (IPC) в Linux. У каждого канала есть собственный буфер, который защищается семафорами.

В следующем разделе рассматривается другая классическая задача — «Читатели — записыватель».

Задача «Читатели — записыватель»

Не все операции равны. Когда множество задач одновременно читают одни и те же данные, это не создаст проблем конкурентности, если данные не изменяются. Данными может быть файл, блок памяти и даже регистр процессора. Можно разрешить множество одновременных операций чтения при условии, что сторона, которая записывает данные, делает это в монопольном режиме — то есть не бывает нескольких операций записи одновременно.

Например, представьте, что общие данные — это библиотечный каталог. Обычные посетители библиотеки читают каталог, чтобы найти нужную книгу. Каталог могут обновлять один или несколько библиотекарей. В общем случае каждое обращение к каталогу рассматривается как критическая секция, и пользователям приходится читать каталог по очереди, а это, очевидно, приводит к неприемлемым задержкам. В то же время важно, чтобы библиотекари не мешали друг другу и чтобы никто не читал каталог, пока в него происходит запись: иначе пользователи рискуют обратиться к противоречивой информации.

В обобщенной форме можно сказать, что одни задачи (посетители библиотеки) только читают данные, а другие задачи (библиотекари, назовем их «записывателями») только записывают данные:

- Любое количество читателей могут читать общие данные одновременно.
- В каждый момент времени не более одного записывателя могут записывать общие данные.
- Пока записыватель записывает общие данные, ни один читатель не может их читать.

Тем самым предотвращаются любые ситуации гонки или некорректное чередование операций из-за ошибок чтения/записи или записи/записи.

Итак, читатели (readers) — это задачи, которые не исключают друг друга, а записыватели (writers) — это задачи, которые должны исключать любые другие задачи (как читателей, так и других записывателей). Такой подход позволяет эффективно решить проблему совместного доступа, вместо того чтобы просто взаимно исключать обращения к общему ресурсу для любых операций.

Для подобных случаев в библиотеках и языках программирования часто предусмотрена блокировка читателей — записывателя (RWLock). Блокировки такого типа обычно используются в больших операциях и могут значительно повысить производительность, если защищенную структуру данных часто читают, но относительно редко изменяют. Поскольку в Python такой блокировки нет, реализуем ее самостоятельно:

```
# Chapter 9/reader_writer/rwlock.py
from threading import Lock
```

```
class RWLock:
    def __init__(self) -> None:
        self.readers = 0
        self.read_lock = Lock()
        self.write_lock = Lock()

    def acquire_read(self) -> None:
        self.read_lock.acquire()
        self.readers += 1
        if self.readers == 1:
            self.write_lock.acquire()
        self.read_lock.release()

    def release_read(self) -> None:
        assert self.readers >= 1
        self.read_lock.acquire()
        self.readers -= 1
        if self.readers == 0:
            self.write_lock.release()
        self.read_lock.release()

    def acquire_write(self) -> None:
        self.write_lock.acquire()

    def release_write(self) -> None:
        self.write_lock.release()
```

Устанавливает блокировку чтения для текущего потока. Если записыватель ожидает блокировки, метод блокируется до тех пор, пока записыватель не снимет блокировку

Снимает блокировку чтения, которую удерживает текущий поток. Если никакие другие читатели не удерживают блокировку, метод снимает блокировку записи

Устанавливает блокировку записи для текущего потока. Если читатель или записыватель удерживает блокировку, метод блокируется до тех пор, пока блокировка не снимется

Снимает блокировку чтения, которую удерживает текущий поток

В ходе нормальной работы к блокировке могут обращаться несколько читателей одновременно. Но когда поток захочет обновить общие данные, он блокируется до тех пор, пока все читатели не снимут блокировку, после чего записыватель устанавливает блокировку и обновляет общие данные. Когда это происходит, новые потоки-читатели блокируются, пока поток-записыватель не завершит операцию.

Пример реализаций потоков читателей и записывателя:

```
# Chapter 9/reader_writer/reader_writer.py
import time
import random
from threading import Thread
```

```
from rwlock import RWLock

counter = 0
lock = RWLock()
```

Общие данные

```
class User(Thread):
    def __init__(self, idx: int):
        super().__init__()
        self.idx = idx

    def run(self) -> None:
```

```

        while True:
            lock.acquire_read()
            print(f"Посетитель {self.idx} читает: {counter}")
            time.sleep(random.randrange(1, 3))
            lock.release_read()
            time.sleep(0.5)

class Librarian(Thread):
    def run(self) -> None:
        global counter
        while True:
            lock.acquire_write()
            print("Библиотекарь записывает...")
            counter += 1
            print(f"Новое значение: {counter}")
            time.sleep(random.randrange(1, 3))
            lock.release_write()

if __name__ == "__main__":
    threads = [
        User(0),
        User(1),
        Librarian()
    ]

    for thread in threads:
        thread.start()

    for thread in threads:
        thread.join()

```

Здесь два потока (посетители) читают общую память, а один поток (библиотекарь) ее изменяет. Результат выглядит так:

```

Посетитель 0 читает: 0
Посетитель 1 читает: 0
Библиотекарь записывает...
Новое значение: 1
Посетитель 0 читает: 1
Посетитель 1 читает: 1
Библиотекарь записывает...
Новое значение: 2
Посетитель 0 читает: 2
Посетитель 1 читает: 2
Посетитель 0 читает: 2
Посетитель 1 читает: 2
Посетитель 0 читает: 2
Посетитель 1 читает: 2
Посетитель 0 читает: 2
Библиотекарь записывает...
Новое значение: 3

```

Из вывода видно, что ни один посетитель не читает в то время, когда библиотекарь записывает, а библиотекарь не записывает, пока хотя бы один посетитель читает общую память.

Несколько слов напоследок

Это была длинная глава! Давайте припомним основные моменты.

Лучшее, что может сделать разработчик, чтобы обеспечить потоковую безопасность, — это качественно спроектировать приложение. Если избегать совместного использования ресурсов и минимизировать коммуникацию между задачами, то у них будет меньше шансов вмешиваться в работу друг друга. Тем не менее не всегда возможно создать приложение, которое не будет обращаться к общим ресурсам. В таких случаях нужна качественная синхронизация.

Синхронизация помогает обеспечить корректное выполнение кода, но за это приходится расплачиваться производительностью. Использование блокировок создает задержки, даже если нет конфликтов. Чтобы задача могла обратиться к общим данным, она сначала должна установить блокировку, связанную с этими данными. А чтобы установить блокировку, синхронизировать ее между задачами и отслеживать общие объекты, процессору приходится выполнять значительную работу, скрытую от программиста. Блокировки и атомарные операции обычно используют барьеры памяти и синхронизацию на уровне ядра, чтобы обеспечить необходимую защиту кода. Если несколько задач пытаются установить одну и ту же блокировку, то затраты еще больше возрастают. Глобальные блокировки также ухудшают масштабируемость.

Следовательно, по возможности следует проектировать код без какой-либо синхронизации. В случае коммуникаций вместо общей памяти попробуйте применить IPC с передачей сообщений — в таком случае задачам не придется совместно использовать память, и у каждой задачи будет собственная копия данных, с которой можно безопасно работать. Всего этого можно добиться эффективными алгоритмами, хорошим выбором моделей для проектирования, подходящими структурами данных или классами, которые не зависят от синхронизации.

Итоги

- Конкурентность — не самое простое понятие, и когда разработчики реализуют ее в своих приложениях, они могут столкнуться с разными проблемами, например:
 - Если неосторожно использовать примитивы синхронизации, могут возникнуть взаимные блокировки. При взаимной блокировке одни задачи ожидают ресурсов, которые заняты другими, и ни одна задача не может выполняться дальше.
 - Активная блокировка — еще одна частая проблема реализации конкурентности, похожая на взаимную блокировку. В ситуации активной блокировки запрос на монопольную блокировку многократно отклоняется из-за того, что

несколько перекрывающихся блокировок мешают друг другу. Задачи продолжают выполняться, но не могут завершить свою работу.

- Поток приложения также может столкнуться с нехваткой ресурсов, когда он не получает нужного процессорного времени или доступа к ресурсам, потому что другие «жадные» потоки поглощают эти ресурсы. Задачи «голодают» без ресурсов и поэтому не выполняют свою работу. Нехватка ресурсов может возникнуть из-за ошибок в алгоритме планирования или некорректной синхронизации.
- Конкурентность — не новая область. Многие распространенные проблемы проектирования уже были решены; появились рекомендуемые практики и паттерны, которые необходимо изучить. Среди самых известных задач можно выделить задачи «Производитель — потребитель» и «Читатели — записыватель». Они наиболее эффективно решаются с помощью семафоров и мьютексов.

Часть III

Асинхронные осьминоги. История о конкурентном выполнении со вкусом пиццы



Представьте себе, что вы пришли в пиццерию и видите, как повар (конечно, осьминог) работает на кухне, готовя несколько пицц одновременно. Движения повара стремительны, его щупальца извиваются в идеальной гармонии, и вам остается только восхищаться его многозадачными способностями: как ловко он подбрасывает основу, распределяет соусы и накладывает добавки. Но как ресторан справляется с десятками или даже сотнями одновременных заказов? Секрет успеха настоящих пиццайоло — асинхронные коммуникации!

В начале последней части книги, посвященной конкурентности, мы перейдем к новой породе осьминогов — асинхронным. Как и их синхронные собратья, эти существа отлично разбираются в многозадачности и умеют одновременно жонглировать множеством задач. Но их отличительная особенность — в том, что они способны делать это без блокирования и не ожидая, пока завершится одна задача, чтобы запустить другую.

В главах с 10-й по 13-ю мир неблокирующего ввода/вывода, событийной конкурентности и асинхронных коммуникаций рассматривается на примере пиццерии. Вы узнаете, как разные подходы к конкурентности влияют на скорость и эффективность приложений и как писать конкурентные приложения, которые справляются с высокими объемами запросов.

Не бойтесь потеряться в ошеломляющем разнообразии методов и подходов к конкурентности. Вскоре вы начнете видеть общую картину — подобно дирижеру, который сводит воедино разные инструменты, чтобы создать гармоничную симфонию.

Так что подкрепите свежеприготовленной пиццей — и вперед!

.....

10 | Неблокирующий ввод/вывод



В этой главе

- ✓ Межпроцессные коммуникации с передачей сообщений в распределенной сети компьютеров
- ✓ Приложения «клиент — сервер»
- ✓ Ограничения на использование множественных потоков или процессов в операциях ввода/вывода
- ✓ Что такое неблокирующие операции и как они помогают маскировать операции с интенсивным вводом/выводом

Исторически скорость процессоров росла, что позволяло выполнять больше команд в единицу времени, а скорость ввода/вывода за ними не успевала. Современные приложения больше зависят от операций ввода/вывода, нежели от операций процессора, отчего такие задачи, как запись на жесткий диск или чтение из сети, занимают гораздо больше времени, чем операции процессора. Соответственно, процессор вынужден простаивать в ожидании ввода/вывода, что не дает приложению выполнять другие задачи. Это ограничение создает узкое место в высокопроизводительных приложениях.

В этой главе мы рассмотрим возможное решение этой проблемы, основанное на механизме межпроцессных коммуникаций (IPC) с передачей сообщений. Опираясь на то, что нам уже известно о потоковой модели, мы изучим, как она применяется в сценариях ввода/вывода с высокой нагрузкой, а в качестве популярного примера рассмотрим разработку веб-сервера. Веб-сервер — отличный пример для того, чтобы

продемонстрировать функции асинхронного программирования и базовые понятия, которые позволяют разработчикам наиболее эффективно применять конкурентность в таких задачах. В следующих главах мы дополнительно усовершенствуем это решение.

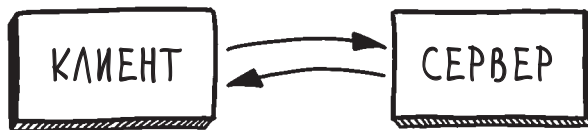
Мир распределенных вычислений

Конкурентность давно вышла за рамки одного компьютера. Интернет и Всемирная паутина стали неотъемлемой частью нашей жизни, а современные технологии позволяют связывать между собой сотни и тысячи отдельных компьютеров. В результате бурно развиваются распределенные системы и распределенные вычисления. В таких системах задачи могут выполняться как на одном компьютере, так и на разных, которые могут находиться в одной локальной сети, а могут располагаться за тысячи километров друг от друга. Все это возможно благодаря совокупности взаимосвязанных технологий, важнейшей из которых является механизм IPC с передачей сообщений (о котором шла речь в главе 5).

В этом контексте компонент представляет собой задачу на одной машине, а ресурсами являются все аппаратные мощности компьютера и отдельные функции, делегированные заданному вычислительному узлу. Данные хранятся в памяти процесса приложения, а коммуникация между узлами происходит по сети с помощью специализированных протоколов. Самая распространенная архитектура для взаимодействия между такими узлами — *модель «клиент — сервер»*.

Модель «клиент — сервер»

В этой модели используются процессы двух видов: *клиенты* и *серверы*. Серверные приложения предоставляют услуги клиентским приложениям. Клиент инициирует взаимодействие, соединяясь с сервером. После этого клиент может запросить обслуживание, для чего отправляет сообщение серверу. Сервер постоянно получает запросы на обслуживание от клиентов, выполняет запрашиваемые действия и (если потребуется) возвращает клиенту сообщение о том, что работа завершена. Наконец, клиент отсоединяется.



Так работают многие сетевые приложения: веб-браузеры являются клиентами для веб-серверов, программа для работы с электронной почтой является клиентом для сервера электронной почты и т. д. Клиент и сервер могут взаимодействовать через сетевые сокеты.

Сетевые сокеты

Мы рассматривали понятие сокетов, когда обсуждали IPC с передачей сообщений в главе 5, но здесь речь пойдет о другой разновидности сокетов — *сетевых сокетах*. Они похожи на доменные сокеты UNIX, но используются для того, чтобы отправлять сообщения по сети. Это может быть логическая сеть, локальная сеть компьютера или сеть, которая физически соединена со внешней сетью, а через нее — с другими сетями. Наиболее очевидный пример такого рода — интернет.

Существуют разные виды сетевых сокетов, но в этой главе мы сосредоточимся на сокетах TCP/IP, которые гарантируют доставку данных и поэтому пользуются наибольшей популярностью. Когда используются сокеты TCP/IP, устанавливается *соединение*; это означает, что два процесса должны согласовать взаимодействие до того, как смогут обмениваться информацией. Оба процесса поддерживают соединение на протяжении *сеанса* связи (session).



Сетевой сокет — это абстракция, с помощью которой ОС взаимодействует с сетью. Для разработчиков сокет представляет конечную точку этого соединения. Он обеспечивает чтение данных из сети и их запись, после чего отправляет данные в сеть. У каждого сокета есть два важных атрибута: IP-адрес и порт.

IP-адреса

У каждого устройства (узла), которое подключено к сети, есть уникальный идентификатор, который представляется в виде IP-адреса. IP-адреса (версии 4) устроены по стандартной схеме: это группа из четырех чисел, разделенных точками (например, 8.8.8.8). С помощью IP-адреса можно соединить сокет с конкретным узлом в любом месте сети — в том числе с принтером, кассовым аппаратом, холодильником, сервером, мейнфреймом или персональным компьютером.

Во многих отношениях IP-адреса напоминают почтовый адрес дома на улице. Обычно у улицы есть название (например, Пятая авеню), и на ней могут находиться несколько домов. У каждого дома свой номер; таким образом, адрес «Пятая авеню, 175» отличается от адреса «Пятая авеню, 350» номером дома.

Порты

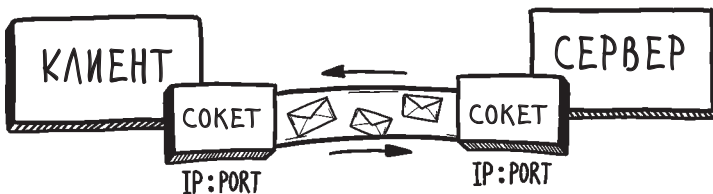
Чтобы на одной машине работали сразу несколько серверных приложений, к которым могут подключаться клиенты, нужно как-то маршрутизировать трафик из одного сетевого интерфейса к этим приложениям. Для этой цели на каждом компьютере функционируют разные *порты*.

Каждый порт служит точкой входа для того или иного приложения: через этот порт оно активно прослушивает входящие запросы. Процесс сервера ассоциируется с конкретным портом и остается в состоянии прослушивания, готовый к обработке клиентских соединений. В свою очередь, чтобы установить соединение, клиенты должны знать номер порта, на котором прослушивает сервер.

Некоторые общеизвестные порты зарезервированы для процессов системного уровня и работают как стандартные порты для конкретных служб. Зарезервированные порты предоставляют согласованный и понятный механизм, с помощью которого клиенты могут соединяться с соответствующими службами. Представьте себе офисы в бизнес-центре: у каждой компании есть собственное помещение, в котором она принимает клиентов.

И у клиента, и у сервера есть собственный сокет, соединенный с другим сокетом. Серверный сокет прослушивает конкретный порт, а клиентский сокет соединяется с серверным через этот порт. После того как соединение установлено, начинается обмен данными.

Процесс-отправитель помещает нужную информацию в сообщение и целенаправленно отправляет его по сети сокету-получателю; процесс-получатель читает сообщение, как было описано ранее для сокетов UDS в главе 5. Процессы, которые участвуют в этом обмене, могут выполняться на одной и той же машине или на разных компьютерах, соединенных сетью.



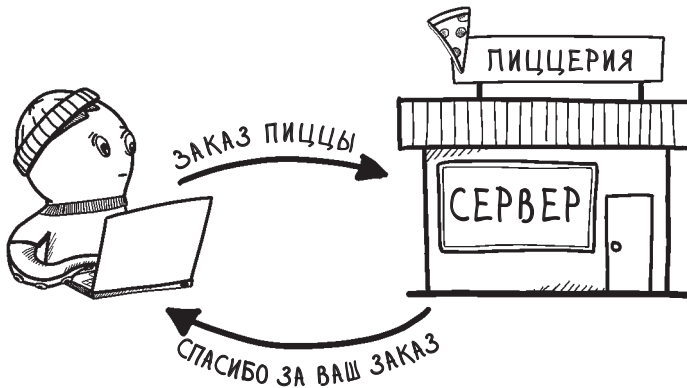
Реализация сервера станет не только хорошим упражнением, но и конкретным примером, который поможет понять, как развивалась конкурентность и какие новые проблемы она создает. Впрочем, мы не будем углубленно изучать сетевую модель и стек протоколов, которые задействованы в этом обмене данными.

Сети и сокеты — обширная тема, по которой написаны целые тома. Если у вас еще мало опыта работы с сокетами или сетевыми взаимодействиями, от обилия терминов и подробностей голова может пойти кругом. Если вам захочется изучить тему подробнее, обращайтесь к книге Таненбаума «Современные операционные системы».

Итак, после того как вы в общих чертах разобрались в том, что такое сетевые сокеты и как устроены взаимодействия «клиент — сервер», вы готовы разработать свой первый сервер. Мы начнем с простейшей, последовательной версии, а позже изменим эту реализацию, чтобы понять, как и почему произошел переход от конкурентности к асинхронности.

Служба заказа пиццы

В 1980-е годы компания Santa Cruz Operation (которая сделала для развития интернета больше, чем Эл Гор¹) постоянно заказывала пиццу для разработчиков в одной и той же пиццерии в деловой части города Санта-Круз (Калифорния). Заказ по телефону занимал слишком много времени, поэтому разработчики создали первое в мире коммерческое приложение, которое позволяло заказывать и оплачивать пиццу. Для этого терминал, установленный в компании, обменивался данными с другим терминалом, установленным в пиццерии. Это была эпоха простых терминалов, которые подключались к местным вычислительным сетям; время персональных компьютеров еще не наступило. В наши дни все стало сложнее. Давайте попробуем воспроизвести аналогичную систему с помощью более современных технологий. Мы реализуем службу заказа пиццы из местной пиццерии: сервер получает заказы и благодарит клиентов в ответном сообщении.



Серверное приложение должно предоставить серверный сокет, к которому будут подключаться клиенты. Для этого серверный сокет связывается с IP-адресом и портом на серверной машине. После этого серверное приложение должно прослушивать входящие соединения:

```
# Chapter 10/pizza_server.py
from socket import socket, create_server
```

Задаёт максимальный объем данных, принимаемый за раз

```
BUFFER_SIZE = 1024
```

Определяет адрес и порт на узле сервера

```
ADDRESS = ("127.0.0.1", 12345)
```

¹ Эл (Альберт) Гор, вице-президент США в 1993–2001 гг., действительно внес большой вклад в развитие Сети: по его инициативе было принято несколько законов, регулирующих глобальные вычислительные сети, и выделены средства на значимые разработки в этой сфере. Однако во время президентской кампании 2000 г. предвыборная агитация и пресса неумеренно преувеличивали заслуги Гора (даже называя его «изобретателем интернета»), что стало поводом для многочисленных шуток. — *Примеч. пер.*

```

class Server:
    def __init__(self) -> None:
        try:
            print(f"Сервер запущен на узле {ADDRESS}")
            self.server_socket: socket = create_server(ADDRESS)
        except OSError:
            self.server_socket.close()
            print("\nСервер остановлен.")

    def accept(self) -> socket:
        conn, client_address = self.server_socket.accept()
        print(f"Установлено соединение с узлом {client_address}")
        return conn

    def serve(self, conn: socket) -> None:
        try:
            while True:
                data = conn.recv(BUFFER_SIZE)
                if not data:
                    break
                try:
                    order = int(data.decode())
                    response = f"Спасибо за заказ пиццы ({order} шт.)!\n"
                except ValueError:
                    response = "Недопустимое количество пицц.\n"
                    print(f"Отправляется сообщение для узла {conn.getpeername()}")
                    conn.send(response.encode())
            finally:
                print(f"Соединение с узлом {conn.getpeername()} закрыто")
                conn.close()

    def start(self) -> None:
        print("Сервер прослушивает входящие соединения")
        try:
            while True:
                conn = self.accept()
                self.serve(conn)
            finally:
                self.server_socket.close()
                print("\nСервер остановлен.")
        except KeyboardInterrupt:
            self.server_socket.close()
            print("\nСервер остановлен.")

if __name__ == "__main__":
    server = Server()
    server.start()

```

Создает объект серверного сокета, привязанный к конкретному адресу

Блокируется, пока клиент не подключится к серверному сокету, и возвращает новое соединение и сокет для этого клиента

Непрерывно принимает данные от клиентского сокета, пока не поступят входящие данные

Отправляет ответ клиентскому сокету

Закрывает клиентский сокет после того, как метод serve завершит выполнение для этого клиента

Принимает входящие соединения и обслуживает каждый клиент, пока сервер не будет остановлен

Здесь мы используем адрес локального компьютера 127.0.0.1 и порт 12345. Чтобы привязать сокет к этому адресу узла и порту, вызывается `create_server()`. Это позволяет серверу принимать входящие соединения от клиентов. Метод `accept()` ожидает клиентского соединения, и сервер ожидает в этой точке, пока не поступит входящее соединение.

Когда клиент подключается, `accept()` возвращает новый объект `Socket`, который представляет соединение и адрес клиента. После этого серверный сокет создает но-

вый сокет, с помощью которого сервер будет взаимодействовать с этим клиентом. Собственно, это все: мы установили соединение с клиентом и можем обмениваться данными.

Сервер готов, и теперь его можно запустить:

```
$ python pizza_server.py
```

Когда мы запускаем эту команду, терминал зависает, потому что сервер блокируется по вызову `accept()` — он ожидает, пока подключится новый клиент.

В качестве клиента мы будем использовать Netcat (<http://netcat.sourceforge.net>; также можно воспользоваться сценарием `Chapter 10/pizza_client.py`). Чтобы запустить клиент, откройте другое окно терминала и дайте соответствующую команду для UNIX/macOS:

```
$ nc 127.0.0.1 12345
```

ПРИМЕЧАНИЕ В Windows используйте команду `ncat`: `$ ncat 127.0.0.1 12345`.

Когда клиент запустится, можно начинать вводить сообщения — то есть заказы пиццы. Если сервер работает, вы увидите от него ответ:

```
$ nc 127.0.0.1 12345
10
Спасибо за заказ пиццы (10 шт.)!
```

Также отобразится вывод сервера:

```
Сервер запущен на узле 127.0.0.1:12345
Сервер прослушивает входящие соединения
Установлено соединение с узлом ('127.0.0.1', 52856)
Отправляется сообщение для узла ('127.0.0.1', 52856)
Соединение с узлом ('127.0.0.1', 52856) закрыто
```

Сервер прослушивает входящие соединения; когда клиент подключается, сервер обменивается с ним данными, пока соединение не закроется (для этого закройте клиент). Затем сервер продолжает прослушивать новые соединения. Не поленитесь потратить немного времени, чтобы внимательно изучить этот код.

Наш сервер работает: теперь клиент может заказать пиццу! Однако в реализации есть проблема, которую мы упустили.

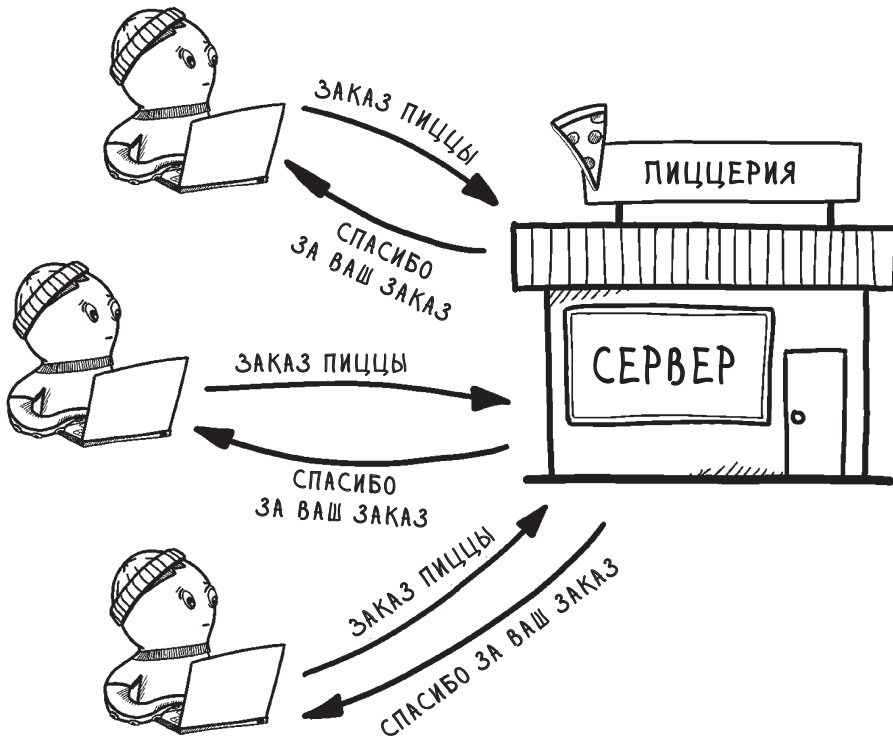
Необходимость в конкурентности

Как и в приложении Santa Cruz Operation, эта версия сервера не конкурентна. Если несколько клиентов пытаются подключиться к серверу одновременно, один клиент подключается и занимает сервер, а остальные ждут, пока текущий клиент отключится. В этом коде сервер фактически блокируется по одному клиентскому соединению!

Чтобы убедиться в этом, попробуйте запустить новый клиент в отдельном терминале. Вы увидите, что второй клиент ждет подключения, пока первый не

отключится. Без конкурентности сервер не может обрабатывать сразу несколько клиентских соединений.

Однако в реальном веб-приложении конкурентность неизбежна: множество клиентов и серверов соединяются по сети, все они одновременно отправляют и получают сообщения и ожидают своевременных ответов. Таким образом, веб-приложение по своей природе является конкурентной системой и требует конкурентных решений. Следовательно, конкурентность — не только особенность веб-архитектуры, но и необходимый и определяющий принцип для реализации крупномасштабных веб-приложений, который позволяет использовать оборудование наиболее эффективно.



Многопоточный сервер

В качестве стандартного решения можно использовать потоки или процессы. Как упоминалось ранее, потоки в общем случае потребляют меньше ресурсов, поэтому реализуем сервер с их помощью:

```
# Chapter 10/threaded_pizza_server.py
from socket import socket, create_server
from threading import Thread
```

```
BUFFER_SIZE = 1024
ADDRESS = ("127.0.0.1", 12345)
```

```

class Handler(Thread):
    def __init__(self, conn: socket):
        super().__init__()
        self.conn = conn

    def run(self) -> None:
        print(f"Установлено соединение с узлом {self.conn.getpeername()}")
        try:
            while True:
                data = self.conn.recv(BUFFER_SIZE)
                if not data:
                    break
                try:
                    order = int(data.decode())
                    response = f"Спасибо за заказ пиццы ({order} шт.)!\n"
                except ValueError:
                    response = "Недопустимое количество пицц.\n"
                    response += "Попробуйте еще раз\n"
                print(f"Отправляется сообщение для узла {self.conn.getpeername()}")
                self.conn.send(response.encode())
            finally:
                print(f"Соединение с узлом {self.conn.getpeername()} "
                      f"закрыто")
                self.conn.close()

class Server:
    def __init__(self) -> None:
        try:
            print(f"Сервер запущен на узле {ADDRESS}")
            self.server_socket = create_server(ADDRESS)
        except OSError:
            self.server_socket.close()
            print("\nСервер остановлен")

    def start(self) -> None:
        print("Сервер прослушивает входящие соединения")
        try:
            while True:
                conn, address = self.server_socket.accept()
                print(f"Запрос на подключение клиента от узла {address}")
                thread = Handler(conn)
                thread.start()
            finally:
                self.server_socket.close()
                print("\nСервер остановлен.")

if __name__ == "__main__":
    server = Server()
    server.start()

```

*Когда сервер получает
запрос на подключение
клиента, для каждого
клиента создается
новый поток, который
будет обслуживать это
подключение*

В этой реализации главный поток содержит прослушивающий серверный сокет, который принимает входящие соединения от клиентов. Каждый клиент, который подключается к серверу, обслуживается в отдельном потоке. Остальной код остается без изменений.

Конкурентность обеспечивается благодаря тому, что работают несколько потоков. ОС выполняет их в перемежающемся режиме с вытесняющим планированием. Мы уже знакомы с этим подходом; он упрощает модель программирования, потому что все потоки, которые нужны, чтобы обрабатывать запросы, можно писать по единой схеме. Более того, такой подход предоставляет простую абстракцию, благодаря которой разработчики избавляются от низкоуровневых подробностей планирования и могут полагаться на поддержку ОС и среды выполнения.

ПРИМЕЧАНИЕ Этот принцип применяется во многих продуктах — в частности, в популярном модуле `prefork` веб-сервера Apache, сервлетах Jakarta EE (до версии 3), фреймворке Spring (до версии 5), Phusion Passenger для Ruby on Rails и Python Flask.

Многопоточный сервер, который мы описали, вроде бы идеально решает проблему обслуживания многих клиентов — но за это приходится платить.

Проблема C10k

Современное серверное приложение обрабатывает сотни, тысячи и даже десятки тысяч клиентских запросов (потоков) в конкурентном режиме, ожидая своевременных ответов. И хотя на создание потоков расходуется относительно мало ресурсов, ОС тратит заметное время, драгоценную оперативную память и другие мощности, чтобы управлять потоками. Для таких мелких задач, как обработка одиночных запросов, затраты на управление потоками могут превысить выгоду от конкурентного выполнения.

ПРИМЕЧАНИЕ Многие ОС с трудом управляются более чем с несколькими тысячами потоков (а обычно предел наступает еще раньше). Вы можете испытать свою машину, воспользовавшись кодом из файла `Chapter 10/thread_cost.py`.

ОС постоянно распределяет процессорное время между всеми потоками независимо от того, готов ли поток продолжить выполнение. Например, поток может ожидать данных из сокета, но планировщик ОС будет тысячу раз переключаться на этот поток и обратно, прежде чем тот выполнит какую-то полезную работу. Когда приходится реагировать на тысячи одновременных запросов на соединение, используя множество потоков или процессов, это отнимает у системы немало ресурсов и ухудшает ее отзывчивость.

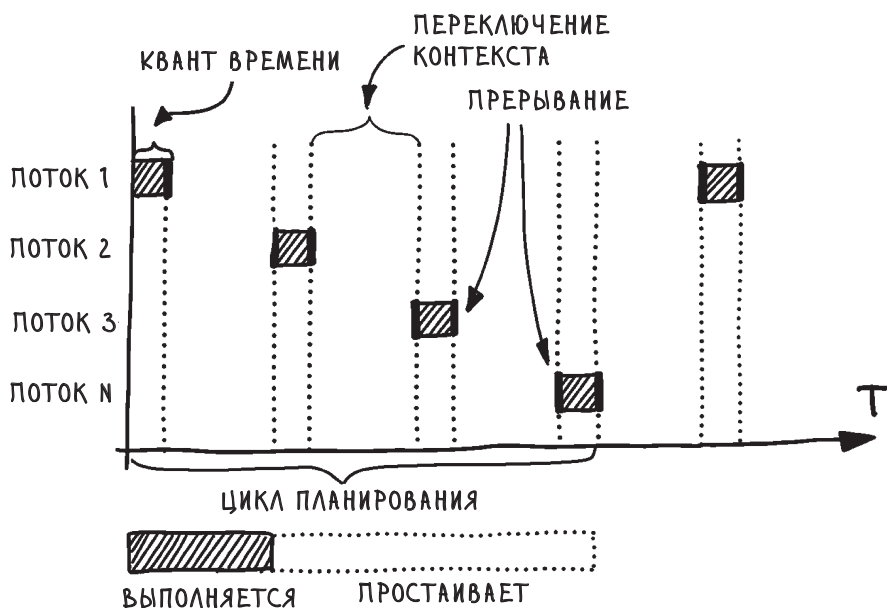
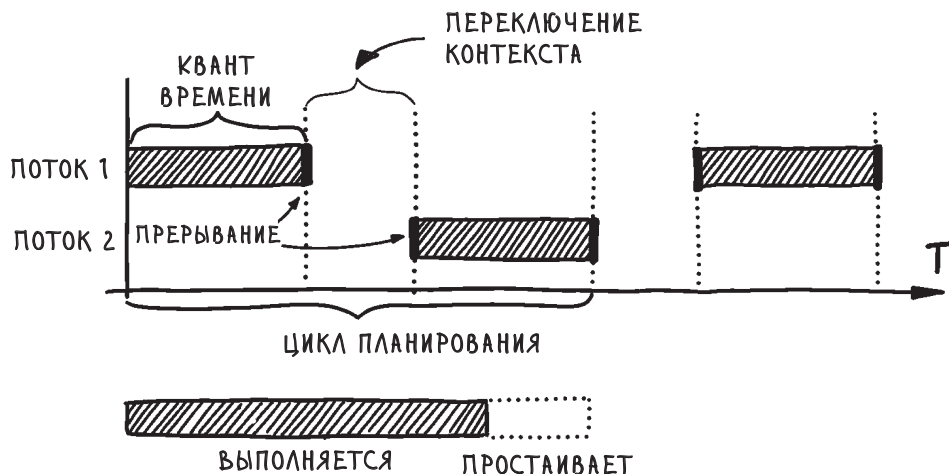
Вспомните вытесняющий планировщик из главы 6, который подбирает процессорное ядро для потока. Если машина сильно загружена, это может потребовать непродолжительного ожидания. А когда оно закончится, обычно оказывается, что время, выделенное для потока, уже исчерпано, и поток возвращается в состояние готовности, ожидая новых порций процессорного времени.

Теперь представьте, что вы задали период планирования в 10 мс и у вас есть два потока; тогда каждому из них достанется по 5 мс. Если у вас 5 потоков, каждому достанется по 2 мс. Но что произойдет, если у вас 1000 потоков? Каждый из них получит квант времени продолжительностью 10 мкс. В таком случае потоки будут

проводить все время за переключением контекстов и не смогут выполнять никакой реальной работы.

Необходимо ограничить минимальную продолжительность кванта. В последнем случае, если она равна 2 мс и используется 1000 потоков, то цикл планирования нужно увеличить до 2 с. Если потоков 10 000, то цикл составит 20 с.

Если в нашем простом примере каждый поток полностью использует свой квант времени, то конкурентное выполнение всех потоков займет 20 с. Это слишком долго.



Переключение контекста между потоками занимает драгоценное процессорное время. Чем больше потоков, тем больше времени уйдет на переключения вместо реальной работы. Таким образом, запускать и останавливать потоки может оказаться весьма затратно.

С высоким уровнем конкурентности (допустим, 10 000 потоков, если ваша ОС сможет столько создать) большое количество потоков может ухудшить пропускную способность из-за частых затрат на переключение контекста. Эта проблема масштабирования называется *проблемой C10k*¹ и не позволяет серверам обрабатывать более 10 000 одновременных соединений.

ПРИМЕЧАНИЕ Поскольку технология не стоит на месте, к нашему времени проблема расширилась до C10m — иначе говоря, до поддержки 10 миллионов одновременных соединений или 1 миллиона соединений в секунду.

К сожалению, потоки не помогают решить проблему C10k. Чтобы с ней бороться, придется менять подход. Но сперва вспомните, для чего изначально были нужны потоки: они позволяли работать с блокирующими операциями.

Блокирующий ввод/вывод

Ожидая данные от ввода/вывода, вы получаете ответ с задержкой. Она может быть небольшой, если вы обращаетесь к файлу на жестком диске, или более долгой, если данные передаются по сети на значительное расстояние. Например, файл, который хранится на жестком диске, должен достичь процессора по кабелям SATA и шинам системной платы, а данным из сетевого ресурса, который находится на удаленном сервере, приходится преодолевать много километров через сетевые кабели и маршрутизаторы и в конечном счете проходить через сетевой адаптер вашего компьютера, прежде чем попасть к процессору. Это означает, что приложение *блокируется*, пока системный вызов ввода/вывода не завершится. Вызывающее приложение находится в состоянии, в котором оно не использует процессор, а просто ждет ответа, что неэффективно с точки зрения обработки. И чем больше операций ввода/вывода, тем заметнее известная проблема: процессор простаивает и не выполняет никакой реальной работы.

ПРИМЕЧАНИЕ Любая операция ввода/вывода последовательна по своей природе: она сводится к тому, чтобы отправить сигнал и ждать ответа. В этом процессе нет ничего конкурентного, поэтому закон Амдала (см. главу 2) здесь работает в полной мере.

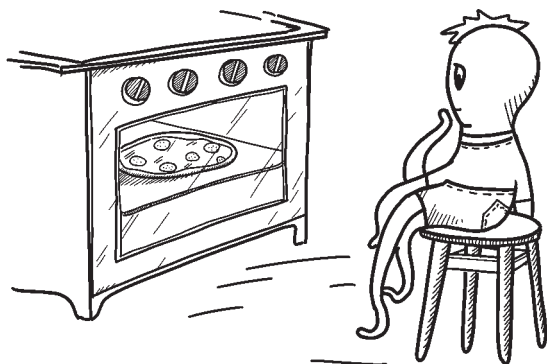
Пример

Допустим, вместо того чтобы заказывать пиццу по телефону, вы решили приготовить ее дома. Для этого вы выложили на основу соус, сыр, пепперони и оливки

¹ В буквальном переводе — «проблема 10 000 соединений», см. Daniel Kegel, «The C10K problem», <http://www.kegel.com/c10k.html>.

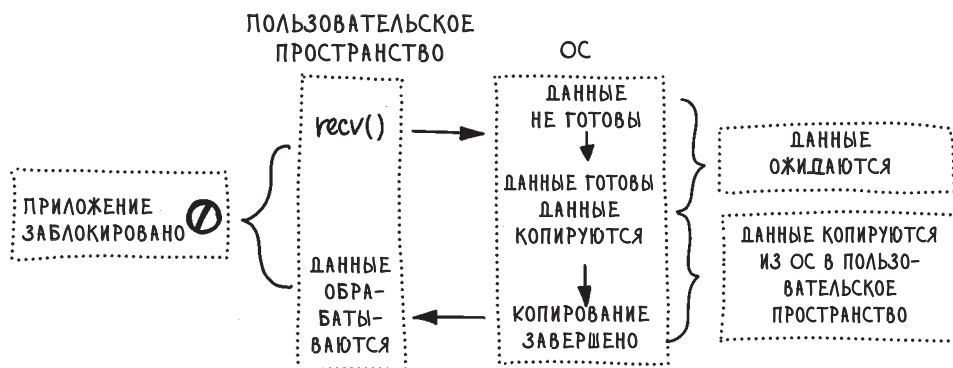
(пицца с ананасами в нашем доме строго запрещена). Вы ставите пиццу в духовку и ждете, пока сыр расплавится, а основа подрумянится. С этого момента вам больше нечего делать; духовка сама все готовит. Вам остается лишь дождаться правильного момента, чтобы достать готовую пиццу.

Итак, вы ставите табуретку перед духовкой, усаживаетесь и бдительно следите за пиццей, стараясь не упустить критический момент, когда она начнет подгорать.



При таком подходе вы не сможете заняться ничем другим, потому что большую часть времени сидите в ожидании перед духовкой. Это *синхронизированная* задача; вы как бы «синхронизируетесь» с духовкой. Вам приходится ждать, чтобы оказаться на месте в тот момент, когда пицца будет готова.

Аналогичным образом традиционные вызовы сокетов `send()` и `recv()` являются блокирующими по своей природе. Если нет сообщений, которые можно принять, то системные вызовы `recv()` блокируют программу, пока не поступят данные. Программа просто берет табуретку, садится и ждет, пока клиент пришлет данные.



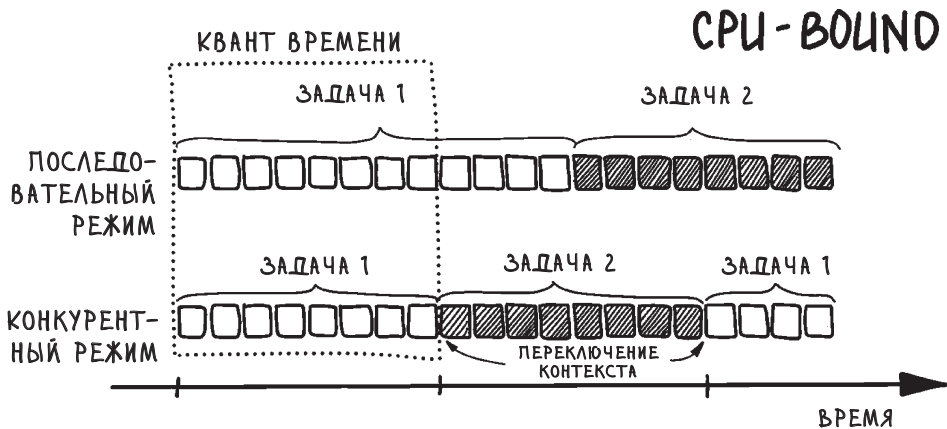
Если не указано обратное, почти все интерфейсы ввода/вывода (включая интерфейсы сетевых сокетов) являются блокирующими. В традиционных настольных

приложениях операции с интенсивным вводом/выводом обычно выполняются лишь изредка. Но для веб-серверов ввод/вывод является основной задачей, и оказывается, что сервер не использует процессор, пока ожидает ответа от клиента. Такая коммуникация крайне неэффективна, потому что основана на блокировании.

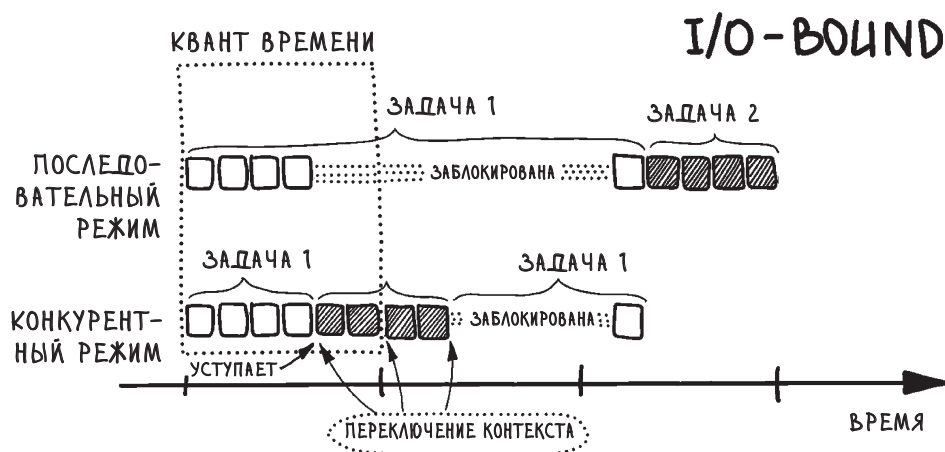
Оптимизация ОС

Но зачем заставлять процессор бездействовать в ожидании запроса? Когда задача блокируется, ОС помещает ее в заблокированное состояние, пока не завершится операция ввода/вывода. Чтобы эффективно использовать физические ресурсы, ОС немедленно «паркует» заблокированную задачу (снимает ее с ядра процессора и сохраняет в системе) и выделяет процессорное время другой задаче, которая находится в состоянии готовности. Сразу же после того, как завершится ввод/вывод, задача переходит из заблокированного состояния в состояние готовности — и, возможно, в состояние выполнения, когда планировщик ОС примет такое решение.

Как мы уже видели, в программах с интенсивными вычислениями (CPU-bound) переключение контекста становится катастрофой для производительности. Поскольку вычислительным задачам всегда найдется, чем заняться, им не нужно ничего ждать; переключение контекста мешает выполнять эту полезную работу.



Если программа выполняет много операций с интенсивным вводом/выводом (I/O-bound), то переключение контекста становится преимуществом. Как только задача переходит в заблокированное состояние, ее место занимает другая задача в состоянии готовности. Это позволяет процессору не простаивать, если для него есть задачи в состоянии готовности. Эта ситуация принципиально отличается от ситуации с интенсивными вычислениями.



Таким образом, если функция блокируется (по любой причине), она может задерживать выполнение других задач, и общий прогресс всей системы пострадает. Если функция блокируется из-за того, что она выполняет задачу с интенсивными вычислениями, то здесь практически ничего не поделать. Но если она блокируется из-за интенсивного ввода/вывода, значит, процессор простаивает, и его можно использовать, чтобы выполнять другую задачу.

Блокирование происходит во всех конкурентных программах и не только из-за операций ввода/вывода: оно возникает из-за входных и выходных взаимодействий процессов (в том числе по сети), чтения и записи файлов, взаимодействия с пользователем в командной строке или графическом интерфейсе и т. д. Конкурентные модули не работают синхронно, как последовательные программы. Обычно им приходится ждать друг друга там, где нужно координировать действия.

Теперь представьте, что можно создать операцию, которая не будет блокироваться.

Неблокирующий ввод/вывод

Как говорилось в главе 6, конкурентность можно реализовать и без параллелизма. В частности, это может принести пользу, если у вас много задач с интенсивным вводом/выводом. Тогда можно отказаться от конкурентности на базе потоков, чтобы добиться большей масштабируемости, и обойти проблему C10k благодаря *неблокирующему вводу/выводу*.

Его идея состоит в том, чтобы запросить операцию ввода/вывода, но не дожидаться ответа, а перейти к другим задачам. Например, в случае неблокирующего чтения можно запросить данные из сетевого сокета и переключиться на поток, который выполняет другую работу (например, обслуживает другое соединение), до того момента, когда данные поступят в буфер и будут готовы к потреблению. У такого подхода есть свой минус: приходится периодически проверять, готовы ли данные к чтению.

Если в неблокирующей реализации на Python вызывается `send()`, `recv()` или `accept()`, но у устройства нет данных для чтения, то выполнение не блокируется, а вместо этого возбуждается исключение `BlockingIOError`. Это означает, что на этом месте могло бы возникнуть блокирование, и вызывающая сторона должна попытаться повторить операцию позже.

Также можно вообще не создавать новых потоков: в случае неблокирующего ввода/вывода они не дают особых преимуществ. Наоборот, они только потребляют больше памяти и тратят процессорное время на переключение контекста. Вот возможная реализация:

```
# Chapter 10/pizza_busy_wait.py
import typing as T
from socket import socket, create_server

BUFFER_SIZE = 1024
ADDRESS = ("127.0.0.1", 12345)

class Server:
    clients: T.Set[socket] = set()
    def __init__(self) -> None:
        try:
            print(f"Сервер запущен на узле {ADDRESS}")
            self.server_socket = create_server(ADDRESS)
            self.server_socket.setblocking(False)
        except OSError:
            self.server_socket.close()
            print("\nСервер остановлен.")

    def accept(self) -> None:
        try:
            conn, address = self.server_socket.accept()
            print(f"Установлено соединение с узлом {address}")
            conn.setblocking(False)
            self.clients.add(conn)
        except BlockingIOError:
            pass

    def serve(self, conn: socket) -> None:
        try:
            while True:
                data = conn.recv(BUFFER_SIZE)
                if not data:
                    break
                try:
                    order = int(data.decode())
                    response = f"Спасибо за заказ пиццы ({order} шт.)!\n"
                except ValueError:
                    response = "Недопустимое количество пицц.\n"
                    response += "Попробуйте еще раз\n"
                print(f"Отправляется сообщение для узла {conn.getpeername()}")
                conn.send(response.encode())
        except BlockingIOError:
            pass
```

Переводит серверный сокет в неблокирующий режим, чтобы он не блокировался, пока ожидает входящих соединений

Это исключение перехватывается, чтобы обрабатывать неблокирующие сокеты, когда нет данных, которые можно было бы читать из сокета. В результате программа не блокируется, а продолжает выполняться для других клиентов, у которых есть данные, доступные для чтения

```

def start(self) -> None:
    print("Сервер прослушивает входящие соединения")
    try:
        while True:
            self.accept()
            for conn in self.clients.copy():
                self.serve(conn)
    finally:
        self.server_socket.close()
        print("\nСервер остановлен.")

if __name__ == "__main__":
    server = Server()
    server.start()

```

В этой реализации сервера сокет переводится в неблокирующий режим вызовом `setblocking(False)`, так что серверное приложение не будет ждать, пока операция завершится. Затем для каждого неблокирующего сокета мы пытаемся выполнять операции `accept()`, `read()` и `send()` в бесконечном цикле `while` — цикле опроса. Он должен пытаться выполнять операции снова и снова, потому что теперь они не блокирующие: во время `send()` мы не знаем, готов ли сокет, так что приходится продолжать попытки, пока одна из них не окажется успешной. То же самое относится к другим вызовам. Таким образом, вызовы `send()`, `recv()` и `accept()` могут просто возвращать управление основному потоку, ничего не сделав.

ПРИМЕЧАНИЕ Существует распространенное заблуждение, будто неблокирующий ввод/вывод ускоряет операции ввода/вывода. На самом деле, хотя он не блокирует задачу, он не обязательно выполняется быстрее; просто он дает возможность приложению выполнять другие задачи, пока операции ввода/вывода не завершатся. Это позволяет лучше использовать процессорное время и эффективнее обрабатывать множественные соединения, что в конечном итоге улучшает общую производительность. Тем не менее скорость операций ввода/вывода прежде всего зависит от характеристик оборудования и быстродействия сети, а неблокирующий ввод/вывод не влияет на эти факторы.

Когда нет блокирующего ввода/вывода, несколько операций ввода/вывода могут перекрываться, даже если используется всего один поток. Таким образом создается иллюзия параллелизма, потому что несколько задач выполняются в конкурентном режиме (по аналогии с тем, как это происходило в главе 6).

Если применять неблокирующий ввод/вывод в подходящих обстоятельствах, он скрывает задержку и улучшает пропускную способность и/или отзывчивость приложения. Он также позволяет работать с одним потоком, а это может избавить нас от проблем синхронизации между потоками и от затрат на управление потоками и сопутствующими системными ресурсами.

Итоги

- В приложениях «клиент — сервер», которые взаимодействуют через ИРС с передачей сообщений, конкурентность неизбежна: несколько клиентов и серверов

связаны по сети, и все они одновременно отправляют и принимают сообщения, ожидая своевременных ответов.

- Когда в программе выполняется код с интенсивным вводом/выводом, процессор часто подолгу бездействует, потому что этот код всего лишь ожидает, пока завершится ввод/вывод.
- *Блокирующие* интерфейсы сначала выполняют всю свою работу, и только после этого возвращают управление вызывающей стороне. А *неблокирующие* интерфейсы начинают свою работу, но немедленно возвращают управление, чтобы дать возможность выполниться другим задачам. Если программа интенсивнее задействует ввод/вывод, чем ресурсы процессора, то выигрыш эффективности из-за неблокирующего ввода/вывода будет намного выше — как и следовало ожидать.
- Потоки и процессы ОС хорошо подходят для небольшого количества задач с высокой продолжительностью, потому что, если потоков становится слишком много, производительность ухудшается из-за постоянного переключения контекстов и потребления памяти в зависимости от размера стека потоков. Один из простых способов избавиться от затрат на создание потоков и процессов — использовать активное ожидание, при котором один поток может в конкурентном режиме обрабатывать множество клиентских запросов с помощью неблокирующих операций.



В этой главе

- ✓ Как бороться с недостатками неэффективного активного ожидания из главы 10
- ✓ Синхронизация IPC с передачей сообщений
- ✓ Событийная конкурентность
- ✓ Паттерн проектирования «Реактор»

Конкурентность — одно из критических качеств современной разработки, которое позволяет выполнять несколько задач одновременно и эксплуатировать оборудование с максимальной отдачей. Хотя традиционная конкурентность на основе потоков/процессов хорошо известна, она подходит не для каждого приложения. Более того, в высоконагруженных приложениях с интенсивным вводом/выводом событийная конкурентность часто оказывается более эффективным решением.

Событийная конкурентность подразумевает, что приложение строится на основе событий или сообщений, а не потоков или процессов. Когда происходит событие, приложение реагирует на него, вызывая функцию-обработчик, которая выполняет необходимые действия. У такого подхода есть ряд преимуществ перед традиционными моделями конкурентности: в частности, расходуется меньше ресурсов, улучшается масштабируемость, и приложение реагирует быстрее.

Реальные примеры событийной конкурентности часто встречаются во многих высокопроизводительных приложениях, таких как веб-серверы, мессенджеры и игровые платформы. Например, благодаря событийной конкурентности веб-сервер может обслуживать множество одновременных соединений с минимальным

потреблением ресурсов, а мессенджеру она помогает эффективно обрабатывать большие объемы сообщений.

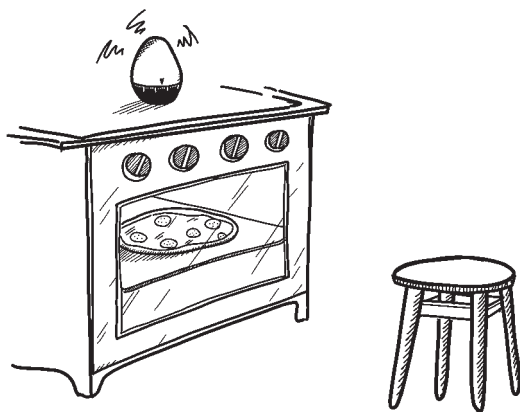
В этой главе мы изучим событийную конкурентность подробнее, сравним ее с традиционной конкурентностью на основе потоков/процессов и рассмотрим, как она чаще всего применяется в клиент-серверных приложениях. Вы узнаете о преимуществах и недостатках событийной конкурентности, а также о том, как эффективно проектировать и реализовывать событийные приложения. К концу главы вы будете достаточно хорошо разбираться в событийной конкурентности и ее практических применениях, чтобы выбирать правильный подход для своих проектов.

События

Возвращаясь к примеру с приготовлением пиццы из главы 10, мы видим, что активное ожидание в этом случае неэффективно и утомительно. Этот метод требует постоянно опрашивать все сокеты независимо от их состояния. Если у вас 10 000 сокетов, из которых только последний готов передавать или принимать данные, вам придется перебрать все сокеты только ради того, чтобы обнаружить сообщение, которое ожидает в последнем соке. Когда мы опрашиваем каждый сокет, чтобы проверить его состояние, процессор непрерывно занят работой. Это значит, что он тратит 99 % своего времени на опрос вместо того, чтобы выполнять другие задачи с интенсивными вычислениями, что неэффективно.

Нужен более совершенный механизм — такой, как *событийная конкурентность*.

Вы хотите знать, когда пицца будет готова, верно? Почему бы не поставить таймер, который оповестит об этом? Тогда в ожидании этого события вы сможете заняться чем-то более полезным. Когда таймер сообщит, что пицца готова, вы сможете обработать это событие и отведать горячей свежей пиццы.



Событийная конкурентность ориентирована на *события*. Мы просто ожидаем, пока произойдет некоторое событие. Это может быть событие ввода/вывода (например, данные готовы к потреблению или сокет готов к записи), или срабатывание таймера,

или что-то еще. Когда это случается, мы проверяем тип события и обрабатываем его должным образом — например, выполняем запросы ввода/вывода, планируем другие события и т. д.

ПРИМЕЧАНИЕ Пользовательские интерфейсы почти всегда проектируются как событийные программы, потому что их задача — реагировать на действия пользователей. Например, JavaScript исторически служил для того, чтобы взаимодействовать с объектной моделью документа (DOM) и с пользователем в браузере, так что для этого языка событийная модель программирования подходит естественным образом. Но этот стиль стал популярным и в некоторых современных системах, включая серверные фреймворки (такие как Node.js). Другой пример продукта с событийной конкурентностью — библиотека React.js, с помощью которой часто реализуют пользовательские интерфейсы. Чтобы обновлять интерфейс в ответ на пользовательский ввод или другие события, React.js не модифицирует DOM напрямую, а использует виртуальную модель DOM и обработчики событий. Благодаря этому React.js минимизирует обновления DOM и экономит ресурсы, группируя обновления.

Обратные вызовы

В событийной программе необходимо указать, какой код нужно выполнять, когда наступает то или иное событие. Этот код называется *обратным вызовом* (callback).

Обратные вызовы работают по тому же принципу, что заказ обратного звонка по телефону. Представьте, что вы звоните, чтобы заказать пиццу, но автоответчик приятным голосом предлагает вам подождать на линии, пока оператор не освободится, или заказать обратный звонок. Если вы его закажете, то освободившийся оператор перезвонит вам и примет заказ. Так что вместо того, чтобы ждать на линии, можно заказать обратный звонок и заняться чем-то еще. Когда оператор перезвонит, вы вернетесь к своему исходному плану и закажете пиццу.



ПРИМЕЧАНИЕ Когда код основан на обратных вызовах, логика программы часто становится менее очевидной, и ее сложнее отлаживать. Код перестает быть понятным и удобочитаемым. Когда-то код можно было читать последовательно, но теперь его логику приходится проследживать среди разрозненных обратных вызовов. Цепочка операций в коде может вызвать каскад вложенных обратных вызовов; такую ситуацию еще называют «*кошмаром обратных вызовов*».

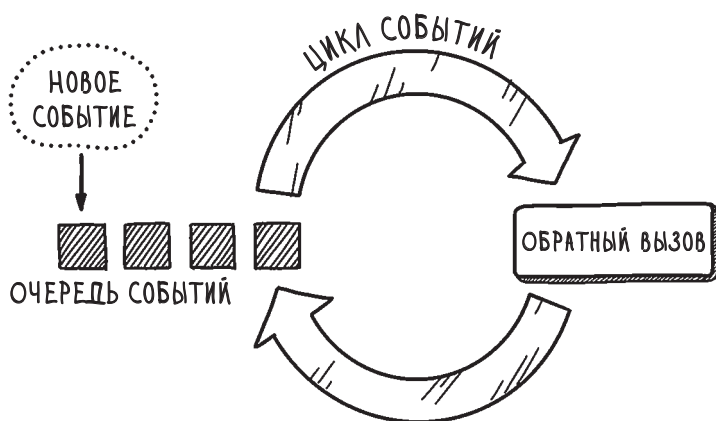
Итак, у нас есть события и обратные вызовы. Как заставить их работать в одной схеме? В основе событийной конкурентности лежит цикл событий.

Цикл событий

Чтобы связать события с обратными вызовами, необходимо ввести управляющую сущность, которая отслеживает разные события и запускает соответствующие обратные вызовы. Она обычно называется *циклом событий*.

Вместо опроса, как это делалось в реализации с активным ожиданием, события помещаются в *очередь событий*. Цикл ожидает событий, непрерывно извлекает их из очереди и запускает соответствующие обратные вызовы.

На следующей диаграмме представлен пример того, как устроена логика выполнения типичной событийной программы. Цикл событий непрерывно извлекает события из очереди событий и выполняет соответствующие обратные вызовы. На диаграмме показаны события только одного типа, которому соответствует один обратный вызов, однако следует заметить, что в некоторых событийных приложениях количество событий и обратных вызовов теоретически может быть бесконечным.



По сути цикл событий делает только одно: он ждет, когда произойдет событие, находит для каждого события обратный вызов, который был зарегистрирован заранее, и выполняет его.

ПРИМЕЧАНИЕ Цикл событий — душа и сердце JavaScript. В этом языке нельзя создавать новые потоки, но конкурентность достигается за счет механизма циклов событий. Так JavaScript заполняет пробел между многопоточностью и конкурентностью, в результате чего он становится серьезным игроком в области, занятой конкурентными языками — такими как Java, Go, Python, Rust и другие. Во многих инструментариях GUI — например, Java Swing — тоже предусмотрен цикл событий.

Попробуем реализовать эту идею в коде:

```
# Chapter 11/event_loop.py
from collections import deque
from time import sleep
import typing as T

class Event:
    def __init__(self, name: str, action: T.Callable[..., None],
                  next_event: T.Optional[Event] = None) -> None:
        self.name = name
        self._action = action
        self._next_event = next_event

    def execute_action(self) -> None:
        self._action(self)
        if self._next_event:
            event_loop.register_event(self._next_event)

class EventLoop:
    def __init__(self) -> None:
        self._events: deque[Event] = deque()

    def register_event(self, event: Event) -> None:
        self._events.append(event)

    def run_forever(self) -> None:
        print(f"Очередь запущена, количество событий: {len(self._events)}")
        while True:
            try:
                event = self._events.popleft()
            except IndexError:
                continue
            event.execute_action()

def knock(event: Event) -> None:
    print(event.name)
    sleep(1)

def who(event: Event) -> None:
    print(event.name)
    sleep(1)
```

Класс Event представляет действие, выполняемое циклом событий

Создает очередь для хранения событий, которые обрабатывает цикл событий

Добавляет событие в очередь событий

Бесконечно выполняет цикл событий и обрабатывает каждое событие в очереди, когда оно становится доступным

```

if __name__ == "__main__":
    event_loop = EventLoop()
    replying = Event("Кто там?", who)
    knocking = Event("Тук-тук", knock, replying)
    for _ in range(2):
        event_loop.register_event(knocking)
        event_loop.run_forever()

```

Регистрирует в цикле событий обратные вызовы: они представляют действия, которые нужно выполнить, когда срабатывают события с соответствующими именами

Запускает цикл событий, который выполняется неопределенно долго, пока не будет прерван, и непрерывно проверяет, не появились ли в очереди новые события, которые нужно обработать

Здесь мы создаем цикл событий и регистрируем два события: `knock` и `who` (обратите внимание, что событие `knock` может порождать событие `who`). Затем мы вручную генерируем два события `knock`, как если бы они только что произошли, и запускаем бесконечный цикл событий. Мы видим, что он обрабатывает одно событие после другого:

Очередь запущена, количество событий: 2
 Тук-тук
 Тук-тук
 Кто там?
 Кто там?

В конечном счете последовательность выполнения приложения зависит от событий. Но как приложение узнает, какое событие нужно обрабатывать следующим?

Мультиплексирование ввода/вывода

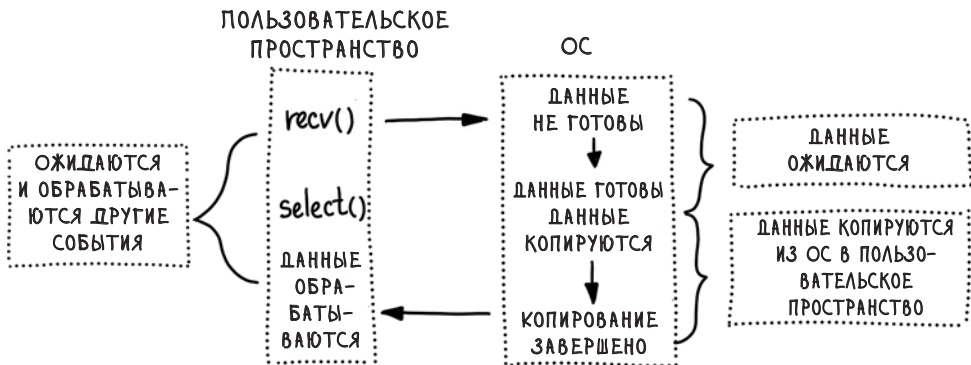
В современных ОС обычно есть подсистемы, которые оповещают о событиях, — так называемые подсистемы мультиплексирования ввода/вывода. Они собирают и ставят в очередь события ввода/вывода от наблюдаемых ресурсов и блокируют их, пока пользовательское приложение не будет готово их обработать. Это позволяет приложению достаточно просто проверять, нет ли входящих событий ввода/вывода, которые требуют внимания.

При мультиплексировании ввода/вывода не нужно отслеживать все события сокетов, как это делалось в предыдущей главе с активным ожиданием. Можно доверять ОС оповещать нас о том, какие события происходят в тех или иных сокетах. Приложение может поручить ОС наблюдать за сокетом и ставить события в очередь до тех пор, пока данные не будут готовы. Приложение может проверять события в любой момент — возможно, попутно занимаясь чем-то другим. Этот механизм предоставляется вызовом `select` — «дедушкой» всех системных вызовов.

Используя `select`, мы не обращаемся с вызовами к тому или иному сокету до того момента, пока `select` не сообщит о том, что с этим сокетом что-то произошло: например, данные поступили и готовы к чтению. Однако самое большое преимущество мультиплексирования ввода/вывода заключается в том, что можно

обрабатывать несколько запросов ввода/вывода к сокетам в конкурентном режиме, используя один поток. Мы можем зарегистрировать несколько сокетов и ждать входящих событий от всех них.

Если при вызове `select` сокет готов, он немедленно возвращает управление циклу событий. В противном случае он блокируется до того момента, когда какие-либо из зарегистрированных сокетов будут готовы. Когда поступает новое событие чтения или сокет доступен для записи, `select` возвращает новые события, помещает их в очередь событий и возвращает управление циклу событий. При этом приложение может получать новые запросы, пока обрабатывает предыдущий запрос. Такая схема обеспечивает, что обработка предыдущего запроса не будет блокироваться, но управление быстро возвращается циклу событий, чтобы обработать новый запрос.



ПРИМЕЧАНИЕ Во многих ОС есть более эффективный интерфейс для оповещения о событиях: POSIX предоставляет `poll`, в Linux есть `epoll`, FreeBSD и macOS используют `kqueue`, в Windows есть `IOCP`, в Solaris — `/dev/poll` и т. д. Эти базовые примитивы позволяют построить неблокирующий цикл событий, который просто проверяет входящие пакеты, читает сообщения сокетов и реагирует соответствующим образом.

Благодаря мультиплексированию ввода/вывода мы в конкурентном режиме выполняем несколько операций ввода/вывода с разными сокетами, используя один поток выполнения без постоянного опроса входящих событий. Вместо этого ОС управляет входящими событиями и оповещает приложение только при необходимости. Системный вызов `select` остается блокирующим, но система не тратит время в ожидании, пока поступят данные, и не расходует ресурсы процессора на бесконечный цикл опроса событий, как в случае с активным ожиданием.

Событийный сервер для заказа пиццы

Теперь мы готовы реализовать однопоточную конкурентную версию сервера для заказа пиццы с мультиплексированием ввода/вывода! Центральное место в программе снова занимает цикл событий — бесконечный цикл, который на каждой итерации

готовится выполнять операции чтения или записи с сокетами из системного вызова `select` и вызывает соответствующие зарегистрированные обратные вызовы:

```
# Chapter 11/pizza_reactor.py
class EventLoop:
    def __init__(self) -> None:
        self.writers = {}
        self.readers = {}

    def register_event(self, source: socket, event: Mask,
                      action: Action) -> None:
        key = source.fileno()
        if event & select.POLLIN:
            self.readers[key] = (source, event, action)
        elif event & select.POLLOUT:
            self.writers[key] = (source, event, action)

    def unregister_event(self, source: socket) -> None:
        key = source.fileno()
        if self.readers.get(key):
            del self.readers[key]
        if self.writers.get(key):
            del self.writers[key]

    def run_forever(self) -> None:
        while True:
            readers, writers, _ = select.select(
                self.readers, self.writers, [])
            for reader in readers:
                source, event, action = self.readers.pop(reader)
                action(source)
            for writer in writers:
                source, event, action = self.writers.pop(writer)
                action, msg = action
                action(source, msg)
```

Отслеживает сокеты, готовые к операциям ввода/вывода (чтению или записи)

Получает уникальный идентификатор, связанный с сокетом

Указывает, что данные доступны для чтения из сокета

Указывает, что сокет готов записывать данные

Удаляет сокет из массива `readers` или `writers`, когда клиент закрывает соединение

Запускает бесконечный цикл, в котором программа с помощью `select` ожидает, пока сокеты из `readers` или `writers` будут готовы к вводу/выводу

Для каждого сокета, готового к чтению, выполняется соответствующее действие, после чего сокет удаляется из массива `readers`

Для каждого сокета, готового к записи, выполняется соответствующее действие, после чего сокет удаляется из массива `writers`

Внутри метода `run_forever` цикла событий мы вызываем системную функцию `select` и ждем, пока она сообщит, что у клиентов появились новые события, которые нужно обработать. Это блокирующая операция, а значит, цикл событий не будет работать «вхолостую»; он ожидает, пока поступит хотя бы одно событие. `select` сообщает, когда сокет готов к чтению или записи, и мы запускаем соответствующий обратный вызов.

Отправку и получение данных необходимо инкапсулировать в независимые функции — обратные вызовы для каждого из ожидаемых типов событий `_on_accept()`, `_on_read()` и `_on_write()`. Затем мы поручаем ОС (а не нашему приложению) отслеживать состояние клиентских сокетов. Все, что для этого нужно, — зарегистрировать все клиентские сокеты со всеми ожидаемыми событиями и соответствующими обратными вызовами. Именно это происходит внутри класса `Server`:

Chapter 11/pizza_reactor.py

class Server:

def __init__(self, event_loop: EventLoop) -> None:

self.event_loop = event_loop

try:

print(f"Сервер запущен на узле {ADDRESS}")

self.server_socket = create_server(ADDRESS)

self.server_socket.setblocking(False)

except OSError:

self.server_socket.close()

print("\nСервер остановлен.")

Обратный вызов, который срабатывает, когда к серверу подключается новый клиент; он регистрирует соединение, чтобы цикл событий отслеживал его на предмет входящих данных

def _on_accept(self, _: socket) -> None:

try:

conn, client_address = self.server_socket.accept()

except BlockingIOError:

return

conn.setblocking(False)

print(f"Установлено соединение с узлом {client_address}")

self.event_loop.register_event(conn, select.POLLIN, self._on_read)

self.event_loop.register_event(self.server_socket, select.POLLIN, self._on_accept)

def _on_read(self, conn: socket) -> None:

try:

data = conn.recv(BUFFER_SIZE)

except BlockingIOError:

return

if not data:

self.event_loop.unregister_event(conn)

print(f"Соединение с узлом {conn.getpeername()} закрыто")

conn.close()

return

message = data.decode().strip()

self.event_loop.register_event(conn, select.POLLOUT, (self._on_write, message))

Обратный вызов, который срабатывает, когда от клиентского соединения поступают данные

Обратный вызов, который срабатывает, когда сервер готов отправить ответ клиенту

def _on_write(self, conn: socket, message: bytes) -> None:

try:

order = int(message)

response = f"Спасибо за заказ пиццы ({order} шт.)!\n"

except ValueError:

response = "Недопустимое количество пицц.\n\nПопробуйте еще раз\n"

print(f"Отправляется сообщение для узла {conn.getpeername()}")

try:

conn.send(response.encode())

except BlockingIOError:

return

self.event_loop.register_event(conn, select.POLLIN, self._on_read)

```

def start(self) -> None:
    print("Сервер прослушивает входящие соединения")
    self.event_loop.register_event(self.server_socket, select.POLLIN,
                                   self._on_accept)

if __name__ == "__main__":
    event_loop = EventLoop()
    Server(event_loop= event_loop).start()
    event_loop.run_forever()

```

Запускает сервер, регистрируя серверный сокет для цикла событий, и назначает функцию обратного вызова, которая будет принимать соединения с новыми клиентами

В этой реализации мы сначала создаем серверный сокет по аналогии с предыдущим решением. Однако вместо монолитного кода приложение организовано в виде набора обратных вызовов, каждый из которых обрабатывает определенный тип событий. После того как все эти компоненты настроены, мы запускаем сервер и он начинает прослушивать входящие соединения.

Центральный компонент реализации — цикл событий, который обрабатывает события из очереди событий и вызывает для них соответствующие обработчики. Цикл событий передает управление соответствующей функции обратного вызова и снова принимает управление, когда та завершит выполнение. Это продолжается до тех пор, пока в очереди остаются необработанные события. Когда все события обработаны, цикл событий возвращает управление функции `select`, которая снова блокируется, ожидая, пока завершатся новые операции.

С помощью такой событийной архитектуры и цикла событий нам удастся успешно обслуживать много клиентов одновременно в конкурентном режиме, запустив цикл событий в рамках одного потока. Ура!

Паттерн «Реактор»

Цикл событий, который ожидает, пока произойдут события, и обрабатывает их, встречается настолько часто, что приобрел статус полноценного паттерна проектирования, который называется «Реактор» (reactor). Когда мы запускаем однопоточный цикл событий, где с помощью мультиплексирования ввода/вывода обрабатывается неблокирующий ввод/вывод и применяются подходящие обратные вызовы, мы фактически реализуем «Реактор».

Этот паттерн обрабатывает входящие запросы, которые поступают в приложение от одного или нескольких клиентов. Здесь приложение — это по сути коллекция обратных вызовов, каждый из которых отвечает за обработку запросов для конкретного типа событий. Для паттерна «Реактор» нужно несколько составляющих: источники событий, обработчики событий, синхронный демультиплексор событий и структура реактора.

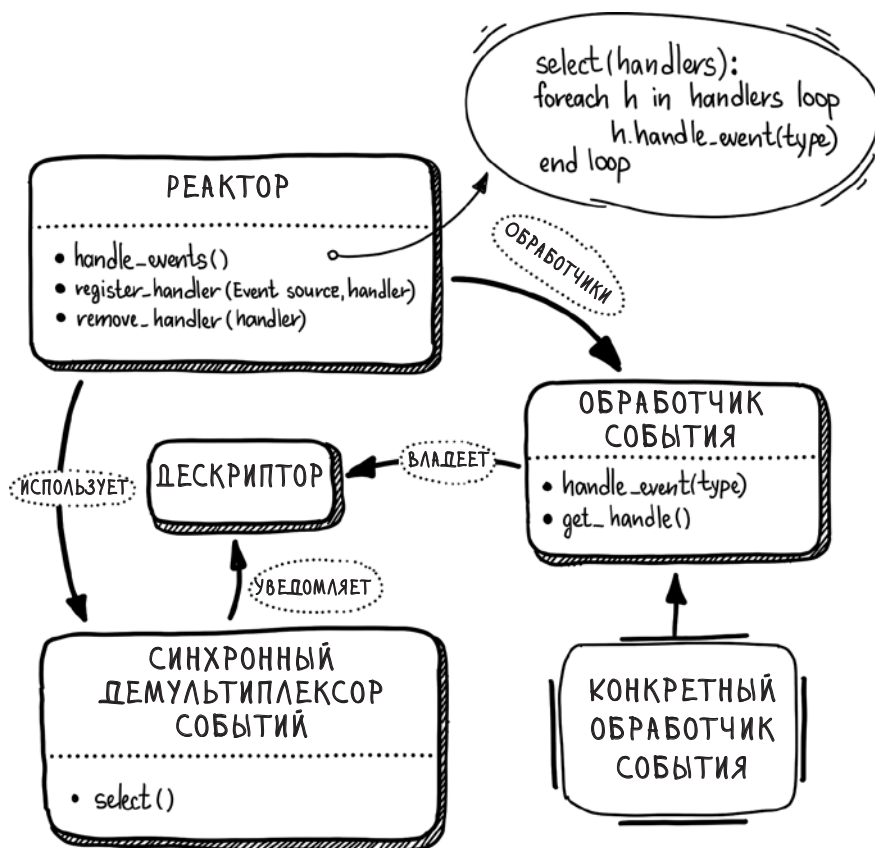
Источники событий — это сущности, которые порождают события, например файлы, сокеты, таймеры или объекты синхронизации. В коде сервера для заказа пиццы есть два источника событий: серверный сокет и клиентские сокеты.

Обработчики событий, которые по сути являются функциями обратного вызова, отвечают за обработку запросов от конкретных источников событий. В нашем коде используются три типа обработчиков:

- `_on_accept` — обслуживает серверный сокет и принимает новое соединение.
- `_on_read` — обрабатывает новое сообщение от клиентского соединения.
- `_on_write` — обрабатывает запись сообщений в клиентское соединение.

Синхронный демультиплексор событий — это ученое название для компонента, который принимает события от механизма оповещения, предоставленного ОС, — например, `select` или любого его аналога. Он ожидает, пока произойдут конкретные события для группы дескрипторов (`handles`).

Всем происходящим управляет *реактор*, он же цикл событий. Он регистрирует обратные вызовы для конкретных событий и реагирует на них, передавая работу подходящему зарегистрированному обработчику событий или обратному вызову. В нашем коде реактором служит класс `EventLoop`, который ожидает событий и реагирует на них. Когда вызов `select` возвращает список ресурсов, которые готовы к операциям ввода/вывода, реактор запускает соответствующие зарегистрированные обратные вызовы.



Таким образом, приложение на основе паттерна «Реактор» регистрирует источники и типы событий, которые его интересуют. Для каждого события оно обеспечивает соответствующий обработчик, который функционирует как обратный вызов. Синхронный демultipлексор ожидает событий и оповещает реактор, который в ответ запускает нужный обработчик, чтобы обработать событие.

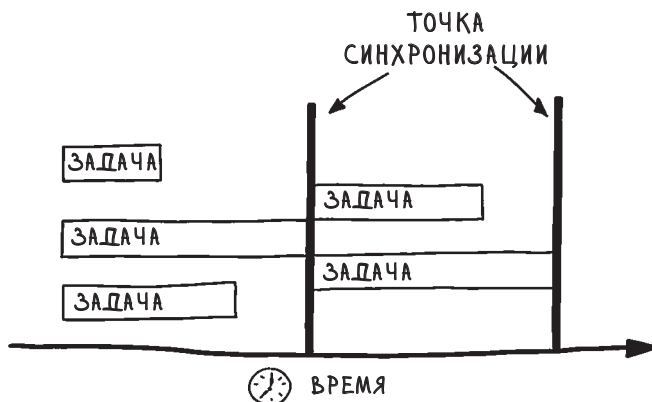
ПРИМЕЧАНИЕ На принципы, описанные в этом разделе, опираются многие популярные библиотеки и фреймворки. Libevent — распространенная и зрелая кроссплатформенная библиотека событий; libuv (абстрактная прослойка поверх libeio, libev, c-ares и iocp) реализует низкоуровневый ввод/вывод в Node.js, Java NIO, nginx и Vert.x, используя неблокирующие модели с реализацией цикла событий, чтобы достичь высокого уровня конкурентности.

Паттерн «Реактор» позволяет использовать модель событийной конкурентности, предотвращая затраты на работу с системными потоками (создание потоков и управление ими) и переключение контекста, а также избегая сложностей, которые связаны с общей памятью и блокировками в традиционных моделях на основе потоков. Конкурентность, основанная на событиях, позволяет потреблять гораздо меньше ресурсов, потому что используется только один поток выполнения. С другой стороны, она требует другого стиля программирования, в котором задействованы обратные вызовы и обработка событий, происходящих позднее.

Подведем итог: паттерн «Реактор» ориентирован на синхронную обработку событий, но асинхронную обработку ввода/вывода и опирается на систему оповещений о событиях ОС. Раз уж мы затронули понятие синхронизации, поговорим о ней подробнее.

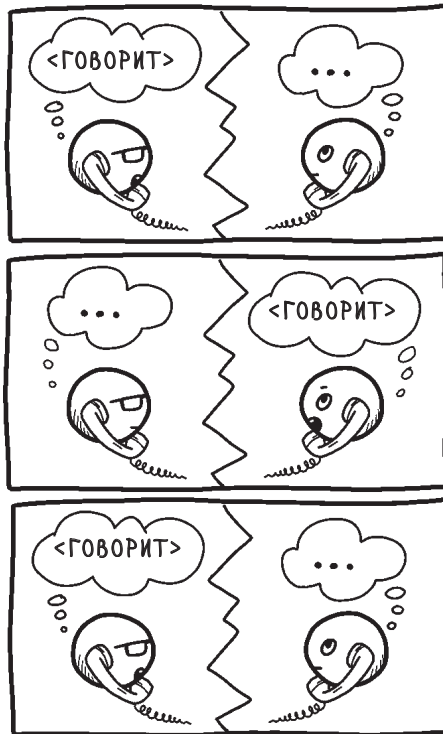
Синхронизация при передаче сообщений

Синхронизация при передаче сообщений заключается в том, чтобы координировать и упорядочивать задачи, которые зависят от конкретного порядка выполнения. Когда задачи синхронизированы, они выполняются в определенной последовательности, и следующие задачи ждут, пока завершатся предыдущие. Важно заметить, что синхронизация относится к начальным и конечным точкам задач, а не к тому, как задачи фактически выполняются между этими точками.



Синхронное взаимодействие требует, чтобы обе стороны были готовы обмениваться данными в одно и то же время, что создает явную точку синхронизации для обеих задач. В таком режиме выполнение программы блокируется, пока взаимодействие не завершится; таким образом, ресурсы системы простаивают. Наоборот, асинхронное взаимодействие происходит тогда, когда вызывающая сторона инициирует задачу, но не ждет, пока она завершится, а продолжает свою работу. Асинхронное взаимодействие не требует синхронизации при отправке и приеме данных, а отправитель не блокируется, если получатель еще не готов. Вызывающее приложение обращается к результатам асинхронно и может проверить события в любое удобное время. Такой подход позволяет процессору тратить время на выполнение других задач, а не простаивать в ожидании.

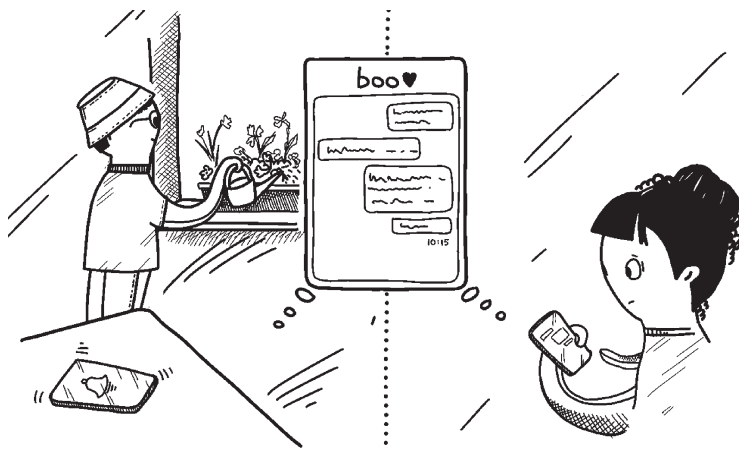
Чтобы продемонстрировать разницу между синхронным и асинхронным взаимодействием, посмотрим, как люди общаются по телефону. Во время звонка, пока один собеседник говорит, другой слушает. Когда первый перестает говорить, второй обычно немедленно начинает отвечать. Пока второй не ответил, первый ждет ответа. Это значит, что один собеседник не может продолжить говорить, пока другой не закончит.



...

В этом примере конечная точка первого собеседника синхронизируется с начальной точкой второго. Хотя такое общение позволяет немедленно воспринимать то, что говорит собеседник, на разговор уходит больше времени, потому что, когда человек читает, он за одно и то же время в среднем усваивает в 10 раз больше информации, чем когда слушает. Это одна из причин, по которым текстовые сообщения завоевали такую популярность среди молодежи.

Текстовые сообщения представляют асинхронный метод коммуникации. Один человек отправляет сообщение, а другой отвечает тогда, когда ему удобно. Тем временем отправитель может заниматься другими делами в ожидании ответа.



В программировании асинхронная коммуникация происходит тогда, когда вызывающая сторона инициирует задачу, но не ждет, пока она завершится, а продолжает работать (как рассеянный собеседник). В этом случае не требуется синхронизации при отправке и получении: отправитель не блокируется, пока получатель не будет готов. Если отправителю нужны результаты задачи, должен быть способ их получить (с помощью обратного вызова или как-то иначе). Независимо от того, что это за способ, мы говорим, что вызывающее приложение обращается к результатам *асинхронно*. Приложение может проверять события в любой удобный момент — возможно, в остальное время занимаясь чем-то другим (или придумывая ответ на коварный вопрос типа «Насколько сильно ты меня любишь?»). Это асинхронный процесс, потому что приложение проявляет интерес к данным в одной точке, а использует их в другой.

У асинхронных задач ни начальные, ни конечные точки не синхронизируются. Вместо ожидания, характерного для синхронной коммуникации, процессор тратит время на другие задачи. Таким образом, он никогда не простаивает, если для него есть работа.

Все асинхронные операции ввода/вывода сводятся к одной схеме: суть не в том, как выполняется код, а в том, где происходит ожидание. Несколько операций ввода/

вывода можно совместить так, чтобы ожидание происходило в одном месте кода. Когда случается событие, асинхронная система возобновляет выполнение той части кода, которая ожидала этого события.

Асинхронный обмен сообщениями ослабляет сцепление в коммуникации между сущностями и позволяет отправлять сообщения, не дожидаясь готовности получателей. В частности, между отправителями и получателями не нужно никакой синхронизации, и обе стороны могут работать независимо. Когда получателей много, преимущества асинхронной коммуникации становятся еще очевиднее. Было бы крайне неэффективно ждать, пока все получатели сообщений окажутся на связи одновременно, не говоря уже о том, чтобы отправлять сообщения в синхронном режиме каждому получателю по отдельности.

Модели ввода/вывода

Термины «блокирующий» и «синхронный» часто используются как синонимы, и то же самое происходит с терминами «неблокирующий» и «асинхронный». Хотя оба термина из каждой пары описывают сходные понятия, они не идентичны, а используются на разных уровнях в разном значении. Мы различаем эти термины, по крайней мере когда говорим об операциях ввода/вывода:

- *Блокирующий — неблокирующий.* С помощью этих свойств приложение может сообщить ОС, как обращаться к устройству. В блокирующем режиме операция ввода/вывода не возвращает управление вызывающей стороне, пока операция не завершится. В неблокирующем режиме все вызовы возвращают управление немедленно, но они только показывают состояние операции. Таким образом, может понадобиться несколько вызовов, чтобы убедиться, что операция завершилась успешно.
- *Синхронный — асинхронный.* Эти свойства описывают высокоуровневую программную логику во время операции ввода/вывода. Синхронный вызов удерживает управление, потому что он не возвращает значения, пока операция не завершится; так в программе возникает точка синхронизации. Асинхронный вызов возвращает значение немедленно, позволяя выполнять другие операции.

Эти свойства в разных сочетаниях дают четыре разные модели операций ввода/вывода, каждая из которых обладает своими свойствами, удобными для тех или иных приложений.

Синхронная блокирующая модель

Это самая распространенная модель операций для многих типичных приложений. В этой модели приложение из пользовательского пространства запускает системный вызов, который заставляет приложение блокироваться до того момента, пока системный вызов не завершится (передачей данных или ошибкой).

Синхронная неблокирующая модель

В этой модели приложение обращается к устройству ввода/вывода в неблокирующем режиме. Это заставляет ОС немедленно вернуть управление из вызова ввода/вывода. Обычно устройство еще не готово, и ответ на вызов говорит о том, что вызов следует повторить позднее. Тем самым приложение часто действует по принципу активного ожидания, что крайне неэффективно. Когда операции ввода/вывода завершаются и в пользовательском пространстве появляются доступные данные, приложение может продолжить работу и использовать их.

Асинхронная блокирующая модель

Примером этой модели служит паттерн «Реактор». Как ни странно, асинхронная блокирующая модель по-прежнему использует для операций ввода/вывода неблокирующий режим. Однако вместо активного ожидания применяется специальный блокирующий системный вызов `select`, который отправляет оповещения о статусе ввода/вывода. Впрочем, блокируются только оповещения, а не вызов ввода/вывода. Если механизм оповещений надежен и эффективен, это хорошая модель для высокопроизводительного ввода/вывода.

Асинхронная неблокирующая модель

Наконец, в асинхронной неблокирующей модели запрос ввода/вывода возвращает управление немедленно, показывая, что операция успешно инициирована. Пока она работает в фоновом режиме, приложение выполняет другие операции. Когда поступает ответ, генерируется сигнал или обратный вызов, чтобы завершить операцию ввода/вывода.

У этой модели есть интересная особенность: на уровне пользователя нет блокирования или ожидания. Вся операция передается другой стороне — ОС или устройству. В результате приложение может воспользоваться дополнительным процессорным временем, пока операции ввода/вывода выполняются в фоновом режиме. Не стоит удивляться тому, что эта модель тоже хорошо работает с высокопроизводительным вводом/выводом.

Эти модели описывают операции ввода/вывода в ОС только на низком уровне. С более абстрактной точки зрения разработчика фреймворк приложения может предоставлять доступ к вводу/выводу с использованием синхронного блокирования через фоновые потоки, но обеспечивать асинхронный интерфейс для разработчиков с помощью обратных вызовов, и наоборот.

ПРИМЕЧАНИЕ Асинхронный ввод/вывод (AIO) относительно недавно появился в ядре Linux. Основной принцип AIO заключается в том, чтобы разрешить процессу инициировать последовательность операций ввода/вывода, ничего не блокируя и не ожидая завершения каких-либо операций. Позднее или после того, как поступит оповещение о том, что ввод/вывод завершен, процесс может получить его результаты. Мы получаем оповещение о том, что сокет доступен для чтения или записи без блокировки, после чего выполняем операцию ввода/вывода, которая не блокируется. В Windows используется сходная модель оповещения о завершении (IOCP).

Итоги

- *Событийная конкурентность* лучше подходит для приложений с интенсивным вводом/выводом, потому что она обеспечивает лучшую масштабируемость при более высокой конкурентности. Такие приложения потребляют меньше памяти, даже если обрабатывают тысячи одновременных соединений.
- *Синхронная коммуникация* характеризует задачи, которые выполняются в определенном порядке и зависят от него. Она блокирует выполнение программы на то время, пока передаются данные, из-за чего ресурсы системы простаивают. При синхронной коммуникации обе стороны должны быть на связи одновременно, а приложение блокируется, пока обмен данными не завершится.
- *Асинхронная коммуникация* происходит тогда, когда вызывающая сторона инициирует задачу и не ожидает, пока она завершится, а продолжает работать. Такая коммуникация не требует синхронизировать отправку и прием данных, а отправитель не блокируется, если получатель еще не готов. При этом процессорное время, которое при синхронной коммуникации потратилось бы на ожидание, можно использовать, чтобы выполнять другие задачи. Приложение может проверять события в любое удобное время, и у асинхронных задач не синхронизируются начальные и конечные точки.
- *Паттерн «Реактор»* — самый популярный паттерн, по которому реализуется событийная конкурентность для приложений с интенсивным вводом/выводом. Он использует однопоточный цикл событий и неблокирующие события, которые передаются соответствующим обратным вызовам.

12 | Асинхронная коммуникация



В этой главе

- ✓ Что такое асинхронная коммуникация и когда стоит использовать асинхронную модель
- ✓ Чем отличается вытесняющая многозадачность от кооперативной
- ✓ Как реализовать асинхронную систему с кооперативной многозадачностью с помощью корутин и предначенных объектов
- ✓ Как объединить событийную конкурентность и примитивы конкурентности, чтобы реализовать асинхронную систему, которая будет эффективно выполнять задачи ввода/вывода и вычисления

Люди нетерпеливы по своей природе и хотят, чтобы системы реагировали мгновенно. Тем не менее это не всегда необходимо. Во многих сценариях программирования можно отложить обработку или переместить ее, чтобы она происходила *асинхронно*. Тем самым мы ослабляем ограничения по задержке в системах, которые должны работать в реальном времени. В частности, на асинхронные операции переходят, чтобы снизить рабочую нагрузку, но это не всегда просто сделать.

Например, в Сан-Хосе (Калифорния) существует популярный стейк-хаус «Henry's Hi-Life», который был открыт еще в 1950 году. Он весьма востребован, но место

в нем ограничено, поэтому владельцы заведения внедрили новый асинхронный метод, чтобы ускоренно обслуживать посетителей, но чтобы при этом им не казалось, будто их торопят.

Посетители заходят через маленький бар, где их приветствует распорядитель. Гости сообщают ему численность компании и получают нужное количество меню. Они могут заказать в баре напитки, пока выбирают блюда. Заказ со всеми специальными пожеланиями записывается и передается распорядителю. После этого заказ отправляется прямо на кухню, и как только он будет готов, распорядитель провожает посетителей к столу; прежде чем они успеют расстелить салфетки, им принесут заказанные блюда с пылу с жару (никаких микроволновок).

Такой процесс ослабляет ограничения по задержке для кухни, улучшает общее впечатление посетителей и обеспечивает максимальную прибыль для ресторана. Асинхронные решения способны улучшить производительность и масштабируемость систем даже в таких сценариях, где пользователи привыкли к мгновенному обслуживанию.

В этой главе вы научитесь реализовывать асинхронные системы, опираясь на модель с циклом событий и обратными вызовами (см. главу 11), которая будет преобразована в отдельную реализацию. Мы подробнее рассмотрим корутины и предназначенные объекты (futures) — популярные абстракции для реализации асинхронных вызовов. Вы узнаете, когда уместно использовать асинхронную модель, и познакомитесь с примерами, которые помогут лучше понять, что такое асинхронность и в каких ситуациях она полезна.

Зачем нужна асинхронность

На первый взгляд событийный подход к программированию кажется отличным решением. С простым циклом событий они обрабатываются по мере возникновения. Однако образуется серьезная проблема, когда событие требует системного вызова, который может блокироваться (например, если это операция с интенсивными вычислениями). Проблема усугубляется тем, что вместо единой, целостной кодовой базы приложение представляет собой набор обратных вызовов, каждый из которых отвечает за особый тип запросов. При таком подходе код трудно читать и сопровождать.

Когда мы имеем дело с серверами, которые используют потоки или процессы, эта проблема легко решается. Пока один поток занят блокирующей операцией, другие потоки могут выполняться параллельно, благодаря чему сервер продолжает функционировать. ОС обеспечивает планирование потоков на доступных ядрах процессора.

Однако при событийном подходе существует только один главный поток с циклом событий, который прослушивает события. Это значит, что никакие операции не должны блокировать выполнение, потому что это остановит всю систему. В результате необходимо применять методы асинхронного программирования, чтобы операции не блокировали цикл событий и система оставалась отзывчивой.

Асинхронные вызовы процедур

По умолчанию в большинстве языков программирования вызовы методов выполняются синхронно. Это значит, что код обрабатывает последовательно, а управление не возвращается окружению, пока весь метод не завершится. Однако это может создать проблемы, если метод выполняется слишком долго — например, если он подразумевает сетевой вызов или длительные вычисления. В таких случаях вызывающий поток блокируется, пока метод не закончит работу. Когда это нежелательно, можно запустить отдельный поток-исполнитель и вызывать методы из него; но в большинстве случаев создание нового потока со всеми его сложностями и затратами не оправдывается.

Представьте себе очень неэффективный пример. Вы приходите в регистратуру поликлиники, чтобы записаться на медицинскую процедуру. Если бы при этом использовалась синхронная коммуникация, вам пришлось бы стоять у окна регистратуры все время, пока вы заполняете необходимые бланки, а администратор просто сидел бы и ждал. Таким образом вы бы замедляли обслуживание других пациентов. Масштабировать такую схему можно только одним способом: нанять дополнительных администраторов и выделить для них место. Это было бы дорого и неэффективно, потому что большую часть времени администраторы ничего бы не делали. К счастью, в жизни никто так не поступает.



Как правило, медицинские учреждения работают в асинхронном режиме. Когда вы обращаетесь в регистратуру и узнаете, что нужно заполнить дополнительные бланки, администратор дает вам все бумаги и ручку и предлагает вернуться, когда вы закончите. Вы садитесь и начинаете заниматься бланками, а администратор обслуживает следующего посетителя в очереди, и вы не блокируете его работу. Когда

все готово, вы возвращаетесь в очередь и ждете, когда снова сможете пообщаться с администратором. Если вы что-то сделали не так или нужно заполнить дополнительные бумаги, администратор вручает их вам или объясняет, что нужно исправить, и весь процесс повторяется: вы садитесь, делаете все необходимое, а потом возвращаетесь в очередь. Такая система уже хорошо масштабируется. Если очередь становится слишком длинной, поликлиника может нанять еще одного администратора, отчего система оказывается еще более масштабируемой.

Модель последовательного программирования можно расширить так, чтобы она поддерживала конкурентность: для этого синхронные вызовы перегружаются асинхронной семантикой. Вызов не создает точку синхронизации; вместо этого планировщик системы выполнения передает результаты обработчику позднее или в асинхронном режиме. Синхронный вызов с добавленной асинхронной семантикой называется *асинхронным вызовом* или *асинхронным вызовом процедуры* (APC, Asynchronous Procedure Call). APC добавляет к потенциально продолжительному (синхронному) методу асинхронную версию, которая возвращает управление немедленно, а также дополнительные методы, которые позволяют легко получить оповещение о завершении или ожидать завершения в будущем.

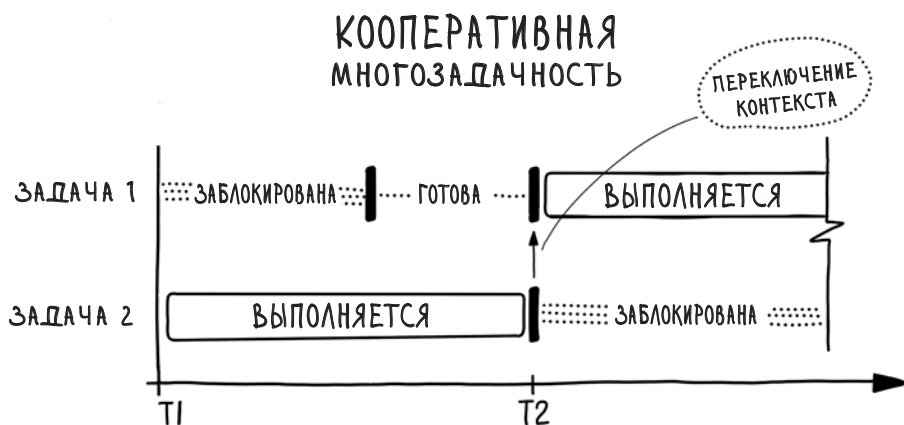
В сфере разработки ПО появились программные конструкции и операции, которые обеспечивают асинхронный режим. Пожалуй, особенно популярной среди них стала кооперативная многозадачность.

Кооперативная многозадачность

Согласно Оксфордскому словарю английского языка, термин «асинхронный» означает «требующий применения управляющего компьютерного временного протокола, в котором та или иная операция начинается по получении сигнала о завершении предшествующей операции». Из этого определения становится ясно, что главная проблема не в том, как и где выполняются операции, а в том, как перезапустить ту или иную часть кода, после того как событие завершится.

До этого момента мы говорили о потоках, которые взаимно однозначно соответствуют потокам системного уровня, которыми управляет сама ОС. Но можно также рассматривать логические потоки на уровне пользователя или приложения; этими потоками управляют разработчики, а ОС про них ничего не знает. Приложения, которые используют такие потоки, ОС рассматривает так, как если бы они были однопоточными процессами. Потоки пользовательского уровня обычно образуют простейшую разновидность многозадачности — *кооперативную многозадачность*, которая также называется *невывесняющей многозадачностью*.

При кооперативной многозадачности ОС никогда не инициирует переключение контекста. Каждая задача сама решает, когда следует вернуть управление планировщику и дать возможность выполняться другим задачам. Для этого она явно сообщает планировщику: «Я приостанавливаю работу на некоторое время; пожалуйста, запускайте другие задачи». Вся работа планировщика заключается в том, чтобы распределить задачи между доступными вычислительными ресурсами.



Таким образом, у нас есть только один поток-исполнитель, и никакой другой поток не может заменить текущий выполняемый поток. Эта схема называется кооперативной многозадачностью, потому что для ее успешной работы программист и среда выполнения должны совместными усилиями обеспечить, чтобы доступные вычислительные ресурсы использовались оптимальным образом.

ПРИМЕЧАНИЕ Этот простой механизм поддерживался во всех версиях macOS вплоть до macOS X, а также в Windows вплоть до Windows 95 и Windows NT.

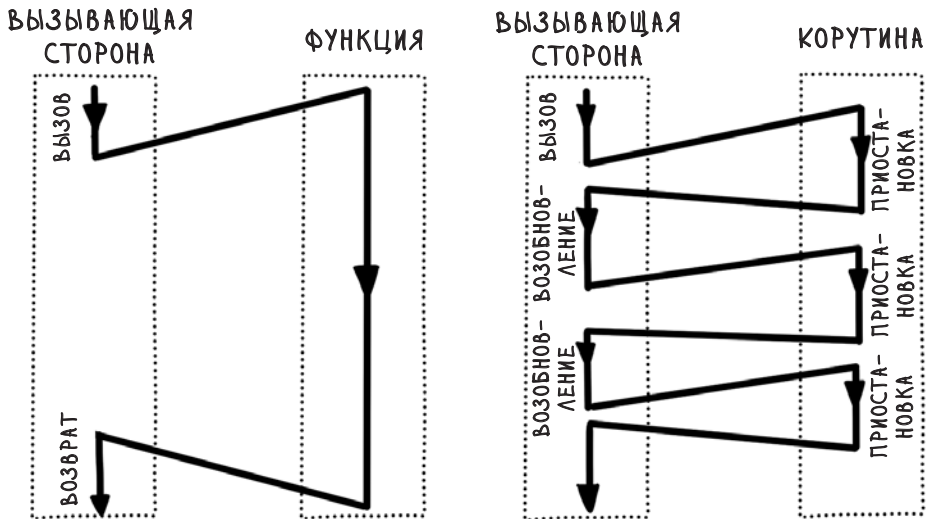
Поскольку используется только один поток выполнения, но нужно выполнить несколько задач, возникает проблема совместного использования ресурсов. В этом случае ресурсом, который должен использоваться совместно, становится управление потоками. Но кооперативный планировщик не может отобрать управление у выполняемой задачи, если только она сама не уступит его добровольно.

Корутины (потоки пользовательского уровня)

В нашей многопоточной реализации сервера (см. главу 10) потоки ОС не возлагали на нас ответственность за передачу управления; они обеспечивали конкурентность, даже если в системе всего одно ядро процессора. Ключевую роль здесь играла способность ОС приостанавливать и возобновлять выполнение потоков с применением вытесняющей многозадачности (см. главу 6). Если бы в нашем распоряжении были функции, которые могут приостанавливать и возобновлять выполнение, как потоки ОС, мы могли бы писать конкурентный однопоточный код. И знаете что? Здесь нам помогут такие программные конструкции, как *корутины*!

Корутины позволяют реализовать кооперативную многозадачность, при которой один поток можно приостанавливать и возобновлять в определенных точках кода. Такой подход обладает рядом преимуществ, включая более эффективный и гибкий код, который способен выполнять асинхронные задачи без явного управления потоками.

Корутины отличаются от потоков ОС прежде всего тем, что переключение корутин выполняется в кооперативном, а не в вытесняющем режиме. Это значит, что разработчик (вместе с языком программирования и средой выполнения) управляет тем, когда происходит переключение между корутинами. В подходящий момент корутину можно приостановить, позволяя другой задаче выполняться вместо нее.



Корутины особенно полезны в ситуациях, когда некоторые операции (например, сетевые запросы) должны блокироваться на значительное время. Вместо того чтобы задействовать системный планировщик, корутины позволяют немедленно переключиться на другую задачу. Их кооперативные качества помогают разработчикам писать более элегантный и удобочитаемый код, который лучше пригоден для повторного использования.

ПРИМЕЧАНИЕ Основная идея корутин происходит из концепции так называемых *продолжений* (continuations). Их можно рассматривать как моментальный снимок контекста выполнения программы в конкретный момент времени, включая текущий стек вызова, локальные переменные и другую релевантную информацию. Сохраняя эти данные, продолжения позволяют зафиксировать состояние выполнения программы и возобновить ее позднее — возможно, в другом потоке или даже на другой машине.

Чтобы понять, чем полезны корутины, рассмотрим пример генерирования последовательности Фибоначчи. Следующий код на Python демонстрирует элегантную и понятную реализацию на основе корутин, которая подчеркивает их преимущества в отношении чистоты, удобочитаемости и повторного использования кода:

```

# Chapter 12/coroutine.py
from collections import deque
import typing as T

Coroutine = T.Generator[None, None, int]

class EventLoop:
    def __init__(self) -> None:
        self.tasks: T.Deque[Coroutine] = deque()

    def add_coroutine(self, task: Coroutine) -> None:
        self.tasks.append(task)

    def run_coroutine(self, task: Coroutine) -> None:
        try:
            task.send(None)
            self.add_coroutine(task)
        except StopIteration:
            print("Задача завершена")

    def run_forever(self) -> None:
        while self.tasks:
            print("Цикл событий")
            self.run_coroutine(self.tasks.popleft())

def fibonacci(n: int) -> Coroutine:
    a, b = 0, 1
    for i in range(n):
        a, b = b, a + b
        print(f"Число Фибоначчи ({i}): {a}")
        yield
    return a

if __name__ == "__main__":
    event_loop = EventLoop()
    event_loop.add_coroutine(fibonacci(5))
    event_loop.run_forever()

```

Содержит список всех корутин, которые будут выполняться

Добавляет новую корутину, которая будет выполняться в цикле событий

Выполняет корутину до тех пор, пока она не дойдет до следующей инструкции yield

Возбуждается исключение, когда корутина закончила выполняться и вернула значение

Входит в цикл, который выполняет корутину из объекта дека цикла событий, пока дек не опустеет

Временно приостанавливает выполнение функции и позволяет выполнять другие корутины

Возвращает итоговое значение, которое вычисляет функция в конце выполнения

Результат выполнения программы выглядит так:

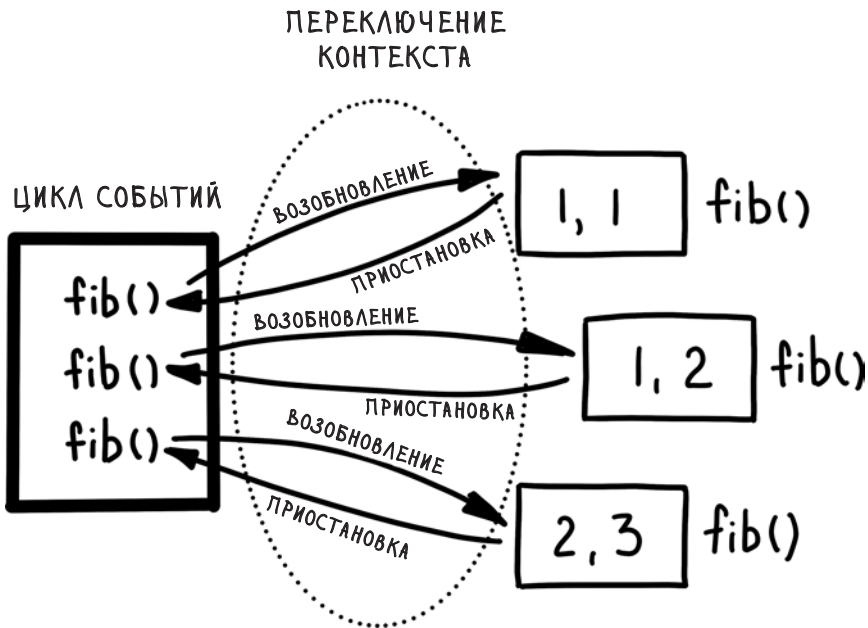
```

Цикл событий
Число Фибоначчи (0): 1
Цикл событий
Число Фибоначчи (1): 1
Цикл событий
Число Фибоначчи (2): 2
Цикл событий
Число Фибоначчи (3): 3
Цикл событий
Число Фибоначчи (4): 5
Цикл событий
Задача завершена

```

В этом коде используется простой цикл событий и корутина. Она вызывается так же, как обычная функция, но выполняет инструкции, пока не достигнет точки приостановки, которая помечена инструкцией `yield`. Эта специальная инструкция временно приостанавливает выполнение текущей функции, возвращает управление вызывающей стороне и сохраняет текущий стек команд и указатель в памяти, фактически консервируя контекст выполнения. В результате цикл событий не блокируется по одной задаче, а выполняет следующую, пока не произойдет ожидаемое событие. Когда это событие завершится, цикл продолжит выполняться с той самой строки, в которой был приостановлен.

Со временем главный поток может снова активизировать ту же корутину, и она будет выполняться не с начала, а с той точки, где приостановилась в последний раз. Таким образом, корутина — это частично выполненная функция, которая позже в определенных условиях может возобновить выполнение, пока не завершится.



В нашем примере корутина `Fibonacci()` приостанавливается и возвращает управление циклу событий, который затем ожидает и делает паузу с маркером возобновления. Возобновившись, корутина `Fibonacci` возвращает результат с помощью `yield`, а цикл событий возобновляет выполнение и передает этот результат соответствующему получателю.

С помощью корутин и цикла событий мы реализуем кооперативную многозадачность, которая позволяет эффективно планировать и выполнять задачи, не полагаясь на множественные потоки или процессы. Корутины помогают писать конкурентный код с улучшенной последовательностью выполнения, благодаря чему

становится проще работать с асинхронными задачами и оптимизировать нагрузку на ресурсы.

ПРИМЕЧАНИЕ *Волокна (fibers), облегченные потоки и зеленые потоки* — другие названия для корутин или аналогичных конструкций. Иногда они выглядят (как правило, намеренно) как потоки ОС, однако они не работают как обычные потоки, а выполняют корутины. В зависимости от языка реализации такие конструкции могут различаться в специфических технических аспектах: встроенные корутины или корутины на основе генераторов в Python, корутины в Scala, горутины в Go, процессы в Erlang и Elixir, потоки Haskell (Haskell GHC) и многие другие.

Преимущества кооперативной многозадачности

У кооперативной многозадачности есть ряд преимуществ перед вытесняющей многозадачностью, из-за чего ее имеет смысл предпочесть в определенных сценариях.

Расходуется меньше ресурсов

Потоки пользовательского уровня потребляют меньше ресурсов. Переключение контекста происходит тогда, когда ОС требуется переключаться между потоками и процессами. Системные потоки относительно тяжеловесны, и переключать контекст между ними довольно затратно. Между тем потоки пользовательского уровня легче в обоих отношениях. Поскольку при кооперативной многозадачности задачи сами управляют своими жизненными циклами, планировщику не нужно отслеживать состояние каждой задачи, и переключение задач требует меньших затрат: оно обходится лишь ненамного дороже, чем вызов функции. Это позволяет создавать миллионы корутин, не расходуя много ресурсов на управление ими. Приложения, в которых используется такой подход, обычно лучше масштабируются, даже если работают в однопоточном режиме (с точки зрения ОС).

Предотвращается блокирование общих ресурсов

С кооперативной многозадачностью задачи могут переключаться между собой в конкретных точках кода, и поэтому у них меньше шансов блокировать общие ресурсы. Если тщательно выбирать эти точки переключения, можно обеспечить, чтобы задачи никогда не прерывали друг друга посреди критических секций.

Повышается эффективность

Переключение контекста при кооперативной многозадачности более эффективно, потому что задача знает, когда нужно приостановиться и передать управление другой задаче. Но для этого задача должна иметь представление о том, что она работает не в одиночку: другие задачи ожидают, пока она решит, что пора уступить управление. Всего одна централизованная последовательность операций может все испортить (см. пример с магазином из главы 2).

Планировщик не может принимать глобальные решения о том, как долго должны выполняться задачи. Поэтому важно, чтобы в кооперативной многозадачности не

было длительных операций — или, если они все-таки есть, чтобы они периодически возвращали управление. Когда несколько программ выполняют небольшие блоки работы и добровольно переключаются между собой, можно достичь уровня конкурентности, который не обеспечит ни один планировщик. В системе могут совместно работать тысячи корутин вместо десятков потоков.

Однако вытесняющая и кооперативная многозадачность не исключают друг друга; они часто используются в одной системе на разных уровнях абстракции. Например, кооперативные вычисления могут периодически вытесняться, чтобы процессорное время распределялось более справедливо.

Преднамеченные объекты

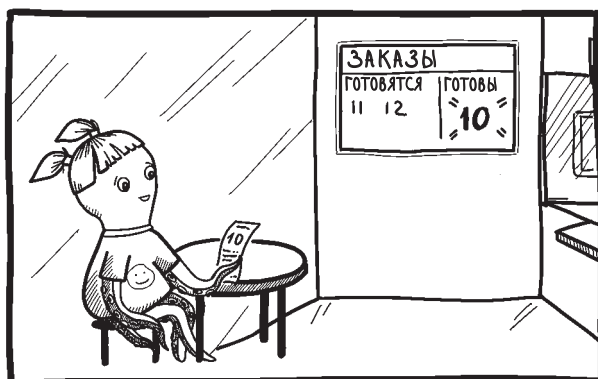
Представьте, что вы зашли в кафе и заказали нестандартный бургер. Кассир обращается к повару на кухне и объясняет ему, как нужно приготовить ваш заказ. Кассир выдает вам номер заказа и обещает, что бургер будет приготовлен и вы получите его в некоторый будущий момент. Когда ваш номер заказа появится на табло, это будет означать, что бургер готов. Во время ожидания вы выбираете столик, садитесь и занимаетесь своими делами. Но если нет метода обратного вызова, то как вы узнаете, когда заказ можно забирать? Другими словами, как получить результат асинхронного вызова?

Возвращаемым значением асинхронного вызова можно сделать объект, который гарантирует результат в будущем (ожидаемый результат или ошибку). Этот объект возвращается как «обещание» будущего результата: объект-заместитель для результата, значение которого пока неизвестно, потому что его вычисление еще не завершено. После того как результат будет получен, его можно будет поместить в объект-заместитель. Такие объекты называются *преднамеченными* (или *объектами Future*).

Преднамеченные объекты можно рассматривать как результаты, которые станут доступными в будущем. Они также действуют как механизм синхронизации, потому что позволяют инициировать независимые вычисления, но синхронизируются с источником и в конечном итоге возвращают результат.

ПРИМЕЧАНИЕ Термины «преднамеченный объект», «фьючерс», «промис» и «отложенный объект» обозначают примерно один и тот же механизм синхронизации в разных языках программирования, где объект выступает как посредник для еще не известного результата. Когда поступает результат, выполняется ожидающий код. За прошедшее время эти термины стали слегка различаться по смыслу в разных языках и экосистемах.

Вернемся к примеру с заказом бургеров. Время от времени вы поглядываете на табло, чтобы узнать, готов ли ваш заказ. В какой-то момент появляется нужный номер. Вы подходите к стойке, забираете бургер, возвращаетесь к столику и наслаждаетесь едой.



В коде это выглядит так:

```
# Chapter 12/future_burger.py
from __future__ import annotations

import typing as T
from collections import deque
from random import randint

Result = T.Any
Burger = Result
Coroutine = T.Callable[[], 'Future']

class Future:
    def __init__(self) -> None:
        self.done = False
        self.coroutine = None
        self.result = None

    def set_coroutine(self, coroutine: Coroutine) -> None:
        self.coroutine = coroutine
```

*Задаёт корутину,
которая связана
с объектом Future*

```

def set_result(self, result: Result) -> None:
    self.done = True
    self.result = result

def __iter__(self) -> Future:
    return self

def __next__(self) -> Result:
    if not self.done:
        raise StopIteration
    return self.result

```

Помечает объект Future как выполненный и заносит в него результат вычислений

```

class EventLoop:
    def __init__(self) -> None:
        self.tasks: T.Deque[Coroutine] = deque()

    def add_coroutine(self, coroutine: Coroutine) -> None:
        self.tasks.append(coroutine)

    def run_coroutine(self, task: T.Callable) -> None:
        future = task()
        future.set_coroutine(task)
        try:
            next(future)
            if not future.done:
                future.set_coroutine(task)
                self.add_coroutine(task)
        except StopIteration:
            return

    def run_forever(self) -> None:
        while self.tasks:
            self.run_coroutine(self.tasks.popleft())

    def cook(on_done: T.Callable[[Burger], None]) -> None:
        burger: str = f"Бургер {randint(1, 10)}"
        print(f"{burger} готов!")
        on_done(burger)

    def cashier(burger: Burger, on_done: T.Callable[[Burger], None]) -> None:
        print("Бургер можно забирать!")
        on_done(burger)

    def order_burger() -> Future:
        order = Future()

    def on_cook_done(burger: Burger) -> None:
        cashier(burger, on_cashier_done)

    def on_cashier_done(burger: Burger) -> None:
        print(f"{burger}? Это мне! Вкуснотища!")
        order.set_result(burger)

```

Запускает корутину: вызывает ее, создает объект Future и выполняет его корутину. Если предназначенный объект еще не выполнен, добавляет корутину в очередь задач, чтобы повторно выполнить ее позже

Готовит бургер и вызывает функцию, которая обрабатывает следующий шаг (действия кассира)

Оповещает клиента, что его бургер можно забирать, и вызывает функцию, которая обрабатывает следующий шаг

Создает объект Future, который представляет заказ клиента

```

    cook(on_cook_done) ←
    return order

if __name__ == "__main__":
    event_loop = EventLoop()
    event_loop.add_coroutine(order_burger)
    event_loop.run_forever()

```

Вызывает функцию для приготовления бургера и возвращает объект Future. Обратные вызовы, которые передаются в качестве аргументов функциям cook и cashier, срабатывают, когда завершается соответствующая операция

Программа вызывает корутину `cook`, в которой повар готовит бургер, а затем передает результат второй корутине — `cashier`, которая оповещает клиента о том, что бургер готов. Каждая корутина возвращает объект `Future` и передает управление главной функции. Функция приостанавливается, пока значение не будет готово, а затем возобновляется и завершает свою операцию. Именно такое поведение делает корутины асинхронными.

Идея объекта `Future` состоит в том, чтобы отделить вычисления от их итогового результата, предоставляя посредника, который возвращает результат, как только он будет готов. Объект `Future` содержит свойство `result`, в котором хранятся результаты будущего выполнения. Также есть метод `set_result`, который присваивает свойству `result` конкретное значение, после того как оно появится.

Ожидая, пока объект `Future` будет заполнен результатом, можно выполнять другие вычисления. Так устроен простой способ вызывать операции, которые выполняются слишком долго или могут задерживаться из-за затратных этапов (например, ввода/вывода), замедляя другие участки программы.

ПРИМЕЧАНИЕ Также существует похожий метод фрагментированного ввода/вывода. Он подразумевает единый вызов процедуры, которая эффективно читает данные из нескольких буферов и записывает их в один поток данных или наоборот. Преимущества такого метода в том, что он удобнее и эффективнее. В частности, такая схема особенно полезна, чтобы одновременно запускать несколько независимых веб-запросов. Если распределить запросы по фоновым задачам и объединить результаты через посредников, то операции можно обрабатывать в конкурентном режиме по аналогии с тем, как в JavaScript работает `promise.all()`. В этот метод можно передать массив предназначенных объектов, и он будет ждать, пока они выполнятся, а затем вернет результаты в виде массива.

Если объединить объекты `Future` с корутинами — функциями, выполнение которых можно приостанавливать и возобновлять, — удастся писать асинхронный код, который по своей форме близок к последовательному.

Кооперативный сервер для заказа пиццы

В главе 10 упоминалось первое приложение электронной коммерции, которое работали в Santa Cruz Operation в 1980-х годах, чтобы заказывать пиццу для программистов. Это была простая синхронная программа, и область ее практического

применения была ограничена из-за нехватки вычислительных ресурсов. С тех пор программисты научились запускать корутины и создавать предназначенные объекты, так что в нашем распоряжении появились все необходимые компоненты для того, чтобы создать асинхронный сервер на основе кооперативной многозадачности.

Цикл событий

Рассмотрим главный компонент — цикл событий:

```
# Chapter 12/asynchronous_pizza/event_loop.py
from collections import deque
import typing as T
import socket
import select

from future import Future

Action = T.Callable[[socket.socket, T.Any], Future]
Coroutine = T.Generator[T.Any, T.Any, T.Any]
Mask = int

class EventLoop:
    def __init__(self):
        self._numtasks = 0
        self._ready = deque()
        self._read_waiting = {}
        self._write_waiting = {}

    def register_event(self, source: socket.socket, event: Mask, future,
                      task: Action) -> None:
        key = source.fileno()
        if event & select.POLLIN:
            self._read_waiting[key] = (future, task)
        elif event & select.POLLOUT:
            self._write_waiting[key] = (future, task)

    def add_coroutine(self, task: Coroutine) -> None:
        self._ready.append((task, None))
        self._numtasks += 1

    def add_ready(self, task: Coroutine, msg=None):
        self._ready.append((task, msg))

    def run_coroutine(self, task: Coroutine, msg) -> None:
        try:
            future = task.send(msg)
            future.coroutine(self, task)
        except StopIteration:
            self._numtasks -= 1

    def run_forever(self) -> None:
        while self._numtasks:
```

```

if not self._ready:
    readers, writers, _ = select.select(
        self._read_waiting, self._write_waiting, [])
    for reader in readers:
        future, task = self._read_waiting.pop(reader)
        future.coroutine(self, task)

    for writer in writers:
        future, task = self._write_waiting.pop(writer)
        future.coroutine(self, task)

task, msg = self._ready.popleft()
self.run_coroutine(task, msg)

```



Проверяет, есть ли готовые к выполнению корутины. Если есть, выполняет следующую. Если нет, ожидает, пока один или несколько зарегистрированных сокетов будут готовы к операциям ввода/вывода, а затем выполняет одну или несколько соответствующих корутин

Помимо уже знакомого цикла оповещения о событиях в главной точке входа — методе `run_forever`, мы запускаем метод `run_coroutine` для всех корутин, которые готовы к запуску. Как только все задачи выполнены (они либо возвратили предначеченный объект и передали управление, либо возвратили результат), мы удаляем все завершенные задачи из очереди. Если нет готовых задач, мы вызываем `select`, как и прежде, блокируя цикл событий, до тех пор пока с зарегистрированными клиентскими сокетами не произойдет то или иное событие. Как только оно происходит, мы запускаем соответствующие обратные вызовы и начинаем новую итерацию цикла.

Как уже упоминалось, кооперативный планировщик не может отобрать управление у выполняемой задачи, потому что цикл событий не может прервать выполняемую корутину. Задача выполняется, до тех пор пока она не передаст управление. Цикл событий выбирает следующую задачу и отслеживает заблокированные задачи, которые не могут выполняться, пока не завершится ввод/вывод, но только если в настоящее время не запущены никакие задачи.

Чтобы реализовать кооперативный сервер, нужно написать корутины для каждого метода серверного сокета (`accept`, `send` и `recv`). Затем мы создаем объект `Future` и возвращаем его циклу событий. Результат помещается в предначеченный объект, после того как нужное событие завершится. Чтобы с асинхронной реализацией сокета было удобнее работать, выделим ее в отдельный класс:

```

# Chapter 12/asynchronous_pizza/async_socket.py
from __future__ import annotations

import select
import typing as T
import socket

from future import Future

Data = bytes

class AsyncSocket:
    def __init__(self, sock: socket.socket):

```

```

self._sock = sock
self._sock.setblocking(False)

def recv(self, bufsize: int) -> Future:
    future = Future()

    def handle_yield(loop, task) -> None:
        try:
            data = self._sock.recv(bufsize)
            loop.add_ready(task, data)
        except BlockingIOError:
            loop.register_event(self._sock, select.POLLIN, future, task)

    future.set_coroutine(handle_yield)
    return future

def send(self, data: Data) -> Future:
    future = Future()

    def handle_yield(loop, task):
        try:
            nsent = self._sock.send(data)
            loop.add_ready(task, nsent)
        except BlockingIOError:
            loop.register_event(self._sock, select.POLLOUT, future, task)

    future.set_coroutine(handle_yield)
    return future

def accept(self) -> Future:
    future = Future()
    def handle_yield(loop, task):
        try:
            r = self._sock.accept()
            loop.add_ready(task, r)
        except BlockingIOError:
            loop.register_event(self._sock, select.POLLIN, future, task)
    future.set_coroutine(handle_yield)
    return future

def close(self) -> Future:
    future = Future()

    def handle_yield(*args):
        self._sock.close()

    future.set_coroutine(handle_yield)
    return future

def __getattr__(self, name: str) -> T.Any:
    return getattr(self._sock, name)

```

Мы сделали серверный сокет неблокирующим и в каждом методе выполняем соответствующую операцию, не ожидая, пока она завершится. Мы просто освобождаем управление, возвращая объект `Future`, в который позднее будет записан результат операции. Мы подготовили общий шаблон кода, и теперь все готово к тому, чтобы создать кооперативное серверное приложение.

Кооперативная реализация сервера для заказа пиццы

Реализуем наш асинхронный сервер по принципу кооперативной многозадачности:

```
# Chapter 12/asynchronous_pizza/cooperative_pizza_server.py
import socket
```

```
from async_socket import AsyncSocket
from event_loop import EventLoop
```

```
BUFFER_SIZE = 1024
ADDRESS = ("127.0.0.1", 12345)
```

Приостанавливает выполнение, пока на серверный сокет не поступит запрос на соединение. Когда он поступит, метод ассерт возвращает новый объект сокета для соединения, и метод возобновляет выполнение

```
class Server:
    def __init__(self, event_loop: EventLoop):
        self.event_loop = event_loop
        print(f"Сервер запущен на узле {ADDRESS}")
        self.server_socket = AsyncSocket(socket.create_server(ADDRESS))
```

```
    def start(self):
        print("Сервер прослушивает входящие соединения")
        try:
```

```
            while True:
                conn, address = yield self.server_socket.accept()
                print(f"Установлено соединение с узлом {address}")
                self.event_loop.add_coroutine(
                    self.serve(AsyncSocket(conn)))
```

```
        except Exception:
            self.server_socket.close()
            print("\nСервер остановлен.")
```

Приостанавливает выполнение, пока от клиента не поступят данные. Когда они поступят, метод возобновляет выполнение и возвращает полученные данные

```
    def serve(self, conn: AsyncSocket):
```

```
        while True:
            data = yield conn.recv(BUFFER_SIZE)
            if not data:
                break
```

```
        try:
            order = int(data.decode())
            response = f"Спасибо за заказ пиццы ({order} шт.)!\n"
        except ValueError:
            response = "Недопустимое количество пицц.\n"
            response += "Попробуйте еще раз\n"
```

```
        print(f"Отправляется сообщение для узла {conn.getpeername()}")
        yield conn.send(response.encode())
        print(f"Соединение с узлом {conn.getpeername()} закрыто")
        conn.close()
```

Приостанавливает выполнение метода serve, пока не появится возможность отправить ответ клиенту. Когда ответ отправлен, метод возобновляет выполнение

```
if __name__ == "__main__":
    event_loop = EventLoop()
    server = Server(event_loop=event_loop)
    event_loop.add_coroutine(server.start())
    event_loop.run_forever()
```

Это решение похоже на предыдущие версии: мы создаем цикл событий и назначаем ему для выполнения функцию `server`. Когда цикл событий начнет выполняться, мы запускаем клиенты и отправляем заказы серверу.

Тем не менее в решениях на основе кооперативной многозадачности мы не полагаемся на потоки или процессы, которые требуют передавать управление, потому что все выполняется в одном потоке. Вместо этого мы управляем несколькими задачами, передавая управление центральной функции, которая их координирует, — циклу событий.

Подведем итог: кооперативная многозадачность значительно сокращает затраты процессорного времени и памяти, особенно если в рабочую нагрузку входит много задач с интенсивным вводом/выводом (например, серверов и баз данных). При прочих равных условиях можно создать на несколько порядков больше задач, чем потоков ОС, потому что кооперативная многозадачность использует всего один дорогостоящий поток, чтобы управлять большим количеством низкозатратных задач.

Асинхронная пиццерия

Возможно, в двух последних главах вы уже задавались вопросом: «Что это за пиццерия, которая не готовит пиццу, а только благодарит за заказ?» Пора надеть фартук и разжечь печь!

Как нетрудно догадаться, приготовление пиццы — долгий процесс. Воспользуемся классом `Kitchen`, чтобы его смоделировать:

```
# Chapter 12/asynchronous_pizza/asynchronous_pizza_joint.py
```

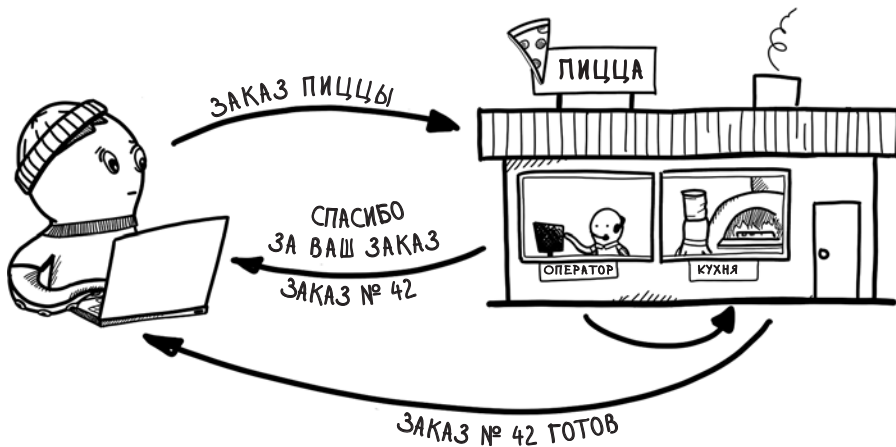
```
class Kitchen:
    @staticmethod
    def cook_pizza(n):
        print(f"Начали готовить пиццу ({n} шт.)")
        time.sleep(n)
        print(f"Свежая пицца готова ({n} шт.)!")
```

Моделирует время, которое необходимо, чтобы приготовить n пицц

Если использовать этот код в нашей реализации кооперативного сервера, он будет долго готовить пиццу для одного клиента и только потом начнет обслуживать остальных! В темном закутке нашей прекрасной асинхронной системы притаился блокирующий вызов. Какая досада!

Мы хотим продолжать принимать заказы от клиентов, пока готовим пиццу в фоновом режиме. Печь и сервер заказов не должны блокировать друг друга. Да, мы возвращаемся к тому, с чего начинали, — к потокам, но на этот раз используем конкурентность с асинхронной коммуникацией. Уже интереснее!

Идея заключается в том, чтобы создать асинхронный метод, который возвращает предназначенный объект, который инкапсулирует длительную операцию, которая завершится в какой-то момент в будущем. После того как задание отправлено, возвращается объект `Future`, и поток выполнения на вызывающей стороне может работать дальше, отдельно от новых вычислений.



В реализации мы используем уже знакомый способ оповещения о событиях; мы можем вернуть объект Future с обещанием результата, который появится когда-нибудь в будущем:

```
# Chapter 12/asynchronous_pizza/event_loop_with_pool.py
import socket
from collections import deque
from multiprocessing.pool import ThreadPool
import typing as T
import select
```

```
from future import Future
```

```
Data = bytes
```

```
Action = T.Callable[[socket, T.Any], None]
```

```
Mask = int
```

```
BUFFER_SIZE = 1024
```

```
class Executor:
```

```
    def __init__(self):
```

```
        self.pool = ThreadPool()
```

```
    def execute(self, func, *args):
```

```
        future_notify, future_event = socket.socketpair()
```

```
        future_event.setblocking(False)
```

```
    def _execute():
```

```
        result = func(*args)
```

```
        future_notify.send(result.encode())
```

```
    self.pool.apply_async(_execute)
```

```
    return future_event
```

```
class EventLoop:
```

Использует пул потоков,
чтобы запускать
блокирующие задачи
в отдельных потоках

Создает пару соединенных
сокетов для межпроцесс-
ной коммуникации. С по-
мощью одного сокета
отправляются оповеще-
ния о завершении задач,
а другой ожидает этих
оповещений

Принимает функцию,
которая будет выполняться
в потоке-исполнителе из пула
потоков, и возвращает сокет
предназначенного события,
который будет ожидать
оповещения

```

def __init__(self):
    self._numtasks = 0
    self._ready = deque()
    self._read_waiting = {}
    self._write_waiting = {}
    self.executor = Executor()

def register_event(self, source: socket.socket, event: Mask, future,
                  task: Action) -> None:
    key = source.fileno()
    if event & select.POLLIN:
        self._read_waiting[key] = (future, task)
    elif event & select.POLLOUT:
        self._write_waiting[key] = (future, task)

def add_coroutine(self, task: T.Generator) -> None:
    self._ready.append((task, None))
    self._numtasks += 1

def add_ready(self, task: T.Generator, msg=None):
    self._ready.append((task, msg))

def run_coroutine(self, task: T.Generator, msg) -> None:
    try:
        future = task.send(msg)
        future.coroutine(self, task)
    except StopIteration:
        self._numtasks -= 1

def run_in_executor(self, func, *args) -> Future:
    future_event = self.executor.execute(func, *args)
    future = Future()

    def handle_yield(loop, task):
        try:
            data = future_event.recv(BUFFER_SIZE)
            loop.add_ready(task, data)
        except BlockingIOError:
            loop.register_event(
                future_event, select.POLLIN, future, task)
    future.set_coroutine(handle_yield)
    return future

def run_forever(self) -> None:
    while self._numtasks:
        if not self._ready:
            readers, writers, _ = select.select(
                self._read_waiting, self._write_waiting, [])
            for reader in readers:
                future, task = self._read_waiting.pop(reader)
                future.coroutine(self, task)
            for writer in writers:
                future, task = self._write_waiting.pop(writer)
                future.coroutine(self, task)

        task, msg = self._ready.popleft()
        self.run_coroutine(task, msg)

```

Запускает операцию в экземпляре Ехесутор и добавляет соответствующий обратный вызов для события, когда поступят данные

Здесь мы объединяем пул потоков с циклом событий. Если поступает задача с интенсивными вычислениями, ее можно запустить в пуле потоков и вернуть объект Future. Когда задача будет выполнена, поток выполнения оповестит об этом и можно будет задать результат объекта Future.

В итоге сервер пиццерии принимает такой вид:

```
# Chapter 11/asynchronous_pizza_joint.py
import socket
import time

from async_socket import AsyncSocket
from event_loop_with_pool import EventLoop

BUFFER_SIZE = 1024
ADDRESS = ("127.0.0.1", 12345)

class Server:
    def __init__(self, event_loop: EventLoop):
        self.event_loop = event_loop
        print(f"Сервер запущен на узле {ADDRESS}")
        self.server_socket = AsyncSocket(socket.create_server(ADDRESS))

    def start(self):
        print("Сервер прослушивает входящие соединения")
        try:
            while True:
                conn, address = yield self.server_socket.accept()
                print(f"Установлено соединение с узлом {address}")
                self.event_loop.add_coroutine(
                    self.serve(AsyncSocket(conn)))
        except Exception:
            self.server_socket.close()
            print("\nСервер остановлен.")

    def serve(self, conn: AsyncSocket):
        while True:
            data = yield conn.recv(BUFFER_SIZE)
            if not data:
                break
            try:
                order = int(data.decode())
                response = f"Спасибо за заказ пиццы ({order} шт.)!\n"
                print(f"Отправляется сообщение для узла {conn.getpeername()}")
                yield conn.send(response.encode())
                yield self.event_loop.run_in_executor(
                    Kitchen.cook_pizza, order)
                response = f"Ваша пицца готова ({order} шт.)!\n"
            except ValueError:
                response = "Недопустимое количество пицц.\n"
                response += "Попробуйте еще раз\n"

            print(f"Отправляется сообщение для узла {conn.getpeername()}")
            yield conn.send(response.encode())
```

Запускает блокирующую операцию в отдельном потоке, чтобы готовить пиццу, продолжая обслуживать других клиентов

```

        print(f"Соединение с узлом {conn.getpeername()} закрыто")
        conn.close()

if __name__ == "__main__":
    event_loop = EventLoop()
    server = Server(event_loop=event_loop)
    event_loop.add_coroutine(server.start())
    event_loop.run_forever()

```

Эта реализация все еще не подходит для реальной эксплуатации из-за различных ограничений: например, исключения обрабатываются недостаточно полноценно, а итерацию цикла событий могут запустить только события сокетов. Тем не менее код дает некоторое представление о том, как работает конкурентность с асинхронными вызовами. Наши текущие знания уже позволяют эффективнее использовать аппаратные ресурсы, повышая производительность. Опираясь на этот пример, вы сможете разрабатывать асинхронные фреймворки следующего поколения на любом языке программирования. Применяя аналогичные средства и приемы, можно создавать более надежные и масштабируемые системы, которые способны одновременно обрабатывать множество задач.

ПРИМЕЧАНИЕ JavaScript — однопоточный язык, так что единственный способ достичь в нем многопоточности — это запустить несколько экземпляров движка JavaScript. Но как организовать коммуникацию между этими экземплярами? На помощь приходят *исполнители*, или *воркеры* (web workers). Они позволяют задачам выполняться в фоновом режиме в отдельном фоновом потоке, который изолирован от главного потока веб-приложения. Эту многопоточную функциональность обеспечивает браузерный контейнер, так что еще не все браузеры поддерживают исполнителей. Node.js — еще один контейнер для движка JavaScript, который обеспечивает многопоточность в координации с ОС.

Асинхронное программирование — сложная область, но многие сложности можно скрыть с помощью асинхронных библиотек и фреймворков. Для примера рассмотрим ту же логику, но с использованием встроенной библиотеки Python `asyncio`:

```

# Chapter 12/asynchronous_pizza/aio.py
import asyncio
import socket

from asynchronous_pizza_joint import Kitchen

BUFFER_SIZE = 1024
ADDRESS = ("127.0.0.1", 12345)

class Server:
    def __init__(self, event_loop: asyncio.AbstractEventLoop) -> None:
        self.event_loop = event_loop
        print(f"Сервер запущен на узле {ADDRESS}")
        self.server_socket = socket.create_server(ADDRESS)
        self.server_socket.setblocking(False)

```

```

async def start(self) -> None:
    print("Сервер прослушивает входящие соединения")
    try:
        while True:
            conn, client_address = \
                await self.event_loop.sock_accept(
                    self.server_socket)
            self.event_loop.create_task(self.serve(conn))
    except Exception:
        self.server_socket.close()
        print("\nСервер остановлен.")

async def serve(self, conn) -> None:
    while True:
        data = await self.event_loop.sock_recv(conn, BUFFER_SIZE)
        if not data:
            break
        try:
            order = int(data.decode())
            response = f"Спасибо за заказ пиццы ({order} шт.)!\n"
            print(f"Отправляется сообщение для узла {conn.getpeername()}")
            await self.event_loop.sock_sendall(
                conn, f"{response}".encode())
            await self.event_loop.run_in_executor(
                None, Kitchen.cook_pizza, order)
            response = f"Ваша пицца готова ({order} шт.)!\n"
        except ValueError:
            response = "Недопустимое количество пицц.\n"
            response = "Попробуйте еще раз\n"
        print(f"Отправляется сообщение для узла {conn.getpeername()}")
        await self.event_loop.sock_sendall(conn, response.encode())
        print(f"Соединение с узлом {conn.getpeername()} закрыто")
        conn.close()

if __name__ == "__main__":
    event_loop = asyncio.get_event_loop()
    server = Server(event_loop=event_loop)
    event_loop.create_task(server.start())
    event_loop.run_forever()

```

Ключевое слово `async` означает, что функция является асинхронной

Благодаря ключевому слову `await` можно ожидать, пока корутина завершится, позволяя другим задачам выполняться в это время

Благодаря ключевому слову `await` можно ожидать, пока корутина завершится, позволяя другим задачам выполняться в это время

Код приложения стал гораздо проще: из него исчезли все шаблонные конструкции. Все — от сокетов до цикла событий и конкурентности — теперь скрыто под библиотечными вызовами, и всем этим управляют разработчики библиотек.

ПРИМЕЧАНИЕ Это не значит, что `async/await` — единственно правильный механизм коммуникации в конкурентных системах. В качестве альтернативного примера можно привести модель CSP, которая реализована в Go и Clojure, или модель акторов в Erlang и Akka. Но, похоже, в Python модель `async/await` является лучшей на сегодняшний день.

Безусловно, это был не самый простой код, так что давайте отвлечемся и поговорим об асинхронной модели в общем случае.

Асинхронная модель: выводы

Асинхронные операции обычно не ждут, пока результаты будут готовы. Они делегируют задачи другим компонентам (устройствам, потокам, процессам или внешним системам), которые могут обрабатывать их независимо. Это позволяет программе выполнять другие задачи без ожидания и получать оповещения, когда делегированные задачи завершаются или сталкиваются с ошибками.

Важно заметить, что асинхронность — это свойство вызова операции или коммуникации; она никак не привязана к конкретной реализации. Существуют разные асинхронные механизмы, но в их основе лежит одна и та же модель. Они различаются тем, как структурируется код, чтобы его можно было приостановить, когда запрошена блокирующая операция, и возобновить, когда она завершится. Такая гибкость позволяет разработчикам выбрать наиболее подходящее решение в зависимости от конкретных требований и программного окружения.

Когда стоит применять асинхронную модель? Асинхронная коммуникация — мощный инструмент для того, чтобы оптимизировать высоконагруженную систему с частыми блокирующими системными вызовами. Но, как и все сложные технологии, эти коммуникации не следует использовать только потому, что они есть.

Асинхронный код сложнее, и его труднее сопровождать. Асинхронная модель подходит лучше синхронной в таких ситуациях:

- Очень много задач. Вероятно, при этом всегда найдется хотя бы одна задача, которая может продвинуться в своем выполнении. Благодаря асинхронной модели часто сокращается время реакции и повышается общее быстродействие, отчего выигрывают конечные пользователи системы.
- Приложение большую часть времени занимается вводом/выводом, а не вычислениями. Например, оно порождает много продолжительных запросов — веб-сокеты, долгое накопление в пуле, медленные внешние синхронные бэкенды, для которых неизвестно, когда завершится обработка запроса.
- Задачи в основном не зависят друг от друга, поэтому между ними не нужны коммуникации (а значит, не нужно ждать, пока завершится одна задача, прежде чем запускать другую).

Эти условия почти идеально описывают типичный нагруженный сервер (например, веб-сервер) в клиент-серверной системе (так что пример с пиццей выглядит вполне логично). В программах на стороне сервера асинхронные коммуникации позволяют эффективно обрабатывать массовые конкурентные операции ввода/вывода, разумно использовать ресурсы во время простоя и не плодить новых ресурсов. Реализация на стороне сервера — основной кандидат для асинхронной модели; именно поэтому в последние годы стали столь популярными библиотека Python `asyncio` и библиотека JavaScript `Node.js`, а также другие асинхронные библиотеки. Приложения фронтенда и пользовательские интерфейсы тоже могут выиграть от асинхронности, потому что она улучшает программную логику, особенно при большом объеме независимых задач ввода/вывода.

Итоги

- Асинхронная коммуникация — метод разработки, благодаря которому процесс может продолжать выполняться, не блокируясь продолжительными задачами, например операциями ввода/вывода или сетевыми запросами. Вместо того чтобы ждать, пока одна задача завершится, прежде чем переходить к другой, асинхронная программа может выполнять другой код, пока задача работает в фоновом режиме. Этот подход оптимизирует потребление системных ресурсов, улучшая производительность и отзывчивость программы.
- Асинхронность — это свойство вызова операции или коммуникации, а не конкретной реализации. Асинхронная модель позволяет эффективно обрабатывать массовые конкурентные операции ввода/вывода, оптимизировать загрузку ресурсов, снизить задержку системы и улучшить ее масштабируемость и пропускную способность. Но без хороших библиотек и фреймворков писать и отлаживать асинхронные программы достаточно сложно.
- *Кооперативная многозадачность* — один из методов реализации асинхронных систем. Она позволяет нескольким задачам совместно использовать ресурсы процессора. В этом режиме задачи должны сотрудничать друг с другом и возвращать управление системе, завершив очередную часть работы.
- У кооперативной многозадачности есть ряд преимуществ по сравнению с вытесняющей многозадачностью. Потоки пользовательского уровня, которые используются в кооперативной многозадачности, тратят меньше ресурсов, чем системные потоки. Это позволяет создавать множество корутин без значительных затрат на управление. При этом абсолютно необходимо, чтобы задачи имели представление о том, что они работают не в одиночку и им необходимо решать, когда уступить управление другим задачам.
- Кооперативная многозадачность значительно сокращает затраты процессорного времени и памяти, особенно для рабочей нагрузки с многочисленными задачами ввода/вывода (серверы или базы данных). Используя небольшое количество потоков для управления множеством задач, кооперативная многозадачность позволяет эффективнее задействовать мощности оборудования. Этот фактор в сочетании с асинхронными взаимодействиями приводит к тому, что ресурсы меньше простаивают.
- Чтобы реализовать асинхронные вызовы, часто используются такие популярные абстракции, как корутины и предназначенные объекты. *Корутина* — это функция, которая выполняется частично и приостанавливается, а при подходящих обстоятельствах возобновляется в какой-то последующий момент времени, и так до тех пор, пока не выполнится до конца. *Предназначенный объект* (*объект Future*) можно рассматривать как обещание будущего результата — объект-заемщик для результата, который изначально неизвестен (обычно потому, что его значение пока еще не вычислено).



В этой главе

- ✓ Фреймворк для проектирования конкурентных систем и два демонстрационных примера
- ✓ Подытоживаем и сводим воедино все, что вы узнали из этой книги

В этой книге мы изучили, какими способами можно реализовывать конкурентные приложения и какие проблемы с ними связаны. Пришло время применить эти знания к сценариям из реального мира.

В текущей главе мы сосредоточимся на том, как применять конкурентное программирование на практике, и рассмотрим систематический подход к проектированию конкурентных систем, проиллюстрировав его на характерных примерах. К концу главы у вас будет достаточно знаний и навыков, чтобы успешно проектировать небольшие конкурентные системы, а также распознавать и решать потенциальные проблемы, которые ухудшают эффективность или масштабируемость.

Но сначала выделим немного времени, чтобы подытожить ключевые понятия и принципы, которые мы рассматривали ранее в контексте конкурентности.

Так что же такое конкурентность?



Конкурентность — огромная, иногда ошеломляющая головоломка. На заре компьютерной истории программы писались для *последовательных* вычислений. Чтобы решить проблему в этой парадигме, алгоритм конструируется и реализуется как последовательный поток инструкций, которые выполняются на процессоре одного компьютера. Это простейший стиль программирования и самая бесхитростная модель выполнения. Все задачи выполняются одна за другой, и каждая задача должна завершиться до того, как начнется следующая. Если задачи всегда выполняются в определенном порядке, то, когда очередная задача начинает выполняться, можно предположить, что все предыдущие задачи завершились без ошибок, а все их результаты доступны для использования — это определенное упрощение логики.

Конкурентное программирование означает, что программа разбивается на задачи, которые выполняются в произвольном порядке, но в конце концов получается один и тот же результат. Из-за этого работать с конкурентностью становится не так просто. За десятилетия исследований и практики появились разнообразные модели конкурентности для разных целей. Эти модели создаются прежде всего для того, чтобы оптимизировать производительность, эффективность и удобство использования, а также минимизировать ошибки. В зависимости от контекста единицы конкурентности обозначаются разными терминами: *задачи*, *корутины*, *процессы*, *потoki* и т. д.

Обработкой могут заниматься разные системы: например, один компьютер с несколькими процессорами, несколько компьютеров, соединенных в сеть, специализированное оборудование или любая комбинация этих ресурсов. Процессом выполнения управляет система выполнения (ОС); в системе с несколькими процессорами или ядрами задачи могут выполняться *параллельно*, а на одном процессорном ядре — в *многозадачном* режиме. Суть в том, что подробности выполнения берет на себя система выполнения, а разработчик просто представляет себе независимые задачи, которые могут выполняться в конкурентном режиме.

Кроме того, что можно безопасно выполнять несколько задач в конкурентном режиме, также нужен способ координировать их с общими ресурсами. Здесь начинаются проблемы с конкурентностью. Если задачи используют устаревшие данные, они могут вносить противоречивые обновления, в системах возникают *взаимные блокировки*, данные из разных систем могут не прийти к согласованным значениям и т. д. Порядок, в котором задачи обращаются к общим ресурсам, полностью зависит не от разработчика, а от того, как задачи распределяются между процессорами. Иначе говоря, когда и как долго выполняется каждая задача — это автоматически решает реализация языка программирования в ОС. Поэтому ошибки конкурентности очень трудно воспроизвести, но их можно избежать, если реализовывать в приложении правильные методы проектирования, сводить к минимуму коммуникацию между задачами и применять эффективные средства синхронизации.

Итак, у нас есть безопасный способ координировать задачи, но им часто приходится взаимодействовать друг с другом. Коммуникация между задачами может быть *синхронной* или *асинхронной*. Синхронный вызов не возвращает управление, пока операция не завершится, создавая тем самым *точку синхронизации*. Асинхронный вызов запрашивает нужную операцию, а затем получает оповещение, когда она завершается; между этими моментами ресурсы остаются доступными для другой работы. В асинхронной модели задача выполняется, до тех пор пока явно не уступит управление другой задаче. Обратите внимание, что асинхронные модели можно сочетать с конкурентными и использовать в одной системе.

А теперь рассмотрим методологию, которая поможет вам создавать конкурентные программы: методологию Фостера.

Методология Фостера

В 1995 году Иэн Фостер (Ian Foster) предложил последовательность шагов для проектирования конкурентных систем, которая теперь известна как *методология проектирования Фостера*¹. Процесс состоит из четырех шагов. Сначала мы разберем их на абстрактном уровне, а потом рассмотрим несколько примеров.

Допустим, вы планируете поездку с друзьями. Нужно позаботиться о том, чтобы поездка была приятной, и обеспечить все необходимые приготовления. Рассмотрим такие четыре шага:

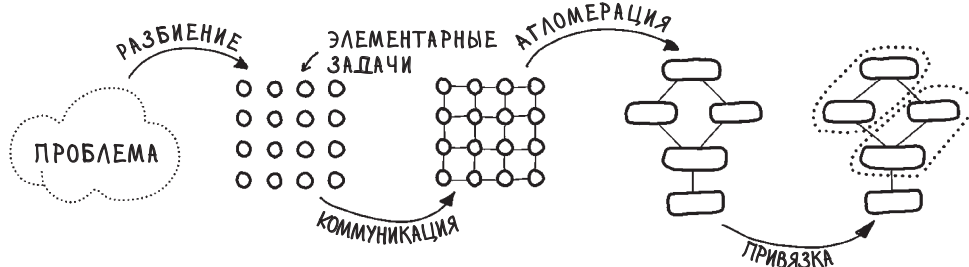
1. *Разбиение* (partitioning). Подготовку можно разбить на меньшие задачи: спланировать маршрут, забронировать жилье и собрать информацию о местах, которые вы собираетесь посетить. Это помогает лучше организовать процесс и гарантировать, что все необходимые задачи будут выполнены.
2. Применительно к конкурентности это означает, что мы выясняем, какие части работы можно выполнять в конкурентном режиме. Проблема разбивается на множество задач, для чего применяются методы декомпозиции данных или декомпозиции задач (см. главу 7). Мы игнорируем практические аспекты — напри-

¹ Ian Foster, «Designing and Building Parallel Programs», <https://www.mcs.anl.gov/~itf/dbpp>.

мер, сколько процессоров на целевом компьютере, — и направляем все внимание на то, чтобы выявить возможности для независимого выполнения.

3. *Коммуникация* (communication). Готовясь к поездке, нужно общаться со всеми ее участниками, чтобы получать необходимые данные. Можно создать групповой чат или рассылку, в которой все смогут обсуждать предпочтения относительно маршрута, жилья и мест, которые хочется посетить.
4. Аналогичным образом организуется коммуникация для того, чтобы получать данные, которые необходимы для выполнения задач. Нужно определить, какая коммуникация нужна, чтобы координировать задачи, а также наладить надлежащие коммуникационные структуры и алгоритмы.
5. *Агломерация* (agglomeration). Агломерация заключается в том, чтобы установить зоны ответственности, разделив задачи и обязанности на области. Имеет смысл сгруппировать сходные или взаимосвязанные задачи — например, бронирование жилья и сбор информации об интересных местах. Это облегчает коммуникацию и координацию между участниками команды, а также упрощает процесс планирования, потому что каждый участник отвечает за свою область.
6. Задачи и коммуникационные структуры, определенные на первых двух шагах, оцениваются в контексте требований к производительности и затрат на реализацию. Чтобы сократить коммуникацию или упростить реализацию, задачи можно группировать в более крупные задачи, по возможности сохраняя гибкость.
7. *Привязка* (mapping). Наконец, надо распределить задачи между участниками поездки. Например, одному человеку можно поручить прокладывать маршрут и вести машину, а другой будет отвечать за бронирование жилья и билеты на мероприятия. Цель этого шага — минимизировать общее время выполнения и обеспечить, чтобы у каждого была своя роль в организации успешной поездки.

Когда мы распределяем задачи между физическими процессорами, цель обычно состоит в том, чтобы минимизировать общее время выполнения. Чтобы привязка была эффективнее, можно использовать методы балансировки нагрузки или планирования задач. Каждая задача назначается тому или иному процессору так, чтобы по возможности удовлетворить конфликтующие требования: максимизировать загрузку процессора и минимизировать коммуникационные затраты. Привязку можно задавать статически, а можно определять на стадии выполнения благодаря алгоритмам балансировки нагрузки.



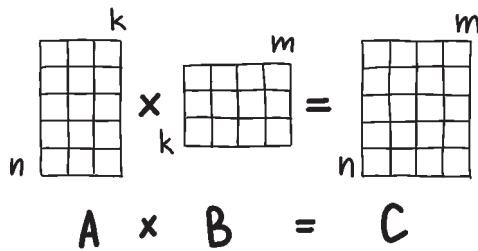
ПРИМЕЧАНИЕ Проектируя конкурентные системы, многие совершают распространенную ошибку: они выбирают конкретные механизмы конкурентности на слишком ранней стадии проектирования. У каждого механизма есть свои достоинства и недостатки, и то, какой механизм лучше подойдет для конкретного сценария использования, часто определяется неочевидными компромиссами и уступками. Чем раньше вы выбираете механизм, тем меньше у вас будет информации, на основании которой вы принимаете решения.

Таким образом, машинно-независимые аспекты методологии проектирования (например, независимость задач) рассматриваются на ранней стадии, а машинно-зависимые аспекты откладываются до завершающих стадий проектирования. На первых двух этапах основное внимание уделяется конкурентности и масштабируемости, а также тому, чтобы найти алгоритмы с этими качествами. На третьем и четвертом шаге фокус смещается на эффективность и производительность. Реализация конкурентной программы — это последний шаг, который должен обеспечить, чтобы предполагаемый алгоритм был эффективно реализован; возможно, при этом понадобится уделить особое внимание аспектам, которые зависят от специфики машины или алгоритма. В оставшейся части главы мы рассмотрим эти шаги подробнее и проиллюстрируем их на практических примерах.

Умножение матриц

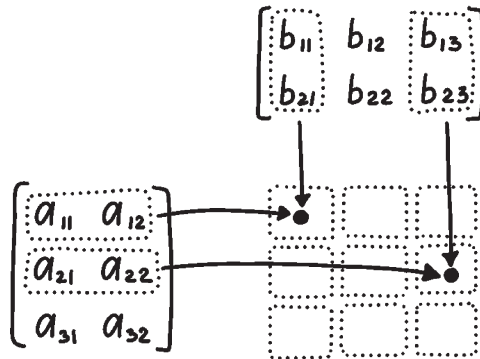
Рассмотрим применение методологии Фостера на примере умножения матриц. Каждая матрица представляется двумерным массивом массивов. Матрицу A можно умножить на матрицу B , если количество столбцов в матрице A равно количеству строк в матрице B .

Размеры произведения A и B , которое мы будем называть матрицей C , определяются количеством строк матрицы A и количеством столбцов матрицы B . Каждый элемент матрицы C — это произведение соответствующей строки A и столбца B .



$$A \times B = C$$

Таким образом, например, элемент $c_{2,3}$ вычисляется как произведение второй строки A и первого столбца B — или, если выразить в виде формулы, $c_{2,3} = a_{2,1} \times b_{1,3} + a_{2,2} \times b_{2,3}$.



Чтобы можно было сравнить разные подходы, сначала рассмотрим пример с последовательным алгоритмом:

```
# Chapter 13/matmul/matmul_sequential.py
import random
from typing import List
```

```
Row = List[int]
Matrix = List[Row]
```

```
def matrix_multiply(matrix_a: Matrix, matrix_b: Matrix) -> Matrix:
```

```
    num_rows_a = len(matrix_a)
    num_cols_a = len(matrix_a[0])
    num_rows_b = len(matrix_b)
    num_cols_b = len(matrix_b[0])
    if num_cols_a != num_rows_b:
```

```
        raise ArithmeticError(
            f"Неверные размеры; не удастся умножить "
            f"{num_rows_a}x{num_cols_a} на {num_rows_b}x{num_cols_b}"
        )
```

Создает новую матрицу, заполненную нулями, с количеством строк из матрицы A и количеством столбцов из матрицы B

```
    solution_matrix = [[0] * num_cols_b for _ in range(num_rows_a)]
    for i in range(num_rows_a):
        for j in range(num_cols_b):
            for k in range(num_cols_a):
                solution_matrix[i][j] += matrix_a[i][k] * matrix_b[k][j]
    return solution_matrix
```

← Для каждой строки матрицы A...

← Для каждой строки матрицы B...

← Для каждого столбца матрицы A...

```
if __name__ == "__main__":
```

```
    cols = 3
```

```
    rows = 2
```

```
    A = [[random.randint(0, 10) for i in range(cols)]
          for j in range(rows)]
```

```
    print(f"матрица A: {A}")
```

```
    B = [[random.randint(0, 10) for i in range(rows)]
          for j in range(cols)]
```

```
    print(f"матрица B: {B}")
```

```
    C = matrix_multiply(A, B)
```

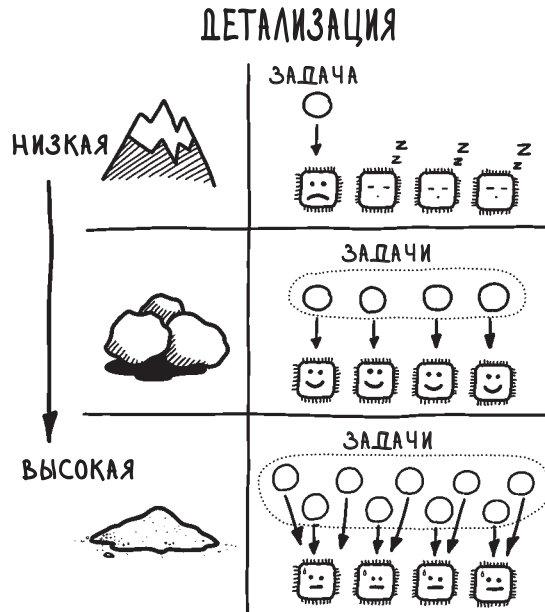
```
    print(f"матрица C: {C}")
```

Генерирует случайные матрицы

Здесь реализована последовательная версия умножения матриц, которая принимает две матрицы, A и B , и возвращает их произведение — матрицу C . Функция использует набор вложенных циклов `for`, которые перебирают строки A и столбцы B . Третий цикл `for` суммирует произведения элементов из строки A и столбца B , заполняя значениями результирующую матрицу C . Наша цель — спроектировать и построить конкурентную программу, которая вычисляет произведение двух матриц, и убедиться, что эта стандартная математическая задача сильно выигрывает, если применить конкурентные вычисления.

Разбиение

Первый шаг методологии Фостера — разбиение — нужен для того, чтобы найти возможности для конкурентного выполнения. Следовательно, цель здесь в том, чтобы выявить множество мелких задач и таким образом получить детализированную декомпозицию проблемы (см. главу 7). Подобно тому как мелкий песок проще собрать в кучу, чем кирпичи, высокодетализированная декомпозиция обеспечивает наибольшую гибкость в контексте потенциально конкурентных алгоритмов.



Цель

Цель разбиения — выявить задачи с максимально возможной степенью детализации. Это можно сделать только на этапе разбиения; другие этапы обычно снижают степень конкурентности, так что здесь необходимо выявить ее в полном объеме. На этой исходной стадии нас не интересуют такие практические вопросы, как коли-

чество процессорных ядер и тип целевой машины, и наше внимание сосредоточено на том, чтобы обнаружить возможности для параллельного выполнения.

ПРИМЕЧАНИЕ Количество задач, полученное на шаге разбиения, должно быть как минимум на порядок больше, чем количество процессоров на целевой машине. В противном случае у вас будет меньше вариантов на более поздних стадиях проектирования.

Декомпозиция данных и декомпозиция задач

Реализовывая конкурентный алгоритм, мы предполагаем, что он будет выполняться на нескольких вычислительных устройствах. Для этого необходимо изолировать в алгоритме группы операций, которые могут выполняться независимо; другими словами, мы его *декомпозируем*. Декомпозиция бывает двух видов: декомпозиция данных и декомпозиция задач (см. главу 7).

Если алгоритм должен обрабатывать большие объемы данных, можно попытаться разбить данные на части, каждую из которых может независимо обрабатывать отдельное вычислительное устройство. Это *декомпозиция данных*. Другой подход подразумевает, что вычисления разделяются в зависимости от их функциональности — это *декомпозиция задач*.

ПРИМЕЧАНИЕ Декомпозиция не всегда возможна. Некоторые алгоритмы не позволяют включать в реализацию несколько исполнителей. Чтобы ускорить такие алгоритмы, можно применять вертикальное масштабирование, но у него есть физические ограничения (см. главу 1).

Не забывайте, что декомпозиция данных и декомпозиция задач — это взаимодополняющие подходы к решению задач, и их можно органично совмещать. Разработчики обычно начинают с декомпозиции данных, потому что на нее опираются многие конкурентные алгоритмы. Но иногда декомпозиция задач позволяет взглянуть на проблему по-иному. Декомпозиция задач может выявить подводные камни или возможности для улучшения оптимизации, которые неопытный программист упустит, если будет просто просматривать данные.

Пример

Вернемся к примеру с умножением матриц. Программа перед вами, и можно задуматься о том, как ее декомпонировать и какие зависимости в ней скрываются. Какие части программы можно выполнять независимо?

Как следует из определения умножения матриц, каждый элемент матрицы C можно вычислять независимо от остальных элементов. В результате один из возможных подходов к разбиению умножения матриц заключается в том, что основная вычислительная подзадача определяется как вычисление одного элемента результирующей матрицы C . В таком случае общее количество подзадач равно $n \times m$ (в соответствии с количеством элементов этой матрицы).

Степень конкурентности, которой можно достичь с таким подходом, может показаться избыточной: подзадач может получиться гораздо больше, чем доступных ядер

процессора. Однако на текущей стадии это нормально; позже на стадии агломерации можно будет укрупнить вычисления в зависимости от конкретных требований.

Коммуникация

Следующий шаг процесса проектирования — организовать коммуникацию. На этом этапе вы определяете, как координировать процесс выполнения и как наладить каналы связи между задачами.

Цель

Когда все вычисления образуют одну последовательную программу, то все данные доступны всем частям программы. А когда вычисления делятся на независимые задачи, которые могут выполняться на разных процессорах или даже на разных ядрах, то одни данные, которые необходимы задаче, могут размещаться в ее локальной памяти, а другие — в памяти других задач. В любом случае этим задачам понадобится обмениваться данными друг с другом. Эффективно наладить такую коммуникацию может оказаться непростым делом. Даже простая декомпозиция может породить сложные коммуникационные структуры. Мы хотим свести к минимуму эти затраты в своей программе, поэтому важно четко их определить.

ПРИМЕЧАНИЕ Как упоминалось ранее, лучший способ реализовать конкурентность — это сократить коммуникацию и взаимозависимости между конкурентными задачами. Если каждая задача работает со своим собственным набором данных, ей не нужно защищать эти данные блокировками. Даже когда две задачи совместно используют один набор данных, можно попробовать разбить его или предоставить каждой задаче собственный экземпляр. Конечно, с копированием данных тоже сопряжены затраты, которые необходимо сопоставить с затратами на синхронизацию, прежде чем принимать решение.

Пример

Наш конкурентный алгоритм на этой стадии формулируется как набор задач, в котором каждая задача вычисляет значение элемента матрицы C и ожидает принять на вход одну строку матрицы A и один столбец матрицы B .

На стадии агломерации можно попытаться объединить задачи так, чтобы они вычисляли не один элемент матрицы C , а целую строку. В таком случае, чтобы выполнить необходимые вычисления, должна быть доступна одна строка матрицы A и все столбцы матрицы B . Простым решением было бы дублировать матрицу B в каждой задаче, но это может оказаться неприемлемым из-за значительных затрат памяти на хранение данных. Другой вариант — использовать общую память, потому что алгоритмы обращаются к матрицам A и B только для чтения, а элементы матрицы C вычисляются независимо друг от друга. На более поздних стадиях мы рассмотрим эти варианты и подумаем, какое решение будет оптимальным в нашей ситуации.

Агломерация

На первых двух шагах проектирования мы разбиваем вычисления, чтобы достичь максимальной конкурентности, и вводим коммуникацию между задачами, чтобы у них были необходимые данные. Итоговый алгоритм все еще остается абстракцией, потому что он не предназначен для выполнения на каком-то конкретном компьютере. Архитектура, которая сложилась на этот момент, вряд ли хорошо подойдет для реальной машины. Если задач гораздо больше, чем процессоров, то ресурсоемкость будет сильно зависеть от того, как задачи распределяются между процессорами. На третьем этапе (агломерация) мы переосмысливаем решения, которые приняли на стадиях разбиения и коммуникации.

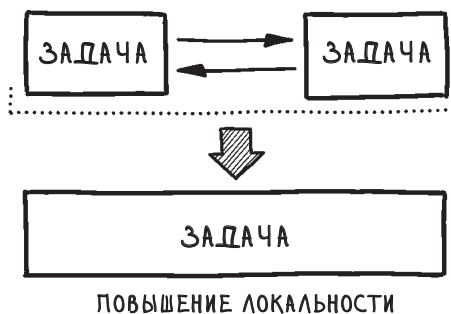
Цель

На этом этапе нужно улучшить производительность и упростить разработку, для чего группы задач часто объединяются в большие задачи. Эти цели часто конфликтуют друг с другом, и приходится идти на определенные компромиссы.

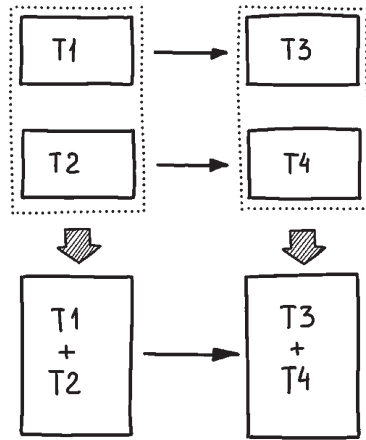
Если объединять задачи, время выполнения которых сильно различается, это в некоторых случаях может создать проблемы с производительностью. Например, если одна продолжительная задача объединяется со множеством коротких, то может оказаться, что коротким задачам придется подолгу ожидать, пока завершится продолжительная. С другой стороны, если разделять задачи, это может упростить архитектуру, но снизить производительность. В таких случаях приходится искать компромисс между простотой и производительностью.

Рассмотрим пример с уборкой снега из главы 7. Расчищать снег сложнее и медленнее, чем посыпать дорожки солью, поэтому работник с мешком соли может дать работнику с лопатой фору по времени, а затем начать разбрасывать соль. Когда он догонит коллегу, они могут поменяться ролями, чтобы уборщик немного отдохнул и взял мешок с солью, дав фору по времени своему напарнику. Эта схема применяется до тех пор, пока все задания не будут выполнены. Тем самым сокращается коммуникация между работниками и улучшается общая производительность.

Сокращение коммуникационных затрат — один из способов повысить производительность. Когда две задачи, которые обмениваются данными друг с другом, объединяются в одну, обмен данными становится частью одной задачи, и коммуникации пропадают вместе с затратами на них. Это называется *повышением локальности*.

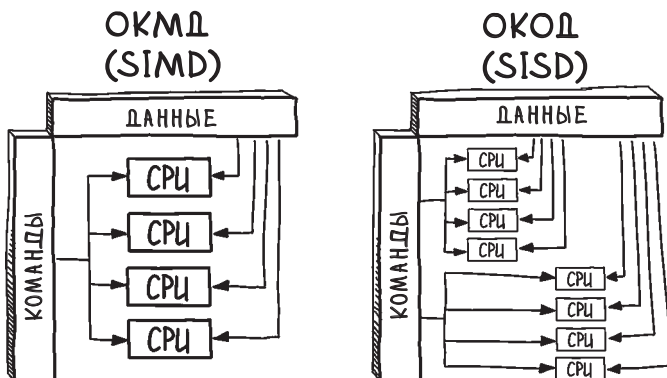


Другой способ сократить коммуникационные затраты — объединить в одну группу задачи, которые передают данные, а в другую — задачи, которые принимают данные (по возможности). Иначе говоря, представьте, что задача T1 передает данные задаче T3, а T2 передает данные T4. Если объединить T1 и T2 в одну задачу T1 + T2, а T3 и T4 — в одну задачу T3 + T4, то коммуникационные затраты снижаются. Время передачи не уменьшится, но общее время ожидания сократится вдвое. Помните, что когда задача ожидает данных, она не может выполнять вычисления, так что время, проведенное в ожидании, теряется впустую.



Пример

Когда выше мы разбивали проблему умножения матриц, мы использовали высокую детализацию. Нужно вычислить каждый элемент итоговой матрицы. Мы декомпозировали задачу умножения на отдельные задачи — по одной задаче для каждого элемента итоговой матрицы. Анализируя коммуникацию, мы определили, что каждой подзадаче должна быть доступна строка матрицы *A* и столбец матрицы *B*. Для компьютера ОКМД (с одним потоком команд и множественными потоками данных — см. главу 3) было бы замечательно, если бы потоки могли совместно использовать матрицы *A* и *B*; на такой машине программа работала бы лучше при большом количестве потоков. Естественным решением было бы поручить каждому потоку вычислять один элемент результата.



Но на обычном оборудовании — компьютере МКМД (с множественными потоками команд и множественными потоками данных — см. главу 3) — количество задач больше, чем количество процессоров (*p*). Если элементов в матрице больше, чем *p*, то задачи можно объединить, если сгруппировать несколько соседних строк и столб-

цов перемножаемых матриц в одну подзадачу. В таком случае исходная матрица A разбивается на горизонтальные полосы, а B — на вертикальные полосы. Размер полосы (d) в идеале должен быть равен $d = n \times m/p$ (при условии, что n кратно p), потому что при этом вычислительная нагрузка равномерно распределится между процессорами. Тем самым сокращается коммуникация между этими задачами, потому что все происходит локально внутри задачи.

ПРИМЕЧАНИЕ Слишком много агломерации — тоже плохо. Легко принять недалёковидное решение, которое ограничит масштабируемость программы. Хорошо спроектированная параллельная программа должна адаптироваться к изменению количества процессоров. Постарайтесь не устанавливать излишне жестких ограничений на количество задач в программе. Проектируйте систему так, чтобы использовать преимущества дополнительных ядер, если они появятся. Сделайте количество ядер входной переменной и учитывайте ее при проектировании.

Привязка

На последнем шаге методологии Фостера каждая задача связывается с вычислительным устройством. Конечно, такой проблемы не возникает на однопроцессорных машинах или компьютерах с общей памятью, где ОС обеспечивает автоматическое планирование задач. Если вы просто пишете программы для выполнения на настольном компьютере, как в примерах из этой книги, учитывать планирование не обязательно. Оно становится важным фактором в распределенных системах или на специализированном оборудовании со множеством процессоров для крупномасштабных задач. Мы рассмотрим эту ситуацию в следующем примере.

Цель

Привязка алгоритма преследует сразу две цели: минимизировать общее время выполнения программы и оптимизировать использование ресурсов. Для этого существуют две основные стратегии: либо размещать на разных процессорах задачи, которые можно выполнять параллельно (это улучшит общую конкурентность), либо размещать на одном процессоре задачи, которые часто взаимодействуют друг с другом (это улучшит локальность, благодаря тому что задачи будут располагаться близко друг к другу). В некоторых ситуациях можно использовать оба подхода, но они часто конфликтуют, а это значит, что при проектировании придется идти на компромиссы. Проектирование хорошего алгоритма привязки сильно зависит от структуры программы и от оборудования, на котором она выполняется; к сожалению, эта тема выходит за рамки книги.

Пример

В нашем примере с умножением матриц привязка и планирование задач делегируются ОС, поэтому нам не придется беспокоиться об этих вопросах.

Реализация

В нашем проектировании остается сделать еще несколько шагов. Во-первых, надо провести какой-нибудь простой анализ производительности, чтобы выбрать один из нескольких альтернативных алгоритмов и убедиться, что проект соответствует требованиям и целям по производительности. Также стоит хорошенько подумать о том, каковы затраты на реализацию алгоритма, можно ли повторно использовать существующий код и как все это вписывается в контекст большой системы, частью которой может быть алгоритм. Все эти вопросы привязаны к конкретному сценарию использования, и в реальных системах, скорее всего, появятся дополнительные сложности, которые придется рассматривать в каждом конкретном случае. Все эти вопросы тоже выходят за рамки книги.

Ниже приведен пример реализации конкурентного умножения матриц:

```
# Chapter 13/matmul/matmul_concurrent.py
from typing import List
import random
from multiprocessing import Pool
```

```
Row = List[int]
Column = List[int]
Matrix = List[Row]
```

```
def matrix_multiply(matrix_a: Matrix, matrix_b: Matrix) -> Matrix:
    num_rows_a = len(matrix_a)
    num_cols_a = len(matrix_a[0])
    num_rows_b = len(matrix_b)
    num_cols_b = len(matrix_b[0])
    if num_cols_a != num_rows_b:
        raise ArithmeticError(
            f"Неверные размеры; не удастся умножить "
            f"{num_rows_a}x{num_cols_a} на {num_rows_b}x{num_cols_b}"
        )
```

Создает новый пул процессов, чтобы вычислять матрицу в конкурентном режиме

```
pool = Pool()
results = pool.map(
    process_row,
    [(matrix_a, matrix_b, i) for i in range(num_rows_a)])
pool.close()
pool.join()
return results
```

Применяет к каждой строке матрицы функцию, которая принимает на вход матрицы A и B и индекс текущей строки i и возвращает список результатов

```
def process_row(args: tuple) -> Column:
    matrix_a, matrix_b, row_idx = args
    num_cols_a = len(matrix_a[0])
    num_cols_b = len(matrix_b[0])
```

Умножает строку матрицы A на каждый столбец матрицы B и возвращает полученный столбец

```
result_col = [0] * num_cols_b
for j in range(num_cols_b):
    for k in range(num_cols_a):
        result_col[j] += matrix_a[row_idx][k] * matrix_b[k][j]
return result_col
```

```

if __name__ == "__main__":
    cols = 4
    rows = 2
    A = [[random.randint(0, 10) for i in range(cols)] for j in range(rows)]
    print(f"матрица A: {A}")
    B = [[random.randint(0, 10) for i in range(rows)] for j in range(cols)]
    print(f"матрица B: {B}")
    C = matrix_multiply(A, B)
    print(f"матрица C: {C}")

```

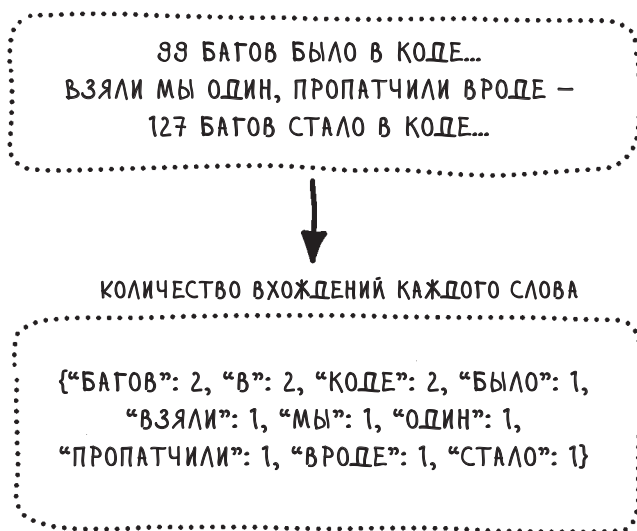
Эта программа определяет функцию `matrix_multiply`, которая принимает на вход две матрицы и вычисляет их произведение в конкурентном режиме. С помощью пула процессов вычисление разбивается на меньшие задачи, каждая из которых конкурентно вычисляет отдельный столбец матрицы решения. Программа собирает результаты этих задач и сохраняет их в итоговой матрице.

Этот пример впечатляет, но многие фреймворки и библиотеки уже умеют решать подобные математические задачи. Рассмотрим другую, более реалистичную проблему: на каких-нибудь курсах по большим данным ее сочтут вводным упражнением уровня «Hello world», но мы реализуем ее исключительно на Python.

Распределенный подсчет слов

Распределенный подсчет слов — классический пример задачи больших данных, которую можно решить с помощью распределенных вычислений. Требуется подсчитать количество вхождений каждого слова в большом наборе данных — обычно в текстовом файле или множестве файлов. Хотя задача кажется простой, она может отнять много времени и ресурсов при работе с крупными объемами данных.

Чтобы продемонстрировать, насколько важна эта задача, вспомним печально известный инцидент при переиздании Библии короля Якова в 1631 году. В процессе печати каждую букву (из 3 116 480) нужно было поместить на нижнюю пластину печатного станка, чтобы сформировать все 783 137 слов Библии. Однако из-за досадной ошибки в знаменитом стихе была пропущена частица «не». Результат стал известен как «Библия грешников», потому что одна из десяти заповедей гласила: «Прелюбодействуй». Если бы печатники могли автоматически подсчитывать все слова и буквы, которые должны войти в готовый продукт, этой фатальной ошибки можно было бы избежать. Этот случай напоминает о том, как важно точно и эффективно подсчитывать слова, особенно при работе с большими наборами данных.



Для начала напишем простую последовательную программу:

```
# Chapter 13/wordcount/wordcount_seq.py
import re
import os
import glob
import typing as T
```

```
Occurrences = T.Dict[str, int]
```

```
ENCODING = "UTF-8"
```

```
def wordcount(filenamees: T.List[str]) -> Occurrences:
    word_counts = {}
```

```
    for filename in filenamees:
        print(f"Обсчитывается {filename}")
        with open(filename, "r", encoding=ENCODING) as file:
```

Для каждой строки в текущем файле...

```
        for line in file:
```

```
            words = re.split("\W+", line)
```

...разбивает строку на отдельные слова с помощью регулярного выражения, которое соответствует промежуткам между словами

```
            for word in words:
                word = word.lower()
                if word != "":
```

```
                word_counts[word] = 1 + word_counts.get(word, 0)
```

```
    return word_counts
```

```
if __name__ == "__main__":
```

```
    data = list(
```

```
        glob.glob(f"{os.path.abspath(os.getcwd())}/input_files/*.txt"))
```

```
    result = wordcount(data)
```

```
    print(result)
```

Включает слово в подсчет, если оно не пусто

Для каждого файла наше приложение читает текст, делит его на слова (игнорируя знаки препинания и регистр) и заносит количество вхождений каждого слова в словарь. Для каждого слова создается пара «ключ — значение» (слово, 1). Таким образом, слово служит ключом, а соответствующее ему число 1 означает, что слово было обнаружено один раз.

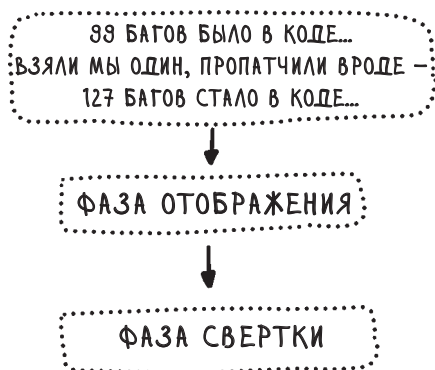
Наша цель — спроектировать и реализовать конкурентную программу, которая вычисляет количество вхождений каждого слова в каждом документе из файлов общим объемом в несколько гигабайт на распределенном компьютерном кластере. Давайте опять пройдем все четыре стадии, но на этот раз для новой задачи.

ПРИМЕЧАНИЕ Задача подсчета слов используется уже в нескольких поколениях продуктов распределенных баз данных, чтобы продемонстрировать их возможности. Она впервые была представлена в MapReduce, а затем задействована во многих других решениях, включая Pig, Hive и Spark.

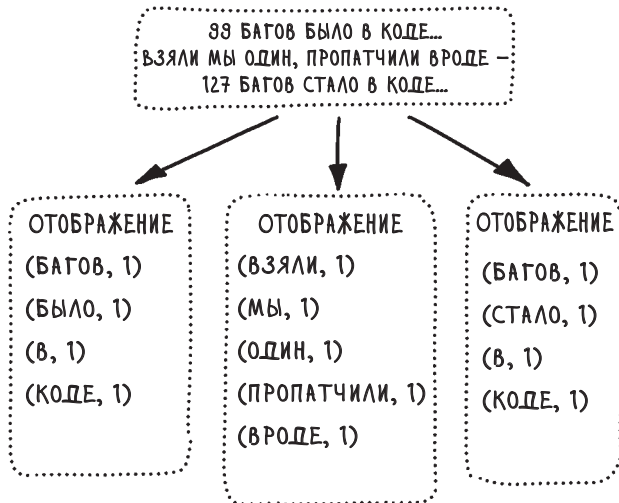
Разбиение

Чтобы создать решение, которое связывает каждое слово с его частотой в наборе данных, нужно справиться с двумя основными задачами: разбить текстовые файлы на отдельные слова и подсчитать количество вхождений каждого слова. Вторая задача зависит от завершения первой, так что нельзя начать подсчитывать вхождения до того, как текст будет разделен на слова. Это типичный пример декомпозиции задач, когда проблема разбивается на меньшие задачи в зависимости от их функциональности. При таком подходе мы уделяем основное внимание тому, какие задачи нужно выполнить, а не тому, какие данные нужны для вычислений.

Эта ситуация также отлично подходит для того, чтобы применить паттерн «Отображение/свертка», который был представлен в главе 7. Вычисления можно разделить на две фазы: *отображение* (map) и *свертка* (reduce).

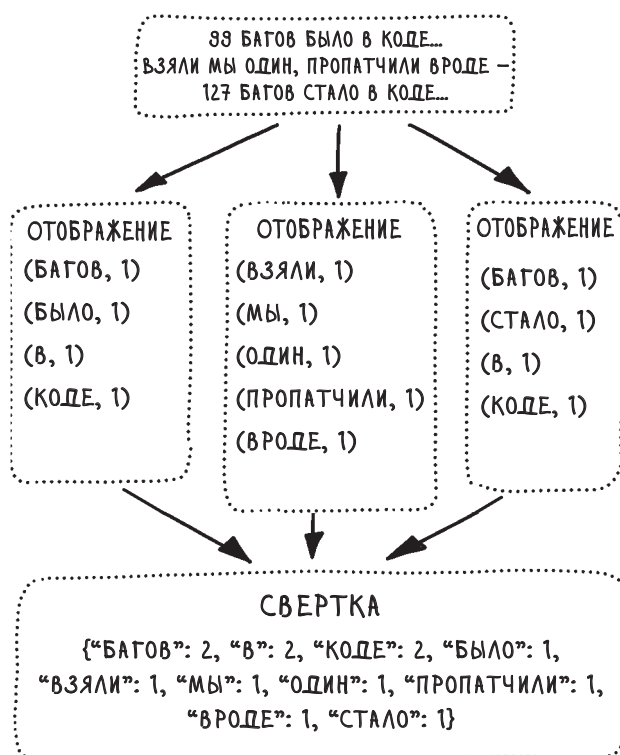


В фазе отображения программа читает текстовые файлы и разбивает их на пары «ключ — значение». Чтобы достичь максимальной конкурентности в этой фазе (к чему мы стремимся на стадии разбиения), можно разделить входные данные на блоки. Если у нас M исполнителей, то желательно получить M блоков, чтобы у каждого исполнителя были данные, с которыми можно работать. Количество исполнителей зависит в основном от того, сколько машин в вашем распоряжении.



Какими бы сложными ни были данные, которые вы пытаетесь обрабатывать, фаза отображения порождает события, которые состоят из ключа и значения. Ключ играет важную роль в фазе свертки.

Задача свертки получает выходные данные задачи отображения (список пар «ключ — значение») и объединяет все значения для каждого уникального ключа. Например, если отображение выработало список [("в", 1), ("коде", 1), ("в", 1)], задача свертки объединяет значения для ключа "в" и получает вывод [("в", 2)]. Это называется *сверткой* или *агрегированием* данных. Выходные данные свертки — список уникальных ключей и связанных с ними счетчиков.



Можно составить и другой алгоритм: создать несколько задач свертки и назначить каждой из них список слов для обработки. Выбрать лучшую реализацию предстоит на следующих этапах.

Невозможно предсказать, какому исполнителю достанется для чтения тот или иной файл. Это может быть любой файл в любом порядке. Таким образом у программы появляется хороший потенциал для горизонтального масштабирования. Просто добавьте больше исполнителей, и можно будет читать больше файлов одновременно. Если бы у нас были неограниченные аппаратные мощности, мы могли бы читать все файлы параллельно, и общее время чтения данных зависело бы от объема самого длинного текста.

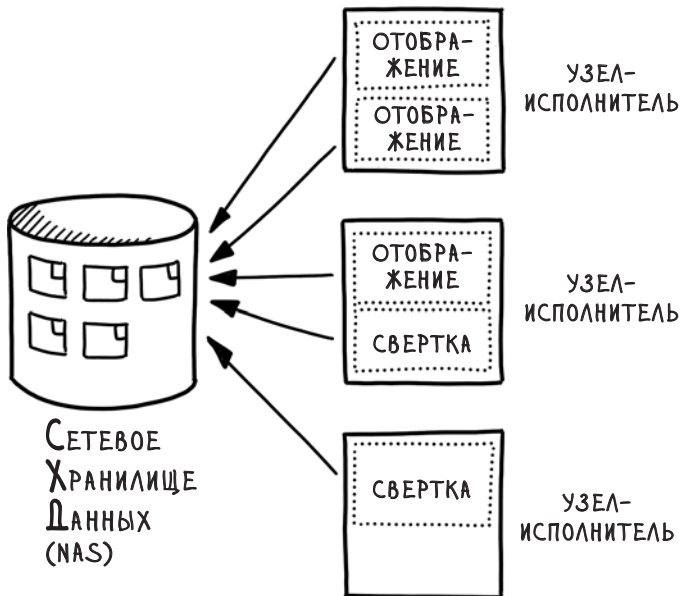
Коммуникация

Всем исполнителям нашего кластера назначаются блоки данных для чтения. В примере с подсчетом слов представьте, будто мы читаем огромное количество текстовых файлов — например, все книги библиотеки, где каждая книга представлена отдельным файлом.

Чтобы хранить и распределять эти текстовые данные, можно воспользоваться *сетевым хранилищем данных*, или *NAS* (network attached storage). Это решение сочетает хранилище данных большого объема со специальной аппаратной платформой, которая позволяет подключить хранилище к локальной компьютерной сети.

При этом нам не придется беспокоиться о сложных протоколах передачи данных, и каждый узел кластера сможет обращаться к файлам так, как если бы они хранились на локальном диске.

Задачи отображения и свертки должны выполняться на произвольных машинах нашего кластера без какого-либо общего контекста. Они могут запускаться как на одной машине, так и на разных. Это значит, что все данные, которые выводит фаза отображения, должны передаваться фазе свертки; возможно, их придется записывать на диск, если они слишком велики, чтобы поместиться в памяти (как это часто бывает). В такой ситуации есть несколько вариантов, и первый из них — межпроцессные коммуникации (IPC) с передачей сообщений (см. главу 5). Также можно хранить промежуточные результаты задач отображения как общие данные, а задачи свертки могут использовать общий том NAS. Мы поступим именно так.



Другой фактор, который необходимо учесть, — будет ли коммуникация синхронной или асинхронной. При синхронной коммуникации всем задачам придется ждать, пока завершится весь обмен данными, прежде чем они смогут продолжить свою работу. Это может привести к тому, что задачи будут проводить много времени в ожидании обмена данными вместо полезной активности.

Между тем при асинхронной коммуникации задача, которая передала асинхронное сообщение, может немедленно выполнять другую работу независимо от того, когда задача-получатель примет сообщение. При этом следует учесть, сколько ресурсов понадобится на обработку, которая необходима для этой конкретной стратегии коммуникации. В конце концов, если процессор тратит время на то, чтобы передавать и принимать данные, — значит, он не тратит это время на то, чтобы обрабатывать данные.

Для нашей проблемы удобно использовать асинхронную коммуникацию. Мы имеем дело с продолжительными задачами, которые не требуют блокирующего выполнения и при этом активно обмениваются данными друг с другом.

Агломерация

Сейчас каждая задача отображения вырабатывает пары слов (слово, 1). Очень простой способ ускорить обработку заключается в том, чтобы предварительно агрегировать эти пары локально для каждой задачи отображения — до того, как завершится фаза отображения и начнется фаза свертки. Этот шаг называется *слиянием* (combine) и напоминает свертку. Он получает произвольный список промежуточных пар «ключ — значение», сгруппированный по ключу, агрегирует значения (если это возможно) и выводит уменьшенный список пар «ключ — значение». Иначе говоря, он может предварительно агрегировать некоторые промежуточные значения, чтобы сократить затраты на коммуникацию между задачами отображения и свертки.

Кроме того, вспомнив наши прежние соображения о том, сколько задач свертки нужно, чтобы упростить алгоритм, мы будем использовать только одну задачу свертки: вся свертка сливается в одну большую задачу свертки. После того как мы добавили задачу слияния, данных для обработки будет не так много, так что эта схема должна работать нормально.

Привязка

После фазы агломерации мы оказываемся на месте композитора, который подготовил все для исполнения симфонии. Однако прекрасные звуки оркестра прозвучат только благодаря дирижеру, который координирует усилия отдельных музыкантов и вносит свой стиль в исполняемое произведение. Да, речь идет о том, как планировать задачи на имеющихся вычислительных ресурсах.



Самое важное (и самое сложное) в алгоритме планирования задач — стратегия распределения задач между исполнителями (воркерами). Обычно стратегия становится

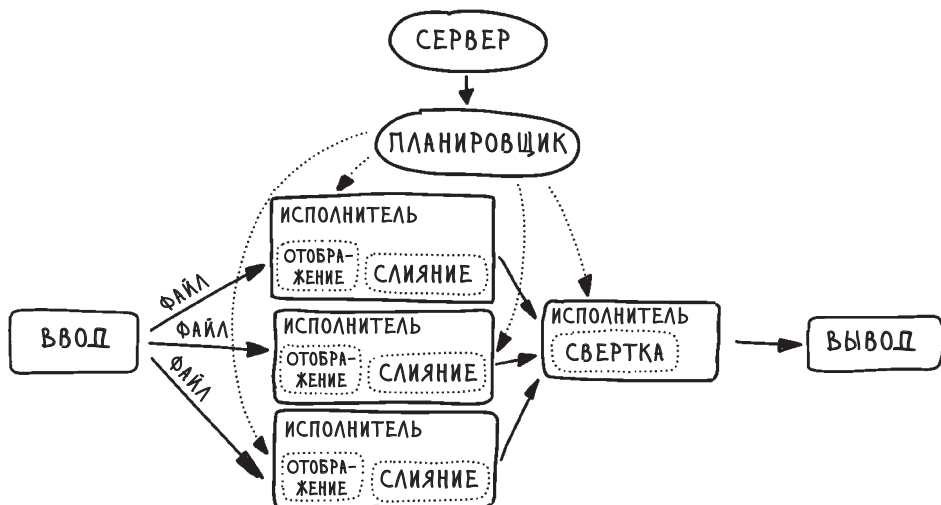
компромиссом между конфликтующими требованиями, одно из которых — независимость работы (чтобы снизить коммуникационные затраты), а другое — глобальное знание о состоянии вычислений (чтобы лучше сбалансировать нагрузку).

Мы реализуем простейшую идею: центральный планировщик, который управляет задачами исполнителям, отслеживает ход их выполнения и возвращает результаты. Он выбирает бездействующих исполнителей и назначает им либо задачу отображения, либо задачу свертки. Когда все исполнители завершат свою задачу отображения, планировщик оповещает их о том, что следует приступить к задаче свертки (в нашем случае исполнитель только один).

Каждый исполнитель в цикле запрашивает у планировщика задачи и завершает их, возвращая результаты работы планировщику. Эффективность этой стратегии зависит от количества исполнителей и от относительных затрат на получение и завершение задач. Мы используем довольно сложную стратегию динамического распределения задач, потому что количество файлов и их размеры заранее неизвестны — а значит, нельзя гарантировать оптимальное распределение задач до начала работы.

Реализация

На следующей диаграмме схематично изображена логика всей программы. Сервер инициирует выполнение и создает центральный планировщик. Каждому исполнителю отображения назначается файл для обработки. Если файлов больше, чем исполнителей, то после того, как исполнитель закончил обрабатывать файл, ему назначается еще один. Перед тем как задача отображения завершается, запускаются задачи слияния, которые агрегируют вывод задач отображения, чтобы сократить коммуникационные затраты. Когда фаза отображения будет завершена, планировщик переходит к фазе свертки, в которой все выходные данные отображения объединяются в один выходной набор.



Основную функциональность сервера мы реализуем так:

```
# Chapter 13/wordcount/server.py
import os
import glob
import asyncio
```

```
from scheduler import Scheduler
from protocol import Protocol, HOST, PORT, FileWithId
```

```
class Server(Protocol):
```

```
    def __init__(self, scheduler: Scheduler) -> None:
        super().__init__()
        self.scheduler = scheduler
```

Определяет метод,
который будет
вызываться, когда
новый исполнитель
подключается к серверу

```
    def connection_made(self, transport: asyncio.Transport) -> None:
        peername = transport.get_extra_info("peername")
        print(f"Новое подключение исполнителя {peername}")
        self.transport = transport
        self.start_new_task()
```

```
    def start_new_task(self) -> None:
        command, data = self.scheduler.get_next_task()
        self.send_command(command=command, data=data)
```

Получает следующую
задачу от плани-
ровщика и передает
команду и данные ис-
полнителю

```
    def process_command(self, command: bytes,
                        data: FileWithId = None) -> None:
        if command == b"mapdone":
            self.scheduler.map_done(data)
            self.start_new_task()
        elif command == b"reducedone":
            self.scheduler.reduce_done()
            self.start_new_task()
        else:
            print(f"Получена неизвестная команда: {command}")
```

```
def main():
```

```
    event_loop = asyncio.get_event_loop()
    current_path = os.path.abspath(os.getcwd())
    file_locations = list(
        glob.glob(f"{current_path}/input_files/*.txt"))
    scheduler = Scheduler(file_locations)
```

Получает цикл
событий

Получает список
файлов во входном
каталоге

```
    server = event_loop.create_server(
        lambda: Server(scheduler), HOST, PORT)
    server = event_loop.run_until_complete(server)
```

Создает экземпляр Scheduler
со списком имен файлов
с данными

Создает сервер

Запускает сервер

```
print(f"Работа на {server.sockets[0].getsockname()}")
try:
    event_loop.run_forever()
finally:
    server.close()
    event_loop.run_until_complete(server.wait_closed())
    event_loop.close()

if __name__ == "__main__":
    main()
```

Пытается запустить бесконечный цикл событий. В секции `finally` останавливает сервер, дожидается, пока он остановится, и останавливает цикл событий

За коммуникацию с каждым процессом-исполнителем отвечает `Server` — основной процесс выполнения. Он также вызывает `Scheduler`, чтобы получать каждую следующую задачу для каждого исполнителя и координировать фазы отображения и свертки.

Функциональность исполнителя реализуется так:

```
# Chapter 13/wordcount/worker.py
import re
import os
import json
import asyncio
import typing as T
from uuid import uuid4
```

```
from protocol import Protocol, HOST, PORT, FileWithId, \
    Occurrences
```

```
ENCODING = "UTF-8"
RESULT_FILENAME = "result.json"
```

Запускается, если соединение с сервером потеряно

```
class Worker(Protocol):
    def connection_lost(self, exc):
        print("Сервер закрыл соединение")
        asyncio.get_running_loop().stop()
```

```
def process_command(self, command: bytes, data: T.Any) -> None:
    if command == b"map":
        self.handle_map_request(data)
    elif command == b"reduce":
        self.handle_reduce_request(data)
    elif command == b"disconnect":
        self.connection_lost(None)
    else:
        print(f"Получена неизвестная команда: {command}")
```

Функция отображения. Принимает на вход имя файла, открывает файл, читает каждую строку, разбивает ее на слова и возвращает каждое слово со счетчиком 1

```
def mapfn(self, filename: str) -> T.Dict[str, T.List[int]]:
    print(f"Запущено отображение для {filename}")
    word_counts: T.Dict[str, T.List[int]] = {}
    with open(filename, "r", encoding=ENCODING) as f:
        for line in f:
```

```

words = re.split("\W+", line)
for word in words:
    word = word.lower()
    if word != "":
        if word not in word_counts:
            word_counts[word] = []
        word_counts[word].append(1)
return word_counts

```

Функция слияния. Принимает словарь результатов, суммирует счетчики для каждого слова и возвращает словарь объединенных результатов

```

def combinefn(self, results: T.Dict[str, T.List[int]]) -> Occurrences:
    combined_results: Occurrences = {}
    for key in results.keys():
        combined_results[key] = sum(results[key])
    return combined_results

```

```

def reducefn(self, map_files: T.Dict[str, str]) -> Occurrences:
    reduced_result: Occurrences = {}
    for filename in map_files.values():
        with open(filename, "r") as f:
            print(f"Запущена свертка для {filename}")
            d = json.load(f)
            for k, v in d.items():
                reduced_result[k] = v + reduced_result.get(k, 0)
    return reduced_result

```

Функция свертки. Принимает словарь имен файлов (где ключи — идентификаторы файлов, а значения — имена файлов), читает каждый файл и объединяет результаты в общий словарь

Запускает функцию отображения

```

def handle_map_request(self, map_file: FileWithId) -> None:
    print(f"Отображается {map_file}")
    temp_results = self.mapfn(map_file[1])
    results = self.combinefn(temp_results)
    temp_file = self.save_map_results(results)
    self.send_command(
        command=b"mapdone", data=(map_file[0], temp_file))

```

Объединяет промежуточные результаты функции отображения

Передает серверу сообщение о том, что фаза отображения завершена

Сохраняет результаты слияния во временный файл и возвращает путь к файлу

```

def save_map_results(self, results: Occurrences) -> str:
    temp_dir = self.get_temp_dir()
    temp_file = os.path.join(temp_dir, f"{uuid4()}.json")
    print(f"Сохраняется в {temp_file}")
    with open(temp_file, "w") as f:
        d = json.dumps(results)
        f.write(d)
    print(f"Сохранено в {temp_file}")
    return temp_file

```

```
def handle_reduce_request(self, data: T.Dict[str, str]) -> None:
    results = self.reducefn(data)
    with open(RESULT_FILENAME, "w") as f:
        d = json.dumps(results)
        f.write(d)
    self.send_command(command="reducedone",
                      data=("0", RESULT_FILENAME))
```

Вызывает функцию `reducefn` для файлов с промежуточными результатами отображения

Сохраняет результаты свертки в файл JSON

Передает серверу сообщение о том, что фаза свертки завершена

```
def main():
    event_loop = asyncio.get_event_loop()
    coro = event_loop.create_connection(Worker, HOST, PORT)
    event_loop.run_until_complete(coro)
    event_loop.run_forever()
    event_loop.close()

if __name__ == "__main__":
    main()
```

В фазе отображения исполнители вызывают функцию `mapfn`, чтобы разобрать данные, а затем функцию `combinefn`, чтобы выполнить слияние результатов и записать промежуточные результаты «ключ — значение». В фазе свертки исполнитель получает промежуточные данные, однократно вызывает функцию `reducefn` для каждого уникального ключа и передает ей список всех значений, сгенерированных для этого ключа. Затем исполнитель записывает окончательные выходные данные в один файл, к которому пользовательский код сможет обращаться, после того как программа завершится.

А вот наша реализация планировщика:

```
# Chapter 13/wordcount/scheduler.py
import asyncio
from enum import Enum
import typing as T

from protocol import FileWithId

class State(Enum):
    START = 0
    MAPPING = 1
    REDUCING = 2
    FINISHED = 3

class Scheduler:
    def __init__(self, file_locations: T.List[str]) -> None:
        self.state = State.START
        self.data_len = len(file_locations)
        self.file_locations: T.Iterator = iter(enumerate(file_locations))
        self.working_maps: T.Dict[str, str] = {}
        self.map_results: T.Dict[str, str] = {}

    def get_next_task(self) -> T.Tuple[bytes, T.Any]:
```

```

if self.state == State.START:
    print("ЗАПУЩЕН")
    self.state = State.MAPPING

if self.state == State.MAPPING:
    try:
        map_item = next(self.file_locations)
        self.working_maps[map_item[0]] = map_item[1]
        return b"map", map_item
    except StopIteration:
        if len(self.working_maps) > 0:
            return b"disconnect", None
        self.state = State.REDUCING

if self.state == State.REDUCING:
    return b"reduce", self.map_results

if self.state == State.FINISHED:
    print("ЗАКОНЧИЛ РАБОТУ")
    asyncio.get_running_loop().stop()
    return b"disconnect", None

def map_done(self, data: FileWithId) -> None:
    if not data[0] in self.working_maps:
        return
    self.map_results[data[0]] = data[1]
    del self.working_maps[data[0]]
    print(f"ОТОБРАЖЕНИЕ: {len(self.map_results)}/{self.data_len}")

def reduce_done(self) -> None:
    print("СВЕРТКА: 1/1")
    self.state = State.FINISHED

```

Получает следующую задачу

Функция обратного вызова для события, когда закончено отображение файла

Функция обратного вызова для события, когда все файлы прошли отображение и свертку

Это центральный планировщик. В нашей реализации он разделен на несколько состояний:

- **Запуск (START)**. Планировщик инициализирует необходимые структуры данных.
- **Отображение (MAPPING)**. Планировщик распределяет все задачи отображения. Каждая задача — это отдельный файл, и когда сервер запрашивает следующую задачу, планировщик просто возвращает следующий необработанный файл.
- **Свертка (REDUCING)**. Планировщик останавливает все выполнение, кроме одной задачи свертки.
- **Завершение (FINISHED)**. Планировщик останавливает сервер, а значит, и всю программу.

ПРИМЕЧАНИЕ Я тестировал это приложение на книгах проекта «Гутенберг» (<https://www.gutenberg.org/help/mirroring.html>), и вся система работала достаточно быстро с несколькими гигабайтами данных.

Итоги

- В первых 12 главах этой книги рассматривались отдельные части головоломки, которая называется конкурентностью. В этой главе сведены воедино все знания, которые вы получили ранее.
- Прежде чем начинать писать конкурентную программу, мы сначала анализируем задачу, которую надо решить, и убеждаемся, что она оправдывает затраты на разработку конкурентной программы.
- Затем необходимо убедиться в том, что задачу можно разбить на подзадачи, а также обеспечить необходимую коммуникацию и координацию этих подзадач.
- На третьем и четвертом шагах абстрактный алгоритм становится осязаемым и эффективным, когда мы ориентируемся на конкретное параллельное оборудование, на котором должна выполняться программа. Это централизованная многопроцессорная система или мультимикомпьютер? Какие каналы коммуникации поддерживаются? Как объединить задачи, чтобы эффективно распределить их между процессорами?

Эпилог



На протяжении этой книги мы использовали много абстрактных понятий, чтобы продемонстрировать нюансы проектирования конкурентных систем. Мы приводили разные аналогии — от симфонических оркестров до регистратуры в поликлинике, от приготовления бургеров до уборки снега; мы проводили параллели, чтобы помочь вам понять сложные темы. И хотя мы признаём, что эта книга лишь затронула основы этой необъятной области, даже на этом простейшем уровне можно выделить разные стратегии разработки конкурентных приложений.

После того как вы прочитали все 13 глав, у вас наверняка сформировался надежный фундамент для того, чтобы еще глубже исследовать область конкурентности. Вам предстоит еще многое узнать! А теперь вступает оркестр! (♪♪♪ Музыка! ♪♪♪)